



SoftUni

**SVETLIN NAKOV, PhD**  
& TEAM

# Java

PROGRAMMING  
BASICS WITH **Java**



# Contents

|                                                            |     |
|------------------------------------------------------------|-----|
| Preface .....                                              | 11  |
| Chapter 1. First Steps in Programming.....                 | 25  |
| Chapter 2.1. Simple Calculations .....                     | 67  |
| Chapter 2.2. Simple Calculations – Exam Problems .....     | 87  |
| Chapter 3.1. Simple Conditions.....                        | 103 |
| Chapter 3.2. Simple Conditions – Exam Problems.....        | 123 |
| Chapter 4.1. More Complex Conditions .....                 | 135 |
| Chapter 4.2. More Complex Conditions – Exam Problems ..... | 155 |
| Chapter 5.1. Loops .....                                   | 175 |
| Chapter 5.2. Loops – Exam Problems .....                   | 203 |
| Chapter 6.1. Nested Loops.....                             | 221 |
| Chapter 6.2. Nested Loops – Exam Problems.....             | 237 |
| Chapter 7.1. Complex Loops .....                           | 251 |
| Chapter 7.2. Complex Loops – Exam Problems.....            | 273 |
| Chapter 8.1. Practical Exam Preparation – Part I.....      | 285 |
| Chapter 8.2. Practical Exam Preparation – Part II .....    | 309 |
| Chapter 9.1. Problems for Champions – Part I.....          | 325 |
| Chapter 9.2. Problems for Champions – Part II.....         | 339 |
| Chapter 10. Methods .....                                  | 353 |
| Chapter 11. Tricks and Hacks.....                          | 379 |
| Conclusion .....                                           | 391 |

# Free Book "Programming Basics with Java"

The free book **Programming Basics with Java** introduces readers to writing entry-level programming code (working with data, conditional statements, loops, and methods) using the Java programming language.

## Book Editions

This book is available in several versions in different programming languages:

- [Programming Basics with C# \(English\)](#)
- [Programming Basics with Java \(English\)](#)
- [Programming Basics with JavaScript \(English\)](#)
- [Programming Basics with Python \(English\)](#)
- [Programming Basics with C# \(Bulgarian\)](#)
- [Programming Basics with Java \(Bulgarian\)](#)
- [Programming Basics with JavaScript \(Bulgarian\)](#)
- [Programming Basics with Python \(Bulgarian\)](#)
- [Programming Basics with C++ \(Bulgarian\)](#)

## Book Details

Brief information about this edition:

- Title: **Programming Basics with Java**
- Authors: **Svetlin Nakov & Team**
  - Main authors: Asya Dikova-Kirova, Daniel Tsvetkov, Dimitar Dalev, Ivaylo Ivanov, Kristiyan Pamidov, Milena Georgieva, Nikolay Bankin, Petar Ivanov, Rositsa Nenova, Svetlin Nakov, Teodor Kurtev, Tsvetan Georgiev, Vaydin Kachanov, Ventsislav Petrov, Zhulieta Atanasova
  - Other contributors: Aleksander Krastev, Aleksander Lazarov, Aleksandar Peev, Alen Paunov, Angel Dimitriev, Daniel Tsvetkov, Denis Milanov, Desislava Dahterova, Dimitar Tatarski, Dimitar Vlahov, Dimo Dimov, Diyan Tonchev, Elena Rogleva, Hristiyan Hristov, Hristo Hristov, Iskra Nikolova, Ivelin Kirilov, Julieta Atanasova, Kalin Primov, Kiril Kolarov, Kristiyan Pamidov, Luboslav Lubenov, Mariya Nikolova, Monika Petkova, Nikolay Bankin, Nikolay Dimov, Pavlin Petkov, Petar Ivanov, Rositsa Nenova, Rozalina Zaharieva, Ruslan Filipov, Stefka Vasileva, Svetlin Nakov, Teodor Kurtev, Tonyo Zhelev, Tsvetan Iliev, Vanya Voykova, Vasko Viktorov, Venelin Bekyarov, Ventsislav Petrov, Yanitsa Vuleva, Yoana Maksimova, Yulian Linev, Zahariya Pehlivanova, Zhivko Nedyalkov
- ISBN: **978-619-00-1402-7** (9786190014027)
- Edition: **Faber Publishing, Sofia, Oct 2021**
- License: **CC-BY-SA**
- Source code: <https://github.com/SoftUni/Programming-Basics-Book-Java-EN>



# Programming Basics with Java

Svetlin Nakov and Team

Aleksander Krastev

Aleksander Lazarov

Aleksander Peev

Alen Paunov

Angel Dimitriev

Asya Dikova-Kirova

Daniel Tsvetkov

Daniel Tsvetkov

Denis Milanov

Desislava Dahterova

Dimitar Dalev

Dimitar Tatarski

Dimitar Vlahov

Dimo Dimov

Diyan Tonchev

Elena Rogleva

Hristiyan Hristov

Hristo Hristov

Iskra Nikolova

Ivaylo Ivanov

Ivelin Kirilov

Julieta Atanasova

Kalin Primov

Kiril Kolarov

Kristiyan Pamidov

Luboslav Lubenov

Mariya Nikolova

Milena Georgieva

Monika Petkova

Nikolay Bankin

Nikolay Dimov

Pavlin Petkov

Petar Ivanov

Rositsa Nenova

Rozalina Zaharieva

Ruslan Filipov

Stefka Vasileva

Svetlin Nakov

Teodor Kurtev

Tonyo Zhelev

Tsvetan Georgiev

Tsvetan Iliev

Vanya Voykova

Vasko Viktorov

Vaydin Kachanov

Venelin Bekyarov

Ventsislav Petrov

Yanitsa Vuleva

Yoana Maksimova

Yulian Linev

Zahariya Pehlivanova

Zhivko Nedyalkov

Zhulieta Atanasova

ISBN: 978-619-00-1402-7

<https://java-book.softuni.org>

Sofia, 2021

# “Programming Basics with Java” Book and Video Lessons

© Svetlin Nakov and Team, 2021

First Edition, October 2021

This book and video lessons are distributed **freely** under the [CC-BY-SA](#) open source license, which defines the following **rights** and **obligations**:

- **Sharing** – you can copy and distribute the book and videos freely in any format or media.
- **Adaptation** – you can copy, remix and modify portions of this book and video lessons and produce new content based on them.
- **Attribution** – when you use portions of this book and video lessons you should attribute the original source, along with link to it and this license, but not in any way that suggests the licensor endorses you or your use.
- **Share Alike** – if you remix, transform, or build upon this book and video lessons, you must distribute your contributions under the same license as the original.

All trademarks used in this book are the property of their respective owners.

Publisher: Faber Publishing, Veliko Tarnovo

ISBN: 978-619-00-1402-7

Official Web Site: <https://java-book.softuni.org>

Official Facebook Page: <https://fb.com/IntroProgrammingBooks>

Cover: Marina Shideroff – <https://behance.net/marinashideroff>

Source Code: <https://github.com/SoftUni/Programming-Basics-Book-Java-EN>

# Table of Contents

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| Contents .....                                                             | 11        |
| Table of Contents .....                                                    | 7         |
| Preface .....                                                              | 11        |
| Who is This Book Intended for? .....                                       | 11        |
| Why Did We Choose Java? .....                                              | 11        |
| The Book in Other Programming Languages: C#, JavaScript, Python, C++ ..... | 12        |
| Programming is Learned by a Lot of Writing, Not Reading! .....             | 12        |
| About Software University (SoftUni) .....                                  | 13        |
| How to Become a Software Developer? .....                                  | 16        |
| The History Behind This Book .....                                         | 21        |
| <b>Chapter 1. First Steps in Programming .....</b>                         | <b>24</b> |
| Videos .....                                                               | 25        |
| What Does It Mean to Program? .....                                        | 25        |
| Computer Programs .....                                                    | 25        |
| How Do We Write a Console Program? .....                                   | 29        |
| Java Development Kit (JDK) .....                                           | 29        |
| Installing JDK .....                                                       | 38        |
| Development Environment (IDE) .....                                        | 39        |
| Installing IntelliJ IDEA .....                                             | 31        |
| Typical Mistakes in Java Programs .....                                    | 38        |
| What Have We Learned from This Chapter? .....                              | 39        |
| First Steps in Programming – Exercises .....                               | 40        |
| Console, Graphical, and Web Applications .....                             | 45        |
| Exercises Graphical and Web Applications .....                             | 46        |
| <b>Chapter 2.1. Simple Calculations .....</b>                              | <b>66</b> |
| Videos .....                                                               | 67        |
| The System Console .....                                                   | 67        |
| Reading Integers from The Console .....                                    | 68        |
| Calculations in Programming .....                                          | 69        |
| Data Types and Variables .....                                             | 69        |
| Reading Floating-Point Numbers from The Console .....                      | 69        |
| Reading and Printing Text .....                                            | 71        |
| Problem: Greeting by Name .....                                            | 72        |
| Problem Concatenate Data .....                                             | 72        |
| Arithmetic Operations .....                                                | 73        |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| Concatenating Text and Numbers.....                              | 75         |
| Numerical Expressions.....                                       | 76         |
| Problems Simple Calculations.....                                | 76         |
| <b>Chapter 2.2. Simple Calculations – Exam Problems.....</b>     | <b>87</b>  |
| Reading Numbers from The Console .....                           | 87         |
| Printing Text Using Format Specifiers .....                      | 87         |
| Arithmetic Operators .....                                       | 88         |
| Concatenation.....                                               | 88         |
| Exam Problems .....                                              | 88         |
| <b>Chapter 3.1. Simple Conditions.....</b>                       | <b>103</b> |
| Video .....                                                      | 103        |
| Comparing Numbers .....                                          | 103        |
| Simple If Conditions .....                                       | 104        |
| If-Else Conditions.....                                          | 105        |
| The Curly Brackets {} After If / Else.....                       | 105        |
| Variable Scope .....                                             | 108        |
| Sequence of If-Else Conditions .....                             | 109        |
| Problems Sequence of If-Else Conditions .....                    | 110        |
| Debugging.....                                                   | 114        |
| Problems: Simple Conditions .....                                | 115        |
| <b>Chapter 3.2. Simple Conditions – Exam Problems .....</b>      | <b>123</b> |
| Exam Problems .....                                              | 123        |
| <b>Chapter 4.1. More Complex Conditions.....</b>                 | <b>135</b> |
| Video .....                                                      | 135        |
| Nested Conditions .....                                          | 135        |
| More Complex Conditions.....                                     | 138        |
| Logical "OR" .....                                               | 140        |
| Logical Negation.....                                            | 141        |
| The Parenthesis ( ) Operator .....                               | 142        |
| More Complex Conditions - Problems .....                         | 142        |
| Switch-Case Conditional Statement .....                          | 146        |
| Problems More Complex Conditions.....                            | 150        |
| <b>Chapter 4.2. More Complex Conditions – Exam Problems.....</b> | <b>155</b> |
| Nested Conditions .....                                          | 155        |
| Switch-Case Conditions .....                                     | 155        |

|                                                               |            |
|---------------------------------------------------------------|------------|
| Exam Problems .....                                           | 156        |
| <b>Chapter 5.1. Loops .....</b>                               | <b>174</b> |
| Video .....                                                   | 175        |
| Repetitions of Blocks of Code (For Loop) .....                | 175        |
| Code Snippet for The <b>for</b> Loop in IntelliJ IDEA .....   | 176        |
| Problems: Loops .....                                         | 184        |
| Problem: Graphical and Web Applications .....                 | 191        |
| <b>Chapter 5.2. Loops – Exam Problems .....</b>               | <b>203</b> |
| Exam Problems .....                                           | 203        |
| <b>Chapter 6.1. Nested Loops .....</b>                        | <b>220</b> |
| Video .....                                                   | 221        |
| Nested Loops .....                                            | 223        |
| Drawing More Complex Figures .....                            | 229        |
| <b>Chapter 6.2. Nested Loops – Exam Problems .....</b>        | <b>237</b> |
| Exam Problems .....                                           | 237        |
| <b>Chapter 7.1. Complex Loops .....</b>                       | <b>250</b> |
| Video .....                                                   | 251        |
| Loops With a Step .....                                       | 251        |
| While Loop .....                                              | 254        |
| Greatest Common Divisor (GCD) .....                           | 255        |
| The Euclidean Algorithm .....                                 | 256        |
| Do-While Loop .....                                           | 257        |
| Infinite Loops and <b>break</b> Operator .....                | 259        |
| Nested Loops and the <b>break</b> Operator .....              | 262        |
| Error Handling: <b>try-catch</b> Construction .....           | 264        |
| <b>Chapter 7.2. Complex Loops – Exam Problems .....</b>       | <b>272</b> |
| Exam Problems .....                                           | 273        |
| <b>Chapter 8.1. Practical Exam Preparation – Part I .....</b> | <b>284</b> |
| Practical Exam .....                                          | 285        |
| The Online Evaluation System (Judge) .....                    | 285        |
| Simple Calculations – Problems .....                          | 285        |
| Simple Conditions – Problems .....                            | 289        |
| Complex Conditions – Problems .....                           | 293        |
| Simple Loops – Problems .....                                 | 296        |
| Drawing Figures on The Console – Problems .....               | 300        |
| Nested Loops with More Complex Logic – Problems .....         | 305        |

|                                                                |            |
|----------------------------------------------------------------|------------|
| <b>Chapter 8.2. Practical Exam Preparation – Part II</b> ..... | <b>309</b> |
| Exam Problems .....                                            | 309        |
| <b>Chapter 9.1. Problems for Champions – Part I</b> .....      | <b>325</b> |
| More Complex Problems on The Studied Material.....             | 325        |
| <b>Chapter 9.2. Problems for Champions – Part II</b> .....     | <b>339</b> |
| More Complex Problems on The Studied Material.....             | 339        |
| <b>Chapter 10. Methods</b> .....                               | <b>353</b> |
| Video .....                                                    | 353        |
| What is a Method? .....                                        | 353        |
| Methods with Parameters .....                                  | 357        |
| Returning Result from a Method .....                           | 361        |
| Method Overloading .....                                       | 365        |
| Naming Methods. Best Practices When Working with Methods ..... | 368        |
| Problems.....                                                  | 371        |
| <b>Chapter 11. Tricks and Hacks</b> .....                      | <b>379</b> |
| Code Formatting .....                                          | 379        |
| The Official Java Code Conventions .....                       | 379        |
| Code Formatting Shortcuts in IntelliJ IDEA.....                | 380        |
| Naming Code Elements.....                                      | 381        |
| Shortcuts in IntelliJ IDEA .....                               | 382        |
| Code Snippets in IntelliJ IDEA.....                            | 383        |
| Code Debugging Techniques.....                                 | 385        |
| Tricks for Java Developers .....                               | 387        |
| <b>Conclusion</b> .....                                        | <b>391</b> |
| Developer Skills .....                                         | 391        |
| This Book is Only The First Step! .....                        | 391        |
| How to Proceed After This Book?.....                           | 391        |
| Recommended Resources for Developers.....                      | 393        |
| Good Luck to All! .....                                        | 394        |

# Preface

The book **Programming Basics with Java** introduces the readers to writing **programming code** at a beginner's level (basic coding skills), working with **the development environment** (IDE), using **variables** and data, **operators** and **expressions**, working with the **console** (reading input data and printing output), using **conditional statements** (**if**, **if-else**, **switch-case**), **loops** (**for**, **while**, **do-while**, **foreach**) and **methods** (declaring and calling methods, passing parameters, and returning values). This book teaches **basic coding skills**, using the programming language **Java** and the development environment **IntelliJ IDEA**. The learning material covered in this book delivers the basic training that is needed for a furthermore in-depth study of programming and prepares the reader for the practical exam at SoftUni.



This book only gives you **the first steps to programming**. It covers basic skills that you must develop for years, to reach a high enough level and start working as a programmer.

The book is used as an unofficial **textbook for school-level programming courses**: in the high schools, studying professions like **Programmer**, **Application Programmer** and **System Programmer**, as well as an additional teaching tool in the initial programming courses at the **secondary schools**, **mathematical** and **professional high schools**.

## Who is This Book Intended for?

This book is suitable for **absolute beginners in programming** who want to try what programming is and learn the basic constructions for writing program code used in software development, regardless of the programming language and the technologies used. The book gives a **solid basis** of practical skills that you can use in any future training in programming and software development.

## Why Did We Choose Java?

For this book, we choose the **Java** because it is a **modern programming language** for high-level programming, open-source, easy to learn, and **suitable for beginners**. Using Java is **widespread**, with a well-developed ecosystem, numerous libraries and technology frameworks, and accordingly, it gives many **perspectives** for development. Java combines paradigms of procedural, object-oriented, and functional programming in a modern way with easy syntax although it is more descriptive than Python and JavaScript. In this book, we will use **Java** language and **IntelliJ IDEA** development environment, which are available for free.

As we will explain more about later, **the programming language you start with does not make a significant difference** in the process of learning to code, but you still need to choose one. For this book, we will focus specifically on Java. You can find the principles and concepts explained here illustrated with other programming languages like C#, Python, and JavaScript

## The Book in Other Programming Languages: C#, JavaScript, Python, C++

This book on programming for beginners is available in several programming languages (or is in the process of being adapted for them):

- [Programming Basics with JavaScript \(English\)](#)
- [Programming Basics with Python \(English\)](#)
- [Programming Basics with C# \(English\)](#)
- [Programming Basics with Java \(English\)](#)
- [Programming Basics with JavaScript \(Bulgarian\)](#)
- [Programming Basics with Python \(Bulgarian\)](#)
- [Programming Basics with C# \(Bulgarian\)](#)
- [Programming Basics with Java \(Bulgarian\)](#)
- [Programming Basics with C++ \(Bulgarian\)](#)

If you prefer a different language, select it from the list above.

## Programming is Learned by a Lot of Writing, Not Reading!

If someone assumes they will learn to program by just reading a book, without writing code and solving problems, they are deluding themselves. Programming needs **a lot of practice**, with code writing every day and solving hundreds, even thousands of problems, seriously and persistently for years.

You need to: **solve a lot of problems**, to make mistakes, to fix, to search for solutions and information from the Internet, to try, to experiment, to find better solutions, to get used to the code, syntax, the programming language, the development environment, to search for errors and debugging the broken code, the algorithmic thinking, breaking the problems into smaller parts, gaining experience and raising your skills every day, because when you learn to write code, this is only the **first step to the profession of the "software engineer"**. You have a lot to learn!

We advise the reader, as a minimum, **to try out all the examples from the book**, to play with them, to change them, and test them. Even more important than the examples are **the exercises** because they develop the programmer's practical skills. This book provides nearly **150 practical coding exercises**, so it is a good foundation for developing coding and algorithmic thinking skills.

You need to **solve all the problems in the book** because programming is learned with practice! The exercises after each topic are carefully selected to cover in-depth the learning material. The purpose of solving all the problems is to provide **complete set of skills for writing programming code** at a beginner's level (which is the purpose of this book).





Solve all the exercises in the book. Otherwise, you won't learn anything! Programming is learned by writing a lot of code and solving thousands of problems!

## About Software University (SoftUni)

[The Software University \(SoftUni\)](#) is the largest training center for software engineers in South-Eastern Europe. Tens of thousands of students pass through the university every year. SoftUni was founded in 2014 as a continuation of the hard work of [Dr. Svetlin Nakov](#) in training **skillful software engineering professionals** by a practical, contemporary and high-quality education that combines fundamental knowledge with modern software technologies and a lot of practice.

## SoftUni: High-quality Practical Tech Education

The Software University provides **quality education, profession, job and diploma** for programmers, software engineers and IT professionals. SoftUni builds an extremely successful and strong **connection between education and industry** by collaborating with hundreds of software companies, provides job and internships of its students, creates quality professionals for the software industry, and directly responds to the needs of employers via the training process.

## The SoftUni Interactive Classroom

SoftUni's self-paced programs teach software developers worldwide in the innovative **Interactive Classroom** that combines **video lessons, coding sessions, examples, and exercises** in a single platform on the Web.

Using the **SoftUni Interactive Platform**, you **write, execute and test code directly into the browser** and your exercise solutions are automatically evaluated by the integrated **Judge system**. If you have difficulties with an assignment, you can **get assistance** in multiple ways: with automated hints and guidelines or through SoftUni's **help center**.

This is how the **SoftUni Interactive Classroom** looks like:

**Statements**

- The **actions** that a program takes, are expressed as **statements**
- Common **statements** (actions / commands) include:
  - Declaring a variable**  
`int counter;`
  - Assigning a value**  
`counter = 1;`
  - Declaring + initializing**  
`int counter = 1;`
  - Printing a value**  
`System.out.println(counter);`
  - Modifying a value**  
`counter++;`  
`sum = a + b;`

**Calculate Speed**

```

1 import java.util.Scanner;
2
3 public class Program {
4     public static void main(String[] args) {
5         // Write code here
6     }
7 }
  
```

**Description**

Create a program that calculates the speed of a moving vehicle given the distance and time travelled.

- Read 2 floating-point numbers: distance and time
- Calculate the speed at which the vehicle needs to be travelling in order to get from point A to point B for the specified time.
- Print out the result
- Use the formula:  $\text{speed} = \text{distance} / \text{time}$

**Example**

| Input | Output |
|-------|--------|
| 15    | 7.5    |
| 2     |        |

**Submit**

## Video: SoftUni and SoftUni Judge

Watch a **video lesson** about SoftUni and the SoftUni Judge system here:

<https://youtube.com/watch?v=lwLET8SDBE4>.

## The Automated Judge System

The SoftUni Judge system (<https://judge.softuni.org>) is an automated Internet system for checking the solutions of programming exercises via series of tests. The submission and verification happen in **real time**: you submit the solution and within seconds you get an answer whether it is correct. You earn points for each task depending on your answer.

For a completely right one, you get the maximum points for the problem. With a partially correct solution, you receive a fraction of the points. And with a completely wrong answer, you get 0 points. This how the **SoftUni Judge** looks like:

The screenshot shows a web browser window with the URL `judge.softuni.org/Contests/Practice/Index/503#1`. The page title is "02. Expression". Below the title is a code editor with the following C# code:

```

1 using System;
2
3 namespace Expression
4 {
5     class Program
6     {
7         static void Main (string[] args)
8         {
9             Console.WriteLine((3522 + 52353) * 23 - (2336*501 + 23432 - 6743) * 3);
10        }
11    }
12 }

```

Below the code editor, there are submission details and a "Submit" button:

- Allowed working time: 0.100 sec.
- Allowed memory: 16.00 MB
- Size limit: 16.00 KB
- Checker: Numbers Checker ?
- A dropdown menu showing "C# code" with a downward arrow.
- A "Submit" button.

All problems from the current book are available for testing in **SoftUni Judge** and we strongly recommend testing them after you solve them to be sure you don't miss anything and that your solution works correctly according to the task requirements.

Keep in mind some **specifics about SoftUni Judge**:

- For each task the **Judge system keeps the best score you had**. Therefore, if you upload a solution with wrong code or lower score compared to the previous one, the system won't take away your points.
- The output of your program is **compared** by the system to a strictly expected result. Every **unnecessary symbol, missing comma or space** may lead to 0 points on a particular test. The **output** that the Judge system expects is **described in the requirements of every task** and **nothing else should be added**.
- **Example:** If the output requires to print a number (ex. **25**), do not display any descriptive messages such as **The result is: 25**, just print as it is required, i.e. only the number.

The SoftUni Judge system is **available any time** via its website: <https://judge.softuni.org>.

- Using the SoftUni system is **free** and it's not bound with the participation in SoftUni's courses.

We are convinced that after sending a few tasks **you will like getting instant feedback** for your solutions and the Judge system will become your favorite assistant in your programming practice.

## How to Become a Software Developer?

Dear readers, probably many of you have the ambition to become programmers and develop software for a living, or work in the IT area. That's why we have prepared for you a **short guide on "How to become a programmer"**, so we can help you take the first steps towards this desired profession.

You can become a programmer (working in a software company) after **at least 1-2 years of intensive training and coding every day**, solving thousands of programming tasks, development of several more serious practical projects and gaining a lot of experience with code writing and software development. You can't become a programmer for a month or two! The profession of software engineer requires **a lot of knowledge**, covered with **a lot of practice**.

### Video: Become a Software Engineer – 4 Essential Skills

Watch a **video lesson** about the 4 essential developer skills, needed to become a developer: <https://youtu.be/Ds5PD3UW57k>.

## The 4 Essential Skills of the Software Developers

There are **4 main skill groups** where all programmers must have. Most of these skills are resistant in time and are not influenced by the development in specific technologies (that are changing constantly). These are the skills that **any good programmer** has and to which every beginner must strive:

- Coding (20%)
- Algorithmic thinking (30%)
- Computer science and software engineering concepts (25%)
- Languages and software technologies (25%)

### Skill #1 – Coding (20%)

**Writing code** forms around 20% of the minimum knowledge and skills of a programmer, needed for starting a job in a software company. The skill of coding includes the following components:

- Working with variables, conditional statements, loops
- Using functions, methods, classes and objects
- Data manipulation: arrays, lists, hash tables, strings

The skill of coding **can be acquired in a few months** of hard learning and solving practical problems by writing code every day. This book covers only the first point of coding:

**working with variables, conditional statements and loops.** The rest remains to be learned in follow-up trainings, courses and books.

The book (and the courses based on it) gives only the beginning of one long and serious training on the way to professional programming. If you don't learn perfectly the material from this book, you can't become a programmer. You are going to miss fundamentals and it will be harder in the future. For this reason, **give enough time to programming basics:** solve many problems and write code every day for months until you learn to **solve every problem from the book very easily.** Then go ahead.

We specifically note that **the programming language does not matter** for the ability to code. You can code or not. If you can code with C#, you will easily learn to code with Java, C++ or other languages. That's why **the coding skills** are being studied quite seriously at the in all [SoftUni software engineering programs](#), and each programming book for beginners starts with them, including this one.

## Skill #2 – Algorithmic Thinking (30%)

The algorithmic (logical, engineering, mathematical, abstract) thinking forms around 30% of the minimum skills of a programmer needed to start the profession. **Algorithmic thinking** is the ability to break a particular problem into a logical sequence (algorithm), to find a solution for every separate step and then assemble the steps into a working solution. That is the most important skill of any programmer.

How to **build algorithmic thinking**?

- Algorithmic thinking is developed by solving **multiple programming (1000+) problems**, as diverse as possible. That is the recipe: solving thousands of practical problems, building algorithms, and implementing the algorithms, along with debugging the issues that come up in the process.
- Sciences like physics, mathematics and similar ones helps a lot, but they are not mandatory! People with **engineering and technical skills** usually learn very easily to think logically, because they already **have problem solving skills**, even if it is not algorithmic.
- The ability of **solving programming problems** (for which algorithmic thinking is needed) is extremely important for a programmer. Many companies test particularly this skill during their job interviews.

The current book develops the **beginner level of algorithmic thinking**, but it's not enough to make you a good programmer. To become good at this profession you must add **logical thinking and problem-solving skills**, beyond the range of this book. For example, working with **data structures** (arrays, lists, matrices, hash-tables, binary trees) and basic **algorithms** (searching, sorting, searching in tree structures, recursion, etc.).

**Algorithmic thinking skill** can be developed while studying in the [Software Developer programs](#) at SoftUni as well as in specialized algorithmic courses data structures and algorithms.

As you may have guessed, the choice of **programming language does not matter** for the development of algorithmic thinking. It is a skill unrelated to Programming. Because of

their well-developed logical thinking, there is the misconception that all programmers are smart people and having a high IQ is a requirement for entering into the profession.

## Skill #3 – Computer Science and Software Engineering Concepts (25%)

**Fundamental knowledge and skills** for programming, software development, software engineering and computer science comprise around 25% of the developer's minimum skills to start a job. Here are the more important of these skills and knowledge:

- **Basic mathematical concepts** related to programming: coordinate systems, vectors and matrices, discrete and non-discrete mathematical functions, state automata and state machines, combinatorics and statistics concepts, algorithm complexity, mathematical modeling, and others.
- **Programming skills** – code writing, working with data, using conditional statements and loops, working with arrays, lists and associative arrays, strings and text processing, working with streams and files, using programming interfaces (APIs), working with IDE, debugger, developer tools, etc.
- **Data structures and algorithms** – lists, trees, hash-tables, graphs, search, sorting, recursion, binary search trees, etc.
- **Object-oriented programming (OOP)** – working with classes, objects, inheritance, polymorphism, abstraction, interfaces, data encapsulation, exceptions management, design patterns.
- **Functional programming (FP)** – working with lambda functions, higher order functions, functions that return a function as a result, closure, etc.
- **Databases** – relational and non-relational databases, database modeling (tables and links between them), SQL query language, object-relational mapping (ORM), transactions and transaction management.
- **Network programming** – network protocols, network communication, TCP/IP, concepts, tools and technologies from computer networks.
- **Client-server** interaction, peer to peer communication, back-end technologies, front-end technologies, MVC architectures.
- **Technologies for server-side development (back-end)** – Web server architecture, HTTP protocol, MVC architecture, REST architecture, web development frameworks, templating engines.
- **Front-end technologies (client-side development)** – HTML, CSS, JS, HTTP, DOM, AJAX, communication with back-end, calling REST API, front-end frameworks, basic design and UX (user experience) concepts.
- **Mobile technologies** – mobile apps, Android and iOS development, mobile user interface (UI), calling server logic.

- **Embedded systems** – microcontrollers, digital and analog input and output control, sensor access, peripheral management.
- **Operating systems (OS)** – working with operating systems (Linux, Windows, etc.), installation, configuration and basic system administration, process management, memory, file system, users, multitasking, virtualization and containers.
- **Parallel and asynchronous programming** – thread management, asynchronous tasks, promises, common resources, and access synchronization.
- **Software engineering** – source control systems, development management, task planning and management, software development methodologies, software requirements and prototypes, software design, software architectures, software documentation.
- **Software testing** – unit testing, test-driven development, QA engineering, error reporting and error tracking, automation testing, build processes and continuous integration.

We need to once again mention that **the programming language does not matter** for acquiring all these skills. You will accumulate them slowly, over years. Some fundamental knowledge can be learned theoretically, but to get an in-depth understanding of it you need a lot of practice.

Fundamental knowledge and skills for programming, software development, software engineering, and computer science are taught during the [Software Developer Program](#), as well as a number of additional training courses. Working with a variety of software libraries, APIs, frameworks and software technologies and their interaction gradually builds this knowledge and skills, so do not expect that you will understand them from a single course, book or project.

Having **basic knowledge in the areas listed above** is enough to start a job as a programmer. Your further improvement will come as a result of the technology and development tools you will use in your day-to-day work.

## Skill #4 – Programming Languages and Software Technologies (25%)

**Programming languages and software development technologies** form around 25% of a programmer's skills. They have the largest learning content, but they change very fast over time. If we look at the **job offers** in the software industry, they usually mention words like the ones below, but in the job offers they secretly mention **the three main skills**: coding, algorithmic thinking and knowing the fundamentals of computer science and software engineering.

For those clearly technological skills **the programming language does matter**.

- **Note**: only for these 25% of the profession the programming language does matter!
- **For the rest 75% of the skills the programming language doesn't matter**, and these skills are resistant in time and transferable between different languages and technologies.

Here are some commonly used software development stacks which software companies are looking for (as of Jan 2019):

- **C#** + OOP + FP + classes from .NET + SQL Server databases + Entity Framework (EF) + ASP.NET MVC + HTTP + HTML + CSS + JS + DOM + jQuery + cloud + containers.
- **JavaScript (JS)** + OOP + FP + databases + MongoDB or MySQL + HTTP + web programming + HTML + CSS + JS + DOM + jQuery + Node.js + Express + Angular or React + cloud + containers.
- **Python** + OOP + FP + databases + MongoDB or MySQL + HTTP + web development + HTML + CSS + JS + DOM + jQuery + Django or Flask + cloud + containers.
- **Java** + Java API classes + OOP + FP + databases + MySQL + HTTP + web programming + HTML + CSS + JS + DOM + jQuery + JSP / Servlets + Spring MVC or Java EE / JSF + cloud + containers.
- **PHP** + OOP + databases + MySQL + HTTP + web development + HTML + CSS + JS + DOM + jQuery + Laravel or Symfony or other MVC framework for PHP + cloud + containers.
- **C++** + OOP + STL + Boost + native development + databases + HTTP + other languages and technologies.
- **Swift** + OOP + MacOS + iOS + Cocoa + Cocoa Touch + XCode + HTTP + REST + other languages and technologies
- **Go** + OOP + Linux + gRPC + cloud + containers + other languages and technologies.

If the words above look scary and incomprehensible, then you are at the very beginning of your career and you need **many years of learning** until you reach the profession of a software engineer. Do not worry, every programmer goes through one or several technology stacks and needs to study **a set of interconnected technologies**, but it is imperative for you is know how to write **programming logic** (to code) and **think algorithmically** (to solve programming problems). Becoming a good software engineer is impossible without those skills!

## The Programming Language Doesn't Matter!

As it already became clear, **the difference between programming languages** and more specifically between the skills of developers in different languages and technologies forms around **10-20% of the skills**.

- All programmers have around **80-90% of the same skills** that do not depend on the programming language! These are the skills to program and to design and develop software, that are very similar in different programming languages and development technologies.
- The more languages and technologies you know, the faster you will learn new ones, and the less you will feel the difference between them.

Let us state once again that the **choice of programming language (mostly) does not matter**, you just need to learn to program. This process starts with **coding** (by reading this



book or enrolling in a [Software Developer program](#)), continues with mastering more complex **programming concepts** (like data structures, algorithms, OOP, and FP), and includes using **fundamental knowledge and skills for software development, software engineering, and computer science**.

Only when you start working with a specific technology into a software project you will need a **specific programming language**, knowledge about specific programming libraries (APIs), frameworks and software technologies (front-end UI technologies, back-end technologies, ORM technologies, etc.). Keep calm, you will learn them, all programmers are learning them, but first you need to learn the foundation: **to program and do it well**.

This book uses the C# language, but it is not required and can be replaced with Java, JavaScript, Python, PHP, C++, Ruby, Swift, Go, Kotlin, or any other language. To be a **software developer**, you need to learn **coding** (20%), learn **algorithmic thinking**, and **solve problems** (30%), to have **fundamental knowledge of programming and computer science** (25%) and to master a **specific programming language and the technologies around it** (25%). Be patient, for a year or two all this can be mastered on a good starting level, if you are serious.

## The History Behind This Book

The main driver and project manager for the creation of this **free open-source programming book** are [Dr. Svetlin Nakov](#).

In 2014, the SoftUni initiative was launched. In the beginning, these lessons had a larger range and covered more theory. In 2016 Dr. Svetlin Nakov completely reworked, updated, and simplified the whole method of teaching, strongly emphasizing practice. This is how the core of the **learning content of this book was created**.

Following the principle of **free software and free knowledge**, Svetlin Nakov led a **team of volunteers** and started this open-source project. At first, the idea was to create a **free book** for the basics of programming in the C# programming language and later extend the initiative with other similar books in various popular programming languages (like Java, JavaScript and Python). This book follows the structure of [Programming Basics with C#](#).

The project is part of the hard work of the [Software University Foundation](#) to create and distribute open learning content to teach software engineers and IT professionals.

## Authors Team

This book is developed by a broad author's team of **volunteers** who dedicated their time to give away the systematized knowledge and guide you at the start of programming. Below is a list of the main authors (in alphabetical order):

Asya Dikova-Kirova, Daniel Tsvetkov, Dimitar Dalev, Ivaylo Ivanov, Kristiyan Pamidov, Milena Georgieva, Nikolay Bankin, Petar Ivanov, Rositsa Nenova, Svetlin Nakov, Teodor Kurtev, Tsvetan Georgiev, Vaydin Kachanov, Ventsislav Petrov, Zhulieta Atanasova

The book is based on its initial C# variant ([Programming Basics with C#](#)), which is developed by a large team of authors that has a significant contribution to the current book. Below is a list of the additional book contributors (editors, translators, others):

Aleksander Krastev, Aleksander Lazarov, Aleksander Peev, Alen Paunov, Angel Dimitriev, Daniel Tsvetkov, Denis Milanov, Desislava Dahterova, Dimitar Tatarski, Dimitar Vlahov, Dimo Dimov, Diyan Tonchev, Elena Rogleva, Hristiyan Hristov, Hristo Hristov, Iskra Nikolova, Ivelin Kirilov, Julieta Atanasova, Kalin Primov, Kiril Kolarov, Kristiyan Pamidov, Luboslav Lubenov, Mariya Nikolova, Monika Petkova, Nikolay Bankin, Nikolay Dimov, Pavlin Petkov, Petar Ivanov, Rositsa Nenova, Rozalina Zaharieva, Ruslan Filipov, Stefka Vasileva, Svetlin Nakov, Teodor Kurtev, Tonyo Zhelev, Tsvetan Iliev, Vanya Voykova, Vasko Viktorov, Venelin Bekyarov, Ventsislav Petrov, Yanitsa Vuleva, Yoana Maksimova, Yulian Linev, Zahariya Pehlivanova, Zhivko Nedyalkov.

Book cover design: Marina Shiderova.

## Official Web Site

The book **Programming Basics with Java** is available for free at the following web address: <https://java-book.softuni.org>.

The current book also has an [official Facebook page](#), from which you can keep track of news about the books from the series "Basics of Programming", new editions, events, and initiatives:

## License and Distribution

The book is distributed **freely** in electronic format under an open license [CC-BY-NC-SA](#). The book is published and distributed **on paper** by SoftUni, and you can buy a hard copy from online bookstores like Amazon.

You can find the **source code** of the book in its GitHub official repo: <https://github.com/SoftUni/Programming-Basics-Book-Java-EN>.

International Standard Book Number (ISBN): 978-619-00-1402-7.

## The Book That Helps Teachers

If you are a **teacher of programming**, informatics, or information technology or want **to teach programming**, this book gives you more than a well-structured learning material with many examples and tasks. **Free of charge** with the book you receive **quality educational content** for teaching in school, in the **Bulgarian language**, by the school requirements:

- **Educational presentations** (PowerPoint slides) for each learning topic tailored to 45-minute hours in schools – free of charge.
- Well-designed **tasks** for class and homework, with detailed conditions and sample entrance and exit – free of charge.
- **An automated task and homework verification system** (Online Judge System) to be used by students, also free of charge.
- **Video lessons** with methodological instructions from the free course for programming teachers, which is held regularly by the SoftUni Foundation.

## Reporting Bugs

If you find **mistakes**, inaccuracies or bugs in the book, you can report them in the official bug tracker of the project: <https://github.com/SoftUni/Programming-Basics-Book-Java-EN/issues>.

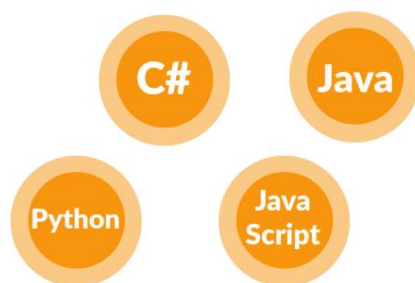
We do not promise to fix everything you send us, but we do want to **continually improve the quality** of the book, so that reported undoubted errors and all reasonable suggestions will be considered.

## Enjoy Reading

Don't forget **to write code** in large quantities, to **test the examples** that you'll find at the end of each chapter and more importantly **to solve the problems from the exercises**. You won't learn to program from just reading the book, so solve as many problems as you can find!

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



Apply now

[softuni.org/apply](https://softuni.org/apply)

# Chapter 1. First Steps in Programming

In this chapter, we are going to figure out **what programming is** at its core. We will get familiar with the idea of **programming languages**, and we will look at the **integrated development environments** (IDE) and how to work with them, particularly with **IntelliJ IDEA**. We will write and execute our **first program** with the programming language **Java**, and afterward, we will exercise with a few problems we will create a console program, a graphic application, and a web application. We are going to learn how to check for correctness the solutions from this book in the **Judge system of SoftUni**, and at the end, we will get familiar with the typical errors, which are made when writing code and how to avoid them.

## Videos

Watch a video lesson about getting started with Java:

<https://youtube.com/watch?v=sXM31yfsj04>.

Watch a video lesson about IntelliJ IDEA: <https://youtube.com/watch?v=3Hny8Mz2agQ>.

## What Does It Mean to Program?

To program means to give instructions to the computer, for example, *play a certain sound*, *print something on the screen*, or *multiply two numbers*. When there is a series of commands, we have a computer program (script). The computer program's text is called **program code** (source code or just code).

## Computer Programs

**Computer programs** represent a **series of commands** written in a particular **programming language**, like Python, C#, Java, JavaScript, Ruby, PHP, C, C++, Go, etc. To write commands, we need to be familiar with the **syntax and semantics of the language** we'll be using, in our case – **Java**. In this book, we will cover programming in general, as well as Python's particular syntax and semantics. We will examine each step of writing code, starting with the simplest and eventually reaching more complex programming constructs.

## Algorithms

Computer programs usually execute some sort of algorithm. **Algorithms** are a series of steps used to complete a problem and achieve an expected result - something like a recipe. For example, if we need to fry some eggs, we follow a specific recipe (algorithm): we heat some oil in a pan, then we break the eggs, we wait until they are cooked, and finally, move the pan away from the heat. Similarly, in programming, **computer programs execute algorithms** – a series of commands needed to complete a specific problem, for example, if we want to print a series of numbers in ascending order, we need an algorithm. It will go through all the numbers, find the smallest one and print it, then go through the rest of them and do the same until there are no more numbers left.

To make creating programs, writing program code (commands), executing it, and other operations that have to do with programming more convenient, we need a **development environment** (IDE) like IntelliJ.

## Programming Languages, Compilers, Interpreters, and Environments for Development

**Programming languages** are artificial languages (syntax for expression), designed to **issue commands** which we expect the computer to read, process, and execute. With the help of programming languages, we write a series of commands (**programs**) that **tell the computer what to do**. Execution of computer programs can be achieved by using either a **compiler** or an **interpreter**.

The **compiler** translates code from a programming language into **machine code** and for each construct (command) in the code, it chooses an appropriate, predefined fragment of machine code while simultaneously checking the program's text for **errors**. Together, all the compiled fragments represent the original program translated in machine code, exactly how the computer's microprocessor is expecting it. After it's been compiled, the program can be directly executed by the microprocessor in cooperation with the operating system. Compiled programming languages **compile the program** before executing it and find syntactic errors (incorrect commands) during compile time. Languages like C++, C#, Java, Swift, and Go work with compilers.

Some programming languages don't use a compiler but are **directly interpreted** by specialized software called an interpreter. The **interpreter** is a **program that executes programs**, written in some programming language. It executes the program's commands in sequence and understands not only single commands and series of commands but also other language constructs (checks, iterations, functions, etc.). PHP, Python, and JavaScript are languages that work with an interpreter and are executed without the need to be compiled. Due to lack of preliminary compilation, **interpreted languages' errors are found during the run time** after the program has already started working.

The **development environment** (Integrated Development Environment – IDE) combines traditional instruments for software development. In the development environment we write code, compile and execute programs. Development environments incorporate a **text editor** for writing code, a **programming language**, a **compiler** or **interpreter**, an **execution environment** where our program is executed, a **debugger** that traces our program and looks for errors, **instruments for design**, and **user interface** as well as other instruments and add-ons.

**Development environments** are convenient because they incorporate everything necessary to develop a program, so there's no need to leave the environment. If we don't use a development environment, we'll have to write code in a text editor, compile it with a command from the console, execute it with another command from the console and write additional commands, when needed, which will cost a lot of time. This is why most programmers use IDEs for everyday work.

For programming with the **Java language**, the most commonly used development environment is **IntelliJ IDEA**, which is developed and distributed by JetBrains and it can be downloaded free from their website: <https://jetbrains.com/idea>.

Alternatives of IntelliJ IDEA are **Eclipse** <https://eclipse.org/downloads/packages>, **NetBeans** <https://netbeans.apache.org/download>, and **JDeveloper** <https://www.oracle.com/tools/downloads/jdeveloper-12c-downloads.html> and many others. In the current book, we are going to use IntelliJ IDEA.

If you don't have an opportunity to install an IDE, you can use some online Java IDE like **Repl.it**: <https://repl.it/languages/java>.

## Low- and High-Level Languages, Runtime Environments

A program is, in essence, a **set of instructions** that ask the computer to carry out specific problems. They are entered by the programmer and **executed unconditionally by the machine**.

There are different types of **programming languages**. **Lowest-level** languages are used to write the **instructions commanding the processor** - **Assembler** is one such language. Higher-level languages are utilized to create an operating system, drivers for managing hardware (video card drivers, for example), web browsers, compilers, engines for game graphics (game engines), and other system components and programs. Even higher-level languages like **JavaScript**, **C#**, and **Python** are used to create application software, like programs for reading mail or chatting.

**Low-level languages** command hardware directly and require a lot of effort, and a vast number of commands, to carry out a specific problem. **Higher-level languages** require less code to achieve the same result but have no direct access to hardware. They are used to develop application software like web and mobile applications.

Most of the software we use daily like music players, video players, GPS programs, etc., is written on high-level **languages for application programming** like Python, JavaScript, C#, Java, C++, PHP, etc.

**Java is a compiled language**, and this means that we are writing commands, which are being compiled before they are executed. The code, written in Java, is not compiled to machine code for a specific processor, but for a specific language, **called Java bytecode**. Due to that, the execution of a program, written in Java, requires a virtual machine - **Java Virtual Machine (JVM)**.

The Java compiler, the virtual machine, and many libraries stand inside the core of the **Java platform**. The Java platform is developed in a few variations

- **Java Standard Edition (Java SE)** – the basic version, which is used for the development of client applications and the so-called Java applets.
- **Java Enterprise Edition (Java EE)** – based on the Standard Edition, it is mostly used with server applications on the internet, but besides that, it includes many other software technologies.



- **Java Micro Edition** (Java ME) – a standalone light version for working on devices with limited computing power like mobile phones, smartphones, PDA devices, and others.

## Computer Programs - Compilation and Execution

As we have already mentioned, the program is a **sequence of commands**, in other words, it describes a sequence of calculations, evaluations, iterations, and all kinds of similar operations, which aim to accomplish some result.

The program is written in text format, and the text of the program itself is called a **source code**. It is compiled to an executable file (for example, **Program.cs** is compiled to **Program.exe**), or it is executed directly from the .NET environment.

The compilation process of the code, before its execution, is used only in compiled languages like C#, Java, and C++. Scripts and interpreted languages, like JavaScript, Python, and PHP, the source code gets executed step by step by an interpreter.

## Computer Programs – Problems

Let us start with a simple example Java program, which consists of a single command.

### Problem: A Command Which Prints a Given Text

Our first program will consist of a **single Java command** which prints the word *"Hello"*

```
System.out.println("Hello");
```

In a moment, we will get to know **how to execute this command**, but for now, we will just stick to what commands in programming consist of. Let us have a look at a few more examples.

### Problem: A Program Which Checks Whether a Word Contains Another Word

We can check whether the word "alright" consist of the word "right" using the **contains** method:

```
System.out.println("alright".contains("right"));
```

### Problem: A Program Which Converts Bulgarian BGNs (leva) into Euro

Let us have a look at another simple program that reads a number of leva (Bulgaria's currency) (whole number) from the user, converts them into euro (divides them by the euro exchange rate), and prints the result. This is a program of three consecutive commands.

```
Scanner console = new Scanner(System.in);  
double leva = Double.parseDouble(console.nextLine());  
double euro = leva / 1.95583;
```



```
System.out.println(euro);
```

We looked at **three examples of computer programs**: two with a single command, and one consisting of a sequence of three commands. Let us now move on to the more exciting bit: writing, compiling, and executing our programs in **Java**.

## How Do We Write a Console Program?

Let's go through the necessary **steps to create and execute a computer program** that uses a text console (window for entering and printing text) to read and write its data. Such programs are called **console programs**. But first of all, we need to **install and prepare the development environment** where we will write and execute the Java programs from this book and the exercises that go along with it.

## Java Development Kit (JDK)

To create programs using the Java programming language, we need to install the **Java Development Kit**. It includes a **virtual machine**, a **Java compiler**, and numerous **helper tools**. JDK should not be confused for a runtime environment – **Java Runtime Environment (JRE)**. Installing JRE will allow us to open and work with Java programs but won't give us the ability to develop our own.

## Installing JDK

JDK is distributed freely and can be downloaded from this link:

<https://oracle.com/technetwork/java/javase/downloads>. Choose the latest version of Java SE (Java Standard Edition).

| Java SE Development Kit 8u144                                                                                           |           |                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| You must accept the <b>Oracle Binary Code License Agreement for Java SE</b> to download this software.                  |           |                                                                                                                                           |
| <input checked="" type="radio"/> <b>Accept License Agreement</b> <input type="radio"/> <b>Decline License Agreement</b> |           |                                                                                                                                           |
| Product / File Description                                                                                              | File Size | Download                                                                                                                                  |
| Linux ARM 32 Hard Float ABI                                                                                             | 77.89 MB  |  <a href="#">jdk-8u144-linux-arm32-vfp-hflt.tar.gz</a> |
| Linux ARM 64 Hard Float ABI                                                                                             | 74.83 MB  |  <a href="#">jdk-8u144-linux-arm64-vfp-hflt.tar.gz</a> |
| Linux x86                                                                                                               | 164.65 MB |  <a href="#">jdk-8u144-linux-i586.rpm</a>              |
| Linux x86                                                                                                               | 179.44 MB |  <a href="#">jdk-8u144-linux-i586.tar.gz</a>           |
| Linux x64                                                                                                               | 162.1 MB  |  <a href="#">jdk-8u144-linux-x64.rpm</a>               |
| Linux x64                                                                                                               | 176.92 MB |  <a href="#">jdk-8u144-linux-x64.tar.gz</a>            |
| Mac OS X                                                                                                                | 226.6 MB  |  <a href="#">jdk-8u144-macosx-x64.dmg</a>              |
| Solaris SPARC 64-bit                                                                                                    | 139.87 MB |  <a href="#">jdk-8u144-solaris-sparcv9.tar.Z</a>       |
| Solaris SPARC 64-bit                                                                                                    | 99.18 MB  |  <a href="#">jdk-8u144-solaris-sparcv9.tar.gz</a>      |
| Solaris x64                                                                                                             | 140.51 MB |  <a href="#">jdk-8u144-solaris-x64.tar.Z</a>           |
| Solaris x64                                                                                                             | 96.99 MB  |  <a href="#">jdk-8u144-solaris-x64.tar.gz</a>          |
| Windows x86                                                                                                             | 190.94 MB |  <a href="#">jdk-8u144-windows-i586.exe</a>            |
| Windows x64                                                                                                             | 197.78 MB |  <a href="#">jdk-8u144-windows-x64.exe</a>             |

We need to accept the license agreement and choose the installation file which corresponds to our operating system. Windows 64-bit, for instance.

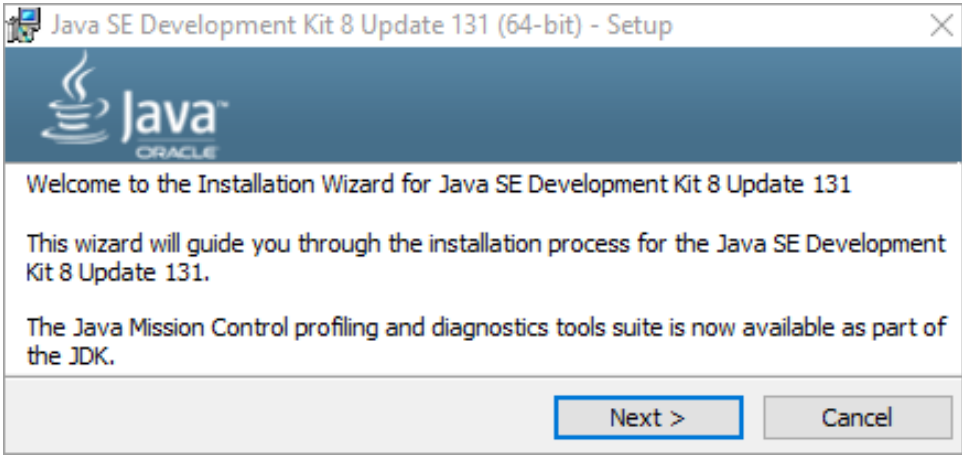
## Java SE Development Kit 8u144 Demos and Samples Downloads

You must accept the [Oracle BSD License](#). to download this software.

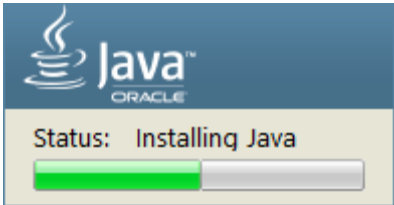
☒ **Accept License Agreement** ☐ Decline License Agreement

| Product / File Description  | File Size | Download                                                    |
|-----------------------------|-----------|-------------------------------------------------------------|
| Linux ARM 32 Hard Float ABI | 9.95 MB   | <a href="#">jdk-8u144-linux-arm32-vfp-hflt-demos.tar.gz</a> |
| Linux ARM 64 Hard Float ABI | 9.94 MB   | <a href="#">jdk-8u144-linux-arm64-vfp-hflt-demos.tar.gz</a> |
| Linux x86                   | 52.66 MB  | <a href="#">jdk-8u144-linux-i586-demos.rpm</a>              |
| Linux x86                   | 52.52 MB  | <a href="#">jdk-8u144-linux-i586-demos.tar.gz</a>           |
| Linux x64                   | 52.72 MB  | <a href="#">jdk-8u144-linux-x64-demos.rpm</a>               |
| Linux x64                   | 52.54 MB  | <a href="#">jdk-8u144-linux-x64-demos.tar.gz</a>            |
| Mac OS X                    | 53.09 MB  | <a href="#">jdk-8u144-macosx-x86_64-demos.zip</a>           |
| Solaris x64                 | 13.52 MB  | <a href="#">jdk-8u144-solaris-x64-demos.tar.Z</a>           |
| Solaris x64                 | 9.31 MB   | <a href="#">jdk-8u144-solaris-x64-demos.tar.gz</a>          |
| Solaris SPARC 64-bit        | 13.58 MB  | <a href="#">jdk-8u144-solaris-sparcv9-demos.tar.Z</a>       |
| Solaris SPARC 64-bit        | 9.34 MB   | <a href="#">jdk-8u144-solaris-sparcv9-demos.tar.gz</a>      |
| Windows x86                 | 53.8 MB   | <a href="#">jdk-8u144-windows-i586-demos.zip</a>            |
| Windows x64                 | 53.82 MB  | <a href="#">jdk-8u144-windows-x64-demos.zip</a>             |

Then we can download the installation file and run it. The installer which opens should look like this on a **Windows** environment:



Press the **[Next]** button a few times until the Java development Kit starts installing:



When it's finished, close the installer with the **[Close]** button. Installation on a Linux environment may be completely different, but most often, you need to install a standard package from the central repository `sudo apt install default-jdk`.

## Development Environment (IDE)

As was already noted, we need a development environment (IDE – Integrated Development Environment) to program. The IDE is a program editor, where we write program code and execute it, discover our mistakes, correct them and start the program again.

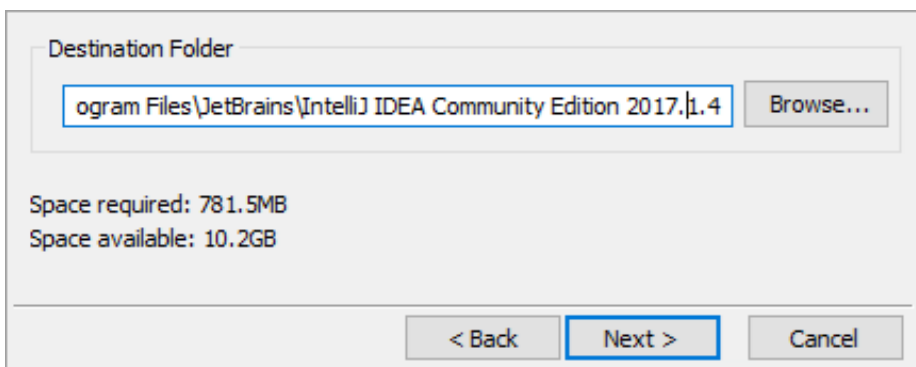
- Suitable environments for Java programming are the IntelliJ IDEA, Eclipse, and NetBeans.

## Installing IntelliJ IDEA

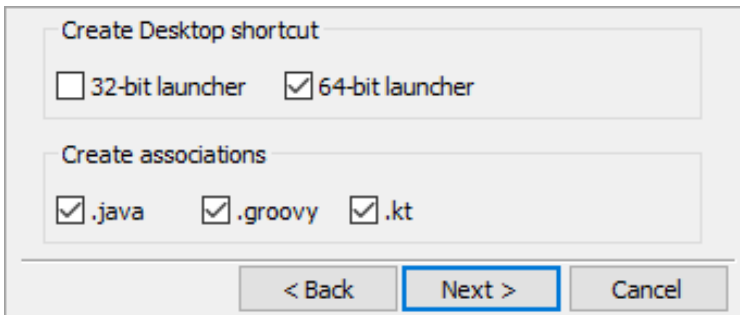
Let us start installing the **integrated environment IntelliJ IDEA Community** (version 2021, current as of September 2021). The **community version** of IntelliJ IDEA is **free distributed** by JetBrains and can be downloaded from this link: <https://jetbrains.com/idea/download>. The following paragraphs contain a detailed description of the **steps needed to install IntelliJ IDEA** (version Community 2021). After we download and run the installation file, appear the following screen:



We click the **[Next]** button, and we will see the following window displayed:

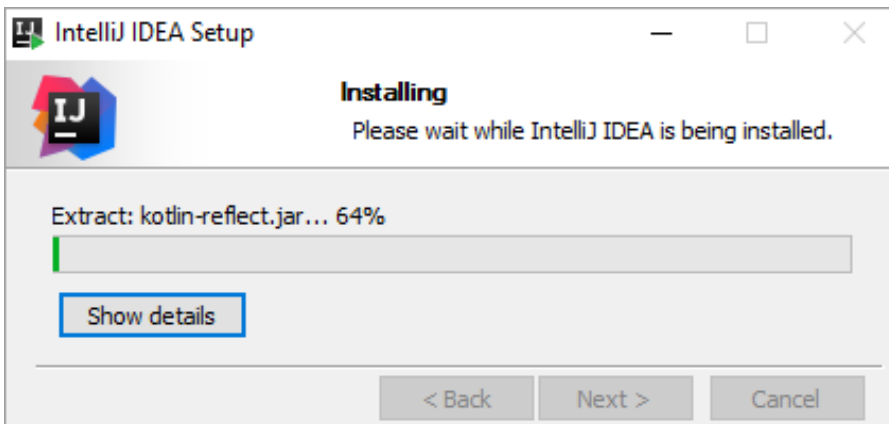


We click the **[Next]** button one more time, and IntelliJ IDEA's installation panel gets loaded on the screen.

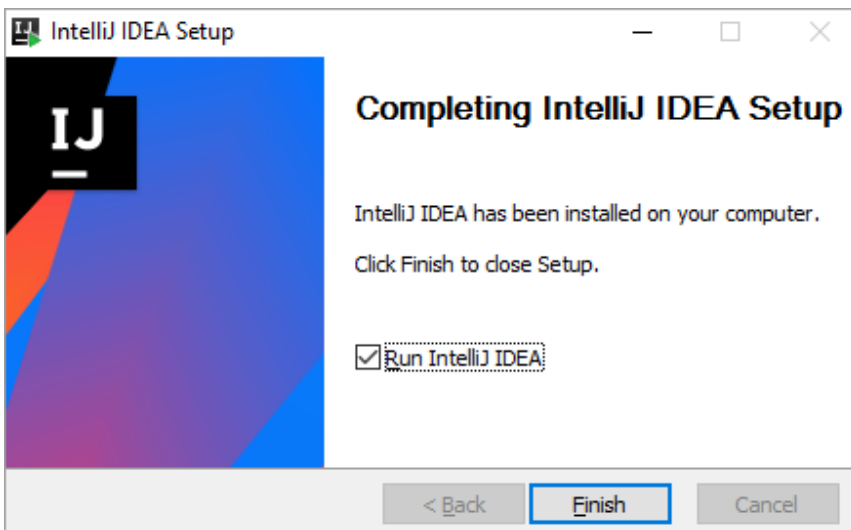


Depending on our operating system, we choose the 32 or the 64-bit launcher. Then we tick the [.java], [.groovy] and [.kt] boxes and click [Next].

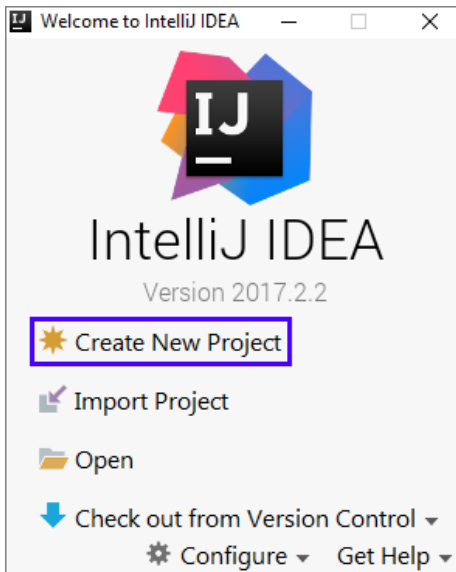
We save the folder's name in the start menu and click [Install]. The following screen should appear once IntelliJ IDEA has started installing:



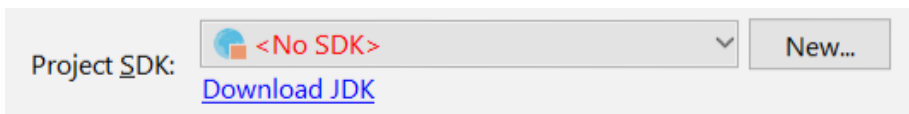
When the process is finished, tick the [Run IntelliJ IDEA] box and hit [Finish], which will wrap up the installation and run the development environment.



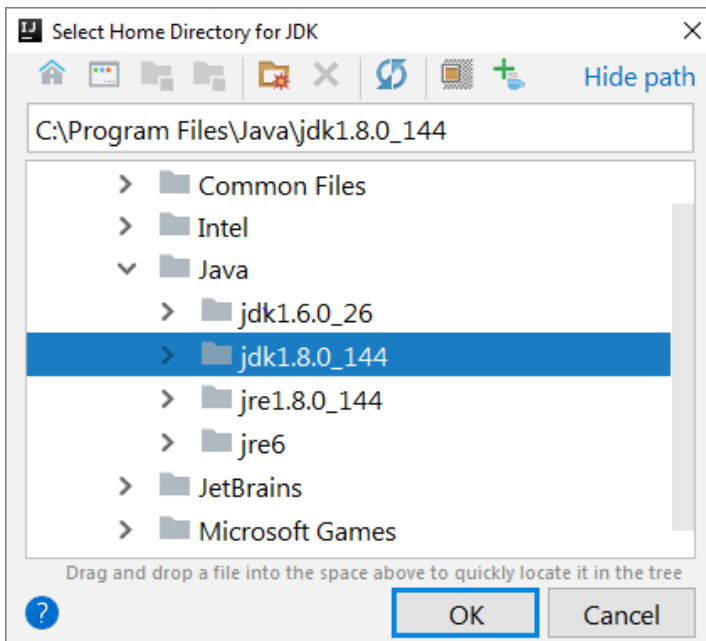
We choose [Create New Project] from IntelliJ IDEA's welcome screen:



A window that shows we haven't chosen a software development kit (SDK) gets displayed:



We click [New] and find the route to the **Java Development Kit (JDK)** we already installed, being careful not to choose **JRE**.



Then we select the JDK installation folder and click [OK]. That is it. We are ready to start working with **IntelliJ IDEA**.

## Older Versions of IntelliJ IDEA

Using older versions of IntelliJ IDEA is possible but **not advisable**. Older versions might not contain some of the newest development options, and some of the examples from this book may not be compiled.

## Online Development Environments

There are also **online development environments** that allow you to write code directly in your browser. They are not the most convenient, but if you have no other options, you can use them at the start of your education and install IntelliJ IDEA later. You can find useful links here:

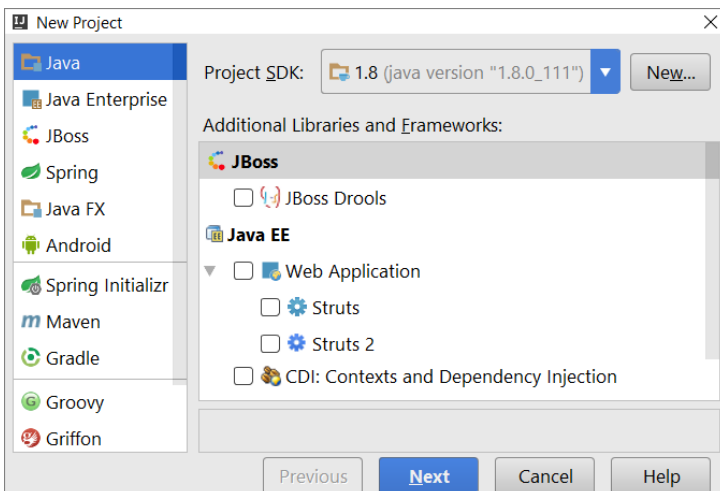
- For Java, we can use the following online Java IDE: <https://www.compilejava.net>.
- For C# we can use .NET Fiddle: <https://dotnetfiddle.net>.
- For JavaScript, we can write JS code directly in our browser by pressing [F12].

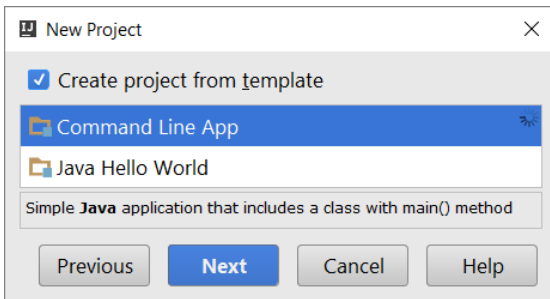
## Project Solutions and Projects in IntelliJ IDEA

The IntelliJ IDEA project uses **logic to group the numerous files** that make up a specific application or component. A single **IntelliJ IDEA project** consists of several **Java source files**, configuration files, and other resources. Each Java source file can have several **definitions and types** (classes or other definitions). **Classes** contain methods (actions), which consist of a **series of commands**. It may seem complicated, but in larger projects, this structure is extremely convenient and allows for a good organization of our work files.

## Problem: Creating a 'Hello Java' Console Program

Let us get back to our console program. Now that we have IntelliJ IDEA installed, we can run it. Then we will create a new project: [Create New Project]. Choose [Java] -> [Command Line App] from the dialog window and give our project an appropriate name – **HelloJava**, for instance.

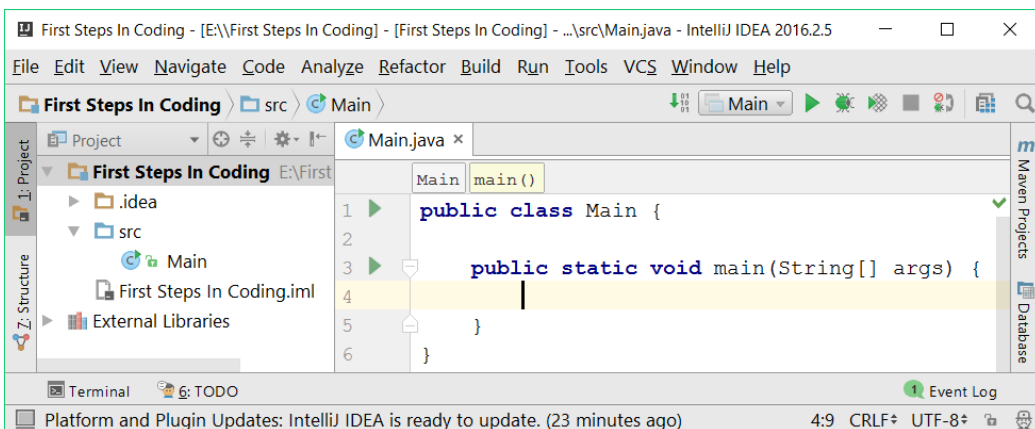




IntelliJ IDEA will create a blank Java program for us and we will need to write the code for it.

## Writing Program Code

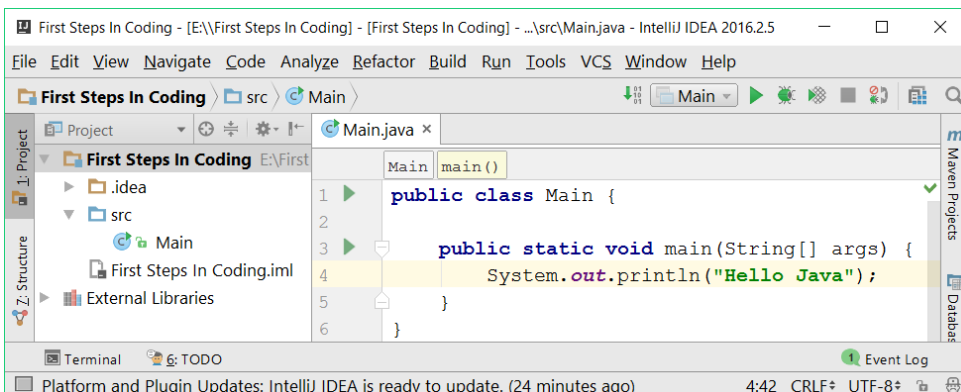
Java programs' source code is written between the curly brackets of the **main(String[] args)** section. It is the main method (action) executed when starting a Java program. Press [Enter] after the opening bracket, and we can start writing. In regards to text formatting, program code is written with an indentation which helps when reviewing and debugging it.



We can write the following command:

```
System.out.println("Hello Java");
```

And here's what our program in IntelliJ IDEA should look like:



The `System.out.println("Hello Java")` command in the Java language allows us to print the text message "Hello Java" on the console ('out.println(...)'). The message needs to be enclosed in quotes that signify text. At the end of every Java command, we place a semicolon `;` that indicates the end of the command (meaning it doesn't continue on the next row).

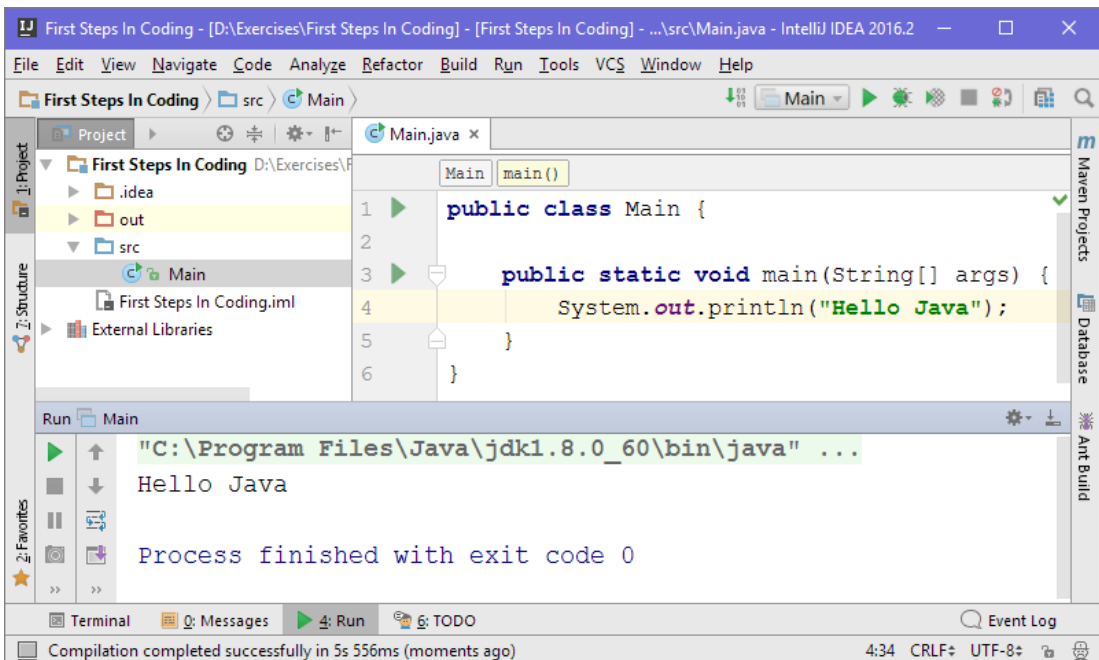
This command is quite typical for programming: signify we need a certain **object** (a **PrintStream** object named **out** located in the object **System**, in this case) to be found, and a certain **action** (printing something, passed within brackets, in this case) to be performed on it. In more technical terms, we call the method `println(...)` from the class **PrintStream**, part of the **System** class, and pass as a parameter to it the text **Hello Java**.

## Classes and Files in Java

Java is an **object-oriented language** hence why every Java program consists of at least one **class**. If the Java class is public, its name needs to match the name of the file + `.java`. In the above example, the **Main** class, defined with `public class Main {...}`, is located in a file named **Main.java** in the project's **src** folder. Were it called **HelloJava**, its source code would need to be located in a file named **HelloJava.java**. Otherwise, we'd get a compile-time error.

## Starting The Program

We start the program by pressing `[Ctrl + Shift + F10]`. If there are no errors, the program will execute. The result will be displayed on the console (the window at the very bottom of IntelliJ IDEA).





The result of the program is the following message:

Hello Java

The additional message **Process finished with exit code 0** is written last in the console and signifies that the program has ended with no errors.

Testing in The Judge System

Testing the solutions to the problems in this book is completely automated via the Judge System's website: <https://judge.softuni.org>. The solutions are judged in real-time by the system. Each solution goes through a series of hidden tests, and every test that passes grants the user certain points. The program that we just wrote can be tested here: <https://judge.softuni.org/Contests/Practice/Index/646#0>. We need to paste the program's entire source code in the black field and make sure we've selected **Java code** from the dropdown menu.

01. Hello Java

```
1 public class Main {
2     public static void main(String[] args)
3     {
4         System.out.println("Hello Java");
5     }
6 }
```

Allowed working time: 0.100 sec.  
Allowed memory: 16.00 MB  
Size limit: 16.00 KB  
Checker: Trim ?

Java code

Submit

Then we can send our solution by clicking the [Submit] button. The system returns a result in a few seconds, which is displayed in the table of submitted solutions. If necessary, we can hit the [Refresh] button at the upper right-hand corner of the table of submitted solutions:

| Submissions                                                                                    |                                  |                                        |
|------------------------------------------------------------------------------------------------|----------------------------------|----------------------------------------|
| <div><div><div>⏮</div><div>⏪</div><div>1</div><div>⏩</div><div>⏭</div></div><div>🔄</div></div> |                                  |                                        |
| Points                                                                                         | Time and memory used             | Submission date                        |
| ✔ 100 / 100                                                                                    | Memory: 7.38 MB<br>Time: 0.014 s | 11:34:30 14.01.2016 <div>Details</div> |
| ✖ 0 / 100                                                                                      | Memory: 7.40 MB<br>Time: 0.016 s | 11:34:19 14.01.2016 <div>Details</div> |
| <div><div><div>⏮</div><div>⏪</div><div>1</div><div>⏩</div><div>⏭</div></div><div>🔄</div></div> |                                  |                                        |

The Judge System will display one of the following possible results in the table:

- A **number of points** (between 0 and 100), when the submitted code is compiled successfully (there are no syntactic errors) and can be tested.
  - If our solution is **correct**, all tests are marked in green, and we receive **100 points**.
  - If our solution is **incorrect**, some tests are marked in red, and we receive less than 100 or 0 points.
- If there are syntactic errors in our program, we receive a **compile-time error message**.

## How Do I Register at SoftUni Judge?

It's just a standard website registration and takes no more than a minute.

## Typical Mistakes in Java Programs

One mistake that often sees with beginners is that they write their **code outside the body of the `main(String[] args)` method**. It causes an error because the integrated environment or the compiler cannot read the commands issued in the program correctly. Here is an example of an incorrect program:

```
static void main(String[] args)
{
}
System.out.println("Hello SoftUni");
```

Another usual mistake is mixing up **capital and lowercase letters**. They matter when we call commands and can impede proper functioning. Here is an example of such a mistake:

```
static void main(String[] args)
{
    System.out.Println("Hello SoftUni");
}
```

In the example above, **PrintIn** is written incorrectly, and the capital letter **P** needs to be changed to lowercase.

The missing **semicolon (;)** at the end of commands is one of the timeless issues for beginner programmers. Skipping this symbol leads to the program functioning improperly, and it is very often left unnoticed\*. Here is an example of incorrect code:

```
static void main(String[] args)
{
    System.out.println("Hello SoftUni")
}
```

A missing **quotation mark** or **closing bracket** can also cause issues. As with the *semicolon*, this will lead to the program either **functioning improperly** or not executing at all. Such mistakes are hard to notice, especially with large amounts of code. Here is an example of an incorrect program with a lonely opening quotation mark:

```
static void main(String[] args)
{
    System.out.println("Hello SoftUni);
}
```

This program will return a **compile-time error**, and even before that, the code will be underlined to draw the programmer's attention towards the error (missing closing quotation mark). Another beginner's frequent mistake is trying to write the **main(...)** method outside the class:

```
public class Example {
}

static void main(String[] args)
{
    System.out.println("Method outside of the class");
}
```

The correct way to do this is by placing the **main(...)** method within the **Example** class and saving the entire code in the **Example.java** file:

Put **this** code in the file **Example.java**

```
public class Example {
    static void main(String[] args) {
        System.out.println(Correct class);
    }
}
```

## What Have We Learned from This Chapter?

First of all, we learned **what programming is** – issuing commands written in a computer language that the machine can understand and carry out. We also found out **what a computer program is** – a **series of commands** that aim to achieve a certain result. We gained some basic knowledge of the **Java programming language** and learned **how to create simple console programs using IntelliJ IDEA**. We followed Java's program code structure, one example of which is that issuing of commands mainly happens in the **static void main(String[] args)** section, within the curly brackets. We looked at printing with the function **System.out.println(...)** and starting the program with **[Ctrl + Shift + F10]**. And last but not least, we now know how to test our code in **SoftUni's Judge System**.

Top work! Let us now tackle the **exercises**. You remember that learning how to program involves a lot of code writing and problem-solving, right? Let us do just that and put what we have learned into practice.

## First Steps in Programming – Exercises

Welcome to the exercises. We are now going to write a couple of console applications, which will help us make a few more steps into programming. After that, we will show you how to program something more complex – programs with graphical and web user interfaces.

### Problem: Expression

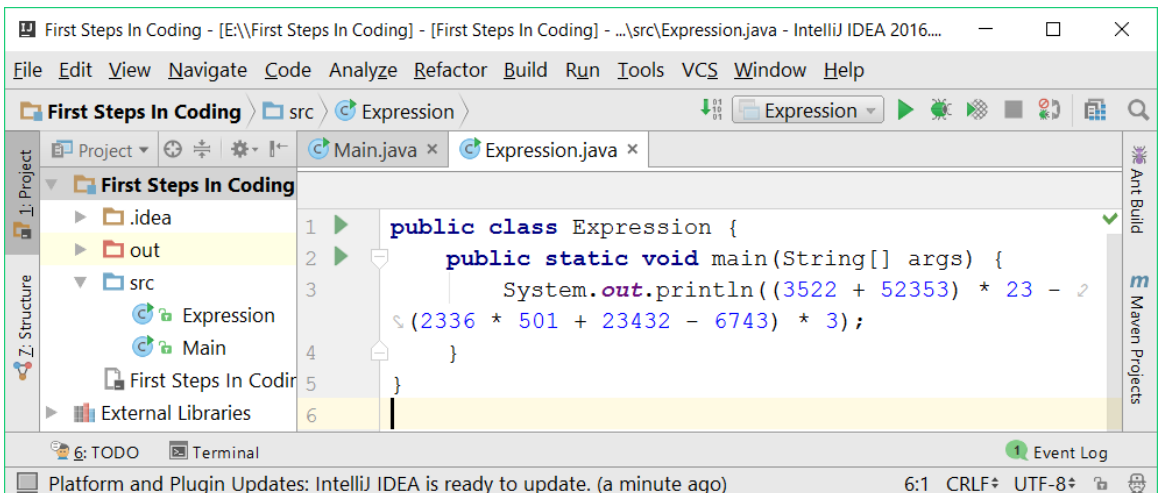
Write a console-based Java program that **calculates** and **prints** the value of the following numerical expression:

- $(3522 + 52353) * 23 - (2336 * 501 + 23432 - 6743) * 3$

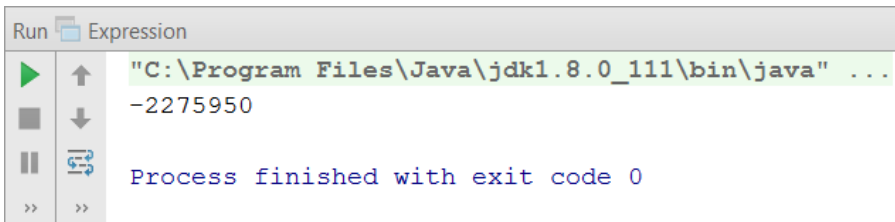
Note: **you are not allowed to previously calculate the value** (for example, with Windows Calculator).

### Hints and Guidelines

Create a new Java file and name it **Expression**. Then we find the method **static void main(String[] args)** and start writing in its body (the space between the opening and closing curly braces). We need to **write the code**, which will calculate the numeric expression above and print its value on the console. Pass the expression to the **System.out.println(...)** command by writing it within its brackets:



Start the program with **[Ctrl + Shift + F10]** and check whether the result matches the one from the picture:



## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/646#1>.

**Expression**

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println((3522 + 52353) * 23 - (2336 * 501 + 23432 - 6743) * 3);
4     }
5 }
6

```

Allowed working time: 0.100 sec.  
 Allowed memory: 16.00 MB  
 Size limit: 16.00 KB  
 Checker: Numbers Checker

| Participant     | Result    |
|-----------------|-----------|
| a.aleksandrov91 | 100 / 100 |
| A.Atanasov      | 100 / 100 |
| a.kolev         | 100 / 100 |
| A.stankov       | 100 / 100 |

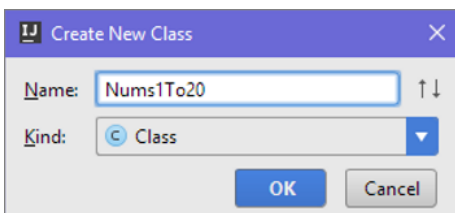
| Points      | Time and memory used             | Submission date                             |
|-------------|----------------------------------|---------------------------------------------|
| ✓ 100 / 100 | Memory: 1.09 MB<br>Time: 0.001 s | 19:57:28 06.05.2016 <a href="#">Details</a> |

## Problem: Nums 1...20

Write a Java console program that prints the numbers from 1 to 20 on separate lines on the console.

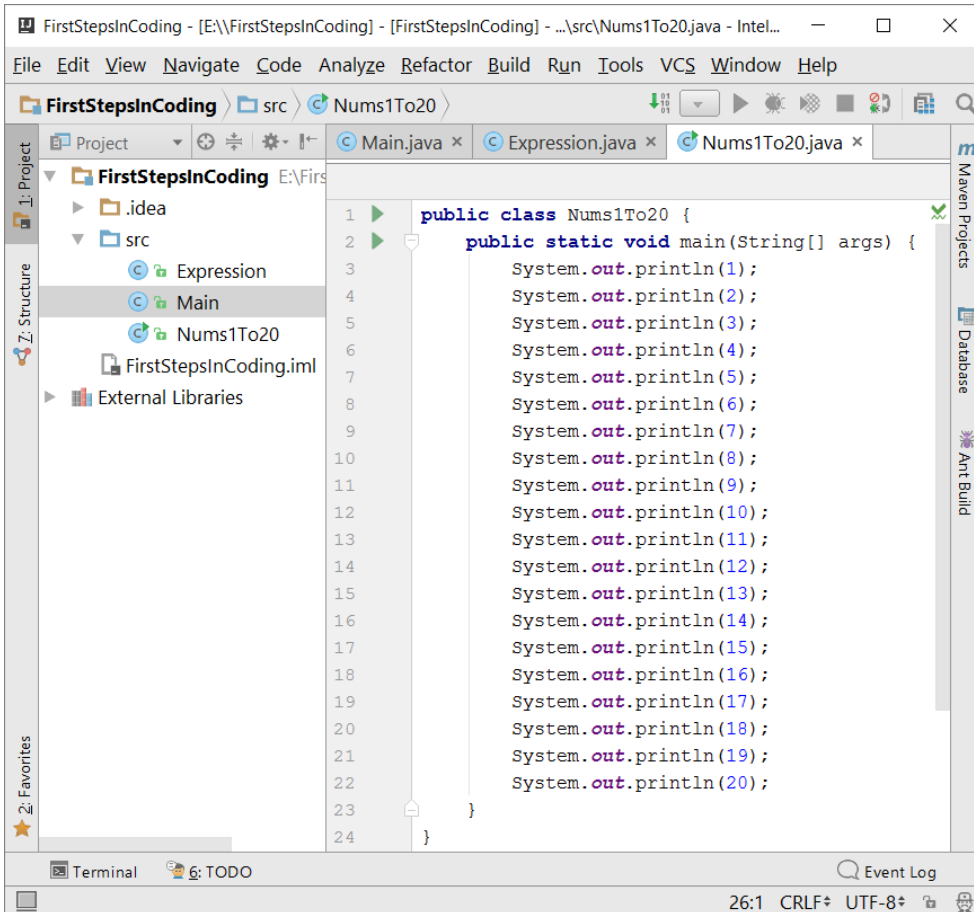
## Hints and Guidelines

Create a new Java class and name it **Nums1to20** (right click on the **src** folder -> [New] -> [Java Class])



Inside the **static void main(String[] args)** method, write 20 **print()** commands, each on a separate line, to print the numbers from 1 to 20 one after another. Some of you

may be wondering if there is a cleverer way. Don't worry, there is, but we will mention it later on.



Let's now **start the program** and check whether the result is what we're expecting:

```

1
2
...
20

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/646#2>. Now try and think of a cleverer way we can write this program, to avoid repeating the same command so many times. Look up information for "[for loop Java](#)" on the Internet.

## Problem: Triangle of 55 Stars

Write a Java console program that **prints a triangle of 55 stars** on 10 lines:

```

*
**
***
****
*****
*****
*****
*****
*****
*****
*****

```

## Hints and Guidelines

Create a new Java console application and name it **TriangleOf55Stars**. Inside we need to write code that prints the triangle of stars, using commands like the ones below:

```

System.out.println();
System.out.println();
...

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/646#3>.

Try and **improve your solution** to avoid repeating the same command so many times. Can this be achieved with a **for**-loop? Did you manage to invent a better solution (with a for-loop, for example) to the previous problem? The current problem can be solved with a similar but more complex approach (a loop within another loop). It is completely fine if you can't figure it out, you will remember this problem when we learn about loops in a few chapters.

## Problem: Rectangle Area

Write a Java program that receives two numbers, **a** and **b**, then calculates and prints the area of a rectangle with sides **a** and **b**.

### Sample Input and Output

| a  | b | area |
|----|---|------|
| 2  | 7 | 14   |
| 7  | 8 | 56   |
| 12 | 5 | 60   |

## Hints and Guidelines

Create a new Java console application. We will use the following code to read our numbers:

```
import java.util.Scanner;

public class RectangleArea {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        int a = Integer.parseInt(console.nextLine());
        int b = Integer.parseInt(console.nextLine());

        // TODO: calculate the area and print it
    }
}
```

To read input data from the console, we import and use the `java.util.Scanner` class, which we will look at in detail a little later. For now, just accept that the reading number is achieved as in the example above.

What is left is to finish the program to calculate the rectangle's area and print it. Pass the product of **a** and **b** to the already familiar `System.out.println()` command. In programming, we multiply using the `*` operator.

## Test Your Solution

Test your solution with a few examples. You should receive a result identical to the one you see below (we enter 2 and 7 as input, and the program prints 14 – their product, as output):

```
2
7
14
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1046#4>.

## Problem: Square of Stars

Write a Java console program that reads a whole positive number **N** from the console and prints a square of **N** stars on the console, like in the examples below.



## Sample Input and Output

| Input | Output                 |
|-------|------------------------|
| 3     | <pre>*** * * ***</pre> |

| Input | Output                       |
|-------|------------------------------|
| 4     | <pre>**** * * * * ****</pre> |

| Input | Output                                         |
|-------|------------------------------------------------|
| 5     | <pre>***** *   * *   * *   * *   * *****</pre> |

## Hints and Guidelines

Create a new **Java console program**. To read the number  $N$  ( $2 \leq N \leq 100$ ), we use the following code:

```
Scanner console = new Scanner(System.in);
int n = Integer.parseInt(console.nextLine());

// TODO: print the rectangle
```

Don't forget to import the `java.util.Scanner` class and put your code in the `main(...)` method of your program's main class. Finish the program so that it prints a square made up of stars. You may need to use **for-loops**. Look up information on the Internet.

**Caution:** this problem is more complex than the rest and is presented to you at this point purposefully. It is marked with a star to provoke you to **look for information on the Internet**. It is one of the most important skills you have to develop while you're learning to program. It is what you'll be doing every day if you work as a developer, so don't be scared to try it out.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/646#5>.

## Console, Graphical, and Web Applications

With **console applications**, as you can figure out yourselves, **all operations** for reading input and printing output are done through the **console**. There the **input data is inserted**, which is read from the application also in it, is **printed the output data** after or during the runtime of the program.

While the console application uses the **text console**, web applications use a **web-based user interface**. To receive their performance are needed two things - a **web server** and a **web browser**, as the browser plays the main role in the **visualization of the data and the interaction with the user**. Web applications are much more pleasant for the user, they visually look better, and a mouse and touch screen can be used (for tablets and smartphones), but behind all of that stands programming. And this is why we **have to learn to program**. We have already made our first steps.

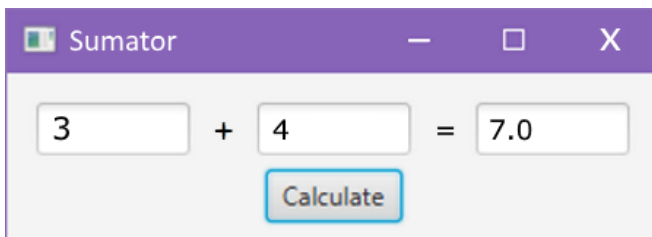
**Graphical (GUI) applications** have a **visual user interface**, directly into your computer or mobile device, without a web browser. Graphical applications (or otherwise said, desktop applications) **contain one or more graphical windows**, which consist of specific **controllers** (text fields, buttons, pictures, tables, and others), **serving for dialog with the user** more intuitively. Similar to them are the mobile applications in your telephone or your tablet, we use forms, text fields, buttons, and other controls, and we control them by programming code. It is why we are learning how to write code now **that code is everywhere** in software development.

## Exercises Graphical and Web Applications

Now we are about to build one simple **web application** and one simple **graphical application**, to take a look at what we will be able to create when we progress with programming and software development. We are not going to look through the details about the used techniques and constructions fundamentally, rather we are just going to take a look at the arrangement and functionality of our creation. After we progress with our knowledge, we will be able to do bigger and more complex software applications and systems. We hope that the examples given below will **enlighten your interest**, rather than make you give up.

### Problem: Graphical Application "Sumator for Numbers"

Write a **graphical (GUI) application**, which **calculates the sum of two numbers**:



By entering two numbers in the first two fields and pressing the button [Calculate], their sum is being calculated, and the result is shown in the third text field.

Unlike console applications, which read and write their data as text on the console **graphical (GUI) applications** have a visual user interface. Graphical applications (desktop applications) consist of one or more graphic windows, in which there are controls text fields, buttons, pictures, tables, and others.

For our application, we will use the technology **JavaFX**. It allows us to create graphical applications for every platform using **Java** language. For the environment for development, we are going to use the program **IntelliJ IDEA**.

To create graphical applications with JavaFX, we will use the program that we can download from: <https://gluonhq.com/products/javafx>

## Download Scene Builder

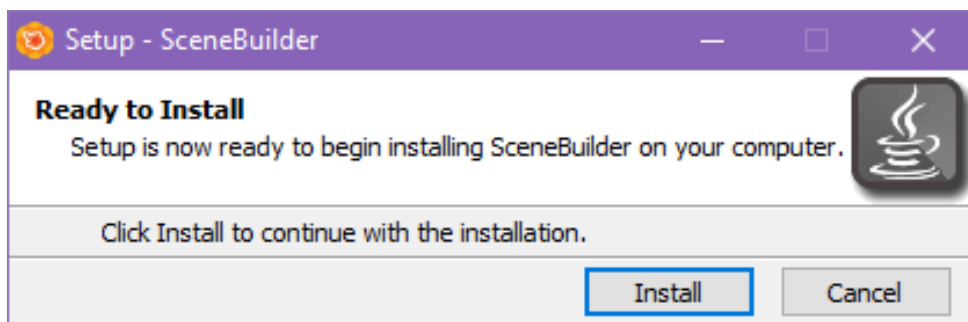
The latest version of Scene Builder is **8.2.0**, it was released on **May 18, 2016**.

To be kept informed of Scene Builder releases, consider subscribing to the [Gluon Newsletter](#).

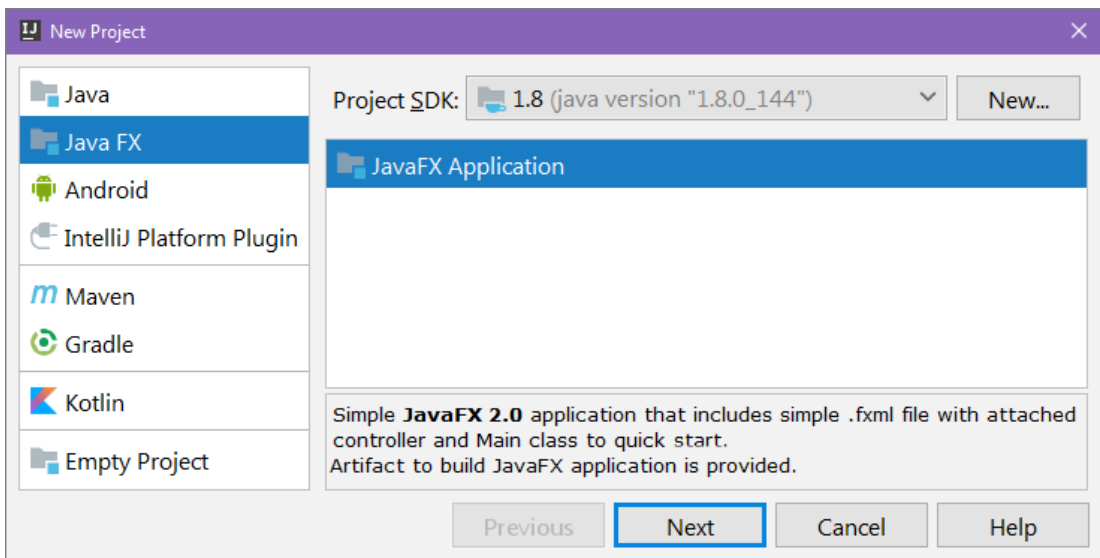
| Product                                | Platform                                     | Download                 |
|----------------------------------------|----------------------------------------------|--------------------------|
| Scene Builder                          | Executable Jar                               | <a href="#">Download</a> |
| Scene Builder                          | Windows Installer (x86) <a href="#">info</a> | <a href="#">Download</a> |
| Scene Builder                          | Windows Installer (x64) <a href="#">info</a> | <a href="#">Download</a> |
| Scene Builder                          | Mac OS X dmg                                 | <a href="#">Download</a> |
| Scene Builder                          | Linux RPM (64-bit)                           | <a href="#">Download</a> |
| Scene Builder                          | Linux Deb (64-bit)                           | <a href="#">Download</a> |
| Scene Builder                          | Linux RPM (32-bit)                           | <a href="#">Download</a> |
| Scene Builder                          | Linux Deb (32-bit)                           | <a href="#">Download</a> |
| Scene Builder Kit <a href="#">info</a> | Jar File                                     | <a href="#">Download</a> |

**License:** Scene Builder is licensed under the BSD license.

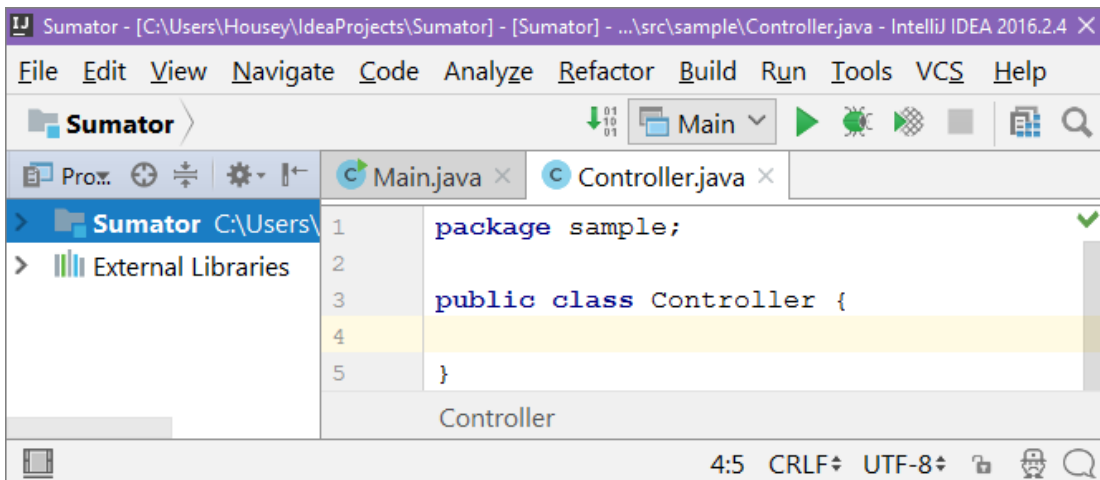
We install **SceneBuilder** for the design of the JavaFX user interface.



In IntelliJ IDEA, create a new Java project of type **JavaFX Application**



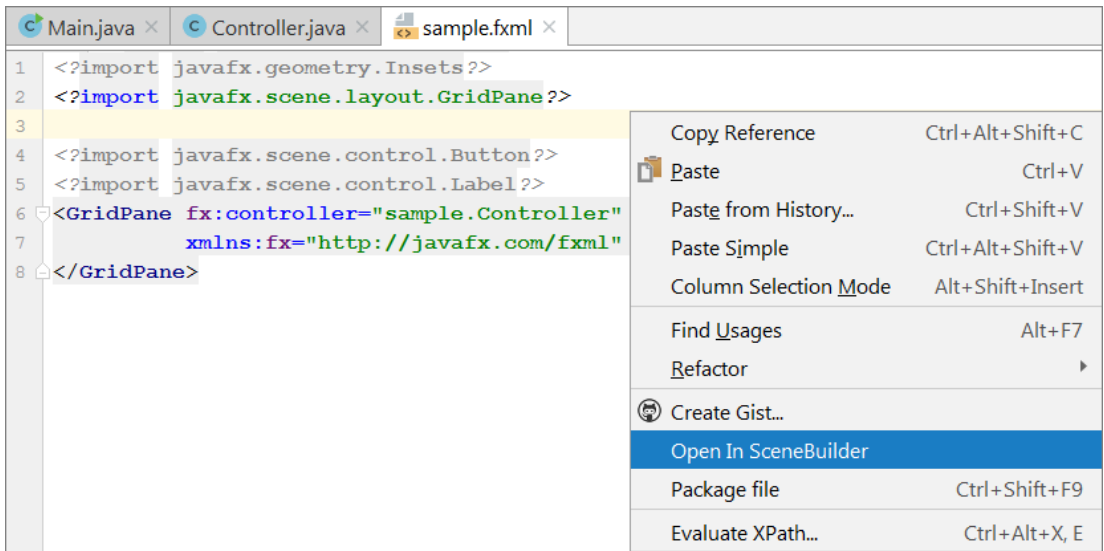
When creating a **JavaFX application**, a project with a few files will be generated.



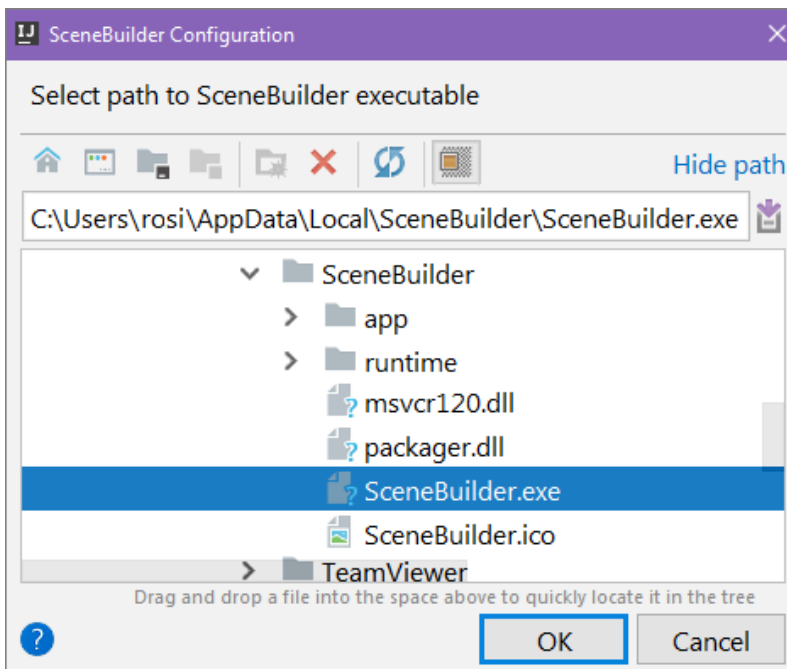
The file, inside which is the view of our application, can be found on the path `[src/sample/sample.fxml]`. Let us go there and open it



The file looks like this. We will not work with it directly later, but we will use the mentioned above **SceneBuilder**, which will generate code instead of us. We click the right mouse button on the file and choose `[Open in SceneBuilder]`.

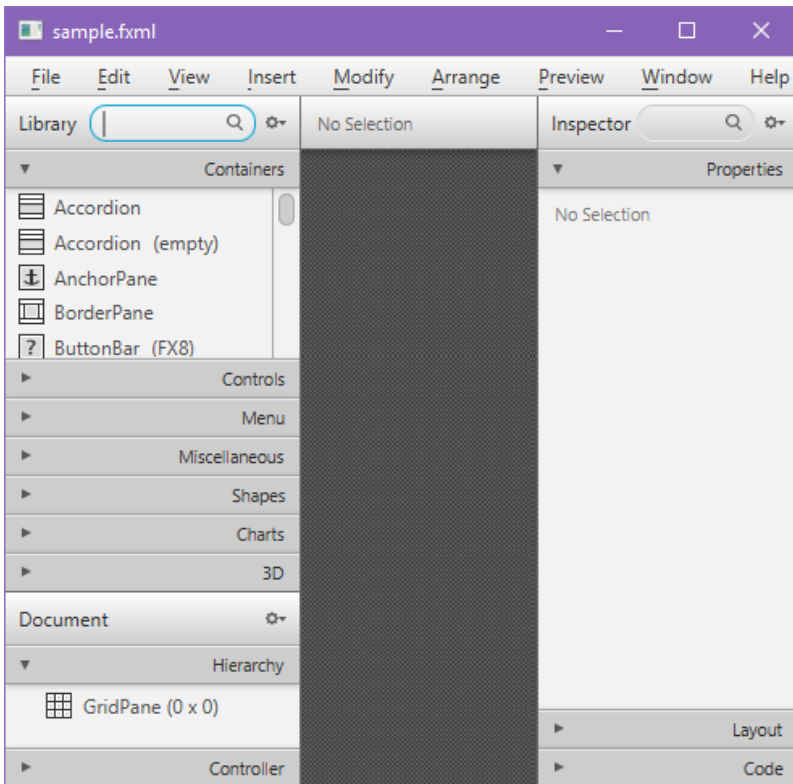


By choosing the option for the first time, IntelliJ IDEA will ask us to enter the path towards **SceneBuilder.exe**:

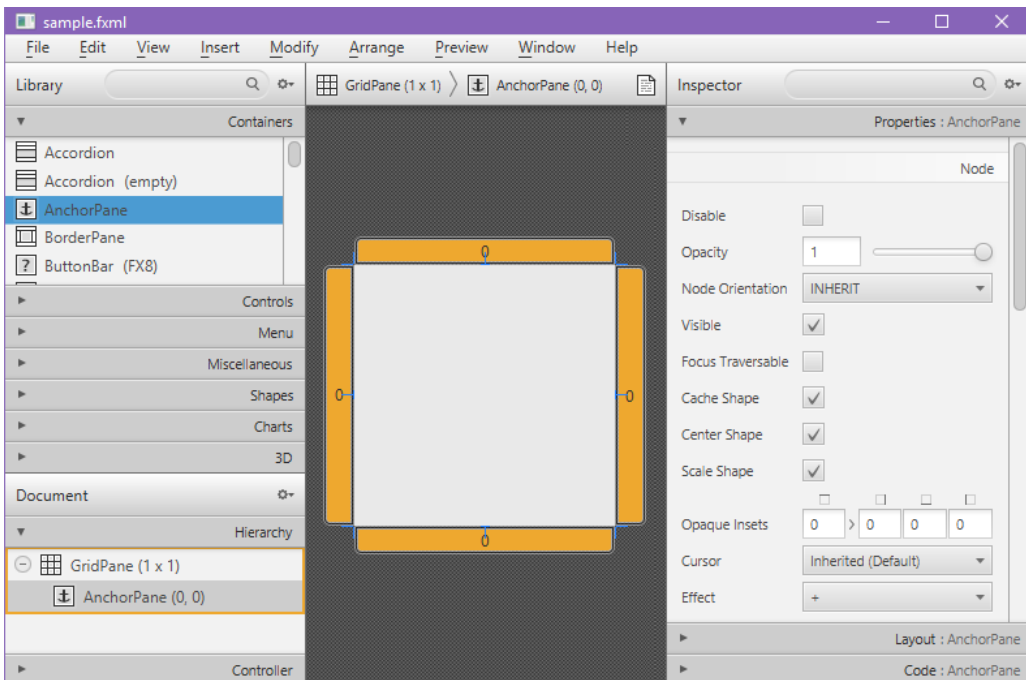


With the usual Windows installation of **SceneBuilder**, it is found in [C:\Users[username]\AppData\Local\SceneBuilder].

After we have pointed the right path, **SceneBuilder** opens, and we see the following screen:



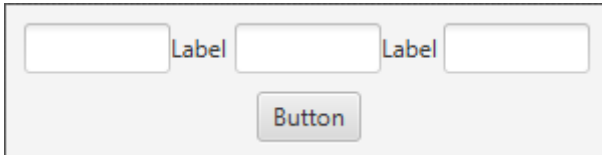
We have the types of controls we can add on the left-hand side of the screen, so let us find the **[AnchorPane]** window and add it by dragging it to the environment where our design is located.



It is our window where we can now start adding controls located in the same place we added [AnchorPane] from (in the [Containers] menu). For our interface we will need:

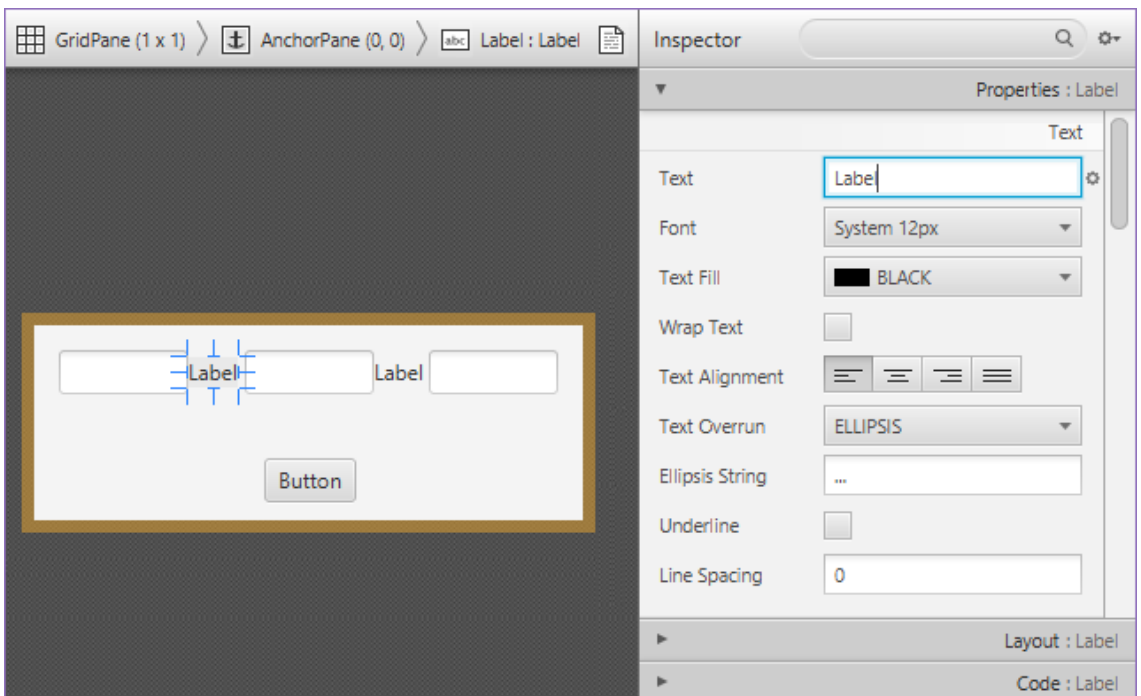
- 3 text fields: **TextField**
- 2 labels between the text fields (for **+** and **=**)
- 1 button for calculation the result: **Button**

After we have added them, our application should look like this:

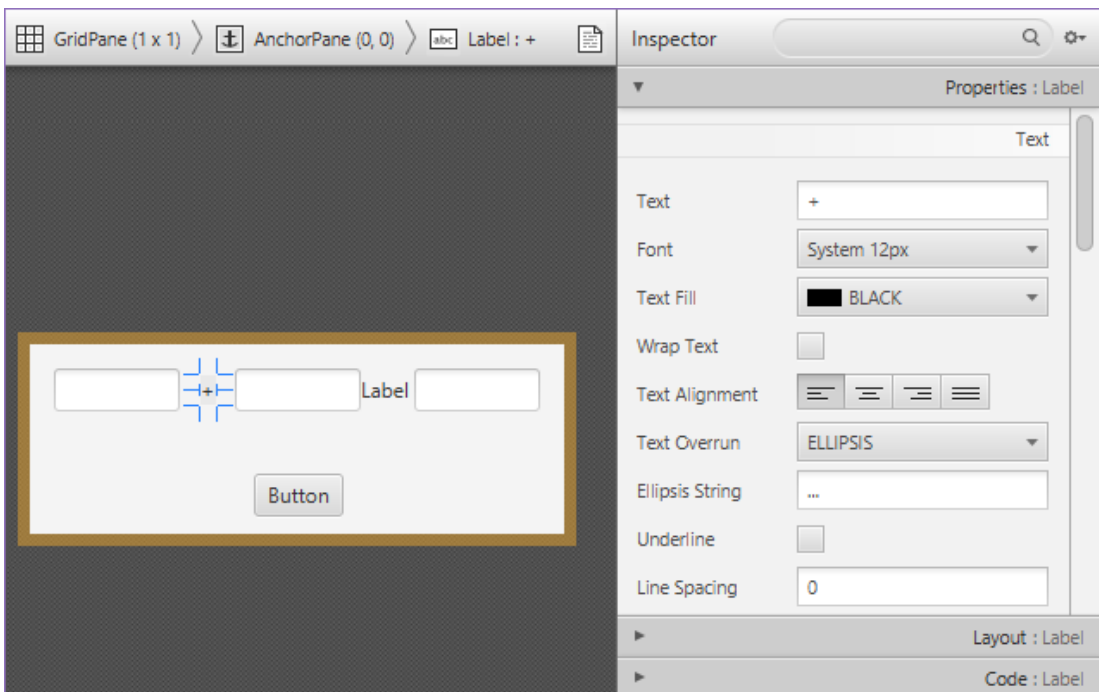


We will rename the controls by choosing more appropriate names, reflecting our application's functionality ("**+**" and "**=**"). It is achieved by clicking the control we wish to change and going to its [Properties] on the right.

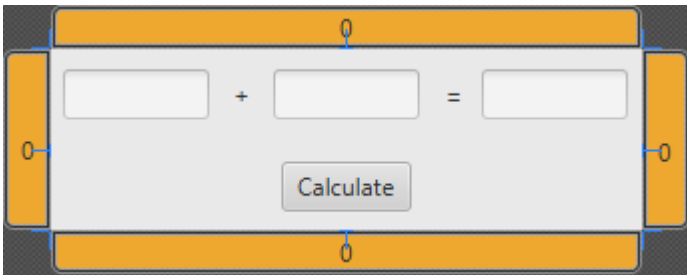
First, we will change the **Text** property of the tickets. Their name is **Label** by default:



We will choose more meaningful names which will reflect our program's logic better. Change the first text **Label** with **+**:



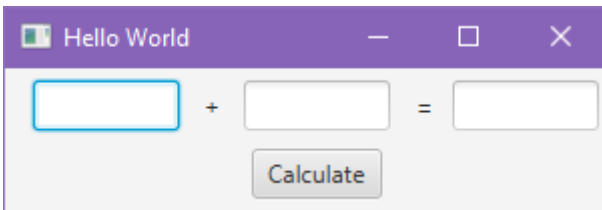
Then do the same with the second ticket and the button - change the former's text to **=** and the latter's to **Calculate**



We will save our design changes using **[Ctrl+S]**:

Changes saved to 'sample.fxml'

Now go back to IntelliJ IDEA and start the application using **[Shift+F10]**:



The application's size is most likely not what you expected it to be. It is due to a piece of our code explicitly stating what that size needs to be. Let us resolve the issue: Go to the **Main** class and find the piece of code



```

@Override
public void start(Stage primaryStage) throws Exception{
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Hello World");
    primaryStage.setScene(new Scene(root, width: 300, height: 275));
    primaryStage.show();
}

```

Delete the code responsible for window size, so our program can use the sizing we chose in SceneBuilder.

```

@Override
public void start(Stage primaryStage) throws Exception{
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Hello World");
    primaryStage.setScene(new Scene(root, width: 300, height: 275));
    primaryStage.show();
}

```

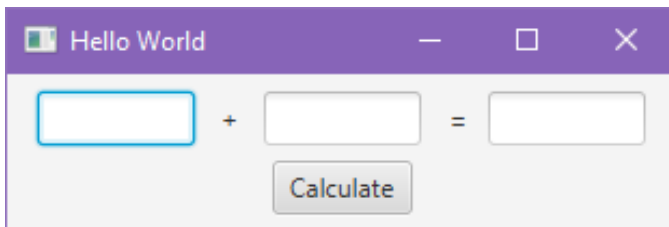
Now the code looks like this:

```

@Override
public void start(Stage primaryStage) throws Exception{
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Hello World");
    primaryStage.setScene(new Scene(root));
    primaryStage.show();
}

```

We start the program once more using [Shift+F10] and check whether the sizing is correct.



The final step in our design is to go back to our code and change the application's title, located on this line:

```

@Override
public void start(Stage primaryStage) throws Exception{
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Hello World");
    primaryStage.setScene(new Scene(root));
    primaryStage.show();
}

```

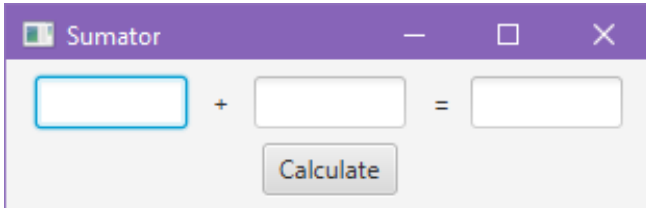
Change the name from "Hello World" to "Sumator":

```

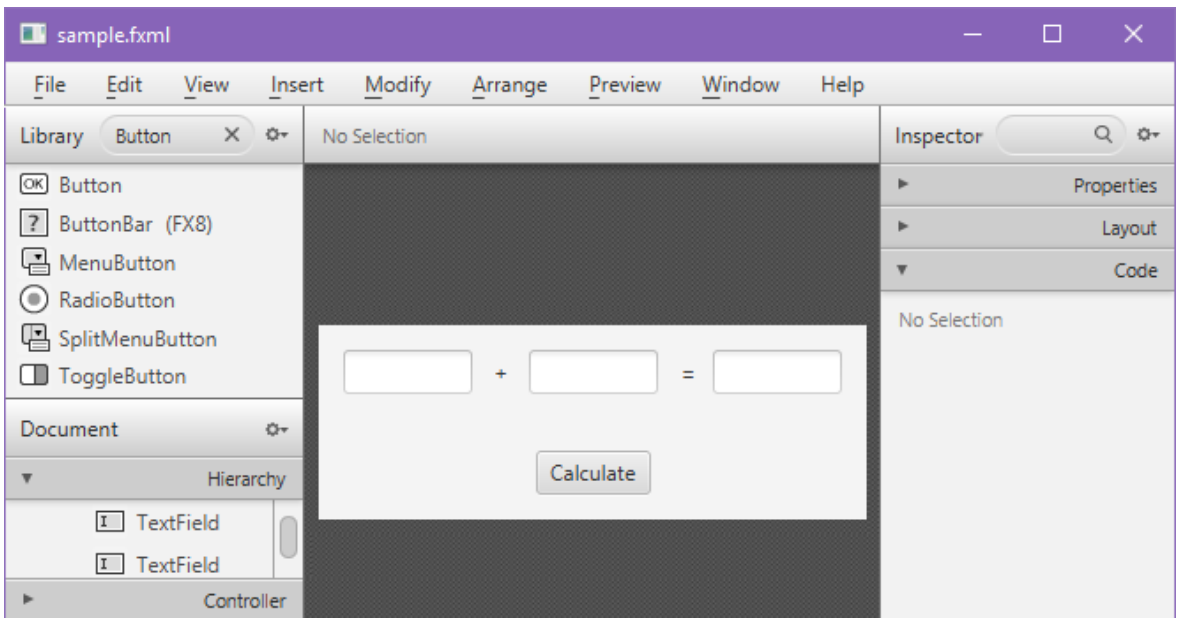
@Override
public void start(Stage primaryStage) throws Exception{
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Sumator");
    primaryStage.setScene(new Scene(root));
    primaryStage.show();
}

```

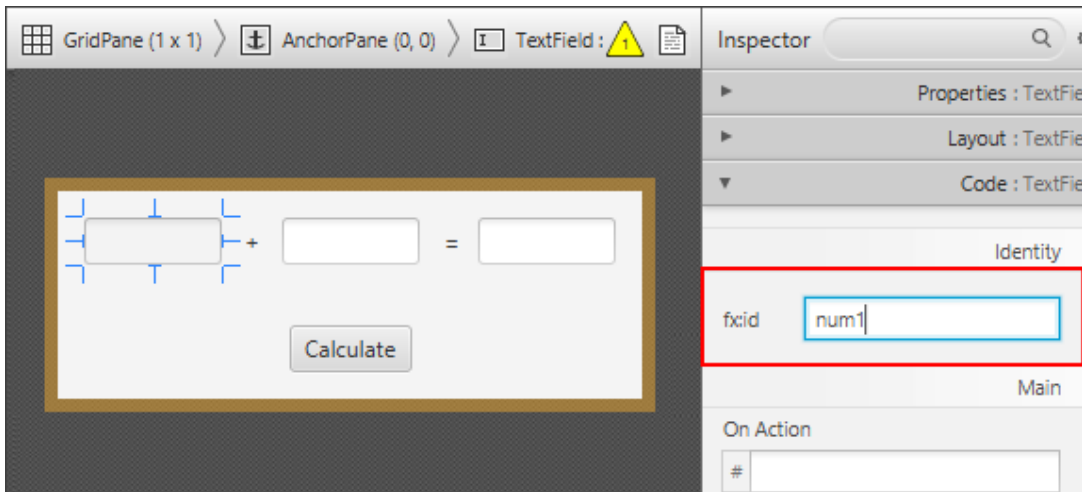
And start the application:



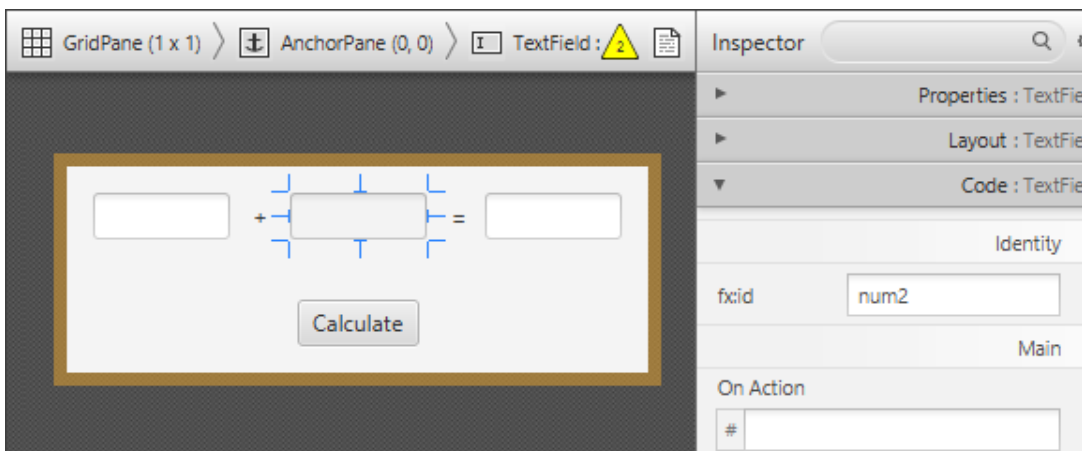
The graphical part's design looks ready. It is now time for something a lot more interesting – **the program logic**. Time to write the code which sums the numbers from the first two fields and displays the result in the third field. To do that, we need to go back to **SceneBuilder** and give the text fields names so we can access them in the code:



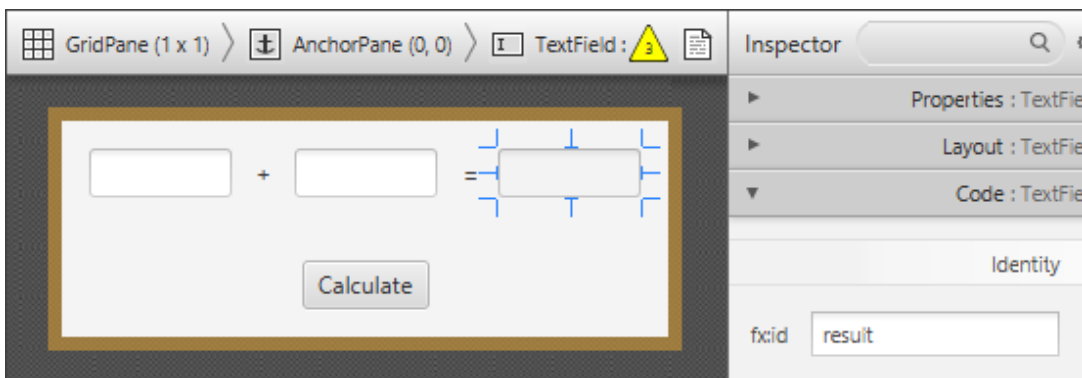
Click the first text field and go to the **[Code]** section on the right, where we will enter the name **num1** in the **fxid** field:



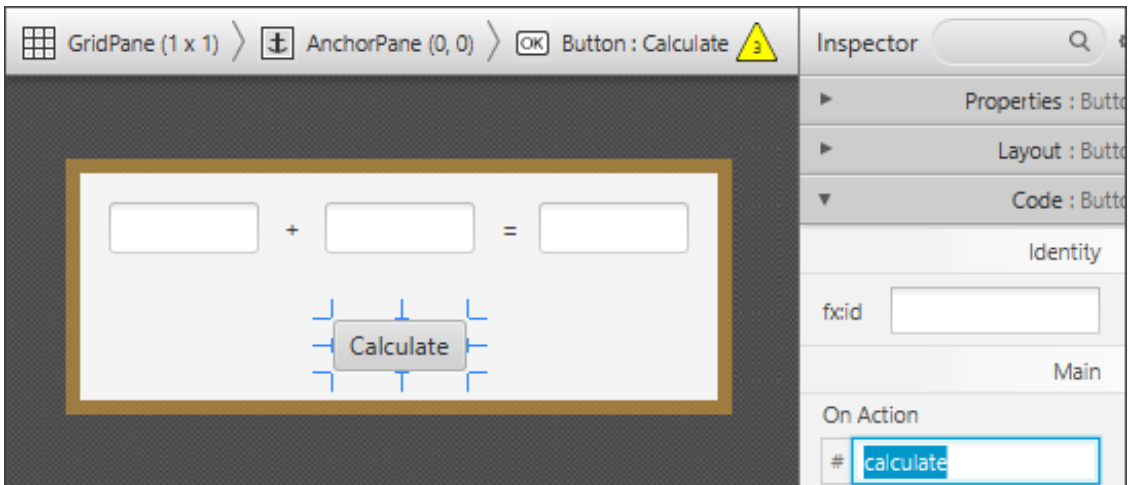
Then rename the second field to **num2**:



And the third field to **result**:



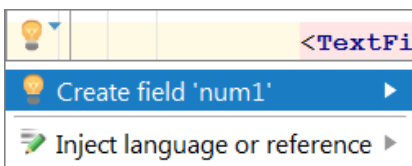
Now we will tell the button what function to perform once it is clicked. Go to the **On Action** field and enter the name of the function **calculate**, which we will create in a moment:



Save the changes in SceneBuilder using [Ctrl+S] and go back to the **sample.fxml** file

```
<GridPane alignment="center" hgap="10" vgap="10"
    xmlns="http://javafx.com/javafx/8.0.111"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="sample.Controller">
    <children>
        <AnchorPane prefHeight="110.0" prefWidth="310.0">
            <children>
                <TextField fx:id="num1" layoutX="14.0" layoutY="14.0"
                    prefHeight="25.0" prefWidth="73.0" />
                <TextField fx:id="num2" layoutX="119.0" layoutY="14.0"
                    prefHeight="25.0" prefWidth="73.0" />
                <TextField fx:id="result" layoutX="223.0" layoutY="14.0"
                    prefHeight="25.0" prefWidth="73.0" />
                <Label layoutX="99.0" layoutY="18.0" text="+" />
                <Label layoutX="205.0" layoutY="18.0" text="=" />
                <Button layoutX="123.0" layoutY="71.0" mnemonicParsing
                    ="false" onAction="#calculate" text="Calculate"/>
            </children>
        </AnchorPane>
    </children>
</GridPane>
```

We can see that our fields have names, and our button has a function to perform once it is clicked. So far, so good, but these fields and the button's function don't yet exist in our code. To generate them, we will go to each one of them and press [Alt+Enter] -> [Create field] (the field's name)



Once we create one of the fields, we will get redirected to the **Controller.java** file, where the field will be generated in the code in the following manner:

```
import javafx.event.ActionEvent;
import javafx.scene.control.TextField;

public class Controller {

    public TextField num1;
    public TextField num2;
    public TextField result;

    public void calculate(ActionEvent actionEvent) {

    }

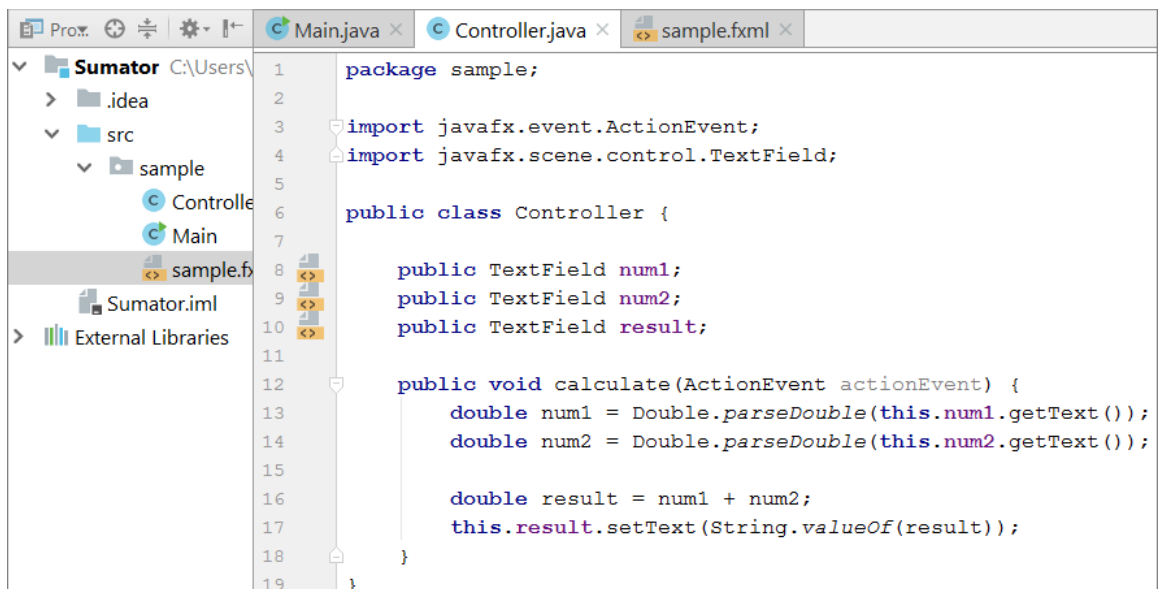
}
```

Write the following Java code between the curly braces of the **calculate** function:

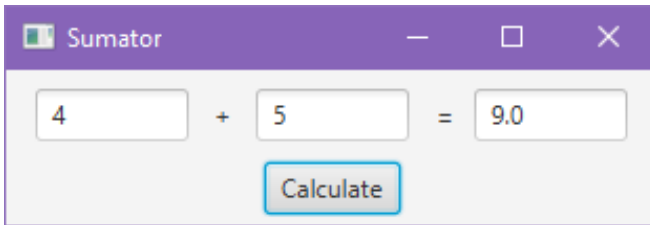
```
double num1 = Double.parseDouble(this.num1.getText());
double num2 = Double.parseDouble(this.num2.getText());

double result = num1 + num2;
this.result.setText(String.valueOf(result));
```

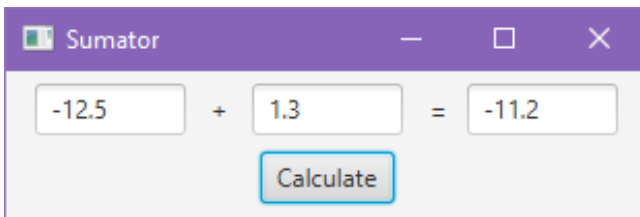
This code takes the first number from the field **num1** and stores it in the variable **num1**, then does the same with the **num2** field and stores it in the variable **num2**. The numbers are then summed, storing their value in the variable **result**, and finally, the latter's text value is visualized in the **result** field.



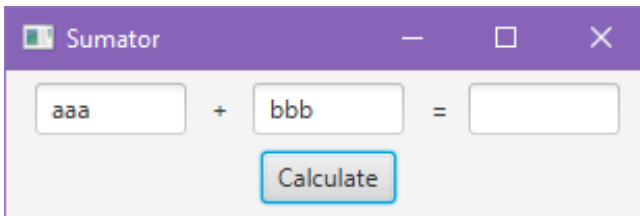
**Start** the program again with [Shift+F10] and **check if it is working**. Check whether we can calculate the sum of different values correctly. First, we enter two whole numbers – 4 and 5, for example:



Then we try summing negative and positive floating-point numbers – for example, -12.5 and 1.3:



How about entering invalid input such as **aaa** and **bbb\*\***:



We have a problem when the user enters **invalid input data**:

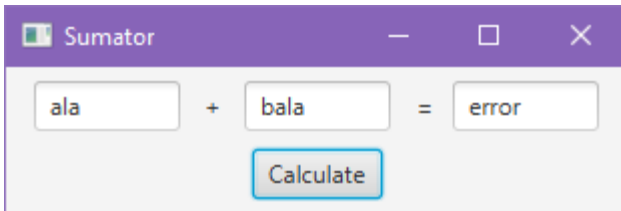
```
at com.sun.java.tk.quantum.GlassViewEventHandler.handleMouseEvent (GlassViewEventHandler.java)
at com.sun.glass.ui.View.handleMouseEvent (View.java:555)
at com.sun.glass.ui.View.notifyMouse (View.java:937)
at com.sun.glass.ui.win.WinApplication._runLoop (Native Method)
at com.sun.glass.ui.win.WinApplication.lambda$null$148 (WinApplication.java:191)
at java.lang.Thread.run (Thread.java:748)
Caused by: java.lang.reflect.InvocationTargetException <9 internal calls>
at javafx.fxml.FXMLLoader$MethodHandler.invoke (FXMLLoader.java:1769)
... 48 more
Caused by: java.lang.NumberFormatException: For input string: "aaa"
at sun.misc.FloatingDecimal.readJavaFormatString (FloatingDecimal.java:2043)
at sun.misc.FloatingDecimal.parseDouble (FloatingDecimal.java:110)
at java.lang.Double.parseDouble (Double.java:538)
at sample.Controller.calculate (Controller.java:13)
... 58 more
```

It is due to the transformation of our text field into a number. If the value in the field isn't a number, the program crashes and **returns an error** (exception). We can fix the code by doing the following:

```
try {
    double num1 = Double.parseDouble(this.num1.getText());
    double num2 = Double.parseDouble(this.num2.getText());

    double result = num1 + num2;
    this.result.setText(String.valueOf(result));
} catch (Exception e) {
    this.result.setText("error");
}
```

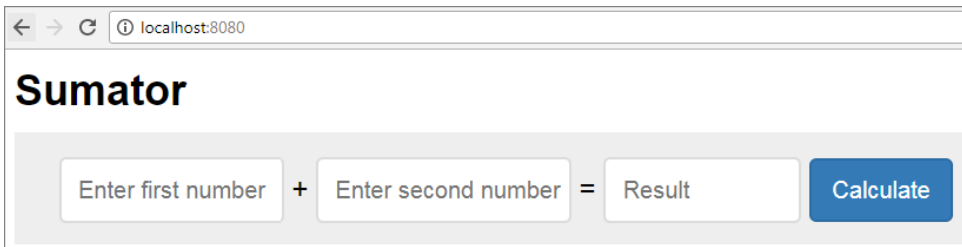
With this code, we intercept errors when working with numbers (**catch exceptions**), and in case of an error, we display the value "error" in our result field. Start the program again with [Shift+F10] and check if it's working. This time, if an incorrect number is entered, the result is "error" and the program doesn't crash:



Does all this seem complicated? If it does, that's completely normal. We are at the beginning of our programming journey. The example above requires much more knowledge and skills, which we will develop using this book and later on. Just allow yourself to have some fun with desktop programming. If something doesn't work, feel free to ask for help in the SoftUni discussion forum: <https://forum.softuni.org>, or bravely move forward to the next example or the next chapter in the book. A time will come when this will be easy for you, but you have to put in some effort and be persistent. Learning programming is a slow process involving lots and lots of practice.

## Web Application: 'Numbers Summator'

Now, we will create something even more complex but more interesting: a Web application that **calculates the sum of two numbers**. By **entering two numbers** in the first two text fields and clicking the [Calculate] button, **their sum is calculated**, and the result is displayed in the third text field. The application is expected to look like as is shown below:



Unlike the console applications that read and write data as text on the console, **Web applications** have a **Web-based user interface**. Web applications are loaded from some

**Internet address** (URL) through a standard **web browser**. Users write input data on a page, visualized by the web browser, the data is processed on a web server, and the results are shown again on a page in the web browser.

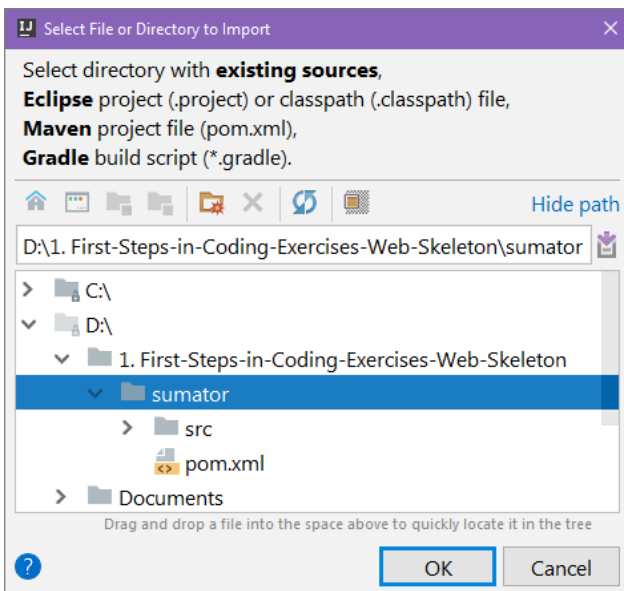
For our web application, we will use **Spring MVC**. It enables us to create a web application with the **Java** programming language in the **IntelliJ IDEA** development environment.

Take the project (the application's skeleton) from this link:

<https://github.com/SoftUni/Programming-Basics-Book-Java-EN/blob/master/assets/chapter-1-assets> and load it in IntelliJ IDEA using [Import Project]:

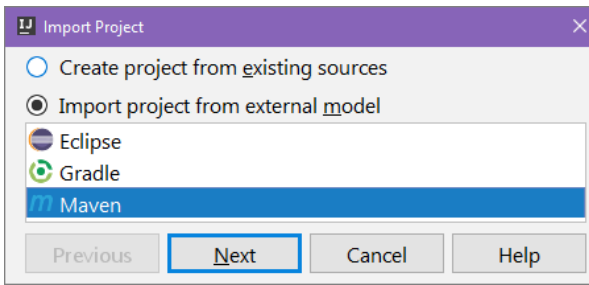


Find the folder where we have downloaded the skeleton:

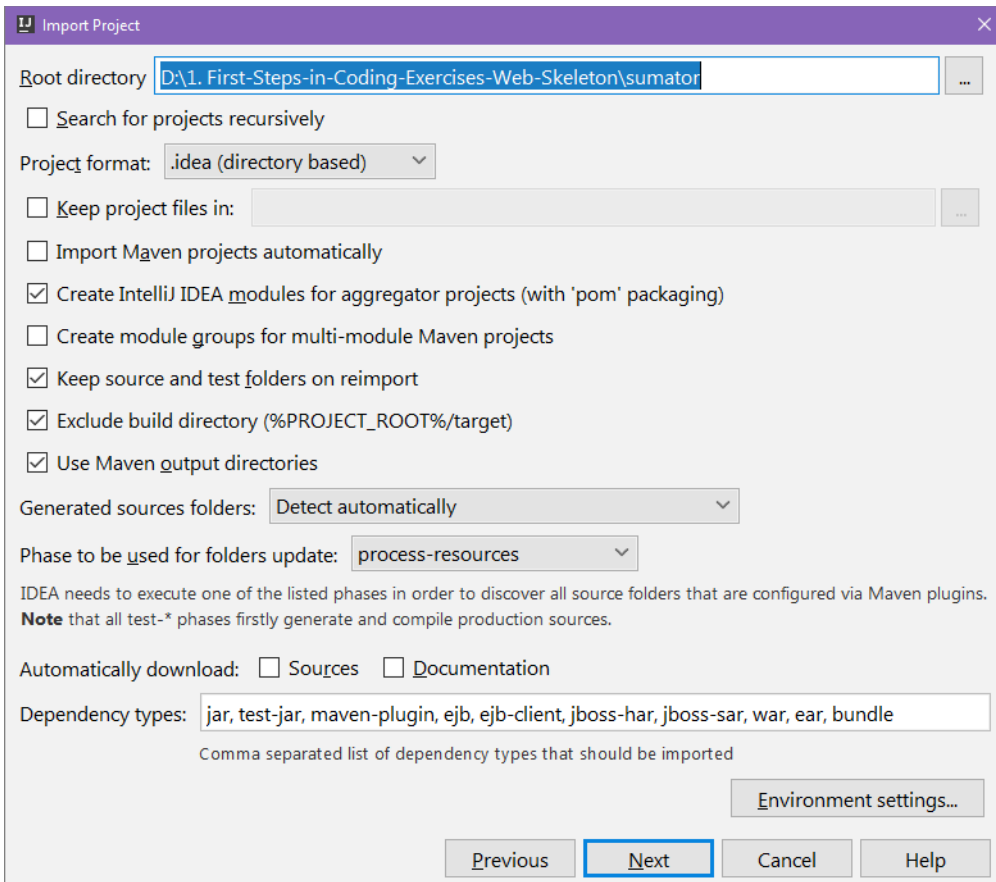


Choose [Import project from an external model] and then [Maven]:

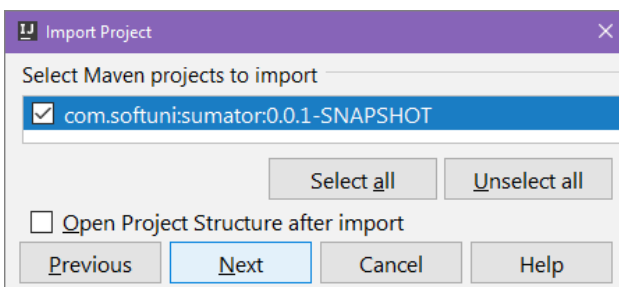




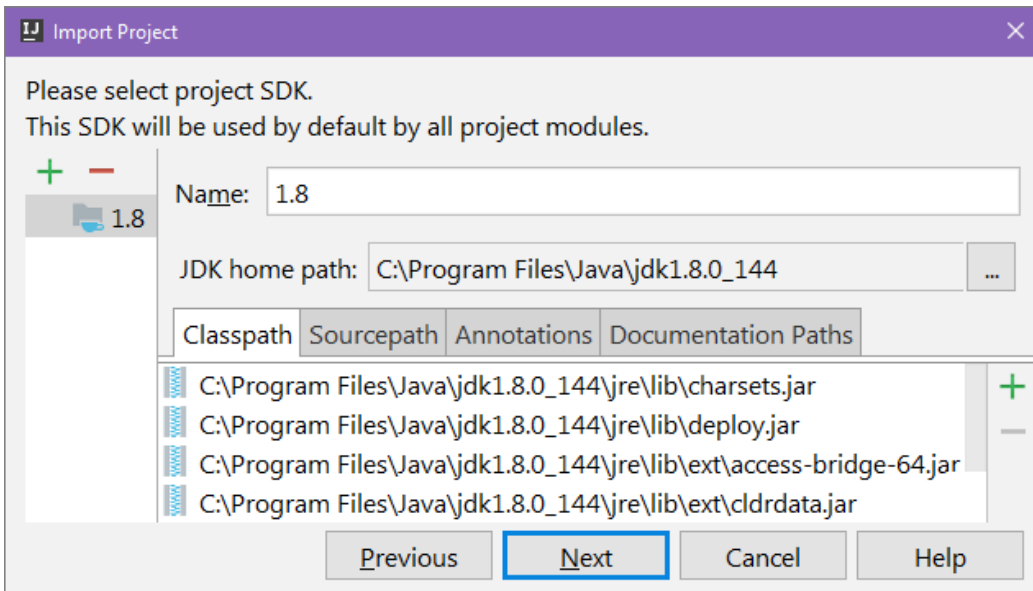
It is the screen that comes up next:



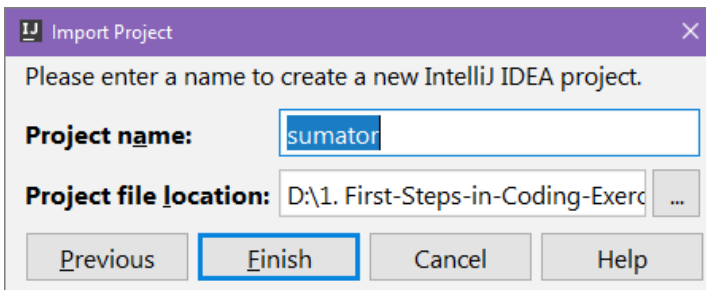
Proceed by clicking [Next]:



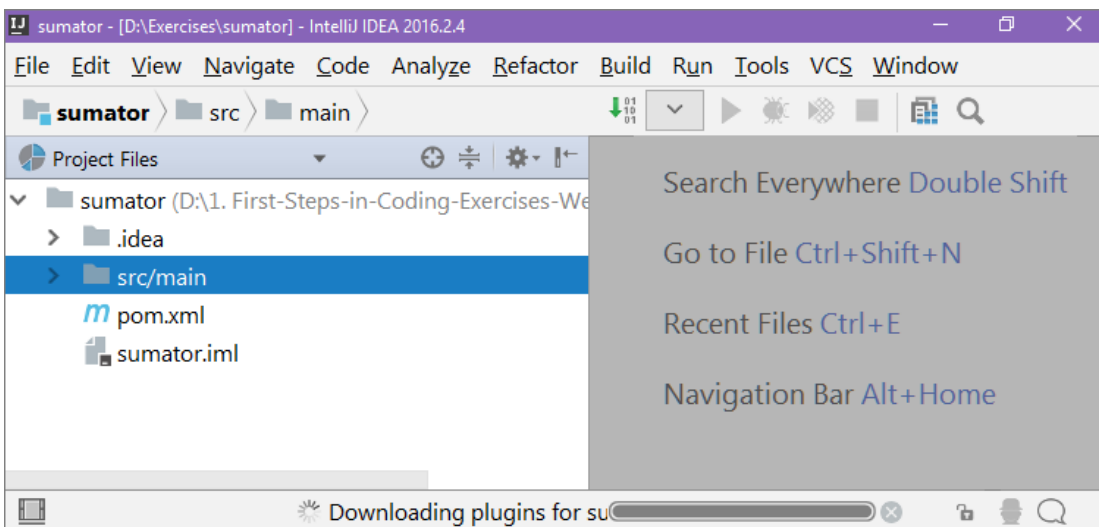
And again, continue with [Next]:



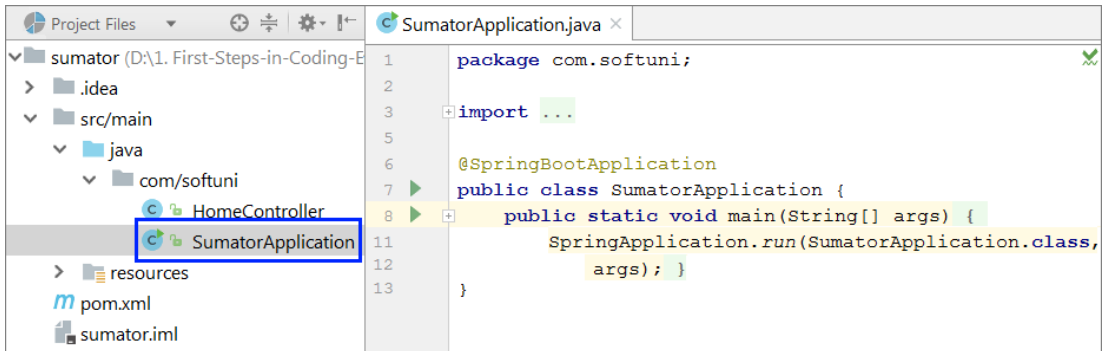
One more time – [Next]:



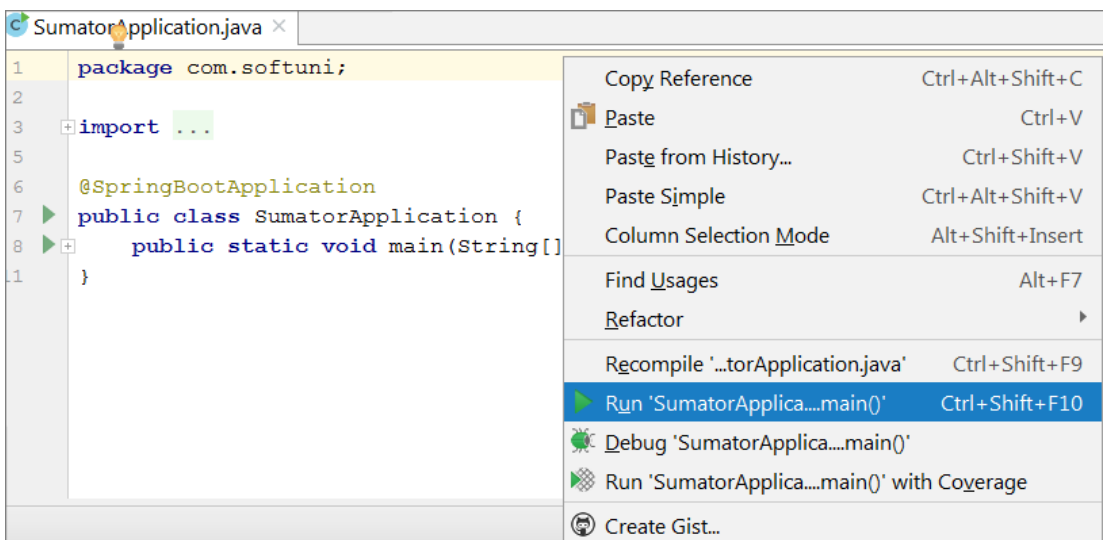
Once the project is loaded, in IntelliJ, it should look something like this:



Let us start the project and make sure it runs without errors. To do that, we will go to the [src/main/javacom.softuni] folder and open the **SumatorApplication** file:



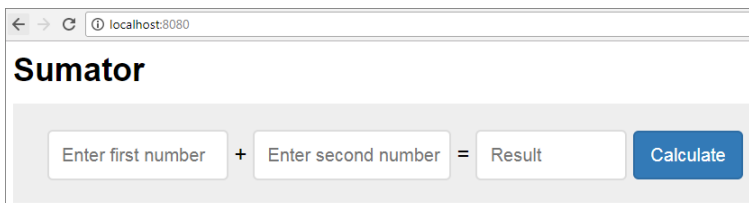
To run the program, we can either use the [Ctrl+Shift+F10] key combination, or select [Run 'SumatorApplication.main()'] from the context menu:



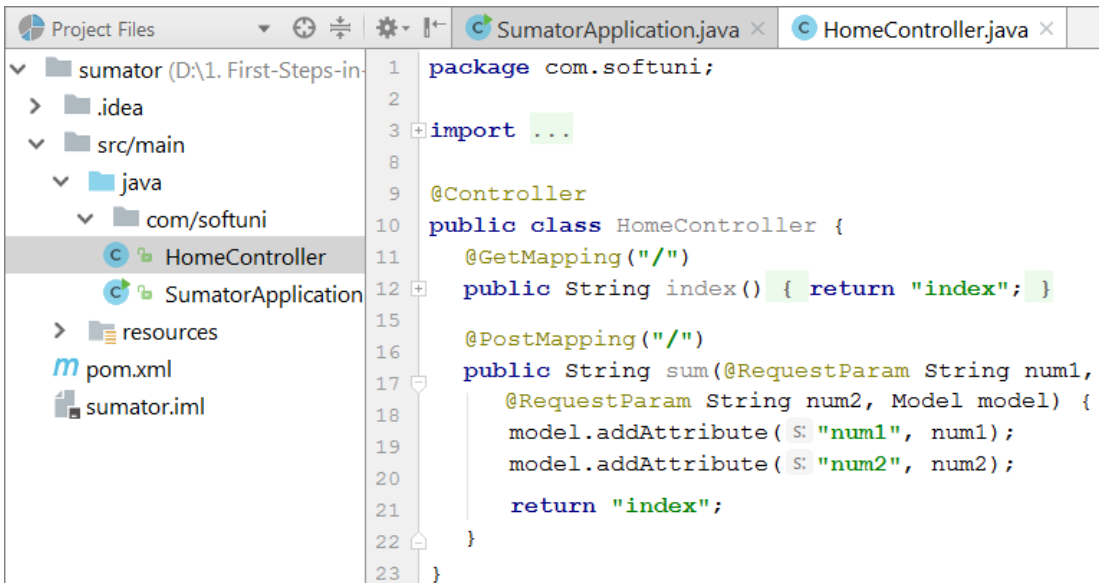
Once the web application is loaded, we should see the following message at the bottom of the console which has appeared:

**Started SumatorApplication in 14.296 seconds**

We can run the application and check whether it's working by opening our web browser and typing **localhost:8080** in the address bar:



All seems fine but when we enter the two numbers and hit [Calculate], nothing happens. Let us write the logic needed to sum both numbers to work as expected. Open the **HomeController** file in the same folder, and we should be able to see the following:



The part we are interested in is the **sum** function:

```

@PostMapping("/")
public String sum(@RequestParam String num1, @RequestParam String num2, Model model) {
    model.addAttribute(S: "num1", num1);
    model.addAttribute(S: "num2", num2);

    return "index";
}

```

Currently, this function accepts two pieces of text **num1** and **num2**, passes them to the server, and returns the file **index** to the user. As you have probably realized, there is no code to calculate the numbers from the two text fields and enter the result in the third field. Let us write the logic to make it happen.

Go between the curly braces and write the following code:

```

double result = Double.parseDouble(num1) + Double.parseDouble(num2);
model.addAttribute(S: "result", result);

```

Here is what the **HomeController** file should look like after the change:

```

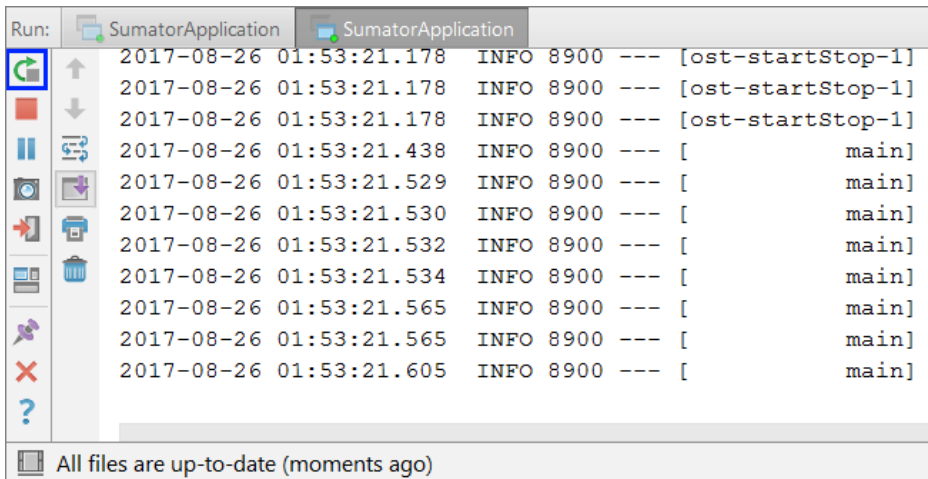
@PostMapping("/")
public String sum(@RequestParam String num1, @RequestParam String num2, Model model) {
    model.addAttribute(S: "num1", num1);
    model.addAttribute(S: "num2", num2);

    double result = Double.parseDouble(num1) + Double.parseDouble(num2);
    model.addAttribute(S: "result", result);

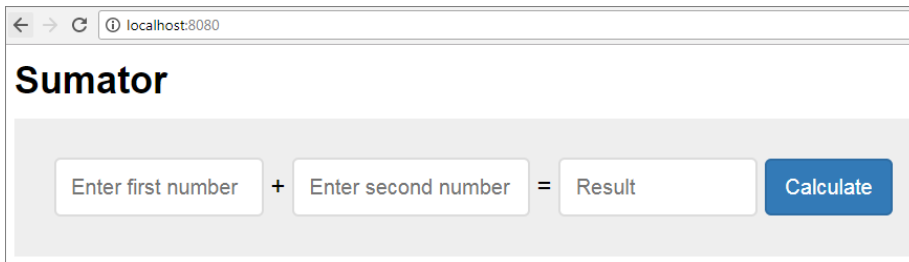
    return "index";
}

```

Before we go back to the application, we need to apply our changes. We'll do that by going to the console window and clicking the **[Rerun application]** button.



The application is completed. If we enter the two numbers in the text fields and click the [Calculate] button, we should see the result come up in the third text field:

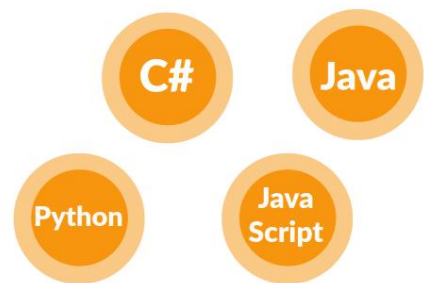


Does all this seem scary? **There's no need to be afraid!** We have a lot to learn to reach the knowledge and skills required to write web-based applications, with ease like in the example above, and much bigger and more complex applications. If it all makes little sense, just keep going without worrying. In time, you will remember with a smile how incomprehensible and exciting your first collision with web programming was.

The purpose of both examples (the graphical desktop application and web application) is not to teach you, but to allow you to dive a little deeper into programming, **to fuel your interest** in software development, and to inspire you to study hard. **You have a lot to learn**, but it is interesting, isn't it?

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



[Apply now](#)

[softuni.org/apply](https://softuni.org/apply)

## Chapter 2.1. Simple Calculations

In this chapter, we are going to get familiar with the following concepts and programming techniques:

- What is a **system console**?
- How to **read numbers** from the system console?
- How to work with **data types and variables**, which are necessary to process numbers and the operations between them?
- How to **print** (a number) on the console?
- How to do simple **arithmetic operations**: add, subtract, multiply, divide, string concatenation?

### Videos

Watch a video lesson on this chapter here:

[https://www.youtube.com/watch?v=JcT7eh\\_X\\_sM](https://www.youtube.com/watch?v=JcT7eh_X_sM).

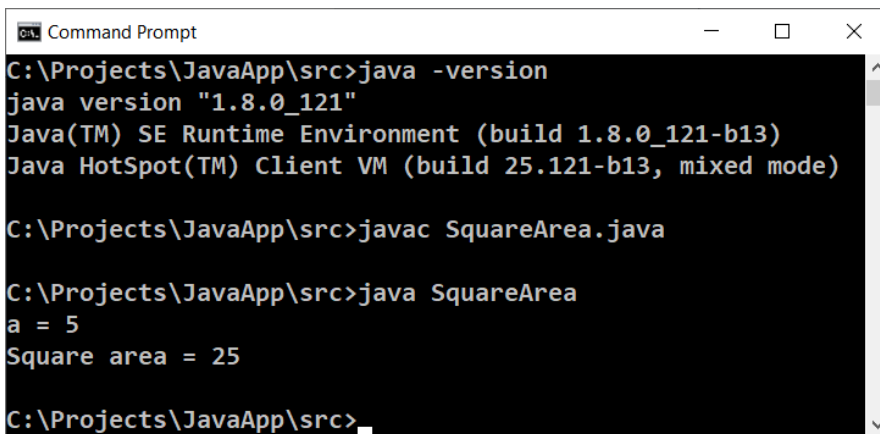
Watch a video lesson on this chapter here:

<https://www.youtube.com/watch?v=tuLo1BPRCqM>.

### The System Console

The system console or the computer console, usually called just a **console**, represents the tool by which we give the computer commands in a text format and get the results from their execution again as a text.

Generally, the system console represents a text terminal which means that it accepts and visualizes just **text** without any graphical elements like buttons, menus, etc. Usually, it looks like a black-colored window, in which we write and perform **console commands** and print the result in the form of a text. In **Windows**, the console looks like this:



```
Command Prompt
C:\Projects\JavaApp\src>java -version
java version "1.8.0_121"
Java(TM) SE Runtime Environment (build 1.8.0_121-b13)
Java HotSpot(TM) Client VM (build 25.121-b13, mixed mode)

C:\Projects\JavaApp\src>javac SquareArea.java

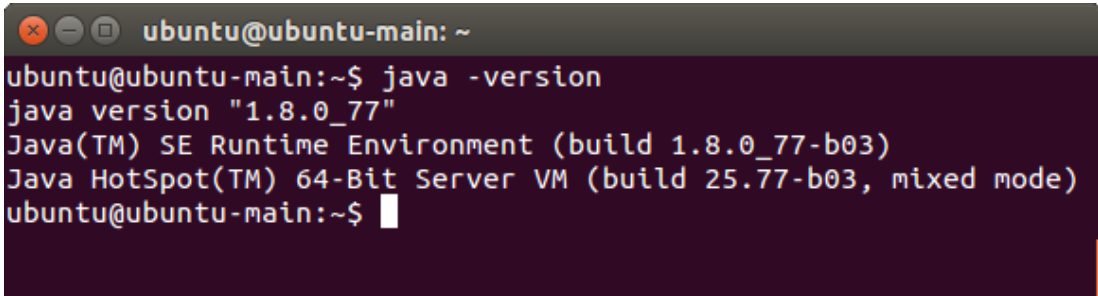
C:\Projects\JavaApp\src>java SquareArea
a = 5
Square area = 25

C:\Projects\JavaApp\src>
```



In most operating systems, generally, **the console** is available as a standalone application on which we write different console commands. In Windows, it is called **Command Prompt**, while in Linux and macOS, it is called **Terminal**. The console runs console applications. They read the text as input from the command line and print the output as text on the console. In this book, we are going to learn programming mostly through creating **console applications**.

This is how the console looks in **Linux**:

A screenshot of a Linux terminal window. The title bar shows window control buttons and the text 'ubuntu@ubuntu-main: ~'. The terminal prompt is 'ubuntu@ubuntu-main:~\$'. The command 'java -version' has been entered, and the output is displayed in four lines: 'java version "1.8.0\_77"', 'Java(TM) SE Runtime Environment (build 1.8.0\_77-b03)', 'Java HotSpot(TM) 64-Bit Server VM (build 25.77-b03, mixed mode)', and the prompt 'ubuntu@ubuntu-main:~\$' followed by a cursor.

## Reading Integers from The Console

In **Java**, we have several options to read input from **the console**. The one way is via the built-in class **Scanner**, which we will use now, and the other is via **BufferedReader**. The second one is a bit faster, but we will start using it after we get familiar with the streams in **Java**.

To read an **integer** (not a floating-point number) from the console we have to import the built-in class **Scanner** on the first line of our program even before the base class declaration. We put this line at the top:

```
import java.util.Scanner;
```

Then in the **main(...)** method of the program, we have to create a new **Scanner** instance used to attach the system input. After that (via the created **scanner** reader) we can read integers, floating-point numbers, text, etc. For example, with **.nextInt()** we read the next integer given as input:

```
Scanner scanner = new Scanner(System.in);  
int num = scanner.nextInt();
```

In the example above, in the variable **num** of integer type **int**, we will have a value read as input from **the console**.

Java is a **highly typed language** this means that the variables have type determined at compile-time and can not change at the execution time (as it is in dynamic languages such as JavaScript and Python). In the example above **scanner** is from type **java.util.Scanner** (scanning text reader) and the variable **num** is from type **int** (integer number).

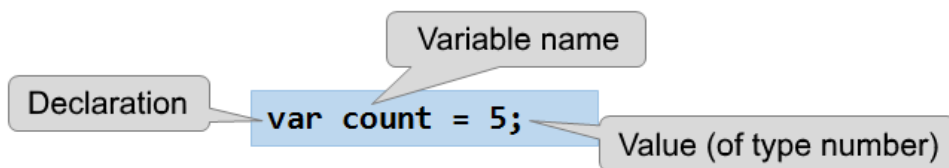


## Reading Floating-Point Numbers from the Console

Try to rewrite the above program to accept as input data not only integers (**int**) but floating-point numbers (**double**). You can use `scanner.nextDouble()`. Later in this chapter, we will learn more about floating-point numbers.

## Calculations in Programming

We know about computers that they are machines that process data. All **data** is stored in the computer memory (Random-access memory (RAM)) in **variables**. A variable is the name of a reserved area allocated in memory that stores data from certain types, for example, numbers or text. Each **variable** in Java has a **name**, **type**, and **value**. In the example below, you can see how to define a variable, and at the same time assign value to it:



After processing, the data is stored again in variables (somewhere in a reserved area allocated in memory by our program).

## Data Types and Variables

In programming, each variable stores a certain **value** of a particular **type**. For example, data types can be a **number**, **letter**, **text** (string), **date**, **color**, **image**, **list**, and others. Here are some examples of data types:

- type **int** number: 1, 2, 3, 4, 5, ...
- type **double** number: 0.5, 3.14, -1.5, ...
- type a **letter** from the alphabet (symbol): 'a', 'b', 'c', ...
- type **text** (string): "Hello", "Hi", "Beer", ...
- type **day of the week**: Monday, Tuesday, ...

## Reading Floating-Point Numbers from The Console

To read a **floating-point** number from the console, again it is necessary to **declare** a **variable**, to point out the **type** of the number, and to use the standard command to read information from the system console:

```
Scanner scanner = new Scanner(System.in);

Double num = scanner.nextDouble();
```

## Problem: Square Area

For example, let's look at the following program which reads an **integer** from the console, multiplies it by itself (squares it), and prints the result from the multiplication. That's how we can calculate square area by side length:

```
// Put this code in the file: SquareArea.java

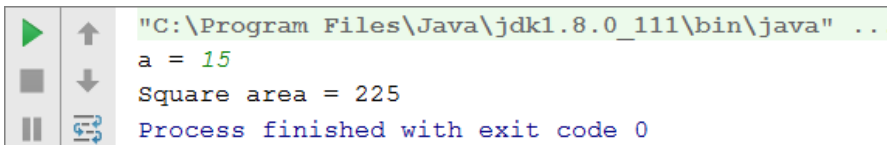
import java.util.Scanner;

public class SquareArea {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("a = ");
        int a = scanner.nextInt();
        int area = a * a;

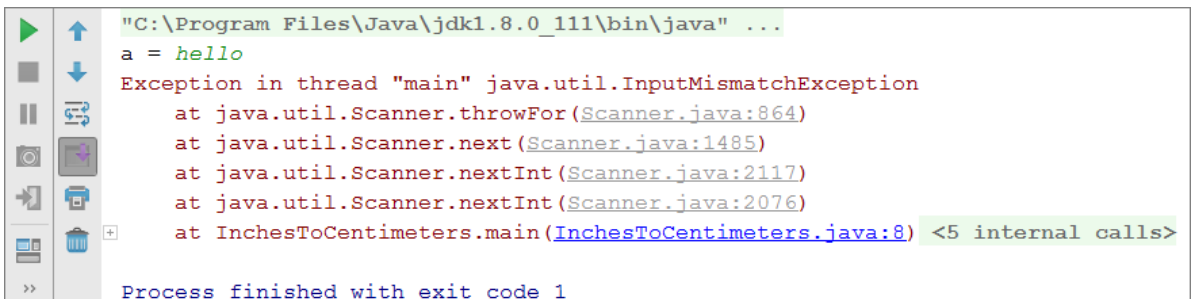
        System.out.print("Square area = ");
        System.out.println(area);
    }
}
```

Here is how the program will work with a side having a size of 15:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
a = 15
Square area = 225
Process finished with exit code 0
```

If we change the number to string - "hello", we will get an exception error.



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
a = hello
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:864)
    at java.util.Scanner.next(Scanner.java:1485)
    at java.util.Scanner.nextInt(Scanner.java:2117)
    at java.util.Scanner.nextInt(Scanner.java:2076)
    at InchesToCentimeters.main(InchesToCentimeters.java:8) <5 internal calls>
Process finished with exit code 1
```

This is a normal phenomenon, because the **Java** language is strongly typed. Later we will learn how we can catch these types of errors and ask the user to type his input again.

## How Does the Example Work?

On the first line **Scanner scanner = new Scanner(System.in);** create a new instance of the class **Scanner** with standart system code.

On the next line `System.out.print("a = ");` prints informative message, that asks the user to type the value for the side `a` of the square. If we want our next input to be on the same row we use `System.out.print(...);`, and not `System.out.println(...);` because this will move it to the next row.

On the next line `int a = scanner.nextInt();` reads the integer number from the console. The result is binded to the variable `a`

The next command `int area = a * a;` defines a new variable `area` with the result `a * a`

The next command `System.out.print("Square area = ");` prints the text without going to a new line. Again we are using `System.out.print(...);`, and not `System.out.println(...);`.

With the last command we will print the area of the square - `System.out.println(area);`

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#0>.

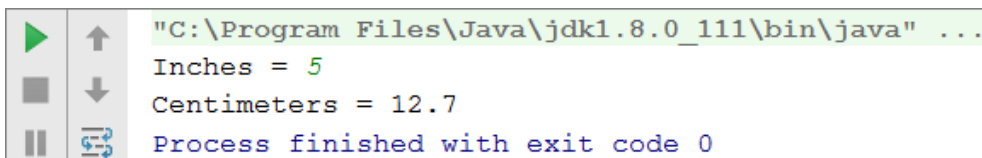
### Problem: Inches to Centimeters

Write a program that reads a floating-point number (representing inches) as an input from the console, converts it to centimeters, and print the result as output:

```
Scanner scanner = new Scanner(System.in);

System.out.print("Inches = ");
Double inches = scanner.nextDouble();
Double centimeters = inches * 2.54;
System.out.print("Centimeters = ");
System.out.println(centimeters);
```

Run the program and make sure that when entering a value in inches, we obtain a correct output in centimeters:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
Inches = 5
Centimeters = 12.7
Process finished with exit code 0
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#1>.

## Reading and Printing Text

To read a text (string) from the console, again we declare a new variable and use the standard command for reading a text from the console:

```
Scanner scanner = new Scanner(System.in);

String str = scanner.nextLine();
```

Let's pay attention to the fact that **reading text does not require converting to another type**. It is this way because, by default, the method `scanner.nextLine(...)` returns as **result text**. Additionally, we can parse the text to an integer by `Integer.parseInt(...)` or floating-point number by `Double.parseDouble(...)`. If we do not parse to a number, each number will be just **text** for the program, and **we can not do arithmetic operations** with them.

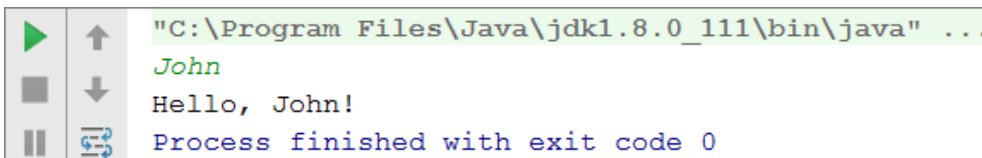
## Problem: Greeting by Name

Write a program that reads the user's name as input from the console and greets him with the text **Hello, *name***, where *name* is the name read as input data.

```
Scanner scanner = new Scanner(System.in);

String name = scanner.nextLine();
System.out.printf("Hello, %s!", name);
```

In this case, the expression `%s` is replaced with the **first** passed argument, which is the variable **name**:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
John
Hello, John!
Process finished with exit code 0
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#2>.

## Problem Concatenate Data

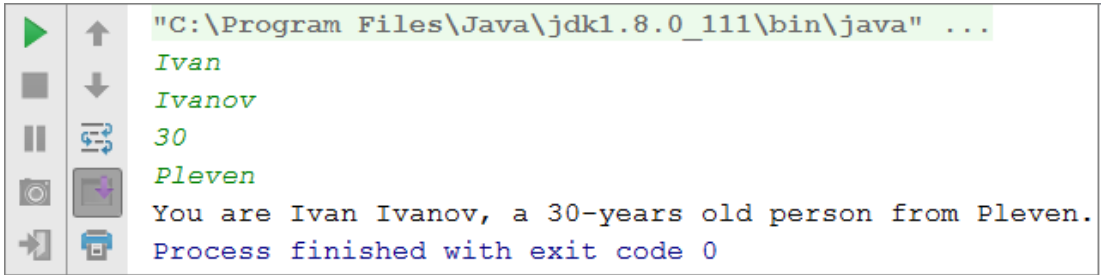
When printing text, numbers, and other data on the console **we can join them** by using templates `%s`, `%d`, `%f`, etc. In programming, these templates are called **placeholders**. Depending on the type of the variable, via different symbols, we declare different types of arguments. The main ones are `%s` for text (string), `%d` for integers, `%f` for floating-point numbers.

Here is **example** for printing formatted text with (placeholders):

```
Scanner scanner = new Scanner(System.in);
```

```
String firstName = scanner.nextLine();
String lastName = scanner.nextLine();
int age = Integer.parseInt(scanner.nextLine());
String town = scanner.nextLine();
System.out.printf("You are %s %s, a %d-years old person from %s.",
    firstName, lastName, age, town);
```

Here is the result, which we will get after the execution of the example:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
Ivan
Ivanov
30
Pleven
You are Ivan Ivanov, a 30-years old person from Pleven.
Process finished with exit code 0
```

Pay attention to the order of variables in the `System.out.printf()` method. Each variable is in the **order we want to print it** and corresponds to the **type in the placeholder**. Essentially, the template (placeholder) accepts variables of **every type**.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#3>.

## Arithmetic Operations

Let's look at the basic arithmetic operations in programming.

### Summing Up Numbers (Operator +)

We can sum up numbers using the operator `+`:

```
int a = 5;
int b = 7;
int sum = a + b; // the result is 12
```

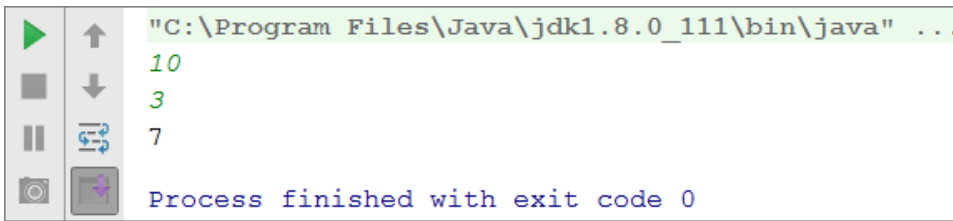
### Subtracting Numbers (Operator -)

Subtracting numbers is done by the operator `-`:

```
Scanner scanner = new Scanner(System.in);

int a = Integer.parseInt(scanner.nextLine());
int b = Integer.parseInt(scanner.nextLine());
int result = a - b;
System.out.println(result);
```

Here is the result of the execution of the program above (with numbers 10 and 3):



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
10
3
7
Process finished with exit code 0
```

## Multiplying Numbers (Operator \*)

For multiplication of numbers, we use the operator `*`:

```
int a = 5;
int b = 7;
int product = a * b; // 35
```

## Dividing Numbers (Operator /)

We divide numbers using the operator `/`. It works differently with integers and floating-point numbers.

- When we divide two integers, an **integer division** is applied, and the obtained output is without its fractional part. Example:  $11 / 3 = 3$ .
- When we divide two numbers, and at least one of them is a floating-point number a **fractional division** is applied, and the obtained result is a floating-point number, just like in math. Example  $11 / 4.0 = 2.75$ . When division is not with exact precision, the result is being rounded, for example,  $11.0 / 3 = 3.666666666666667$ .
- The integer **division by 0** causes an **exception** during runtime (`Java.lang.ArithmeticException`).
- The floating-point number **divided by 0** does not cause an exception and the result is  $\pm$  infinity or the special value **NaN**. Example  $5 / 0.0 = \infty$ .

Here are a few examples with the division operator:

```
int a = 25;
int i = a / 4;           // we perform the so-called integer division:
                        // the result of this operation will be 6 - the
fractional part is cut off,
                        // as we perform division by integers
int f = a / 4.0;        // 6.25 - fractional division. We have
explicitly specified that the number 4 should be interpreted as a
fraction,
                        // adding the decimal point followed by zero
int error = a / 0;      // Error: integer division by 0
```

Let's look at a few examples for **integer division** (remember that when we **divide integers** in Java the result is **an integer**):

```
int a = 25;
System.out.println(a / 4); // Integer result: 6
System.out.println(a / 0); // Error: integer division by 0
```

Let's look at a few examples for **floating numbers division**. When we divide floating-point numbers, the result is always a **float number** and the division never fails and works correctly with the special values  $+\infty$  and  $-\infty$ :

```
int a = 15;
System.out.println(a / 2.0); // Fractional result: 7.5
System.out.println(a / 0.0); // Result: Infinity
System.out.println(-a / 0.0); // Result: -Infinity
System.out.println(0.0 / 0.0); // Result: NaN (Not a Number), i.e.
the result                      // of the operation is not a valid
numeric value
```

When printing the values  $\infty$  and  $-\infty$  the console output may be `?`, because the console in Windows does not work correctly with Unicode and breaks most of the non-standard symbols, letters, and special characters. The example above would most probably give the following result:

```
7.5
?
-?
NaN
```

## Concatenating Text and Numbers

The operator `+` besides for summing up numbers is also used for joining text (concatenation of two strings one after another). In programming, joining text with other text or with number is called "concatenation". Here is how we can concatenate a text with a number with the operator `+`:

```
String firstName = "Maria";
String lastName = "Ivanova";
int age = 19;
String str = firstName + " " + lastName + " @ " + age;
System.out.println(str); // Maria Ivanova @ 19
```

Here is another example:

```
double a = 1.5;
double b = 2.5;
String sum = "The sum is: " + a + b;
System.out.println(sum); // The sum is: 1.52.5
```

Did you notice something strange? Maybe you expected the numbers **a** and **b** to be summed? The concatenation works from left to right and the result above is correct. If we want to sum the numbers we have to use **brackets** to change the order of execution of the operations:

```
double a = 1.5;
double b = 2.5;
String sum = "The sum is: " + (a + b);
System.out.println(sum); // The sum is: 4
```

## Numerical Expressions

In programming, we can calculate **numerical expressions**, for example:

```
int expr = (3 + 5) * (4 - 2);
```

The standard rule for the precedence of arithmetic operations is applied: **multiplying and dividing are always done before adding and subtracting**. In the case of an **expression in brackets**, it is calculated first, but we already know all of that from math.

## Problems Simple Calculations

To consolidate our knowledge of simple calculations, let's solve several problems.

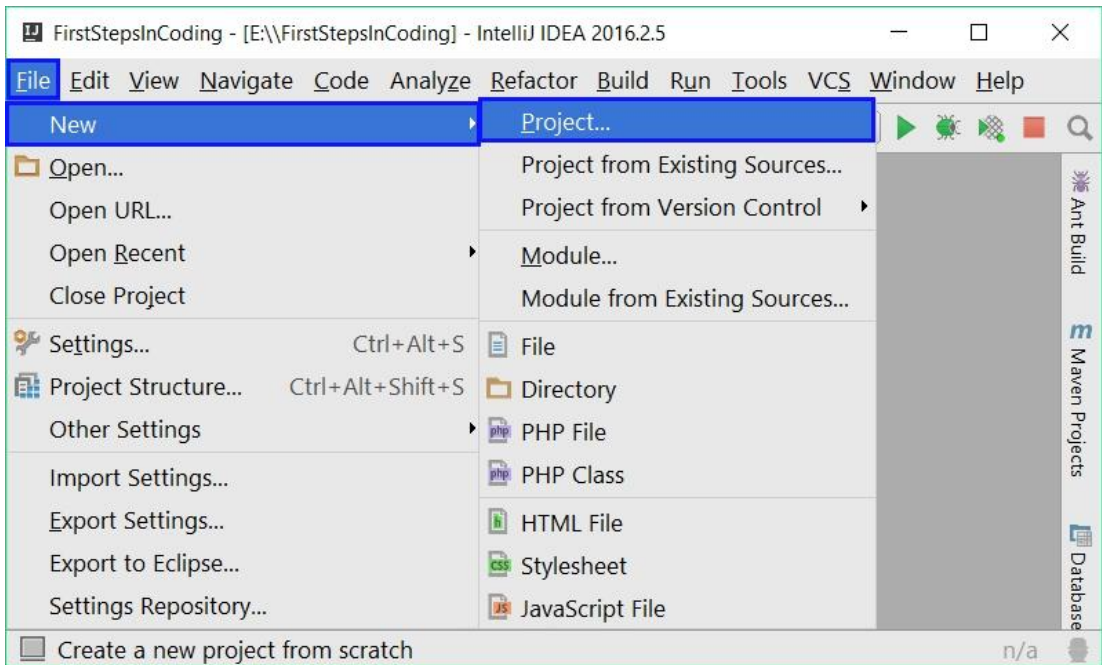
### Blank IntelliJ IDEA solution (Project)

We start by creating an empty solution (**Project**) in IntelliJ IDEA. The solutions (project) in IntelliJ IDEA combine **a group of problems**. This opportunity is **very convenient** when we want to **work on a few projects** and switch quickly between them or we want to **consolidate logically a few interconnected projects**.

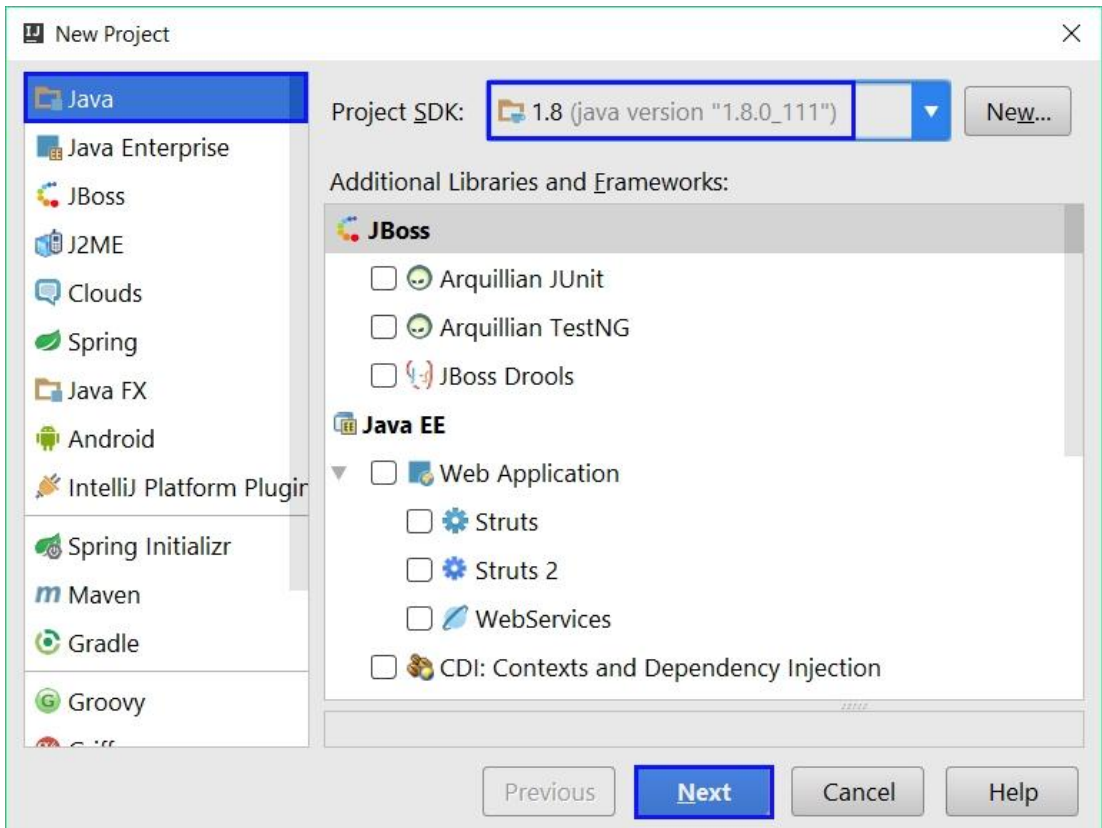
In the current practical problem, we will use a **Project with a couple of problems (Java classes)** to organize the solutions of the problems from the problems – every problem in a separate Java class and all of them in a common project.

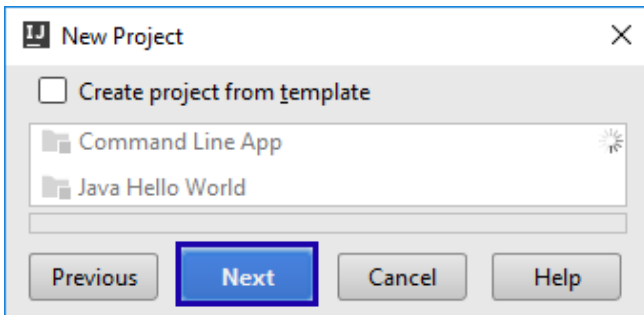
- We start IntelliJ IDEA.
- We create a new **Project**: [File] → [New] → [Project].



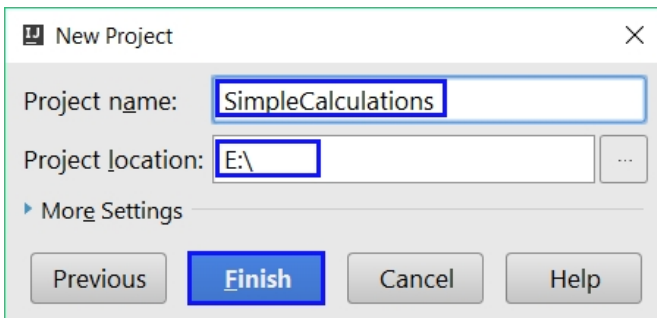


Select from the dialog window [Java], in the field Project SDK we specify which version of Java we will use and finally we click [Next] → [Next]:

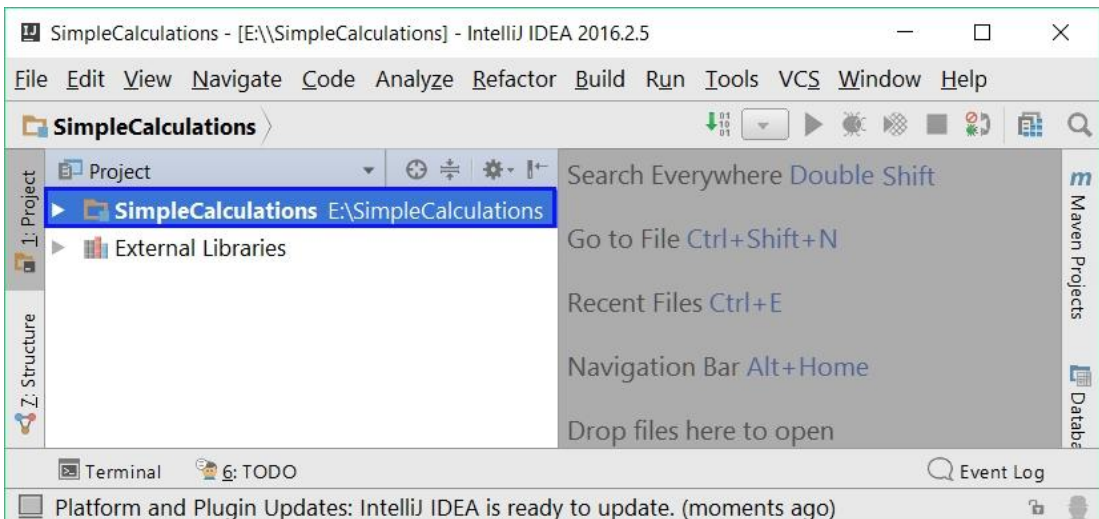




Give an appropriate name for the project, for example "SimpleCalculations" and also where to be stored our project, and after that, we click [Finish]:



Now we have an empty IntelliJ IDEA Project (without any Java classes in it which have to be in the **src** folder):



The purpose of this project is to add in it **.java** class per problem from the problems.

## Problem: Concatenating Text and Numbers

Write a Java program, that reads from the console a first name, last name, age, and city, and prints a message of the following kind: **You are <firstName> <lastName>, a <age>-years old person from <town>..**

## Hints and Guidelines

We add to the existing IntelliJ IDEA Project one more Java class with the name "ConcatenateData". We write the code, which reads the input from the console:

```
Scanner scanner = new Scanner(System.in);
String firstName = scanner.nextLine();
String lastName = scanner.nextLine();
int age = Integer.parseInt(scanner.nextLine());
String town = scanner.nextLine();
```

You should write the code that prints the message described in the requirements.

```
System.out.printf("You are %s %s, a %d-years old person from %s",
    firstName, lastName, age, town);
```

In the picture above, the code is consciously blurred on purpose, for you to think of a way to finish it yourself.

Next, test the solution locally using [Ctrl+Shift+F10] and by entering sample input data.

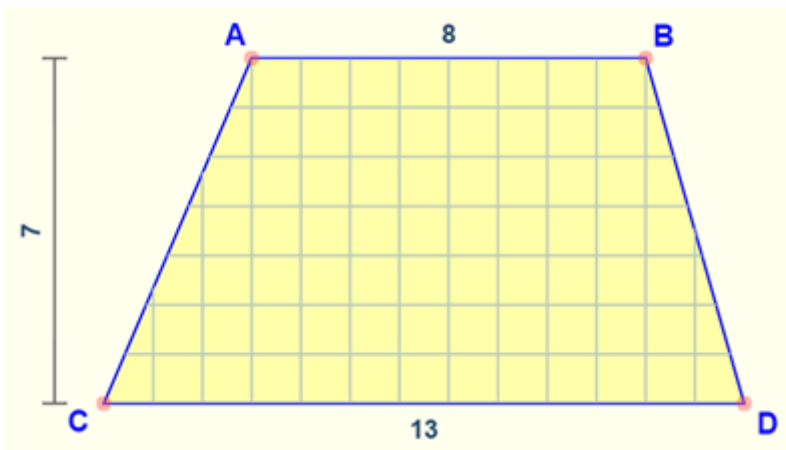
## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#3>.

## Problem: Trapezoid Area

Write a program that reads three numbers from the console **b1**, **b2**, and **h** and then calculates the area of a trapezoid with bases **b1** and **b2** and height **h**. The formula for the area of trapezoid is  $(b1 + b2) * h / 2$ .

The figure below shows a trapezoid with bases 8 and 13 and height 7. It has an area  $(8 + 13) * 7 / 2 = 73.5$ .



## Hints and Guidelines

Again, we have to add to the existing IntelliJ IDEA Project another **Java class** with the name "**TrapezoidArea**" and to write the code that reads the input from the console, then calculates the area of the trapezoid, and prints it as an output on the console:

```
import java.util.Scanner;

public class TrapezoidArea {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double b1 = Double.parseDouble(scanner.nextLine());
        double b2 = Double.parseDouble(scanner.nextLine());
        double h = Double.parseDouble(scanner.nextLine());
        double area = (b1 + b2) * h / 2.0;

        System.out.println("Trapezoid area = " + area);
    }
}
```

The code on the picture is consciously blurred for you to give a thought and finish yourself.

Test your solution locally using [Ctrl+Shift+F10] and by entering sample input data.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#4>.

## Problem: Circle Area and Perimeter

Write a program that reads from the console a number **r** then calculates and prints the area and perimeter of the circle/round with radius **r**.

### Sample Input and Output

| Input | Output                                                  |
|-------|---------------------------------------------------------|
| 3     | Area = 28.2743338823081<br>Perimeter = 18.8495559215388 |
| 4.5   | Area = 63.6172512351933<br>Perimeter = 28.2743338823081 |

## Hints and Guidelines

For the calculations, you can use the following formulas:

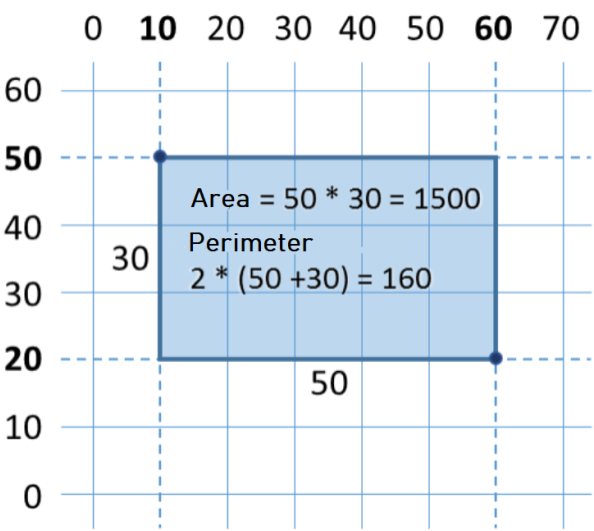
- `Area = Math.PI * r * r.`
- `Perimeter = 2 * Math.PI * r.`

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#5>.

Problem: Rectangle Area

A rectangle is defined by the **coordinates** at two of its opposite angles (x1, y1) – (x2, y2). Calculate its **area** and **perimeter**. The **input** is read from the console. The numbers **x1**, **y1**, **x2**, and **y2** are given one per line. The **output** is printed on the console and it has to contain two lines, each with one number – the area and the perimeter.



Sample Input and Output

| Input | Output      | Input | Output      | Input  | Output              |
|-------|-------------|-------|-------------|--------|---------------------|
| 60    | 1500<br>160 | 30    | 2000<br>180 | 600.25 | 350449.6875<br>2402 |
| 20    |             | 40    |             | 500.75 |                     |
| 10    |             | 70    |             | 100.50 |                     |
| 50    |             | -10   |             | -200.5 |                     |

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#6>.

Problem: Triangle Area

Write a program that reads from the console a **side** and **height** of a **triangle** and calculates its area. Use the **formula** for triangle area: `area = a * h / 2`. Round the result to **2 digits** after the decimal point using `Math.round(area*100.00)/100.00`.

### Sample Input and Output

| Input        | Output                |
|--------------|-----------------------|
| 20<br>30     | Triangle area = 300   |
| 7.75<br>8.45 | Triangle area = 32.74 |

| Input              | Output                |
|--------------------|-----------------------|
| 15<br>35           | Triangle area = 262.5 |
| 1.23456<br>4.56789 | Triangle area = 2.82  |

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#7>.

### Problem: Celsius to Fahrenheit

Write a program that reads **degrees on the Celsius scale** (°C) and converts them to **degrees on the Fahrenheit scale** (°F). Look on the Internet for a proper [formula](#), to do the calculations. Round the result to **2 digits after the decimal point**. Here are a few sample data:

### Sample Input and Output

| Input | Output |
|-------|--------|
| 25    | 77     |

| Input | Output |
|-------|--------|
| 0     | 32     |

| Input | Output |
|-------|--------|
| -5.5  | 22.1   |

| Input | Output |
|-------|--------|
| 32.3  | 90.14  |

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#8>.

### Problem: Radians to Degrees

Write a program, that reads **an angle in radians** (**rad**) and converts it to **degrees** (**deg**). Look for a proper formula on the Internet. The number  $\pi$  in Java programs is available through **Math.PI**. Round the result to the nearest integer using the method **Math.round(...)**.

### Sample Input and Output

| Input  | Output |
|--------|--------|
| 3.1416 | 180    |
| 6.2832 | 360    |

| Input  | Output |
|--------|--------|
| 0.7854 | 45     |
| 0.5236 | 30     |

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#9>.

### Problem: USD to BGN

Write a program for the **conversion of US dollars (USD) into Bulgarian leva (BGN)**. Round the result **2 digits** after the decimal point. Use a fixed rate between a dollar and leva: **1 USD = 1.79549 BGN**.

#### Sample Input and Output

| Input | Output    | Input | Output     | Input | Output    |
|-------|-----------|-------|------------|-------|-----------|
| 20    | 35.91 BGN | 100   | 179.55 BGN | 12.5  | 22.44 BGN |

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#10>.

### Problem \* Currency Converter

Write a program for the **conversion of money from one currency into another**. It has to support the following currencies: BGN, USD, EUR, GBP. Use the following fixed currency rates:

| Exchange rate | USD     | EUR     | GBP     |
|---------------|---------|---------|---------|
| 1 BGN         | 1.79549 | 1.95583 | 2.53405 |

The **input** consists of three lines - the first is a **sum for conversion**, the second is the **input currency**, and the third is the **output currency**. The **output** is one number – the converted value according to the above exchange rates, rounded **2 digits** after the decimal point.

#### Sample Input and Output

| Input             | Output    | Input                | Output     |
|-------------------|-----------|----------------------|------------|
| 20<br>USD<br>BGN  | 35.91 BGN | 12.35<br>EUR<br>GBP  | 9.53 GBP   |
| 100<br>BGN<br>EUR | 51.13 EUR | 150.35<br>USD<br>EUR | 138.02 EUR |

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#11>.

## Problem: 1000 Days After Birth

As an example, let us look at a program. The problem is to calculate the area of a square by a given side's length read as input from the console. The sample source code of the program is below. The code **reads an integer** as input from the console, **multiplies it** by itself (squares it), and as output **prints the result** from the multiplication. Save the code in a file with the name `SquareArea.java`, or else you will have a compile-time error:

```
// Put this code in the file: SquareArea.java

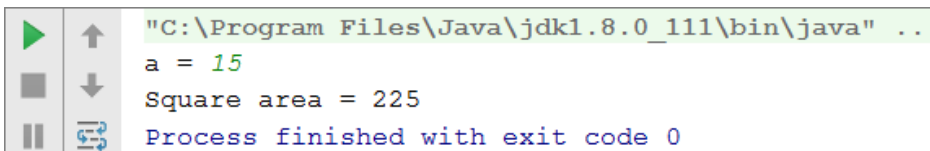
import java.util.Scanner;

public class SquareArea {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("a = ");
        int a = scanner.nextInt();
        int area = a * a;

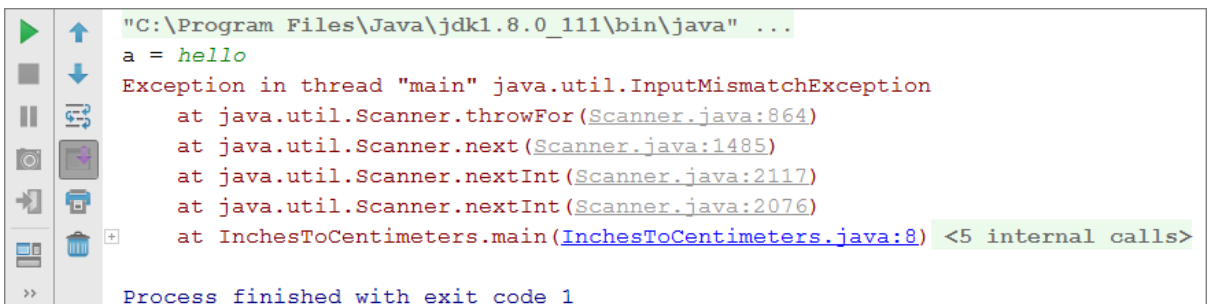
        System.out.print("Square area = ");
        System.out.println(area);
    }
}
```

Here is how the program would work when we have a square with a side's length equal to 15:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
a = 15
Square area = 225
Process finished with exit code 0
```

Test the program with an invalid integer (as an input), for example, "hello". You will get an error message during runtime (exception).



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
a = hello
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:864)
    at java.util.Scanner.next(Scanner.java:1485)
    at java.util.Scanner.nextInt(Scanner.java:2117)
    at java.util.Scanner.nextInt(Scanner.java:2076)
    at InchesToCentimeters.main(InchesToCentimeters.java:8) <5 internal calls>
Process finished with exit code 1
```



It is normal because **Java** is a highly typed language. Later on, we will find out how we can catch these kinds of errors and make the user enter a new input (a number in the example above).

## How Does the Example Work?

The first line **Scanner scanner = new Scanner(System.in);** creates a new instance from the **Scanner** class with standard system input.

The following line **System.out.print("a = ");** prints an informative message which invites the user to enter the side **a** of the square. After the output is printed the cursor stays on the same line. Staying on the same line is more convenient for the user visually. We use **System.out.print(...);** and not **System.out.println(...);** and this way the cursor stays on the same line.

The following line **int a = scanner.nextInt();** reads an integer from the console. The result is kept in a variable with a name **a**.

The following line **int area = a \* a;** keeps the result of the multiplication of **a** by **a** in a new variable with a name **area**.

The following line **System.out.print("Square area = ");** prints the given text without going to the next line. Again, use **System.out.print(...);** and not **System.out.println("...");** and this way the cursor stays on the same line to print the calculated area of the square afterward.

At the last line **System.out.println(area);** prints the calculated value of the variable **area**.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/649#0>.

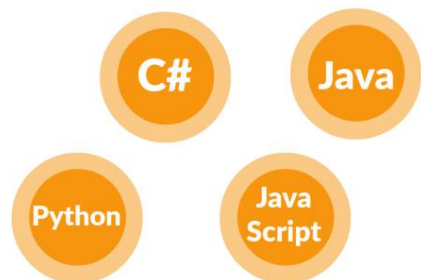
## What Have We Learned from This Chapter?

To summarize what have we learned in this chapter of the book:

- Reading a text: **String str = scanner.nextLine();** (as we have written in advance **Scanner scanner = new Scanner(System.in);**).
- Reading an integer: **int num = Integer.parseInt(scanner.nextLine());**.
- Reading a floating-point number: **double num = Double.parseDouble(scanner.nextLine());**.
- Calculations with numbers and using the relevant arithmetic operators **[+, -, \*, /, (, )]**: **int sum = 5 + 3;**.
- Printing a text by placeholders on the console: **System.out.printf("%d + %d = %d", 3, 5, 3 + 5);**.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**

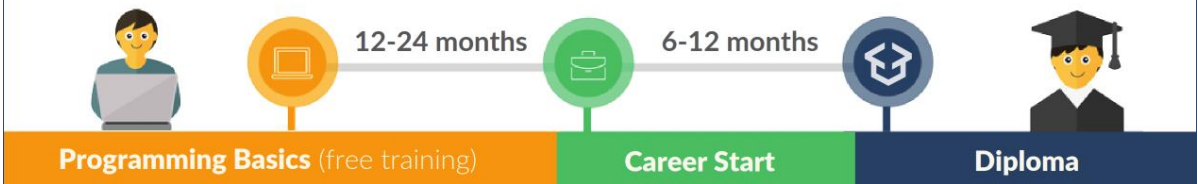


**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)

## Chapter 2.2. Simple Calculations – Exam Problems

In the previous chapter, we got familiar with the system console and how to work with it – how to **read numbers** from the console and **print output** on the console. We went through the basic arithmetical operations and briefly mentioned data types. In this chapter, we will practice and consolidate what we have learned so far. We will solve a few **more complicated problems** given as exams problems.

### Reading Numbers from The Console

Before moving on to the problems, we will recall the most important of the material we have studied in the previous chapter. We will start with reading numbers from the console.

#### Reading an Integer

We need to create a variable to store the number (for example **num**) and to use the standard command to read data from the console in combination with the function **Integer.parseInt(...)** which converts text into an integer:

```
int num = Integer.parseInt(scanner.nextLine());
```

#### Reading a Floating-Point Number

We read a floating-point number the same way we read an integer, but this time we use the function **Double.parseDouble(...)**:

```
double num = Double.parseDouble(scanner.nextLine());
```

### Printing Text Using Format Specifiers

The **format specifier** is an expression that will be replaced with a particular value while printing the output. The method **System.out.printf(...)** supports printing a string using format specifiers with the following format **System.out.printf("format-string", arg1, arg2, ... ])**. As the first parameter of the method, we put the formatted string. As the second parameter of the method, we put so many arguments as the number of specifiers in the formatted string.

```
System.out.printf("You are %s %s, a %d-years old person from %s.",  
    firstName, lastName, age, town);
```

## Arithmetic Operators

Let us recall basic arithmetic operators for simple calculations.

### Operator +

```
int result = 3 + 5; // the result is 8
```

### Operator -

```
int result = 3 - 5; // the result is -2
```

### Operator \*

```
int result = 3 * 5; // the result is 15
```

### Operator /

```
int result = 7 / 3; // the result is 2 (integer division)
double result2 = 5 / 2.0; // the result is 2.5 (fractional division)
```

## Concatenation

When using the operator `+` between string variables (or between a string and a number), the result is so-called concatenation (combining strings).

```
String firstName = "Ivan";
String lastName = "Ivanov";
int age = 19;
String str = firstName + " " + lastName + " is " + age + " years
old";
// Ivan Ivanov is 19 years old
```

## Exam Problems

Now, after we have recalled how to make simple calculations, read and print numbers on the console, let's move to the problems. We will solve several **problems from a SoftUni practical exam**.

### Problem: Training Lab

A **training lab** has a rectangular size `l` and `w` meters without columns on the inside. The classroom is divided into two parts - left and right with a hallway located approximately in

the middle of the room. In both parts, there are **rows with desks**. In the back of the hall, there is a big **entrance door**. In the front, there is a podium with a **chair** for the teacher. A single **working place** occupies **70 x 120 cm** (a desk with a size 70 x 40 cm + space for a chair with a size 70 x 80 cm). **The hallway** has a width of at least **100 cm**. It is known that because of the **entrance door** (which has an opening of 160cm) **exactly one working space is lost** and because of the **podium** (which has a size of 160 x 120 cm) is lost in exactly **2 working places**. Write a program that reads the size of the classroom as input data and calculates the **number of working places in it** as it is described (look at the figure below).

Input Data

As input data, we will read **2 numbers** from the console, each on a separate line: **l** (length in meters) and **w** (width in meters).

Constraints:  $3 \leq w \leq l \leq 100$ .

Output Data

Print on the console one integer number: **the number of working places** in the classroom.

Sample Input and Output

| Input      | Output | Diagram |
|------------|--------|---------|
| 15<br>8.9  | 129    |         |
| 8.4<br>5.2 | 39     |         |

## Explanations to The Problems

In the first problem, the training lab length is 1500 cm. There could be located **12 rows** ( $12 * 120 \text{ cm} = 1440 + 60 \text{ cm residue}$ ). The training lab width is 890 cm. From them, 100 cm is for the hallway in the middle. In the rest 790 cm could be located **11 desks per row** ( $11 * 70 \text{ cm} = 770 \text{ cm} + 20 \text{ cm residue}$ ). **Number of working places** =  $12 * 11 - 3 = 132 - 3 = 129$  (we have 12 rows with 11 working places = 132 minus 3 working places for podium and entrance door).

In the second problem, the training lab length is 840 cm. There could be located **7 rows** ( $7 * 120 \text{ cm} = 840$ , without residue). The training lab width is 520 cm. From them 100 cm are for the hallway in the middle. In the rest, 420 cm could be located **6 desks per row** ( $6 * 70 \text{ cm} = 420 \text{ cm}$ , without residue). **Number of working places** =  $7 * 6 - 3 = 42 - 3 = 39$  (We have 7 rows with 6 working places per row = 42 minus 3 working places for podium and entrance door).

## Hints and Guidelines

Try to solve the problem on your own first. If you fail, use the Hints and Guidelines below.

### An Idea for a Solution

In any programming problem, the **important part is to build an idea for its solution** before starting to write code. Let's carefully go through the problem requirements. We have to write a program that calculates the number of working places in a training lab where the number depends on the length and height of the room. We notice that the provided input data will be in **meters**, but we have the sizes for working places, and the hallway is in **centimeters**. To do the calculations, we must use the same measuring units no matter whether we choose to convert length and height into centimeters or the other data in meters. We use the first option for the presented solution.

Next, we have to calculate **how many columns and how many rows** with desks will fit in the training lab. We can calculate the columns by **subtracting the width by the necessary space for the hallway (100 cm)** and **dividing the difference by 70 cm** (the length of a working place). We will find the rows by dividing the **length by 120 cm**. In both operations, as a result, we can receive a number with integer and fractional parts. **In a variable, we should store only the integer part**. In the end, we multiply the number of rows by the number of columns and divide it by 3 (lost the working places because of the entrance door and podium), and we will receive the required value.

### Choosing Data Types

From the example input data, it looks like we can get a number with integer and fractional parts. For that reason, it is not appropriate to choose the **int** data type. Therefore we will use **double**. Selecting the correct data type for the variables depends on the method we choose to solve the problem. Any programming problem as and this one also has **more than one way to be solved**. We will describe two methods.

## Solution

It is time to go to the solution. We can divide it into three smaller problems:

- Read the input.
- Perform calculations.
- Print the output on the console.

The first thing we have to do is read the input from the console. With `scanner.nextLine()` we read the values from the console and with the function `Double.parseDouble(...)` we convert the string (text) value into `double`.

```
Scanner scanner = new Scanner(System.in);
double length = Double.parseDouble(scanner.nextLine());
double width = Double.parseDouble(scanner.nextLine());
```

Let's move to the calculations. The special part here is that after dividing the numbers, we have to store only the integer part of the result in a variable.



**Search Google!** Whenever we have an idea of how to solve a particular problem, but we do not know how to write it in Java, the easiest way to solve it is by looking for information on the net. If we are dealing with a problem that we assume that many other people have had before us, the easiest way to solve it is by looking for information on the Internet.

In this case, we can try with the following search: *java gets the whole number part of double*. We find out that one possible way is to use the method `Math.floor(...)`. As the method works with a `double` data type, we create variables of the same type for the number of rows and columns.

```
double cols = Math.floor(width / 70);
double rows = Math.floor(length / 120);
double seats = (cols * rows) - 3;
```

As a second solution, we can use: As we already know, the operator for division `/` operates differently on integers and floating-point numbers. **When dividing integer with integer (for example `int`), the result is also an integer.** Therefore, we can search for a way to convert the floating-point numbers of the height and the width of the training lab into integers and then divide them.

In this case, there could be **data loss** after removing the fractional part, so it is necessary for the conversion to be indicated **explicitly** (explicit typecasting). To indicate explicit conversion, we use the operator for converting data **(type)** by replacing the word **type** with the needed **data type** and placing it **before the variable**.

```
int cols = (int) ((width - 100) / 70);
int rows = (int) (length / 120);
int seats = (rows * cols) - 3;
```

With `System.out.println(...)` we print the result on the console.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/650#0>.

## Problem: Vegetable Market

A gardener sells the harvest from his garden on the vegetable market. He sells **vegetables** for **N BGN per kilogram** and **fruits** for **M BGN per kilogram**. Write a program to calculate the earnings from the harvest in EUR (assuming that one EUR is equal to BGN 1.94).

### Input Data

As input data, we will read **4 numbers** from the console, each on a separate line:

- On the first line – the price of the vegetables, per kilogram – a floating-point number.
- On the second line – the price of the fruits, per kilogram – a floating-point number.
- On the third line – total kilograms of vegetables – an integer.
- On the fourth line – total kilograms of fruits – an integer.

**Constraints:** all numbers will be in the range of **0.00** to **1000.00**.

### Output Data

Print on the console **one floating-point number**: the earnings of all fruits and vegetables in EUR.

### Sample Input and Output

| Input                     | Output | Input                  | Output           |
|---------------------------|--------|------------------------|------------------|
| 0.194<br>19.4<br>10<br>10 | 101    | 1.5<br>2.5<br>10<br>10 | 20.6185567010309 |

### Explanations to the first problem:

- The vegetables cost:  $0.194 \text{ BGN} * 10 \text{ kg.} = 1.94 \text{ BGN}$
- The fruits cost:  $19.4 \text{ BGN} * 10 \text{ kg.} = 194 \text{ BGN}$
- Total:  $195.94 \text{ BGN} = 101 \text{ EUR.}$



## Hints and Guidelines

First, we will give a few ideas and particular hints for solving the problem, followed by the essential part of the code.

### An Idea for a Solution

Let's first go through the problem requirements. In this case, we have to calculate the **total income** from the harvest. It is equal to the **sum of the profit from the fruits and vegetables**, which we can calculate by multiplying **the price per kilogram by the quantity**. Our input data is in leva, and the output should be in EUR. According to the condition 1 Euro equals 1.94lv, so to obtain the requested **output value**, we have to divide the sum by 1.94.

### Choosing Data Types

After we have a clear idea of how to solve the problem, we can proceed to choose appropriate data types. Let's go through the **input**: we have **two integers** for total kilograms of vegetables and fruits, so the variables we declare to store their values will be of **int**. The prices of the fruits and vegetables are said to be **floating-point numbers**, so the variables will be of type **double**.

We can also declare two variables to store the income from the fruits and vegetables separately. As we are multiplying a variable of type **int** (total kilograms) with another of **double** type (price), the result should also be of type **double**. Let's clarify that generally, **operators work with arguments of the same type**. Therefore, to multiply values of different data types, we have to convert them into the same type. When there are different types in one expression, conversion is done to the highest type, in this case, in this case **double**. Because this conversion does not suggest data loss **the conversion is implicit** and is automatically done by the compiler.

The **output** should also be a **floating-point number**. This means that the result will be stored in a variable of type **double**.

### Solution

It is time to move on to the solution. We can mentally divide it into three subproblems:

- Read the input.
- Perform calculations.
- Print the output on the console.

To read the input data, we declare variables, being careful to name them in such a way that it tells us what values the variables contain. With **scanner.nextLine()** we read values from the console and with the functions **Integer.parseInt(...)** and **Double.parseDouble(...)** we convert the particular string value into **int** and **double**.

```
Scanner scanner = new Scanner(System.in);
double vegPrice = Double.parseDouble(scanner.nextLine());
double fruitsPrice = Double.parseDouble(scanner.nextLine());
int vegPerKg = Integer.parseInt(scanner.nextLine());
int fruitsPerKg = Integer.parseInt(scanner.nextLine());
```

We do the necessary calculations:

```
double vegTotal = vegPrice * vegPerKg;
double fruitTotal = fruitsPrice * fruitsPerKg;
```

The problem does not specify a special output format, so we just need to calculate the required value and print it on the console. As in mathematics and so in programming, the division has a priority over addition. However, in this problem first, we have to **calculate the sum** of the two input values, and then we have to **divide by 1.94**. To give priority to the addition, we can use brackets. With **System.out.println(...)**, we print the output on the console.

```
System.out.println((fruitTotal + vegTotal) / 1.94);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/650#1>.

## Problem: Change Tiles

On the ground in front of an apartment building **tiles need to be placed**. The place is **square with a side of N meters**. The tiles are **W meters wide and L meters long**. There is a bench in the area with **a width of M meters and a length of O meters**. There is no need to put tiles under it. To place each tile requires **0.2 minutes**.

Write a program that reads from the console the dimensions of the ground, the tiles, and the bench. Then calculates **how many tiles are needed** to cover the place and calculates **the time to place all the tiles**.

**Example:** A **ground** with a size 20 m. has an area of 400 sq.m. A **bench** that is 1 m. wide and 2 m. long has an area of 2 sq.m. One **tile** is 5 m. wide and 4 m. long and has an area of = 20 sq.m. **The area** that needs to be covered is **400 - 2 = 398 sq.m**. The tiles that are needed are **398 / 20 = 19.90**. The needed **time** is **19.90 \* 0.2 = 3.98 minutes**.

## Input Data

As input data, we will read **5 numbers** from the console, each on a separate line:

- **N** – the **length** of a **side** of the ground within the range of [1 ... 100].
- **W** – the **width** of a **tile** in the range of [0.1 ... 10.00].
- **L** – the **length** of a **tile** in the range of [0.1 ... 10.00].
- **M** – the **width** of the **bench** in the range of [0 ... 10].

- O – the length of the bench in the range of [0 ... 10].

## Output Data

Print on the console **two numbers**, each on a new line: **number of tiles** needed for the repair and **the time for placing them**.

## Sample Input and Output

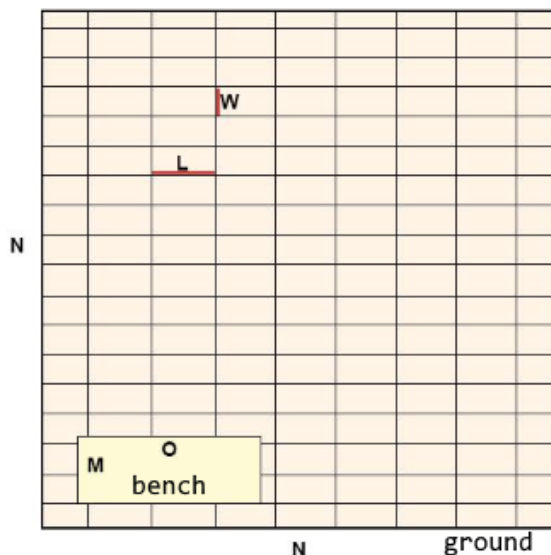
| Input | Output | Input | Output            |
|-------|--------|-------|-------------------|
| 20    |        | 40    |                   |
| 5     | 19.9   | 0.8   | 3302.083333333333 |
| 4     | 3.98   | 0.6   | 660.4166666666667 |
| 1     |        | 3     |                   |
| 2     |        | 5     |                   |

Explanation of the problem:

- Total area =  $20 * 20 = 400$ .
- Bench area =  $1 * 2 = 2$ .
- Coverage area =  $400 - 2 = 398$ .
- Tile area =  $5 * 4 = 20$ .
- Required tiles =  $398 / 20 = 19.9$ .
- Time required =  $19.9 * 0.2 = 3.98$ .

## Hints and Guidelines

Let's make a drawing to clarify the problem requirements. It can look like this:



## An Idea for a Solution

We need to calculate the number of tiles to cover the ground and the time for placing them. To calculate the number of tiles, we have to calculate the area that needs to be covered and divide it by the area per tile. The ground is square, and we find the total area using the formula  $N * N$ . After that, we calculate the area of the bench by multiplying its two sides  $M * O$ . After that, to obtain the area that needs to be covered, we subtract the area of the bench from the area of the ground.

We calculate the area of a single tile by multiplying its two sides with one another -  $W * L$ . As we already said, now we have to divide the area to the cove by the area of a single tile. In this way, we find the number of necessary tiles, which we multiply by 0.2 (the time needed for changing a single tile). Now, we have the required output.

## Choosing Data Types

Input data for the length of the side of the ground, the width, and the length of the bench are integer numbers, so to keep their values, we will declare variables of type **int**. For the width and the length of the tiles we will read floating-point numbers, so we will use variables of type **double**. The output will be a floating-point number, so used variables will be of **double** type.

## Solution

As in the previous problems, we can divide the solution into three subproblems:

- Read the input.
- Perform calculations.
- Print the input on the console.

The first thing we have to do is go through the input data of the problem. It is important to pay attention to the sequence of reading input data. With **scanner.nextLine()** we read values from the console and with **Integer.parseInt(...)** and **Double.parseDouble(...)** we convert the particular string value into **int** and **double**.

```
Scanner scanner = new Scanner(System.in);

// Ground length
int n = Integer.parseInt(scanner.nextLine());
// Tile width
double w = Double.parseDouble(scanner.nextLine());
// Tile length
double h = Double.parseDouble(scanner.nextLine());
// Bench width
int a = Integer.parseInt(scanner.nextLine());
// Bench length
int b = Integer.parseInt(scanner.nextLine());
```

Once we have initialized the variables and saved the corresponding values in them, we proceed to the **calculations**. As the values of the variables **n**, **a**, and **b** that we work with are stored in variables of type **int**, we can also declare for the results **variables of the same type**.

```
int n = ...;
int a = ...;
int b = ...;
```

The variables **w** and **h** are of type **double**, so for the **area of a single tile**, we create a variable of the same type. Finally, **we calculate the values that we have to print** on the console. **The number of needed tiles** we get by **dividing the area that needs to be covered by the area of a single tile**. When dividing two numbers, one of which is the **floating-point number** a result is also a **floating-point number**. For the calculations to be correct we store the result in a variable of type **double**. The problem does not specify special formatting or rounding of the output, so we just print the values with **System.out.println(...)**.

```
double tiles = areaToRepair / (w * h);
```

```
double time = tiles * 0.2;
```

```
System.out.println(tiles);
```

```
System.out.println(time);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/650#2>.

## Problem: Money

Some time ago Pesho bought bitcoins. He plans to go on a trip around Europe and he will need euros. Besides bitcoins, he has Chinese yuan(CNY). Pesho wants to exchange his money in euros for the trip. Write a program that calculates how much euros he can buy depending on the following exchange rates:

- 1 Bitcoin = 1168 BGN
- 1 CNY = 0.15 USD
- 1 USD = 1.76 BGN
- 1 EUR = 1.95 BGN

The exchange office has a commission within 0 to 5 percent from the final amount in EUR.

## Input Data

As input data, we will read **3 numbers** from the console, each on a separate line:

- On the first line – a **number of Bitcoins**. Integer in the range of [0 ... 20].

- On the second line – a **number of Chinese yuan**. Floating-point number in the range of [0.00 ... 50 000.00].
- On the third line – **commission fee**. Floating-point number in the range of [0.00 ... 5.00].

## Output Data

Print the result of the exchange of currencies on the console. Rounding the result is not necessary.

## Sample Input and Output

| Input       | Output               | Input             | Output               | Input                  | Output               |
|-------------|----------------------|-------------------|----------------------|------------------------|----------------------|
| 1<br>5<br>5 | 569.6687179<br>48718 | 20<br>5678<br>2.4 | 12442.24420<br>10256 | 7<br>50200.<br>12<br>3 | 10659.470117<br>7436 |

Explanation:

- 1 Bitcoin = 1168 BGN
- 5 CNY = 0.75 USD
- 0.75 USD = 1.32 BGN
- $1168 + 1.32 = 1169.32$  BGN = 599.651282051282 EUR
- Commission fee: 5% of 599.651282051282 = 29.9825641025641
- Result:  $599.651282051282 - 29.9825641025641 = 569.668717948718$  EUR

## Hints and Guidelines

Let's think again about how we can solve the problem before we start writing code.

### An Idea for a Solution

We see that the **number of bitcoins** and the **number of Chinese yuans** will be given in the input. The **output** should be in **euros**. The exchange rates that we have to work with are in the problem condition. We notice that we can only exchange the sum in BGN to EUR, so we have **first to calculate the whole sum that Peter has in BGN** and **then to calculate the output**.

As we have information for the exchange rate of Bitcoins to BGN, we can directly exchange them. On the other hand, to get the value of **Chinese yuans in BGN**, first, we have to **exchange them in USD** and then **USD to BGN**. Finally, we will **sum the two values** and calculate how much EUR that is.

To calculate the commission fee and subtract the new sum from the total one. The commission is a **floating-point number**, representing a **percentage of the total sum**. Let's divide it from the beginning by 100 to calculate its **percentage value**. We will multiply the percentage value by the sum in EUR, and the result divides with the same sum and print the final sum on the console.

### Choosing Data Types

Bitcoins are given as an **integer**, and for their value, we can declare a **variable of type int**. As a number Chinese yuan and commission fee, we will obtain a **floating-point number**, so we will use **double**. A **double** is the data type with a bigger size and the **output** should also be a **floating-point number**, we will use it for the other variables we create.

### Solution

After we have built an idea on how to solve the problem and we have chosen the data types, it is time to get to **write the code**. As in the previous problems, we can divide the solution into three smaller problems:

- Read the input.
- Perform calculations.
- Print the input on the console.

We declare the **variables** that we are going to use and again we carefully have to choose **meaningful names** in such a way that it tells us what values the variables contain. We initialize their values: with **scanner.nextLine()**, we read the input numbers from the console and convert the string entered by the user to **int** or **double**.

```
Scanner scanner = new Scanner(System.in);
int bitcoins = Integer.parseInt(scanner.nextLine());
double yuans = Double.parseDouble(scanner.nextLine());
double commission = Double.parseDouble(scanner.nextLine()) / 100;
```

We do the necessary calculations:

```
double bitcoinsToLeva = bitcoins * 1168;
double yuansToDollars = yuans * 0.15d;
double dollarsToLeva = yuansToDollars * 1.76d;
```

```
double sum = bitcoinsToLeva + dollarsToLeva;
double euro = sum / 1.936d;
```

Finally, we calculate the commission fee value and subtract it from the sum in EUR. Let's pay attention to this: **euro -= commission \* euro**. It is the shortcut to write **euro = euro - (commission \* euro)**. In this case, we use a **combined assignment operator -=**, which subtracts the right value of the operand from the one to the left. The operator for multiplication **\*** has a **higher priority** than the combined operator than the expression **commission \* euro** is performed first. Then its value is subtracted from the value to the left of the operand.

The problem does not specify special formatting or rounding of the result, so we just have to calculate the output and print it on the console.

```
euro -= commission * euro;  
System.out.println(euro);
```

Let's pay attention to something that applies to all other problems of this type: written like that, the solution of the problem is pretty detailed. As the problem is not too complex, we could write one complex expression, where after we read the input data, we calculate the output. For example, such an expression would look like this:

```
double euro = (bitcoins * 1168 + yuans * 0.15 * 1.76) / 1.95  
              - ((bitcoins * 1168 + yuans * 0.15 * 1.76) / 1.95 * commission);
```

This code would print a correct result, but it is **hard to read**. It won't be easy to find out how it works, and whether it contains any mistakes, as well as finding such and correcting them. It is better to **instead of one complex expression, write a few simpler ones** and store their values in variables with appropriate names. This way, the code is cleaner and easily maintainable.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/650#3>.

## Problem: Daily Earnings

Ivan is a programmer in an **American company** and he works from home **N days per month** and earns **M USD per day**. At the end of the year, Ivan **gets a bonus** which equals **2.5 monthly salaries**. From what he earned during the year, **the withholding tax is 25%**. Write a program that **calculates the amount of Ivan's net average earning** in BGN per day because he spends it in Bulgaria. We accept that one year has exactly **365 days**. The **exchange rate of USD to BGN** will be **read from the console**.

### Input Data

As input data, we will read **3 numbers** from the console, each on a separate line:

- On the first line – **working days of the month**. Integer in the range of [5 ... 30].
- On the second line – **money earned per day**. Floating-point number in the range of [10.00 ... 2000.00].
- On the third line – **exchange rate USD to BGN** /1 USD = X BGN/. Floating-point number in the range of [0.99 ... 1.99].

### Output Data

On the console **print average earning in BGN per day**. The result should be **rounded up to the second digit after the decimal point**.



## Sample Input and Output

| Input | Output | Input | Output | Input  | Output |
|-------|--------|-------|--------|--------|--------|
| 21    |        | 15    |        | 22     |        |
| 75.00 | 74.61  | 105   | 80.24  | 199.99 | 196.63 |
| 1.59  |        | 1.71  |        | 1.50   |        |

Explanation:

- Monthly salary =  $21 * 75 = 1575$  USD.
- Annual income =  $1575 * 12 + 1575 * 2.5 = 22837.5$  USD.
- Taxes = 25% from 22837.5 = 5709.375 USD.
- Net annual income = 17128.125 dollars = 27233.71875 BGN.
- Average earning per day =  $27233.71875 / 365 = 74.61$  BGN.

## Hints and Guidelines

Firstly, we have to analyze the problem and think of a way to solve it. Then we will choose data types, and finally, we will write the code.

### An Idea for a Solution

Let's first calculate how much the monthly salary of Ivan is. We will do this by multiplying the working days per month by the money he earns per day. We multiply the result by 12 to calculate his yearly salary and then multiply his yearly salary by 2.5 to calculate the bonus. Adding the two values obtained, we will calculate his total annual income. We have to subtract 25% from it for taxes. We can do this by multiplying the total income by 0.25 and subtracting the result from it. Then we exchange the USD to BGN and divide the result by the days of a year, which we assume are 365.

### Choosing Data Types

The working days per month are given as an integer, so we can declare a variable of type **int**. For the earned money and the exchange rate of USD to BGN, we will obtain a floating-point number, so for them, we use **double** as type. A **double** is a type with a bigger size and the output should also be a floating-point number, we use **double** for the other variables we create.

### Solution

After we have an idea of how to solve the problem, and we have considered the data types that we will use, we can start writing the code. As in the previous problems, we can divide the solution into three smaller problems:

- Read the input.
- Perform calculations.
- Print the output on the console.

We declare the variables that we are going to use by trying to choose meaningful names. With `scanner.nextLine()` we read the input numbers from the console and we convert the input string to `int` or `double` with `Integer/Double.parseDouble/Double(...)`.

```
Scanner scanner = new Scanner(System.in);
int workdays = Integer.parseInt(scanner.nextLine());
double moneyPerDay = Double.parseDouble(scanner.nextLine());
double moneyPerYear = Double.parseDouble(scanner.nextLine());
```

We do the necessary calculations:

```
double monthSalary = workdays * moneyPerDay;
double moneyPerYear = (monthSalary * 12) + (monthSalary * 2.5);
double taxes = 0.25 * moneyPerYear;
double netSalary = moneyPerYear - taxes;
double salaryInLeva = netSalary * currencyRate;
```

We could write an expression that calculates the annual income without brackets as well. Multiplication is an operation that has a higher priority over addition and performs first. Despite that, it is a good practice to use brackets when using more operators because the code becomes more easily readable and the chances of making a mistake are smaller.

Finally, we have to print the result on the console. We notice that the number has to be rounded up to the second digit after the decimal point. For this purpose, we can use formatted specifiers, an item that will be replaced by a particular value when printing. In Java, for formatting specifier is used `%` followed by a specific letter (specifier). We can format an integer or a floating-point number by using `F/f`, or `D/d`. In the case of a floating-point number, before the letter, we put a dot and a whole positive number specifying the number of digits after the decimal point.

The final result is as follows:

```
double average = salaryInLeva / 365;
System.out.printf("%.2f", average);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/650#4>.

# Chapter 3.1. Simple Conditions

This chapter will discuss **conditional statements in the Java language**. Depending on the condition, a program may have different behavior. First will be explained the syntax of conditional operators **if** and **if-else** with appropriate examples. Then we will see in what range (**scope**) a variable lives. Finally, we will look through **debugging** techniques to track the step-by-step execution of our programs.

## Video

Watch the video lesson about what we will learn in this chapter:

<https://www.youtube.com/watch?v=7H3K4g8BQ3w>.

## Comparing Numbers

In programming, we can compare values using the following **operators**:

- Operator **<** (less than)
- Operator **>** (greater than)
- Operator **<=** (less than or equals)
- Operator **>=** (greater than or equals)
- Operator **==** (equals)
- Operator **!=** (not equals)

When comparing values, the result is of boolean type with a value **true** or **false**, depending on whether the result of the comparison is true or false.

## Problems for Comparing Numbers

```
public static void main(String[] args) {
    int a = 5;
    int b = 10;

    System.out.println(a < b);
    System.out.println(a > 0);
    System.out.println(a > 100);
    System.out.println(a < a);
    System.out.println(a <= 5);
    System.out.println(b == 2 * a);
}
```

## Comparison Operators

In Java, we can use the following comparison operators when comparing numbers:

| Operator description   | Notation |
|------------------------|----------|
| Equals to              | ==       |
| Not Equals to          | !=       |
| Greater than           | >        |
| Greater than or equals | >=       |
| Less than              | <        |
| Less than or equals    | <=       |

The following example demonstrates how to use comparison operators in expressions:

```
boolean result = (5 <= 6);  
System.out.println(result); //true
```

## Simple If Conditions

In programming, we often **check given conditions** and perform different actions depending on the result. This is done by **if** conditional statement, which has the following structure:

```
if (condition) {  
    // body of if conditional statement  
    // single command or block of code to be executed if the  
    condition is true  
}
```

## Problem: Excellent Result

Read the grade from the console and check if it is excellent ( $\geq 5.50$ ).

```
Scanner scanner = new Scanner(System.in);  
double evaluation = scanner.nextDouble();
```

```
if (evaluation >= 5.5) {  
    System.out.println("Excellent!");  
}
```

Test the code (from the example) locally. Test with different grades, like 4.75, 5.49, 5.50, and 6.00. If the grade is **less than 5.50**, the program will not output any result, otherwise (if the grade is **greater than or equals 5.50**), the program will output "Excellent!".

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#0>.

## If-Else Conditions

Simple **if** conditions could be extended with an **else** conditional statement, which specifies a block of code to be executed if the boolean expression (defined at the beginning **if(condition)**) returns **false**. The resulting **conditional statement** is called **if-else** construction and have the following behavior: if the condition returns a **positive (true)** result – the code described in the curly brackets right after the **if** clause will be executed, otherwise if the condition returns a **negative (false)** result – the code described in the curly brackets after the **else** clause will be executed. The format of the construction is:

```
if (condition) {
    // body of if construction
    // single command or block of code to be executed if the
    condition is true
} else {
    // body of else construction
    // single command or block of code to be executed if the
    condition is false
}
```

### Problem: Excellent or Not

Like the example above, read the grade from the console and check if it is excellent, but we should **return the output in both cases**.

```
Scanner scanner = new Scanner(System.in);
double evaluation = scanner.nextDouble();

if (evaluation >= 5.5) {
    System.out.println("Excellent!");
} else {
    System.out.println("Not excellent.");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#1>.

## The Curly Brackets {} After If / Else

When we have **only one command** in the body of the **if** statement, we can **skip the curly brackets**. When we want to execute a **block of code** (group of commands), curly brackets are required, because if we skip them, **only the first line** after the **if** clause will be executed.



It is a good practice to **always** put curly brackets because it makes our code more readable and cleaner.

Here is an example that skipping curly brackets leads to confusion:

```
String color = "red";
if (color.equals("red"))
    System.out.println("tomato");
else if (color.equals("yellow"))
    System.out.println("banana");
System.out.println("lemon");
```

Executing the above code will output the following result on the console:

```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
tomato
lemon
Process finished with exit code 0
```

Here is the same example, but using curly brackets:

```
String color = "red";
if (color.equals("red")) {
    System.out.println("tomato");
}
else if (color.equals("yellow")) {
    System.out.println("banana");
    System.out.println("lemon");
}
```

Executing the code with curly brackets will output the following result on the console:

```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
tomato
Process finished with exit code 0
```



Both use cases are **correct** and could be used depending on the situation and/or the specific requirements, but you always must be careful and check the expected results.

## Problem: Even or Odd

Write a program that checks whether an integer is **even** or **odd**.

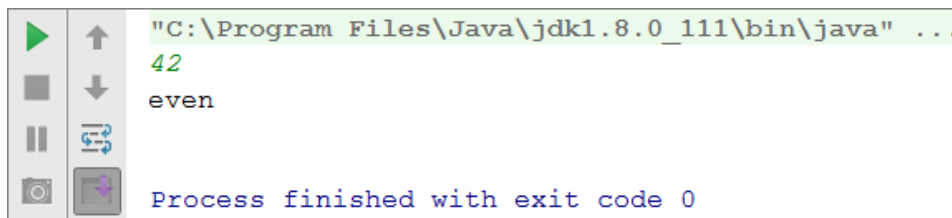
## Hints and Guidelines

We can solve the problem with one **if-else** statement and the operator **%**, which returns the remainder of dividing two numbers.

```
Scanner scanner = new Scanner(System.in);
int number = scanner.nextInt();

if(number % 2 == 0) {
    System.out.println("even");
} else {
    System.out.println("odd");
}
```

Executing the above code will output the following result:



## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#2>.

## Problem: Greater Number

Write a program that reads two integer numbers, from the console, and returns the greater one. Print the output in the following format: "Greater number: x", where the x is the returned number.

## Hints and Guidelines

Our first problem is to **read** both integer numbers from the console. Then we must perform the check using one **if-else** statement in combination with the **operator for greater than (>)**. Part of the code is consciously blurred to test what you have learned so far.

```
System.out.println("Enter two integers:");

Scanner scanner = new Scanner(System.in);

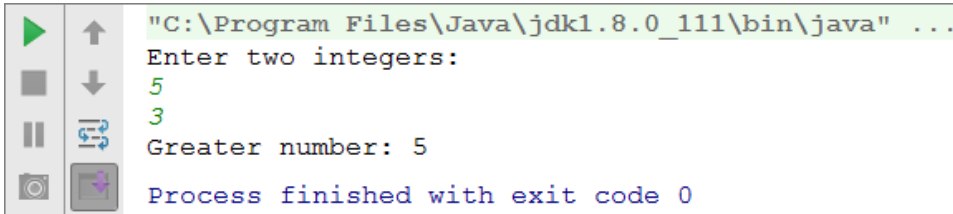
int num1 = scanner.nextInt();
int num2 = scanner.nextInt();
```

```

// Example 3: Greater
System.out.println("Greater number: " + greater);
}
// Example 4: Greater
System.out.println("Greater number: " + greater);
}

```

Executing the above code will output the following result for numbers 3 and 5:



```

"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
Enter two integers:
5
3
Greater number: 5
Process finished with exit code 0

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#3>.

## Variable Scope

Each variable has a scope in which it exists, called **variable scope**. This scope determines where the variable lives, in other words, the scope where you can use it. In Java, a variable scope begins from the line in which we **defined it** and ends with the first closing curly bracket **}** (of the method, of the **if statement**, etc.). Thus, it is important to know that **any variable defined inside the body of an if statement will not be available outside of it** unless we have defined it above in the code.

In the example below we will get an **error** because on the last line we are trying to print the variable **salary** that is defined inside the **if statement**, and we do not have access to it outside the block (in this case we will receive notification from the **IDE** about variable scope).

```

int myMoney = 500;
int payDate = 07;
int todayDate = 10;

if(todayDate >= payDate){
    int salary = 5000;
    myMoney = myMoney + salary;
}

System.out.println(myMoney);
System.out.println(salary);    //Error!

```



## Sequence of If-Else Conditions

Sometimes we need to do a sequence of conditions before we decide what actions our program will execute. In such cases, we can apply the construction **if-else if... -else**. For this purpose, we use the following format:

```
if (condition) {
    // body of if statement
} else if (condition2) {
    // body of if statement
} else if (condition3) {
    // body of if statement
} ... else {
    // body of else statement
}
```

### Problem: Number 0...9 to Text

Print a given digit, in the range from 1 to 9, in English (the digit is read from the console).

#### Hints and Guidelines

First, we read the digit from the console. Then using a **sequence of conditions**, we determine the corresponding English word and finally print it to the console:

```
Scanner scanner = new Scanner(System.in);
int num = Integer.parseInt(scanner.nextLine());

if (num == 1) {
    System.out.println("one");
} else if (num == 2) {
    System.out.println("two");
} else if (...) {
    ...
} else if (num == 9) {
    System.out.println("nine");
} else {
    System.out.println("number too big");
}
```

The program logic from the above example **sequentially compares** the input digit from the console with the numbers from 1 to 9. Each following comparison is being performed **only in case the preceding comparison is false**. If none of the **if** statements return true, then the last **else** clause is executed.

#### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#4>.

## Problems Sequence of If-Else Conditions

To consolidate our knowledge of the conditional statements **if** and **if-else**, let's solve several practical problems.

### Problem: Bonus Score

An **integer** is read from the console - the number of points. A **bonus score** adds to it according to the rules described below. Write a program that calculates the **bonus score** for this integer and **the total number of points** with the bonuses.

- If the integer is **up to 100** inclusive, the bonus score is 5.
- If the integer is **greater than 100**, the bonus score is **20%** from the integer.
- If the integer is **greater than 1000**, the bonus score is **10%** from the integer.
- Additional bonus score (added separately from previous)
  - If the integer is **even** -> +1 bonus score
  - If the integer is odd, with the **last digit equals 5** -> +2 bonus score

### Sample Input and Output

| Input | Output  | Input | Output    | Input | Output          | Input | Output            |
|-------|---------|-------|-----------|-------|-----------------|-------|-------------------|
| 20    | 6<br>26 | 175   | 37<br>212 | 2703  | 270.3<br>2973.3 | 15875 | 1589.5<br>17464.5 |

We can calculate the main and additional bonus score with a sequence of **if-else-if-else** statements. For **the main bonus score**, we have **3 cases** (when the input integer is up to 100, between 100 and 1000, and greater than 1000). For **the additional bonus score** – **2 more cases** (when the integer is even and odd, ended with 5).

```
Scanner scanner = new Scanner(System.in);
int number = scanner.nextInt();
double bonusPoints = 0;

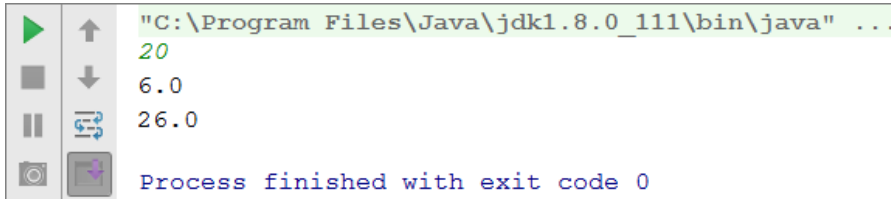
if (number % 2 == 0) {
    bonusPoints = 1;
} else if (number % 5 == 0) {
    bonusPoints = 2;
}

if (number <= 100) {
    bonusPoints += 5;
} else if (number > 100 && number < 1000) {
    bonusPoints += number * 0.2;
} else if (number > 1000) {
    bonusPoints += number * 0.1;
}
```

```
System.out.println(bonusPoints);
System.out.println(number + bonusPoints);

System.out.println(bonusPoints);
System.out.println(number + bonusPoints);
```

Executing the above code will output the following result:



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
20
6.0
26.0
Process finished with exit code 0
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#5>.

## Problem: Sum Seconds

Three athletes finish in a particular number of **seconds** (between **1** and **50**). Write a program that reads the time of each athlete from the console and calculates their **total time** in "minutes:seconds" format. Seconds need to be **formatted with a zero at the front** if they're less than 10 (2 -> "02", 7 -> "07", 35 -> "35").

### Sample Input and Output

| Input | Output | Input | Output | Input | Output | Input | Output |
|-------|--------|-------|--------|-------|--------|-------|--------|
| 35    | 2:04   | 22    | 1:03   | 50    | 2:29   | 14    | 0:36   |
| 45    |        | 7     |        | 50    |        | 12    |        |
| 44    |        | 34    |        | 49    |        | 10    |        |

## Hints and Guidelines

The problem has several solutions, but in the context of this chapter, we can do the following: First, sum up the three numbers to get the total result in seconds. Since **1 minute = 60 seconds**, we will have to calculate the number of minutes and seconds in the range from 0 to 59:

- If the result is between 0 and 59, print 0 minutes + calculated seconds.
- If the result is between 60 and 119, print 1 minute + calculate seconds minus 60.
- If the result is between 120 and 179, print 2 minutes + calculate seconds minus 120.
- If the seconds are less than 10, print the number with zero in front.

```

Scanner scanner = new Scanner(System.in);
int firstCompetition = scanner.nextInt();
int secondCompetition = scanner.nextInt();
int thirdCompetition = scanner.nextInt();
int allSeconds = firstCompetition +
    secondCompetition + thirdCompetition;

```

```

//if seconds < 60
if (allSeconds < 60) {
    seconds = allSeconds;
}
//if seconds < 60
if (allSeconds < 60) {
    seconds = allSeconds;
}
//if seconds < 60
if (allSeconds < 60) {
    System.out.println(seconds + "s" + allSeconds);
} else {
    System.out.println(seconds + "s" + allSeconds);
}

```

A different solution that does not use the **if-else** statement is more appropriate because you can use it for greater time values:

```

Scanner scanner = new Scanner(System.in);
int firstCompetition = scanner.nextInt();
int secondCompetition = scanner.nextInt();
int thirdCompetition = scanner.nextInt();

int allSeconds = firstCompetition + secondCompetition + thirdCompetition;

```

```

String minutes = String.valueOf(allSeconds / 60);
int sec = allSeconds % 60;
String seconds = String.valueOf(sec);
if (sec < 10) {
    seconds = "0" + String.valueOf(sec);
}
System.out.println(minutes + ":" + seconds);

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#6>.

## Problem: Metric Converter

Write a program that **converts the distance** between the following **units**: **m**, **mm**, **cm**, **mi**, **in**, **km**, **ft**, **yd**. Use the conversion table below to convert a value from one unit to another:

| Input Unit  | Output Unit               |
|-------------|---------------------------|
| 1 meter (m) | 1000 millimeters (mm)     |
| 1 meter (m) | 100 centimeters (cm)      |
| 1 meter (m) | 0.000621371192 miles (mi) |
| 1 meter (m) | 39.3700787 inches (in)    |
| 1 meter (m) | 0.001 kilometers (km)     |
| 1 meter (m) | 3.2808399 feet (ft)       |
| 1 meter (m) | 1.0936133 yards (yd)      |

The program will receive three input lines:

- First line: a number for converting.
- Second line: input unit.
- Third line: output unit (for the result).

### Sample Input and Output

| Input          | Output     | Input           | Output           | Input           | Output           |
|----------------|------------|-----------------|------------------|-----------------|------------------|
| 12<br>km<br>ft | 39370.0788 | 150<br>mi<br>in | 9503999.99393599 | 450<br>yd<br>km | 0.41147999937455 |

### Hints and Guidelines

Read the input data. We can use the **toLowerCase()** function, which will make all letters lowercase. As we can see from the table in the problem, we can easily convert values if we **use meters as our base unit**. Afterward calculate the result from the conversion of the input number in meters, performing a set of checks to define the input unit. Then calculate directly the output unit.



Keep in mind that in Java, you cannot use operator **==** for string comparison. For this purpose, you may use the built-in functions.

```
Scanner scanner = new Scanner(System.in);

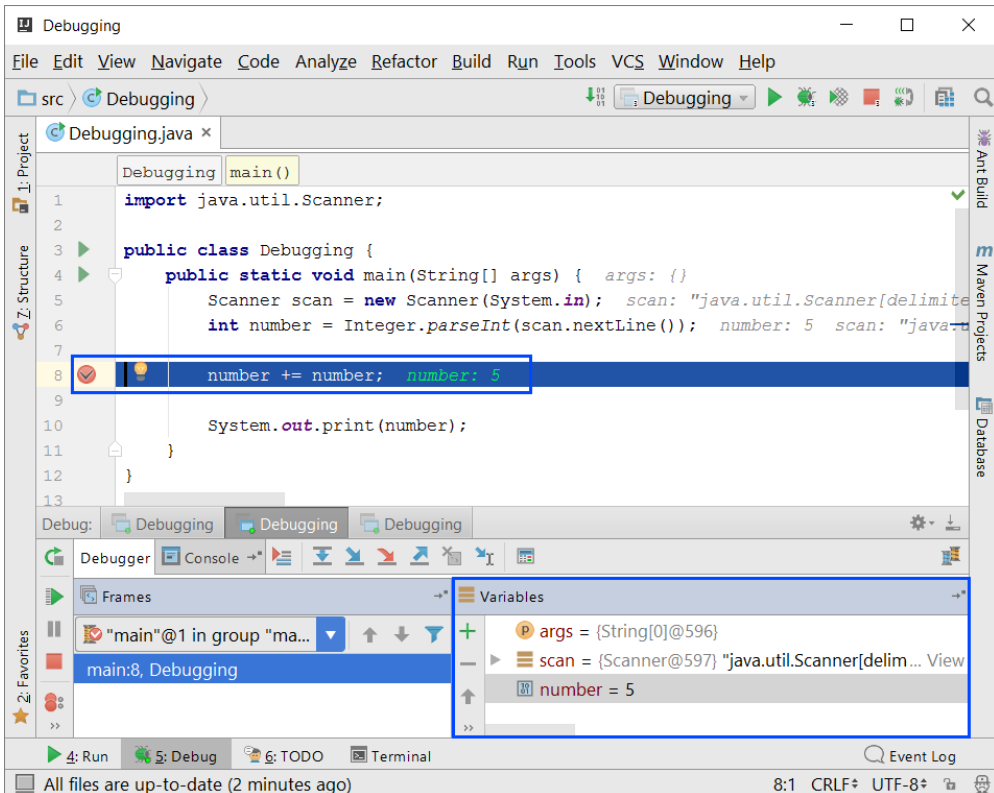
double size = scanner.nextDouble();
String sourceMetric = scanner.next().toLowerCase();
String destinationMetric = scanner.next().toLowerCase();
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#7>.

## Debugging

There were probably errors in the code you wrote so far. There is an easier way to find errors, use a tool. In the following section, we will look at one.

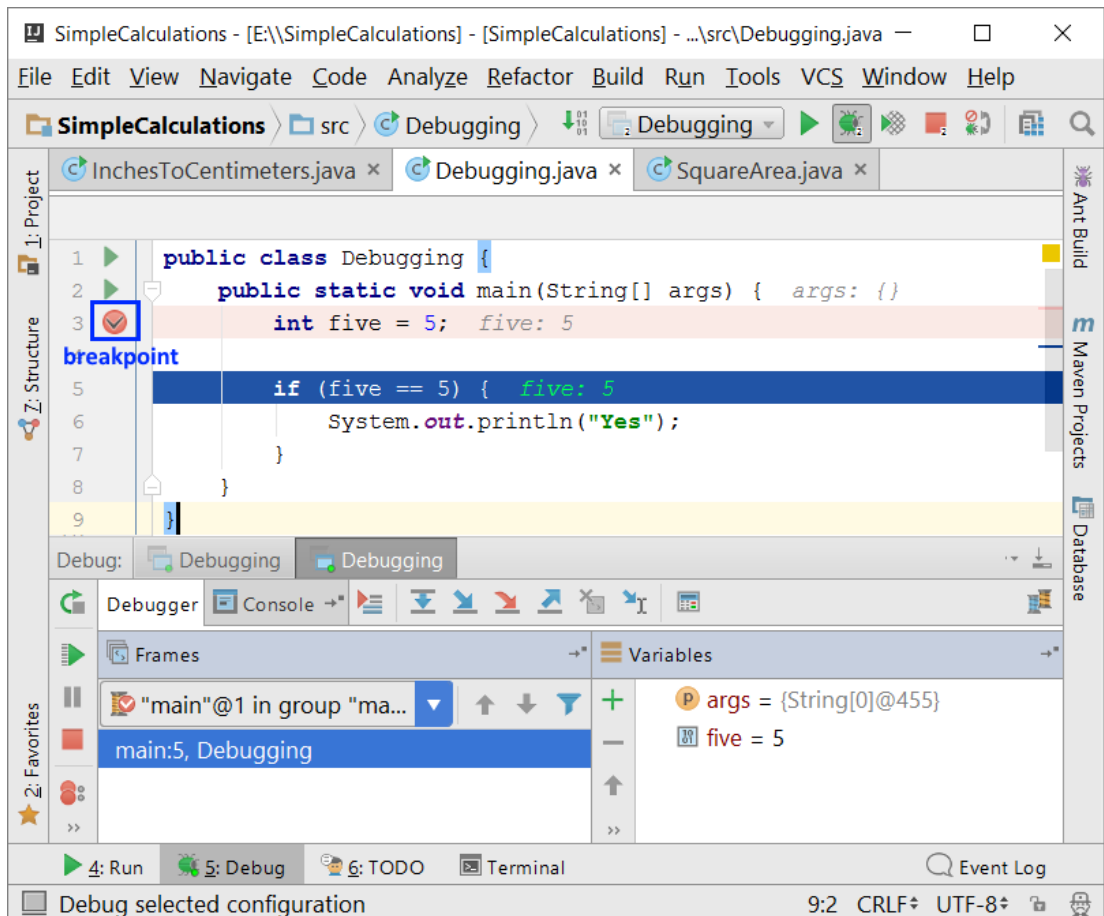


## What is "Debugging"?

**Debugging** lets you find and resolve errors, called **bugs**, a lot faster. Debugging is the process that lets you **track step by step the execution of a program**. This tracking is possible by pausing the execution of the program and **analyzing** its state by thorough examination, **line by line**. Of the logic that follows, the defined variables and how they are changed, and so on.

## Debugging with IntelliJ Idea

By pressing a combination of buttons [**Shift + F9**] you run the current program in **debug mode**. To move to **the next line** in the code, use the [**F7**] button.



By pressing a combination of buttons [**CTRL + F8**] you create special markers called **breakpoints** that suspend program execution at a specific point.

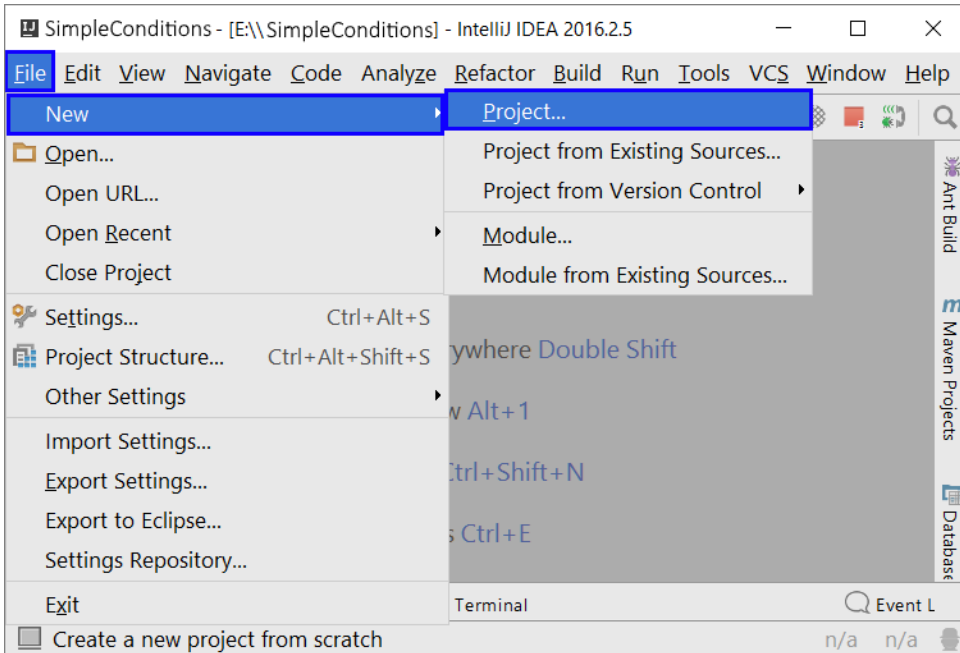
## Problems: Simple Conditions

To get a better understanding of what we have learned, let's solve a few practical exercises.

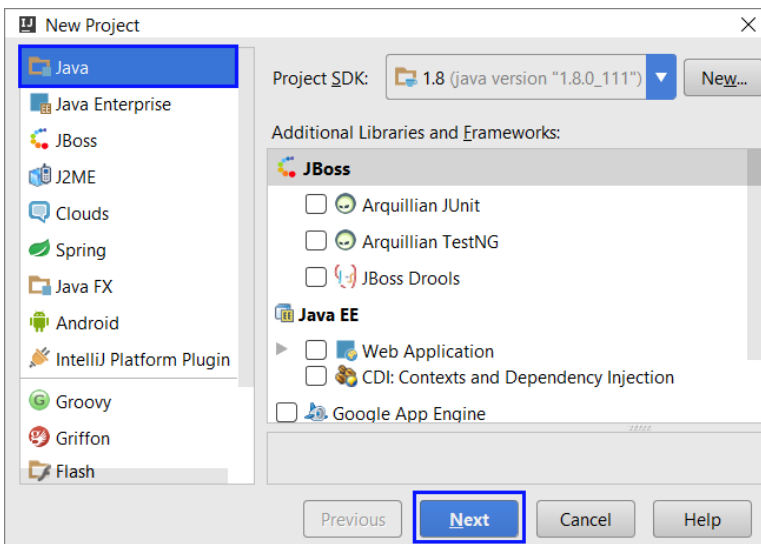
## Empty Project in IntelliJ Idea

Create a new project with the name **Java** in IntelliJ Idea and leave all other options at their default state. To better organize the solutions to the problems from the exercises - each solution will be in a separate class and all classes will be in the **src** directory of the project.

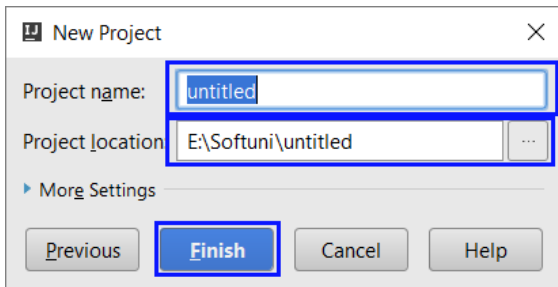
Run IntelliJ Idea. Create a new **Java project**: [File] → [New] → [Project].



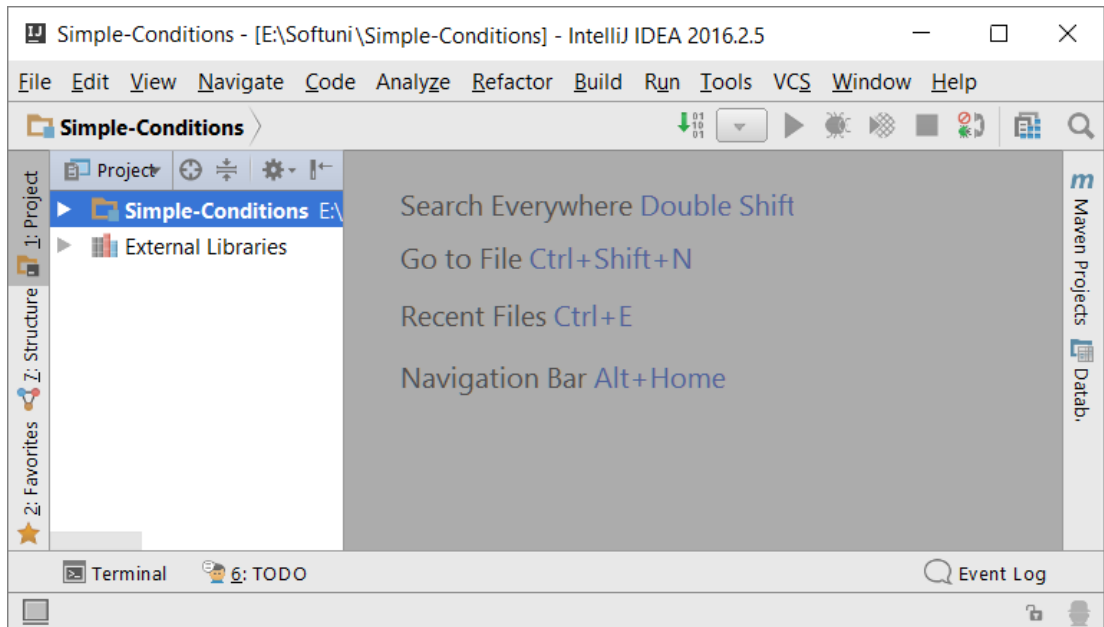
Choose **Java** from the left panel and leave anything else at its default state, press [Next]. In the next dialog box, we have an option to create a project from a template. Usually, we will do this, but now we can skip it and just press [Next]. In the last dialog box, enter the project's name and storage location, and then click [Finish].







We now have an empty Java project:



## Problem: Password Guess

The next problem is to write a **program that reads input data from the console**. As input data, enter a **password** (a single line with random text) and check if the input data is **the same** as the phrase "s3cr3t!P@ssw0rd". Output "Welcome" if the result is true and "Wrong password!" otherwise.

### Sample Input and Output

| Input  | Output          | Input           | Output  | Input       | Output          |
|--------|-----------------|-----------------|---------|-------------|-----------------|
| qwerty | Wrong password! | s3cr3t!P@ssw0rd | Welcome | s3cr3t!p@ss | Wrong password! |

### Hints and Guidelines

To solve the problem, it is necessary to use one **if-else** statement.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#8>.

### Problem: Number 100...200

The next problem is to write a **program that reads input data from the console**. As input data, enter an **integer** and check if the input data is **below 100**, **between 100 and 200**, or **above 200**. Output the corresponding message as shown in the table below.

#### Sample Input and Output

| Input | Output        | Input | Output              | Input | Output           |
|-------|---------------|-------|---------------------|-------|------------------|
| 95    | Less than 100 | 120   | Between 100 and 200 | 210   | Greater than 200 |

## Testing in The Judge System

Test your solution here:: <https://judge.softuni.org/Contests/Practice/Index/651#9>.

### Problem: Equal Words

The next problem is to write a **program that reads input data from the console**. As input data, enter **two words** and check if they are the same. Do not distinguish between uppercase and lowercase letters. Output "yes" or "no".

#### Sample Input and Output

| Input          | Output | Input              | Output | Input       | Output | Input         | Output |
|----------------|--------|--------------------|--------|-------------|--------|---------------|--------|
| Hello<br>Hello | yes    | SoftUni<br>softuni | yes    | Soft<br>Uni | no     | beer<br>vodka | no     |

## Hints and Guidelines

Before comparing words, transform them to lowercase so that the size of the letters (uppercase/lowercase) doesn't affect the comparison: **String wordFirst = scanner.next().toLowerCase();**

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#10>.

### Problem: Speed Info

The next problem is to write a **program that reads input data from the console**. As input data, enter **speed**(a decimal number). Output information about the speed.

- At speed **up to 10** (inclusive), print "slowly".

- At speed **above 10 and up to 50**, print **"average"**.
- At speed **above 50 and up to 150**, print **"fast"**.
- At speed **above 150 and up to 1000**, print **"ultra fast"**.
- At greater speed, print **"extremely fast"**.

### Sample Input and Output

| Input | Output  | Input | Output     | Input | Output         |
|-------|---------|-------|------------|-------|----------------|
| 8     | slow    | 126   | fast       | 3500  | extremely fast |
| 49.5  | average | 160   | ultra fast |       |                |

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#11>.

### Problem: Area of Figures

The next problem is to write a **program that reads input data from the console**. As input data, enter **the dimensions of a geometric figure** and **calculate its area**. The figures are of four types: **square**, **rectangle**, **circle**, and **triangle**.

On the first line of input data, read the type of figure (**square**, **rectangle**, **circle**, **triangle**).

- If the figure is a **square**, on the next line read one number - the length of its side.
- If the figure is a **rectangle**, on the next two lines read two numbers - the lengths of its sides.
- If the figure is a **circle**, on the next line read one number - the radius of the circle.
- If the figure is a **triangle**, on the next two lines read two numbers - the length of its side and the length of the adjacent height.

Format the output to **3 digits after the decimal point**.

### Sample Input and Output

| Input       | Output | Input                 | Output | Input       | Output  | Input                 | Output |
|-------------|--------|-----------------------|--------|-------------|---------|-----------------------|--------|
| square<br>5 | 25     | rectangle<br>7<br>2.5 | 17.5   | circle<br>6 | 113.097 | triangle<br>4.5<br>20 | 45     |

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#12>.

## Problem: Time + 15 Minutes

The next problem is to write a **program that reads input data from the console**. As input data, enter **the hour and minutes** (each on a separate line) of a 24-hour day and calculate **what is going to be the time in 15 minutes**. Print the output in **hh: mm** format. The hours are always between 0 and 23, and the minutes are always between 0 and 59. Output the hours either using one or two digits. Minutes are always displayed with two digits and a **leading zero** when necessary.

### Sample Input and Output

| Input   | Output | Input   | Output | Input    | Output | Input    | Output |
|---------|--------|---------|--------|----------|--------|----------|--------|
| 1<br>46 | 2:01   | 0<br>01 | 0:16   | 23<br>59 | 0:14   | 11<br>08 | 11:23  |

### Hints and Guidelines

To solve the problem, add 15 minutes and perform a few checks. If the minutes exceed 59, **increase the hours by 1** and **decrease the minutes by 60**. Similarly, consider the case when the hours exceed 23. When printing the minutes, check for **leading zero**.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#13>.

## Problem: 3 Equal Numbers

The next problem is to write a **program that reads input data from the console**. As input data, enter **3 integers**. Output if they are equal (**yes/no**).

### Sample Input and Output

| Input       | Output | Input       | Output | Input       | Output |
|-------------|--------|-------------|--------|-------------|--------|
| 5<br>5<br>5 | yes    | 5<br>4<br>5 | no     | 1<br>2<br>3 | no     |

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#14>.

## Problem: Number 0...100 to Text

The next problem is to write a **program that reads input data from the console**. As input data, enter a number in the range [0 ... 100] and convert the number into a text. Output the text in English.

### Sample Input and Output

| Input | Output      |
|-------|-------------|
| 25    | twenty five |

| Input | Output    |
|-------|-----------|
| 42    | forty two |

| Input | Output |
|-------|--------|
| 6     | six    |

### Hints and Guidelines

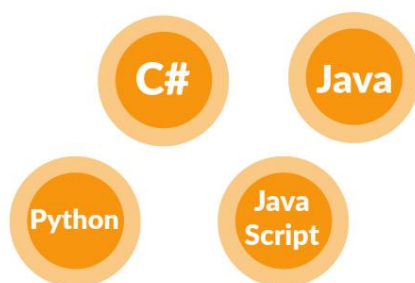
To solve the problem, first, check for **one-digit numbers**, and if the number has only one digit, print the appropriate text for it. Then check for **two-digit numbers**. Print them in two parts: left part (**tens = number / 10**) and right part (**units = number % 10**). If the number has three digits, it must be 100 and considered a special case.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/651#15>.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)

## Chapter 3.2. Simple Conditions – Exam Problems

In the previous chapter, we looked at conditional constructions in the Java language, used to perform various actions depending on a condition. We also mentioned the scope of a variable, as well as how to follow line by line the execution of our program (the so-called **debugging**). In this chapter, we will practice working with conditional statements, and we'll be looking at some problems given in actual exams. Let's first retrieve some of the newly acquired knowledge on how the conditional statement is constructed:

```
if (boolean expression) {
    // body of the conditional block;
} else {
    // body of else-block;
}
```

**if** statements consist of:

- **if** clause
- boolean expression - a variable of boolean type (**boolean**) or boolean logical expression (an expression that returns a result **true** / **false**)
- body - contains any block with source code
- **else** clause and its source code block (optional)

### Exam Problems

After recalling how to write conditional statements, let's solve a few problems to get practical experience with the **if-else** statement.

#### Problem: Transport Price

A student must travel **n** kilometers. He has a choice between **three modes of transport**:

- **Taxi**. Starting fee: **0.70** BGN. Daily rate: **0.79** BGN/km. Nightly rate: **0.90** BGN/km.
- **Bus**. Daily/Nightly rate: **0.09** BGN/km. Can be used for distances minimum of **20** km.
- **Train**. Daily/Nightly rate: **0.06** BGN/km. Can be used for distances at least **100** km

Write a program that enters the number of **kilometers n** and the **period of the day** (day or night) and computes **the price of the cheapest transport** .

#### Input Data

Two lines are read from the console:

- The first line contains the number **n** - number of kilometers - integer in the interval [1... 5000].

- The second line contains the word "day" or "night" - if the trip is during the day or at night.

## Output Data

Print on the console the lowest price for the specified number of kilometers.

## Sample Input and Output

| Input    | Output |
|----------|--------|
| 5<br>day | 4.65   |

| Input      | Output |
|------------|--------|
| 7<br>night | 7      |

| Input     | Output |
|-----------|--------|
| 25<br>day | 2.25   |

| Input        | Output |
|--------------|--------|
| 180<br>night | 10.8   |

## Hints and Guidelines

We will read the input data and depending on the distance we will choose the cheapest transport. For this purpose, we will use several conditional statements.

### Input Data

Our first step is to create an object of type `java.util.Scanner`. We pass in its constructor the "input stream" (`System.in`) that we are going to read from. In the definition of the problem, we have information about the input and output. The first two lines of the solution will contain the declaration and initialization of the two variables that will store the values of the input data.

The first line is mentioned to contain an integer, so the variable that will be declared is of type `int`. For the second line, the instruction is that it contains a word, respectively the variable is of type `String`.

```
Scanner scanner = new Scanner(System.in);  
int distance = Integer.parseInt(scanner.nextLine());  
String dayOrNight = scanner.nextLine();
```

Before we begin with the conditional statements it's necessary to declare one variable that will store the value of the transport price.

```
double price = 0.0d;
```

## Expression Evaluation and Computing The Relevant Calculations

Once we have declared and initialized the input data and the variable that will store the value of the price, we should assess which conditions of the problem will be the first to be checked.

It is evident from the problem that the rates of two of the vehicles do not depend on whether it is day or night, but the rate of the third transport (taxi) does. This is a reason to pick our first condition to verify whether it's day or night and determine which taxi rate will be in effect. To achieve that we declare another variable that will store the value of the taxi fare.



```
double taxiRate = 0.0d;
```

Computing the **taxi rate** is performed with an **if-else** statement. That's how we taxi price variable is assigned a **value**.

```
if (dayOrNight.equals("day")) {
    taxiRate = 0.79;
} else if (dayOrNight.equals("night")) {
    taxiRate = 0.90;
}
```

Once that's done, we can now proceed to the computation of the **full transportation price**. Another condition for the price defined in the problem is the **traveling distance** with a chosen vehicle. Naturally to do that we pick an **if-else** condition that we used to determine the **price** for the transport depending on the entered kilometers.

```
if (distance < 20) {
    price = 0.70 + (distance * taxiRate);
} else if (distance < 100) {
    price = distance * 0.09;
} else {
    price = distance * 0.06;
}
```

First, we check if the kilometers are **below 20** since it's specified in the problem that the student could only use a **taxi** for destinations that are **below 20** kilometers away. If the expression in the first conditional statement is evaluated to be **true** then the variable that stores the transportation price (**price**) will be **assigned** the corresponding value. This value is equal to the **initial rate**, which we **sum up** with its **rate multiplied by the distance** that the student has to travel.

If the expression in the first conditional statement is evaluated to **false** then the next step of the program is to check if the distance is **below 100**. The reason we're doing that is to be possible to use a **bus** as a vehicle in this distance range. **The price per kilometer for the bus is lower** than the taxi price. Therefore, the result of the expression evaluation is **true**, then here's the place to assign the **value** of the transportation price variable (**price**) which equals the result of the **multiplication** of the **distance** and the **bus rate**.

In the case of the second check **not returning true** as a result, the program ends up in the **else** block and assigns a **value** equal to the **result** of the **multiplication** of the **distance** and the **rate of the train**. We do that because the train is the **cheapest** vehicle for the given distance.

### Printing The Result

We **have successfully computed the price of the cheapest transportation** so we're almost at the end of the problem. The only thing left is to print out the result that we have on the console. In our problem, there's no requirement for the resulting output and that's why we choose to keep it simple and just print the **variable (price)**.

```
System.out.println(price);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/652#0>.

## Problem: Pipes in Pool

A Pool with volume  $V$  has two filling pipes. Each pipe has a specified hourly flow rate (liters of water that pass through one pipe per hour). The worker opens the pipes at the same time and leaves the sight for  $N$  hours. Your problem is to write a program that prints out the state of the pool, at the moment the worker comes back.

### Input Data

Four lines are read from the console:

- The first line contains the number  $V$  - pool volume in liters - integer in the range [1... 10000].
- The second line contains the number  $P1$  - hourly flow rate of the first tube - integer in the interval [1 ... 5000].
- The third line contains the number  $P2$  - hourly flow rate of the second tube - integer in the range [1 ... 5000].
- The fourth line contains the number  $H$  - number of hours that the employee is absent - a floating-point number in the interval [1.0 ... 24.00].

### Output Data

Print on the console one of two possible states:

- How full are the pool and each pipe's percentage contribution? Use the whole number before the decimal point of a percentage (i.e. round down).
  - "The pool is [x]% full. Pipe 1: [y]%. Pipe 2: [z]%. "
- If the pool has overflowed - how many liters over the pool's volume has the pool overflowed for the given time, floating-point number.
  - "For [x] hours the pool overflows with [y] liters." \ *Have in mind that due to the percentage rounding some data may be lost and it's okay for \*the sum of the percentages to be 99% and not 100%.*

### Sample Input and Output

| Input                   | Output                                            |
|-------------------------|---------------------------------------------------|
| 1000<br>100<br>120<br>3 | The pool is 66% full. Pipe 1:<br>45%. Pipe2: 54%. |

| Input                    | Output                                               |
|--------------------------|------------------------------------------------------|
| 100<br>100<br>100<br>2.5 | For 2.5 hours the pool<br>overflows with 400 liters. |

## Hints and Guidelines

To solve the problem, we'll read from the standard input stream, utilize several conditional statements and computations, and print our result to the console.

### Input Data

By definition, our program has to have **four lines** that are going to read and store **the input data**. The first **three** are going to store **whole numbers** and this is a reason we pick the **int** type for those **variables**. On the **fourth line**, we are supposed to get a floating-point **number** and that's why the **variable** that we're going to use is of type **double**.

```
Scanner scanner = new Scanner(System.in);
int volume = scanner.nextInt();
int pipe1 = scanner.nextInt();
int pipe2 = scanner.nextInt();
double hours = scanner.nextDouble();
```

Our next step is to **declare and initialize** a variable that will store the computed value for the amount of water that the pool has filled up with while the worker has been away. The calculation is made by **summing up the flow rate of the two pipes and multiplying them by the input hours H**.

```
double water = (pipe1 + pipe2) * hours;
```

### Expression Evaluation and Computing the Relevant Calculations

Once we have **the amount** of water that has passed through the **pipes**, the next step is to **compare** it with the volume of the pool.

We will do this with a simple **if-else** conditional statement, in which the condition will be whether **the amount of water is less than the volume of the pool**. If the check returns **true**, then we need to print a line that contains **the ratio** between **the amount of water that has passed through the pipes** and the volume of pool, as well as the ratio of the amount of water **from each pipe to the pool volume\*\***.

The ratio needs to be expressed in **percentage** that's why we'll **multiply by 100** the result of all the prior calculations on this line. The values will be interpolated using **formatting strings** and rounded down to **integer** with **Math.floor (...)** (it's a requirement of the problem).

```
if (water <= volume) {
    System.out.printf(
        "The pool is %.0f%% full. Pipe 1: %.0f%%. Pipe 2: %.0f%%.",
        Math.floor(water / volume * 100),
        Math.floor(pipe1 * hours / volume * 100),
        Math.floor(pipe2 * hours / volume * 100);
} else {
    System.out.printf(
        "For %f hours the pool overflows with %f liters.",
        hours, water - volume);
}
```

However, if the check returns **false** it means that the amount of water is **greater** than the **volume** of the pool, respectively it has **overflowed**. Again, the output must be **one line** but this time it should contain only **two values** - **away time in hours** of the worker and the **amount of overflowed water**.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/652#1>.

## Problem: Sleepy Cat Tom

The cat Tom enjoys sleeping all day, unfortunately whenever his owner has some free time he comes to play with Tom. To have a good sleeping routine Tom's **norm for playtime** is 30 000 minutes per year\*. Tom's playtime is dependent on his owner's non-working days:

- When Tom's owner is at **work**, Tom's playtime is **63 minutes a day**.
- When Tom's owner has a **day off**, Tom's playtime is **127 minutes a day**.

Your problem is to write a program that takes as input Tom's owner's **number of days off** and prints out whether Tom has a **satisfactory sleeping routine** and how much is the **difference from Tom's norm for playtime** for the current year, assuming that **the year has 365 days**.

**Example:** 20 days off → 345 working days ( $365 - 20 = 345$ ). The actual playing time is 24 275 minutes ( $345 \setminus 63 + 20 \setminus 127$ ). The difference from the norm is 5 725 minutes ( $30\,000 - 24\,275 = 5\,725$ ) or 95 hours and 25 minutes.

## Input Data

The input data is read from the console and consists of an integer - **Tom's owner number of days off**, number in the range [0 ... 365].

## Output Data

**Two lines** are to be printed on the console.

- If Tom's **playing time** is above the norm for the current year:
  - On the first line print: "Tom will run away".
  - On the second line print the difference from the norm in the format: "{H} hours and {M} minutes more for play".
- If Tom's **playing time** is below the norm for the current year:
  - On the first line print: "Tom sleeps well".
  - On the second line print the difference from the norm in the format: "{H} hours and {M} minutes less for play".

## Sample Input and Output

| Input | Output                                                      |
|-------|-------------------------------------------------------------|
| 20    | Tom sleeps well<br>95 hours and 25 minutes<br>less for play |

| Input | Output                                                       |
|-------|--------------------------------------------------------------|
| 113   | Tom will run away<br>3 hours and 47 minutes more<br>for play |

## Hints and Guidelines

To solve the problem, we'll read from the standard input stream, utilize several conditional statements and computations, and print our result to the console.

### Input Data

The **input data** will be read in a **single line** that will contain a **whole number** in the range [0 ... 365]. That's why the variable that'll store the value is going to be of type **int**.

```
Scanner scanner = new Scanner(System.in);
int holidays = scanner.nextInt();
```

Our **first** step is to calculate how many **minutes in total** has Tom and his master played together. The problem states that not only has the sleepy cat played on **days off** but also on **working days**. The number that we read from the console is the number of **non-working days**.

Our next step is to use that number to **calculate** how many are the **working days** of Tom's owner. We need this number because without it we can not get the **total number of playtime minutes**. The total number of days in a year is **365**, the number of days off is **X**, which means that the number of working days is **365 - X**. We will store the difference in a new variable (**workingDays**), which we will use **only** for this value.

```
int workingDays = 365 - holidays;
```

Now that we have the number of days to play, we can now calculate Tom's playing time in minutes. Its value is equal to the result of the multiplication of the working days by 63 minutes (in the problem it's specified that on working days the playing time is 63 minutes per day) added to the result of the multiplication of days off and 127 minutes (in the problem it's specified that on days off the playing time is 127 minutes per day).

```
int totalPlayMinutes = holidays * 127 + workingDays * 63;
```

We need to **print the difference** between hours and minutes. That's why we'll subtract from the **total** playing time Tom's playtime norm of **30 000** minutes and we'll store the result in a **new** variable. After that, we'll use that new variable to **divide without a remainder** to 60 and that's how we'll get the **hours**. To get the number of **minutes** we'll utilize the **modulo operator %** and get the remainder of the division by 60.

Note that if Tom's resulting **playtime** is **less** than **30 000** when we **subtract** the norm we'll get a **negative number**. We can fix that with the method **Math.abs(...)** when subtracting.

```
double difference = Math.abs(totalPlayMinutes - 30000);
double hours = difference / 60;
double minutes = difference % 60;
```

### Conditional Expressions Verification

The playing time is already calculated which leads us to the next step - Tom's playtime and Tom's norm comparison. We'll use an **if-else** conditional statement. It'll verify in its **if** clause whether the playtime is larger than 30 000 (norm).

### Output Data Processing

Depending on the result of the evaluation of the boolean expression our objective is to print the difference in hours and minutes. We'll do that using the formatting specifier and the previously computed hour and minutes values. The given output examples show that the printed values are formatted as a rounded down whole number. This is a cue that we should be using **Math.floor(...)** when we're printing the result.

```
if (totalPlayMinutes > 30000) {
    System.out.println("Tom will run away");
    System.out.printf("%.0f hours and %.0f minutes more for play",
        Math.floor(hours), Math.floor(minutes));
} else {
    System.out.println("Tom sleeps well");
    System.out.printf("%.0f hours and %.0f minutes less for play",
        Math.floor(hours), Math.floor(minutes));
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/652#2>.

## Problem: Harvest

There's a vineyard that has an area of  $X$  sq. M.. 40% of the harvest is allocated for wine production. The yield from 1sq.m. vineyard is  $Y$  kilograms of grapes. To produce 1 liter of wine is used 2.5kg grapes. The desired amount of wine for sale is  $Z$  liters.

Your problem is to write a program that calculates the possible amount of produced wine and validates if it's enough. If the production is satisfying the expectations, then the remainder shall be distributed equally among the vineyard workers.

### Input Data

Four lines are read from the console:

- The first line contains the area of the vineyard  $X$  sq.m - integer in the interval [10 ... 5000].

- The second line contains the grapes yield per square meter **Y** - a real number in the interval [0.00 ... 10.00].
- The third line contains the desired amount of wine **Z liters** - an integer in the range [1 ... 600].
- The fourth line contains the **number of workers** - integer in the interval [1 ... 20].

## Output Data

The following is expected to be printed on the console:

- If the **produced** wine is **less than the required**:
  - "It will be a tough winter! More {insufficient wine} liters wine needed."
  - The result should be **rounded down**.
- If the **produced** wine is **sufficient**:
  - "Good harvest this year! Total wine: {total wine} liters."
  - The result must be **rounded down**.
  - "{Wine left} liters left -> {wine for 1 worker} liters per person."
  - Both results must be **rounded up**.

## Sample Input and Output

| Input                | Output                                                                                     | Input                   | Output                                                  |
|----------------------|--------------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------|
| 650<br>2<br>175<br>3 | Good harvest this year! Total wine: 208 liters.<br>33 liters left -> 11 liters per person. | 1020<br>1.5<br>425<br>4 | It will be a tough winter! More 180 liters wine needed. |

## Hints and Guidelines

To solve the problem, we'll read from the standard input stream, utilize several conditional statements and computations, and print our result to the console.

## Input Data

First, we need to **check** what **input data** is expected and choose an adequate type for each of the **variables** that will store this data.

```
Scanner scanner = new Scanner(System.in);
int vineyardArea = scanner.nextInt();
double grapePerSquare = scanner.nextDouble();
int neededLiters = scanner.nextInt();
int workers = scanner.nextInt();
```

To solve the problem, we need to **compute** the **amount of wine in liters** that the harvest will produce using the **input data**. The problem defines a formula that requires the **weight of the yielded grapes in kg**. We can easily calculate this by **declaring** a **variable** that will store a **value** equal to **40%** of the result of the **multiplication** of the area of the vineyard and the amount of yielded grapes per square meter.

Once we have made these calculations, we are ready to **insert** them into the problem formulation. We **declare** another **variable** that will store the **amount of wine in liters** and will be equal to the **weight of the yielded grapes in kg**, divided by 2.5.

```
double harvestPerVine = (vineyardArea * grapesPerSquare) * 0.4;
double vine = harvestPerVine / 2.5;
```

### Expression Evaluation and Processing the Output Data

Our next step is to check if the produced wine is enough. We'll use a simple **if-else** conditional statement with a clause that **verifies** if the **amount of wine in liters** produced from the harvest is **more than or equal** to the **desired liters**.

If the expression evaluation returns **true**, then on the **first line** we should print the **amount of wine produced by these harvesters**. It's required that **this value** should be **rounded down**. To comply with the requirement, we'll use the method **Math.floor(...)** when printing it using the **formatting specifier**.

The requirement for the second line is to print the interpolated values **rounded up**. We'll use the method **Math.ceil(...)**. The values that we have to print are **the wine that's left** and **the amount of wine that is to be given to each worker**. The wine that's left is equal to the **difference** between **the produced liters of wine** and **the desired liters of wine**. As the result of this computation, we'll store in a new variable that we'll declare and initialize in the **body block** of the **if** statement before printing the first line. The **amount of wine that is to be given to each worker** we'll get by **dividing** the **difference** by the **number of workers**.

```
if ( (producedLitersOfWine >= desiredLitersOfWine) ) {
    double vineLeft = producedLitersOfWine - desiredLitersOfWine;
    System.out.printf(
        "Good harvest this year! Total wine: %.0f liters.\n",
        Math.floor(producedLitersOfWine));
    System.out.printf(
        "%.0f liters left -> %.0f liters per person.",
        Math.ceil(vineLeft), Math.ceil(vineLeft / workers));
}
```

If the expression evaluation returns **false**, then we need to **print the difference** between **desired liters of wine** and **the produced liters of wine from this harvest**. It's required that **this value** should be **rounded down**. To comply with the requirement we'll use the method **Math.floor(...)** when printing it using the **formatting specifier**.



```

} else {
    System.out.printf(
        "It will be a tough winter! More %d liters wine needed.",
        Math.floor(neededHours / hours));
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/652#3>.

## Problem: Firm

A company is requested to develop a project that is estimated for a given number of hours. The company has **a certain number of days** to do that. **In 10% of those days**, its employees are being onboarded and **are not ready to work** on the project. A normal **working day at this company lasts for 8 hours**. The project is important for the company and every **employee is obliged to work on this project overtime for 2 hours per day**.

The hours should be **rounded down** (e.g. 6.98 hours are rounded to 6 hours).

Your problem is to write a program that **calculates if the company is going to finish the project on time and how many hours more are needed for the completion of the project** or if the execution was successful **how many dedicated working hours are left at hand**.

## Input Data

Three lines are read from the console:

- The first line contains the necessary **number of working hours** - an integer in the interval [10 ... 200 000].
- The second line contains the **days that the company has to finish the project** - an integer in the interval [0 ... 20 000].
- The third line contains **the number of all employees** - an integer in the range [1 ... 600].

## Output Data

One line is expected to be **printed** on the console:

- If the time is enough:
  - "Yes! {hours left} hours left."
- If the time is NOT enough:
  - "Not enough time! {hours needed} hours needed."

## Sample Input and Output

| Input        | Output             |
|--------------|--------------------|
| 90<br>7<br>3 | Yes!99 hours left. |

| Input        | Output                           |
|--------------|----------------------------------|
| 99<br>3<br>1 | Not enough time!72 hours needed. |

## Hints and Guidelines

To solve the problem, we'll read from the standard input stream, utilize several conditional statements and computations, and print our result to the console.

### Input Data

First, we need to **check** what **input data** is expected and choose an adequate type for each of the **variables** that will store this data.

```
Scanner scanner = new Scanner(System.in);
int projectHours = scanner.nextInt();
int availableDays = scanner.nextInt();
int workers = scanner.nextInt();
```

### Auxiliary Calculations

The next step is to calculate **the number of working hours**. This we can do by multiplying the working days by 8 (every working day is 8 hours) and the number of the company's employees. On top of that, we should add the overtime too. **The working days** are equal to **90% of the days** that the company has. **The overtime** is equal to the result of the multiplication of the employees and 2 (decided overtime length) and the number of days that the company has. We can see that the problem requires **the working hours** to be **rounded down**. That's something we can easily do by using the method **Math.floor(...)**.

```
double workDays = availableDays * 0.9;
double overtimeHours = workDays * workers * 2;
double workHours = workDays * workers * 8;
double totalHours = Math.floor(workHours + overtimeHours);
```

### Performing Verifications

Once we've made the calculations, we need to compute the value of **the working hours** we should check whether these hours **are enough**.

If **the time is enough**, we print the difference between **the working hours** and the **estimated hours** for the project.

If **the time is not enough**, we print the additional hours that are needed to finish the project. And that is the difference between **the estimated hours** and **the working hours**.

```
if (totalHours < projectHours) {
    System.out.printf("Yes!%.0f hours left.",
        projectHours - totalHours);
} else {
    System.out.printf("Not enough time!%.0f hours needed.",
        projectHours - totalHours);
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/652#4>.

# Chapter 4.1. More Complex Conditions

In the **current** chapter, we are going to examine the **nested conditional statements** in the **Java** language, by which our program can contain **conditions** that contain other **nested conditional statements**. We call them "**nested**" because we put an **if** condition into another **if** condition. We are going to examine the **more complex logical conditions** through proper examples.

## Video

Watch the video lesson about what we will learn in this chapter:

<https://www.youtube.com/watch?v=L4KtlGD5cbk>.

## Nested Conditions

Pretty often the program logic requires the use of **if** or **if-else** statements, which are contained one inside another. They are called **nested if** or **if-else** statements. As implied by the title "**nested**", these are **if** or **if-else** statements that are placed inside other **if** or **else** statements.

```
if (condition1) {
    if (condition2) {
        // body;
    } else {
        // body;
    }
}
```

Nesting of\*more than three conditional statements inside each other is not considered a good practice and has to be avoided, mostly through optimization of the structure/the algorithm of the code and/or by using another type of conditional statement, which we are going to examine below in this chapter.

## Problem: Personal Titles

Depending on **age** (decimal number) and **gender** (m / f), print a personal title:

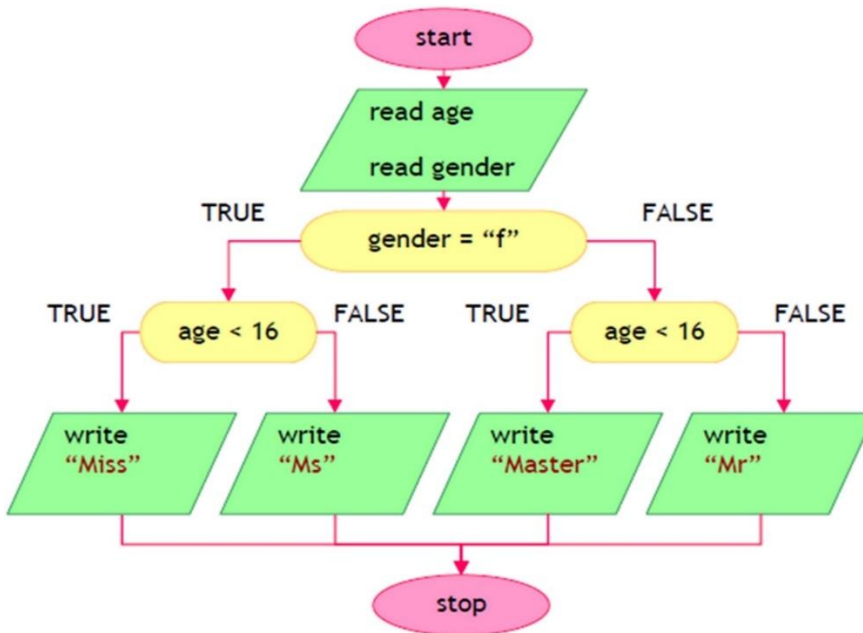
- "Mr." – a man (gender "m") – 16 or more years old.
- "Master" – a boy (gender "m") under 16 years.
- "Ms." – a woman (gender "f") – 16 or more years old.
- "Miss" – a girl (gender "f") under 16 years.

## Sample Input and Output

| Input   | Output | Input   | Output | Input   | Output | Input     | Output |
|---------|--------|---------|--------|---------|--------|-----------|--------|
| 12<br>f | Miss   | 17<br>m | Mr.    | 25<br>f | Ms.    | 13.5<br>m | Master |

## Solution

We should notice that the **output** of the program **depends on a few things**. First, we have to check what is the entered **gender** and **then** check the **age**. Respectively, we are going to use **a few if-else** blocks. These blocks will be **nested**, meaning from **the result** of the first, we are going to **define** which one of the **others** to execute.



After reading the input data from the console, the following program logic should be executed:

```

if (age < 16) {
    if (gender.equals("m")) {
        System.out.println("Master");
    } else if (gender.equals("f")) {
        System.out.println("Miss");
    }
} else if (age >= 16) {
    if (gender.equals("f")) {
        System.out.println("Ms.");
    } else if (gender.equals("m")) {
        System.out.println("Mr.");
    }
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#0>.

## Problem: Small Shop

A Bulgarian entrepreneur opens **small shops** in a **few cities** with different **prices** for the following **products**:

| product / city | Sofia | Plovdiv | Varna |
|----------------|-------|---------|-------|
| coffee         | 0.50  | 0.40    | 0.45  |
| water          | 0.80  | 0.70    | 0.70  |
| beer           | 1.20  | 1.15    | 1.10  |
| sweets         | 1.45  | 1.30    | 1.35  |
| peanuts        | 1.60  | 1.50    | 1.55  |

Calculate the price by the given **city** (string), **product** (string), and **quantity** (double).

### Sample Input and Output

| Input                | Output | Input                   | Output | Input              | Output | Input                 | Output |
|----------------------|--------|-------------------------|--------|--------------------|--------|-----------------------|--------|
| coffee<br>Varna<br>2 | 0.9    | peanuts<br>Plovdiv<br>1 | 1.5    | beer<br>Sofia<br>6 | 7.2    | water<br>Plovdiv<br>3 | 2.1    |

### Solution

```
Scanner console = new Scanner(System.in);
String product = console.nextLine().toLowerCase();
String town = console.nextLine().toLowerCase();
double quantity = Double.parseDouble(console.nextLine());
double price = 0;

if (town.equals("sofia")) {
    if (product.equals("coffee")) {
        price = quantity * 0.5;
    } else if... // TODO - complete the code
} else if (town.equals("plovdiv")) {
    if (product.equals("coffee")) {
        price = quantity * 0.4;
    } else if... // TODO - complete the code
} else if (town.equals("varna")) {
    // TODO - complete the code
}
System.out.println(price);
```

We **convert** all of the letters into **lower register** using the function `.toLowerCase()`, to compare products and cities **no matter** what the letters are – small or capital ones.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#1>.

## More Complex Conditions

Let's take a look at how we can create more complex logical conditions. We can use the logical "AND" (`&&`), logical "OR" (`||`), logical **negation** (`!`) and **brackets** (`()`).

### Logical AND

As we saw, in some problems we have to make **many checks at once**. But what happens when to execute some code **more** conditions have to be executed and we **don't want to** make a **negation** (**else**) for each one of them? The option with nested **if blocks** are valid, but the code would look very **unordered** and for sure – **hard** to read and maintain.

The logical "AND" (operator `&&`) means a few conditions have to be **fulfilled simultaneously**. The following table of truthfulness is applicable:

| a     | b     | a && b |
|-------|-------|--------|
| true  | true  | true   |
| true  | false | false  |
| false | true  | false  |
| false | false | false  |

## How Does The && Operator Work?

The `&&` operator accepts a couple of Boolean (conditional) statements, which have a **true** or **false** value, and returns **one** bool statement as a **result**. Using it **instead** of a couple of nested **if** blocks, makes the code **more readable**, **ordered**, and **easy** to maintain. But how does it **work**, when we put a **few** conditions one after another? As we saw above, the logical "AND" returns **true**, **only** when it accepts as **arguments statements** with the value **true**. Respectively, when we have a **sequence** of arguments, the logical "AND" **checks** either until one of the arguments is **over**, or until it **meets** an argument with value **false**.

Example:

```
boolean a = true;
boolean b = true;
boolean c = false;
boolean d = true;
boolean result = a && b && c && d;
```

```
// false (as d is not being checked)
```

The program will run in the **following** way: It **starts** the check from **a**, **reads** it, and accepts that it has a **true** value, after which it **checks b**. After it has **accepted** that **a** and **b** return **true**, it **checks the next** argument. It gets to **c** and sees that the variable has a **false** value. After the program accepts that argument **c** has a **false** value, it calculates the expression **before c**, **independent** of what the value of **d** is. That is why the evaluation of **d** is being **skipped** and the whole expression is calculated as **false**.

```
if ((x >= x1 && x <= x2) && (y >= y1 && y <= y2))
```

## Problem: Point in Rectangle

Checks whether point  $\{x, y\}$  is placed inside the rectangle  $\{x1, y1\} - \{x2, y2\}$ . The input data is read from the console and consists of 6 lines: the decimal numbers  $x1, y1, x2, y2, x$  and  $y$  (as it is guaranteed that  $x1 < x2$  and  $y1 < y2$ ).

### Sample Input and Output

| Input                         | Output | Visualization |
|-------------------------------|--------|---------------|
| 2<br>-3<br>12<br>3<br>8<br>-1 | Inside |               |

### Solution

A point is internal for a given polygon if the following four conditions are applied **at the same time**:

- The point is placed to the right from the left side of the rectangle.
- The point is placed to the left from the right side of the rectangle.
- The point is placed downwards from the upper side of the rectangle.
- The point is placed upwards from the down side of the rectangle.

```
Scanner console = new Scanner(System.in);
double x1 = Double.parseDouble(console.nextLine());
double y1 = Double.parseDouble(console.nextLine());
double x2 = Double.parseDouble(console.nextLine());
double y2 = Double.parseDouble(console.nextLine());
double x = Double.parseDouble(console.nextLine());
double y = Double.parseDouble(console.nextLine());
```

```
if ((x >= x1 && x <= x2) && (y >= y1 && y <= y2)) {  
    System.out.println("Inside");  
} else {  
    System.out.println("Outside");  
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#2>.

## Logical "OR"

The logical "OR" (operator `||`) means that **at least one** among a few conditions is fulfilled. Similar to the operator `&&`, the logical "OR" accepts a few arguments of **bool** (conditional) type and returns **true** or **false**. We can easily guess that we **obtain** a value **true** whenever at least **one** of the arguments has a **true** value. This is shown at the truth table below:

In school, the teacher said: "Ivan or Peter should clean the board". For completing this condition (the board to be clean), it's possible only Ivan to clean it, only Peter to clean it, or both of them to do it.

| a     | b     | a    b |
|-------|-------|--------|
| true  | true  | true   |
| true  | false | true   |
| false | true  | true   |
| false | false | false  |

## How Does The `||` Operator Work?

We have already learned what the logical "OR" represents. But how is it being achieved? Just like with the logical "AND", the program **checks** from left to right **the arguments** that are given. To obtain **true** from the expression, **just one** argument must have a **true** value. Respectively, the checking **continues** until an **argument** with **such** value is met or until the arguments **are over**.

Here is one **example** of the `||` operator in action:

```
boolean a = false;  
boolean b = true;  
boolean c = false;  
boolean d = true;  
boolean result = a || b || c || d;  
// true (as c and d are not being checked)
```



The program checks **a**, accepts that it has a value **false**, and continues. Reaching **b**, it understands that it has a **true** value and the whole expression is calculated as **true**, without having to check **c** or **d**, because their values wouldn't change the result of the expression.

## Problem: Fruit or Vegetable

Let's check whether a given product is a fruit or a vegetable. The "fruits" are banana, apple, kiwi, cherry, lemon and grapes. The "vegetables" are tomato, cucumber, pepper, and carrot. Everything else is "unknown".

### Sample Input and Output

| Input  | Output    |
|--------|-----------|
| banana | fruit     |
| tomato | vegetable |
| java   | unknown   |

### Solution

We have to use a few conditional statements with logical "OR" (`||`):

```
Scanner console = new Scanner(System.in);
String product = console.nextLine();

boolean isFruit = product.equals("banana") || product.equals("apple")
    || product.equals("kiwi") || product.equals("cherry")
    || product.equals("lemon") || product.equals("grapes");

boolean isVegetable = product.endsWith("tomato") || product.equals("cucumber")
    || product.equals("pepper") || product.equals("carrot");

if (isFruit) {
    System.out.println("fruit");
} else if (isVegetable) {
    System.out.println("vegetable");
} else {
    System.out.println("unknown");
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#3>.

## Logical Negation

Logical negation (operator `!`) means that a given condition is not fulfilled.

| a    | !a    |
|------|-------|
| true | false |

The operator `!` accepts as an **argument** a bool variable and **returns** its value.

## Problem: Invalid Number

A given **number** is **valid** if it is in the range `[100 ... 200]` or it is `0`. Validate an **invalid** number.

### Sample Input and Output

| Input | Output      |
|-------|-------------|
| 75    | invalid     |
| 150   | (no output) |
| 220   | invalid     |

### Solution

```
boolean isInRange = (number >= 100 && number <=200) || number ==0;

if (!isInRange) {
    System.out.println("invalid");
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#4>.

## The Parenthesis `()` Operator

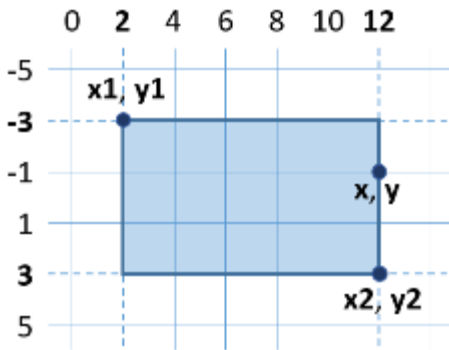
Like the rest of the operators in programming, the operators `&&` and `||` have a priority, as in the case `&&` is with higher priority than `||`. The operator `()` serves for **changing the priority of operators** and is being calculated first, just like in mathematics. Using parentheses also gives the code better readability and is considered a good practice.

## More Complex Conditions - Problems

Sometimes the conditions may be **very complex**, so they can require a long bool expression or a sequence of conditions. Let's take a look at a few examples.

### Problem: Point on Rectangle Border

Write a program that checks whether a **point** `{x, y}` is placed **onto any of the sides** of a **rectangle** `{x1, y1} - {x2, y2}`. The input data is read from the console and consists of 6 lines: the decimal numbers `x1, y1, x2, y2, x`, and `y` (as it is guaranteed that `x1 < x2` and `y1 < y2`). Print **"Border"** (if the point lies on any of the sides) or **"Inside / Outside"** (in the opposite case).



### Sample Input and Output

| Input | Output | Input | Output           |
|-------|--------|-------|------------------|
| 2     | Border | 2     | Inside / Outside |
| -3    |        | -3    |                  |
| 12    |        | 12    |                  |
| 3     |        | 3     |                  |
| 12    |        | 8     |                  |
| -1    |        | -1    |                  |

### Solution

The point lies on any of the sides of the rectangle if:

- $x$  coincides with  $x_1$  or  $x_2$  and at the same time  $y$  is between  $y_1$  and  $y_2$  or
- $y$  coincides with  $y_1$  or  $y_2$  and at the same time  $x$  is between  $x_1$  and  $x_2$ .

```
if ((x == x1) || (x == x2)) && (y >= y1) && (y <= y2)) {
    System.out.println("Border");
} else if (((y == y1) || (y == y2)) && (x >= x1) && (x <= x2)) {
    System.out.println("Border");
} else {
    System.out.println("Inside / Outside");
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#5>.

### Problem: Fruit Shop

A fruit shop during **weekdays** sells at the following **prices**:

| Fruit  | Price |
|--------|-------|
| banana | 2.50  |
| apple  | 1.20  |

| Fruit      | Price |
|------------|-------|
| orange     | 0.85  |
| grapefruit | 1.45  |
| kiwi       | 2.70  |
| pineapple  | 5.50  |
| grapes     | 3.85  |

During the **weekend days** the prices are **higher**:

| Fruit      | Price |
|------------|-------|
| banana     | 2.70  |
| apple      | 1.25  |
| orange     | 0.90  |
| grapefruit | 1.60  |
| kiwi       | 3.00  |
| pineapple  | 5.60  |
| grapes     | 4.20  |

Write a program that **reads** from the console a **fruit** (banana / apple / ...), a **day of the week** (Monday / Tuesday / ...) and a **quantity** (a decimal number) and **calculates** the price according to the prices from the tables above. The result has to be printed **rounded up to 2 digits after the decimal point**. Print **"error"** if it is an **invalid day** of the week or an **invalid name** of a fruit.

### Sample Input and Output

| Input                 | Output |
|-----------------------|--------|
| orange<br>Sunday<br>3 | 2.70   |

| Input                 | Output |
|-----------------------|--------|
| kiwi<br>Monday<br>2.5 | 6.75   |

| Input                     | Output |
|---------------------------|--------|
| grapes<br>Saturday<br>0.5 | 2.10   |

| Input                   | Output |
|-------------------------|--------|
| tomato<br>Monday<br>0.5 | error  |

### Solution

```
if (day.equals("monday") || day.equals("tuesday")
    || day.equals("wednesday") || day.equals("thursday")
    || day.equals("friday")) {

    if (fruit.equals("banana")) {
        price = quantity * 2.50;
    } // TODO - add the rest of the fruits here...
```

```

} else if (day.equals("saturday") || day.equals("sunday")) {

    if (fruit.equals("banana")) {
        price = quantity * 2.70;
    } // TODO - add the rest of the fruits here...
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#6>.

## Problem: Trade Comissions

A company is giving the following **commissions** to its traders according to the **city**, in which they are working and the **volume of sales** *s*:

| City    | $0 \leq s \leq 500$ | $500 < s \leq 1000$ | $1000 < s \leq 10000$ | $s > 10000$ |
|---------|---------------------|---------------------|-----------------------|-------------|
| Sofia   | 5%                  | 7%                  | 8%                    | 12%         |
| Varna   | 4.5%                | 7.5%                | 10%                   | 13%         |
| Plovdiv | 5.5%                | 8%                  | 12%                   | 14.5%       |

Write a **program** that reads the name of a **city** (string) and the volume of **sales** (double) and calculates the rate of the commission fee. The result has to be shown rounded **up to 2 digits after the decimal point**. When there is an **invalid city or volume of sales** (a negative number), print **"error"**.

## Sample Input and Output

| Input         | Output | Input             | Output | Input            | Output |
|---------------|--------|-------------------|--------|------------------|--------|
| Sofia<br>1500 | 120.00 | Plovdiv<br>499.99 | 27.50  | Kaspichan<br>-50 | error  |

## Solution

When reading the input, we could convert the city into small letters (with the function **.toLowerCase()**). Initially, we set the commission fee to **-1**. It will be changed if the city and the price range are found in the table of commissions. To calculate the commission according to the city and volume of sales, we need a few nested **if statements**, as in the sample code below:

```
double commission = -1;

if (town.equals("sofia")) {
    if (volume >= 0 && volume <= 500) {
        commission = volume * 0.05;
    } // TODO - complete the logic
} else if (town.equals("varna")) {
    // TODO - complete the logic
} else if (town.equals("plovdiv")) {
    // TODO - complete the logic
}

if (commission >= 0) {
    System.out.printf("%.2f", commission);
} else {
    System.out.println("error");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#7>.



It is a good practice to use **blocks** that are **enclosed** with curly braces **{ }** after **if** and **else**. Also, it is recommended during writing to **move aside** the code **after if and else** with a single tabulation **inward**, to make the code more easily readable.

## Switch-Case Conditional Statement

The **switch-case** condition works as a sequence of **if-else** blocks. Whenever the work of our program depends on the value of **one variable**, instead of making consecutive conditions with **if-else** blocks, we can **use** the conditional **switch** statement. It is being used for **choosing between a list of possibilities**. The statement compares a given value with defined constants and depending on the result, it takes an action.

We put the **variable** that we want to **compare**, inside the **brackets after the operator switch** and it is called a "**selector**". Here the **type must be comparable** (numbers, strings). Consecutively, the program starts **comparing each value that is found after the case labels**. Upon a match, the execution of the code from the respective place begins and continues until it reaches the operator **break**. In some programming languages (like C#) **break** is necessary for **every case**, which contains a program logic. In **Java** though, the operator **break** can be skipped, until we get to the next **break** operator. Even though it is allowed by the language, it's never used and considered a **bad practice**, because it leads to **collapsing** of the program code. If there is **missing a match**, the **default** construction will run, if there is one.

```
switch (selector) {
    case value1:
        construction;
        break;
    case value2:
        construction;
        break;
    case value3:
        construction;
        break;
    ...
    default:
        construction;
        break;
}
```

## Problem: Day of The Week

Let's write a program that prints the day of the week depending on the given number (1 ... 7) or "Error!" if invalid input is given.

### Sample Input and Output

| Input | Output |
|-------|--------|
| 1     | Monday |
| 7     | Sunday |
| -1    | Error! |

### Solution

```
Scanner console = new Scanner(System.in);
int day = Integer.parseInt(console.nextLine());
```

```
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    ...
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("error");
        break;
}
```



It is a good practice to put at the first place those **case** statements that process the most common situations and leave the **case** constructions processing the more rear situations at the end, before the **default** construction. Another good practice is to arrange the **case** labels in ascending order, regardless of whether they are integral or symbolic.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#8>.

## Multiple Labels in Switch-Cases

In Java, we can use multiple **case** labels in the **switch-case** construction, when they have to execute the same code. This way, when our program finds a **match**, it will execute the **next** code, because **after** the respective **case** label there is no code for execution and a **break** operator.

```
switch (selector) {

    case value1:
    case value2:
    case value3:
        construction;
        break;
    case value4:
    case value5:
        construction;
        break;
    ...
    default:
        construction;
        break;
}
```

## Problem: Animal Type

Write a program that prints the type of the animal depending on its name:

- dog -> **mammal**
- crocodile, tortoise, snake -> **reptile**
- others -> **unknown**



## Sample Input and Output

| Input    | Output  |
|----------|---------|
| tortoise | reptile |

| Input | Output |
|-------|--------|
| dog   | mammal |

| Input    | Output  |
|----------|---------|
| elephant | unknown |

## Solution

We can solve the problem with **switch-case** conditions with multiple labels in the following way:

```
switch (animal) {
    case "dog":
        System.out.println("mammal");
        break;
    case "crocodile":
    case "tortoise":
    case "snake":
        System.out.println("reptile");
        break;
    default:
        System.out.println("unknown");
        break;
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#9>.

## What Have We Learned from This Chapter?

In this chapter, we learned how we can use **more complex conditions**, logical operators inside the condition, and **nested if-constructions**.

## Nested Conditions

```
if (condition1) {
    if (condition2) {
        // body;
    } else {
        // body;
    }
}
```

## Complex Conditions with &&, ||, ! and ()

```
if ((x == left || x == right) && y <= top && y >= bottom)
```

```
System.out.println (...);
```

## Switch-Case Conditions

```
switch (selector) {  
    case value1:  
        construction;  
        break;  
    case value2:  
    case value3:  
        construction;  
        break;  
    ...  
    default:  
        construction;  
        break;  
}
```

## Problems More Complex Conditions

Let's practice using more complex conditions. We will solve a few practical exercises.

### Problem: Cinema

In a cinema hall, the chairs are ordered in a **rectangle** shape in **r** rows and **c** columns. There are three types of screenings with tickets of **different** prices:

- **Premiere** – a premiere screening, with a price of **12.00** BGN.
- **Normal** – a standard screening, with a price of **7.50** BGN.
- **Discount** – a screening for children and students at a reduced price – **5.00** BGN.

Write a program that enters a **type of screening** (string), a number for **rows**, and a number for **columns** in the hall (integer numbers) and calculates **the total income** from tickets from a **full hall**. The result has to be printed in the same format as in the examples below – rounded up to 2 digits after the decimal point.

### Sample Input and Output

| Input                | Output       |
|----------------------|--------------|
| Premiere<br>10<br>12 | 1440.00 leva |

| Input              | Output       |
|--------------------|--------------|
| Normal<br>21<br>13 | 2047.50 leva |

## Hints and Guidelines

While reading the input, we could convert the screening type into small letters (with the function `.toLowerCase()`). We create and initialize a variable that will store the calculated income. In another variable, we calculate the full capacity of the hall. We use a **switch-case** conditional statement to calculate the income according to the type of the projection and print the result on the console in the given format (look for the needed **Java** functionality on the internet).

Sample code (parts of the code are blurred with the purpose to stimulate your thinking and solving skills):

```
Scanner console = new Scanner(System.in);

String projectionType = console.nextLine().toLowerCase();
int rowCount = Integer.parseInt(console.nextLine());
int columnsCount = Integer.parseInt(console.nextLine());

int fullCapacity = rowCount * columnsCount;
double income = 0;

switch (projectionType) {
    case "premiere":
        income = fullCapacity * 12.00;
        break;
    case "normal":
        income = fullCapacity * 7.50;
        break;
    case "discount" :
        income = fullCapacity * 5.00;
        break;
}

System.out.printf("Total: %f BGN", income);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#10>.

## Problem: Volleyball

Vladimir is a student, lives in Sofia, and goes to his hometown from time to time. He is very keen on volleyball, but is busy during weekdays and plays **volleyball** only during **weekends** and on **holidays**. Vladimir plays in **Sofia** every **Saturday** when **he is not working**, and **he is not traveling to his hometown** and also during **2/3 of the holidays**. He travels to his **hometown** **h** times a year, where he plays volleyball with his old friends on **Sunday**. Vladimir is **not working 3/4 of the weekends**, during which he is in Sofia. Furthermore, during **leap years** Vladimir plays **15% more** volleyball than usual. We accept that the year has exactly **48 weekends**, suitable for volleyball. Write a program that calculates **how**

many times Vladimir has played volleyball throughout the year. Round the result down to the nearest whole number (e.g. 2.15 -> 2; 9.95 -> 9).

The input data is read from the console:

- The first line contains the word “**leap**” (leap year) or “**normal**” (a normal year with 365 days).
- The second line contains the integer **p** – the count of holidays in the year (which are not Saturday or Sunday).
- The third line contains the integer **h** – the count of weekends, in which Vladimir travels to his hometown.

Sample Input and Output

| Input          | Output | Input            | Output | Input             | Output | Input          | Output |
|----------------|--------|------------------|--------|-------------------|--------|----------------|--------|
| leap<br>5<br>2 | 45     | normal<br>3<br>2 | 38     | normal<br>11<br>6 | 44     | leap<br>0<br>1 | 41     |

Hints and Guidelines

As usual, we read the input data from the console and, to avoid making mistakes, we convert the text into small letters with the function `.toLowerCase()`. Consequently, we calculate the weekends spent in Sofia, the time for playing in Sofia, and the common playtime. At last, we check whether the year is a leap, we make additional calculations when necessary and we print the result on the console rounded down to the nearest integer (look for a Java class with such functionality).

A sample code (parts of the code are blurred on purpose to stimulate independent thinking and solving skills):

```
Scanner console = new Scanner(System.in);

String year = console.nextLine().toLowerCase();
int holidays = Integer.parseInt(console.nextLine());
int weekendsHome = Integer.parseInt(console.nextLine());

int sofiaWeekends = 45 - weekendsHome;
double playSofia = 2.0 * sofiaWeekends * 12.0 * 60 * 60 / 60;
double playTotal = playSofia + weekendsHome;
```

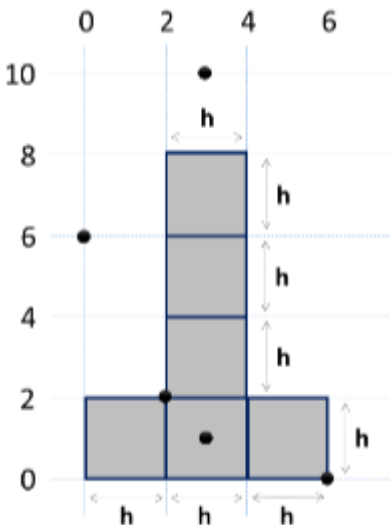
```
if (year.equals("leap")) {
    playTotal = Math.floor(playTotal*11/100+playTotal);
    System.out.println(playTotal);
} else if (year.equals("normal")) {
    System.out.println(Math.floor(playTotal));
}
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#11>.

Problem: Point in The Figure

The figure consists of 6 blocks with size  $h * h$ , placed as in the figure below. The lower left angle of the building is on position  $\{0, 0\}$ . The upper right angle of the figure is on position  $\{2*h, 4*h\}$ . The coordinates given in the figure are for  $h = 2$ :



Write a program that enters an integer  $h$  and the coordinates of a given point  $\{x, y\}$  (integers) and prints whether the point is inside the figure (**inside**), outside of the figure (**outside**), or on any of the borders of the figure (**border**).

Sample Input and Output

| Input | Output  | Input | Output | Input | Output | Input | Output |
|-------|---------|-------|--------|-------|--------|-------|--------|
| 2     | outside | 2     | inside | 2     | border | 2     | border |
| 3     |         | 3     |        | 2     |        | 6     |        |
| 10    |         | 1     |        | 2     |        | 0     |        |

| Input | Output  | Input | Output  | Input | Output | Input | Output  |
|-------|---------|-------|---------|-------|--------|-------|---------|
| 2     | outside | 15    | outside | 15    | inside | 15    | outside |
| 0     |         | 13    |         | 29    |        | 37    |         |
| 6     |         | 55    |         | 37    |        | 18    |         |

Hints and Guidelines

A possible logic for solving the problem (not the only correct one):

- We might split the figure into **two rectangles** with a common side.
- A point is **outer (outside)** for the figure when it is **outside** both of the rectangles.
- A point is **inner (inside)** for the figure if it is inside one of the rectangles (excluding their borders) or lies on their common side.
- In **another case**, the point lies on the border of the rectangle (**border**).

Sample code (parts of the code are blurred to stimulate thinking and solving skills):

```
Scanner console = new Scanner(System.in);

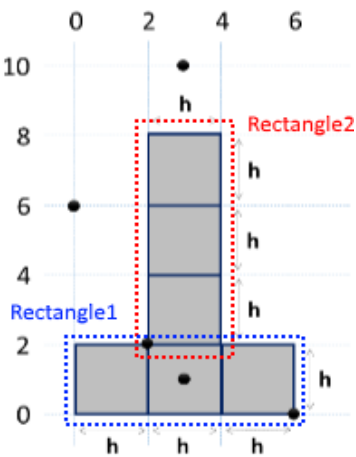
int h = Integer.parseInt(console.nextLine());
int x = Integer.parseInt(console.nextLine());
int y = Integer.parseInt(console.nextLine());

boolean outRectangle1 = /* ... */;
boolean outRectangle2 = /* ... */;

boolean inRectangle1 = /* ... */;
boolean inRectangle2 = /* ... */;

boolean commonBorder = /* ... */;

if ( /* ... */ ) {
    System.out.println("outside");
} else if ( /* ... */ ) {
    System.out.println("inside");
} else {
    System.out.println("border");
}
```



Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/653#12>.

## Chapter 4.2. More Complex Conditions – Exam Problems

In the previous chapter, we got familiar with the **nested conditional** constructions in the Java language. Through them, the logic in a program can be represented by **if conditional statements** that are nested in each other. We also looked at the conditional **switch-case** construction, which allows you to choose from a list of options. We should practice and consolidate what we have learned so far by looking at a few more complex exam problems. Before moving on to the problems, we will recall the conditional constructions:

### Nested Conditions

```
if (condition1) {  
    if (condition2) {  
        // body of if construction  
    } else {  
        // body of else construction  
    }  
}
```



Remember that it is not a good practice to write **deeply nested conditional statements** (with more than three levels of nesting). Avoid nesting of more than three conditional statements inside one another. This complicates the code and makes its reading and understanding difficult.

### Switch-Case Conditions

When the program operation depends on the value of a variable, instead of doing consecutive checks with multiple **if-else** blocks, we can use the **switch-case** conditional statement.

```
switch (selector) {  
    case value1:  
        //code to be executed  
        break;  
    case value2:  
        //code to be executed  
        break;  
    default:  
        //code to be executed  
        break;  
}
```

The structure consists of:

- Selector - an expression that is calculated to some specific value. The type of the selector can be **an integer**, **string**, or **enum**.
- Multiple **case** labels followed by code to be executed, ending with a **break**.

## Exam Problems

Now that we have recalled how conditional statements are used and nested into each other to implement more complex conditions and program logic, let's solve a few exam problems.

### Problem: On Time for The Exam

A student must go **to the exam at a certain time** (for example at 9:30 am). He arrives at the exam hall at a given **hour of arrival** (for example 9:40). It is considered that the student has arrived **on time** if he arrives **at the time when the exam starts or up to half an hour earlier**. If the student arrives **more than 30 minutes earlier**, the student **has come too early**. If he arrives **after the time when the exam starts**, he is **late**.

Write a program that has as input data for the exam starting time and the student arrival time, and prints if the student has arrived **on time**, or if he has **arrived early** or if he is **late**, as well as **how many hours or minutes** the student was early or late.

### Input Data

From the console, we read as input data **four integers** (one per line):

- The first line contains **exam starting time(hours)** – an integer from 0 to 23.
- The second line contains **exam starting time(minutes)** – an integer from 0 to 59.
- The third line contains the **hour of arrival** – an integer from 0 to 23.
- The fourth line contains **minutes of arrival** – an integer from 0 to 59.

### Output Data

Print on the first line:

- **"Late"**, if the student arrives **later** than the exam starting time.
- **"On time"**, if the student arrives **on time** exactly at the exam starting time or up to 30 minutes earlier.
- **"Early"**, if the student arrives more than 30 minutes **before** the exam starting time.

If the student arrives with more than one minute difference compared to the exam starting time, print on the next line:

- **"mm minutes before the start"** for arriving less than an hour earlier.
- **"hh:mm hours before the start"** for arriving 1 hour or earlier. Always print the minutes using two digits, for example, "1:05".



- "mm minutes after the start" for arriving less than an hour late.
- "hh:mm hours after the start" for arriving late with 1 hour or more. Always print the minutes using two digits, for example, "1:03".

### Sample Input and Output

| Input              | Output                             | Input                | Output                               | Input              | Output                                    |
|--------------------|------------------------------------|----------------------|--------------------------------------|--------------------|-------------------------------------------|
| 9<br>30<br>9<br>50 | Late 20 minutes<br>after the start | 16<br>00<br>15<br>00 | Early 1:00 hours<br>before the start | 9<br>00<br>8<br>30 | On time 30<br>minutes before<br>the start |

| Input               | Output                                | Input                | Output                                      | Input                | Output  |
|---------------------|---------------------------------------|----------------------|---------------------------------------------|----------------------|---------|
| 9<br>00<br>10<br>30 | Late 1:30<br>hours after<br>the start | 14<br>00<br>13<br>55 | On time 5<br>minutes<br>before the<br>start | 10<br>00<br>10<br>00 | On time |

### Hints and Guidelines



It is recommended that you read the condition of a problem several times taking notes and sketching examples while thinking about them before you start writing code.

### Input Data

According to the assignment, we expect **four** lines in a row with different **integers**. Looking at the provided parameters, we can use the type **int**, as it is suitable for the expected values. We simultaneously **read** input and **parse** the string value to the selected data type for **integer**.

```
Scanner scanner = new Scanner(System.in);
int examHours = Integer.parseInt(scanner.nextLine());
int examMinutes = Integer.parseInt(scanner.nextLine());
int arrivalHours = Integer.parseInt(scanner.nextLine());
int arrivalMinutes = Integer.parseInt(scanner.nextLine());
```

Looking at the expected output, we can create variables that contain the different output data types to avoid using the so-called "magic strings" in the code.

```
String late = "Late";
String onTime = "On time";
String early = "Early";
```

## Calculations

After reading the input data, we can now start writing the logic for calculating the result. Let's first calculate the **starting time** of the exam in **minutes** for an easier and more accurate comparison.

```
int examTime = (examHours * 60) + examMinutes;
```

Let's also calculate the **arrival time** of the student.

```
int arrivalTime = (arrivalHours * 60) + arrivalMinutes;
```

It remains for us to calculate the difference between the two times, to determine **when** and **what time compared to the exam time** the student has arrived.

```
int totalMinutesDifference = arrivalTime - examTime;
```

Our next step is to do the required **checks and calculations**, and finally, we will print the output. Let's separate the code into **two** parts:

- First, let's show when the student arrived – is he **early**, **late** or **on time**. To do that, we will use an **if-else** statement.
- After that, we will show the **time difference** if the student arrives at a **different time** from the **starting time of the exam**.

To spare one additional check (**else**), we can, by default, assume that the student was late.

According to the condition, we will check whether the difference in times is **more than 30 minutes**. If this is true, we assume that the student is **early**. If we do not match the first condition, we need to check if the **difference is less than or equal to zero ( $\leq 0$ )**, by which we are checking the condition whether the student arrived within the range of **0 to 30 minutes** before the exam.

In all other cases, we assume that the student is **late**, which we did **by default**. Therefore, no additional check is needed.

```
String studentArrival = late;
if (totalMinutesDifference < -30) {
    studentArrival = early;
} else if (totalMinutesDifference <= 0) {
    studentArrival = onTime;
}
```

Finally, we need to find out and print **what is the time difference between exam start time and student arrival time**. Also, whether this difference shows the time of arrival **before or after the exam start**.

We also check whether the time difference is **more than** one hour, to print hours and minutes in the required **format**, or if it is **less than** one hour, to print **only minutes** in the required format and description.

We also need to do one more check – whether the time of the student arrival is **before** or **after** the exam start time.

```
String result = "";
if (totalMinutesDifference != 0) {
    int hoursDifference = Math.abs(totalMinutesDifference / 60);
    int minutesDifference = Math.abs(totalMinutesDifference % 60);

    if (hoursDifference > 0) {
        result = String.format("%d:%02d hours", hoursDifference,
                                minutesDifference);
    } else {
        result = minutesDifference + " minutes";
    }

    if (totalMinutesDifference < 0) {
        result += " before the start";
    } else {
        result += " after the start";
    }
}
```

### Printing The Result

Finally, it remains to print the result on the console. According to the requirements, if the student arrived right in time (**not even a minute difference**), we do not need to print a second result. This is why we do the following **check**:

```
System.out.println(studentArrival);
if (!result.isEmpty()) {
    System.out.println(result);
}
```

Printing the result **on the console** can be done at a much earlier stage – during the calculations. This, however, is not a very good practice. **Why?**

Let's look at the idea that our code is not 10 lines, but 100 or 1000! One day, printing the result will not be done on the console, but will be written in a **file** or displayed as a **web application**. Then, how many places in the code do we need to do changes? Are you sure you won't miss some places?



Always consider the code that contains **logical calculations** as a **separate part** different from the part that processes the input and output data. It must be able to work regardless of how the data is passed to it and where the result will be displayed.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/654#0>.

## Problem: Trip

Most people plan their vacations. A young programmer has a **fixed budget** and spare time in a particular **season**.

Write a program that accepts **as an input the budget and the season**, and **as an output** to print **where the programmer will rest** and **the amount of money he will spend**.

The budget determines the destination and the season what amount of the budget will be spent. If the season is **summer**, he will go **camping**, and if it is **winter** - in a **hotel**. If it is in **Europe**, regardless of the season, the programmer will stay in a **hotel**. Each **camping** or **hotel**, according to the destination, has **its price**, which corresponds to a particular percentage of the budget:

- If **100 BGN or less** – somewhere in **Bulgaria**.
  - Summer – 30% of the budget.
  - Winter – 70% of the budget.
- If **1000 BGN or less** – somewhere on the **Balkans**.
  - Summer – 40% of the budget.
  - Winter – 80% of the budget.
- If **more than 1000 BGN**. – somewhere in **Europe**.
  - Upon traveling in Europe, regardless of the season, the programmer will spend 90% of the budget.

## Input Data

The input data will be read from the console and will consist of **two lines**:

- On the **first line**, we receive the **budget** - a **floating-point number** in the range of [10.00 ... 5000.00].
- On the **second line** – one of the two possible seasons: "**summer**" or "**winter**".

## Output Data

On the console must be printed **two lines**.

- On the **first line** – "**Somewhere in {destination}**", where {destination} is "**Bulgaria**", "**Balkans**" or "**Europe**".
- On the **second line** – "**{Vacation type} – {Spent money}**":
  - The vacation can be "**Camp**" or "**Hotel**".
  - The amount of money has to be rounded up to the second digit after the decimal point.

## Sample Input and Output

| Input        | Output                                |
|--------------|---------------------------------------|
| 50<br>summer | Somewhere in Bulgaria<br>Camp – 15.00 |

| Input        | Output                                 |
|--------------|----------------------------------------|
| 75<br>winter | Somewhere in Bulgaria<br>Hotel – 52.50 |

| Input         | Output                                |
|---------------|---------------------------------------|
| 312<br>summer | Somewhere in Balkans<br>Camp – 124.80 |

| Input          | Output                                 |
|----------------|----------------------------------------|
| 1500<br>summer | Somewhere in Europe<br>Hotel – 1350.00 |

## Hints and Guidelines

Like the other problems, we can separate the solution of our problem into the following parts: reading the input data, doing calculations, printing the result.

### Input Data

While carefully reading the requirements, we understand that we expect **two** lines of input data. The first parameter is a **real number**, for which we need to pick an appropriate variable type. For a higher level of calculation accuracy, we can pick **BigDecimal** as a type for the budget and **string** for the season.

```
Scanner scanner = new Scanner(System.in);
BigDecimal budget = new BigDecimal(scanner.nextLine());
String season = scanner.nextLine();
```



Always consider, what is the **\*\*type\*\*** in the input data, as well as to what data type they need to be converted to, so created program constructs to work correctly!

**Example:** When you need to do calculations related to money, use **BigDecimal** for a higher level of accuracy.

### Calculations

Let's create and initialize the variables needed for applying the logic and calculations.

```
String destinationResult = "";
String holidayInformation = "";
BigDecimal moneySpent = new BigDecimal("0.00");
```

Similar to the example in the previous problem, we can initialize variables with some of the output results, to spare additional initialization.

When examining once again the requirements, we notice that the decision of where the vacation will take place depends on the **amount of the budget** we receive as input data, i.e., in our logic, we have two cases:

- If the budget is **less than or equals** a particular value.

- If it is **less than or equals** other value or is **more than** the specified border value.

Based on the way we arrange the logical scheme (the order in which we will check the border values), we will have more or fewer conditions in the solution. **Think why!**

After that, we need to check the value of the **given season**. Based on the result, we will determine what percentage of the budget will use, and where the programmer will stay – in a **hotel** or a **camp**.

Let's choose and create the needed format for the output data, which we will use further in the code:

```
DecimalFormat formatter = new DecimalFormat("0.00");
```

Here is a sample code that we can use to implement the described logic above:

```
if (budget.compareTo(BigDecimal.valueOf(100.00)) <= 0) {
    destinationResult = "Bulgaria";
    if (season.equals("summer")) {
        moneySpent = BigDecimal.valueOf(0.30).multiply(budget);
        holidayInformation = String.format("Camp - %s",
            formatter.format(moneySpent));
    } else {
        moneySpent = BigDecimal.valueOf(0.70).multiply(budget);
        holidayInformation = String.format("Hotel - %s",
            formatter.format(moneySpent));
    }
} else if {
```

After, we continue with:

```
} else if (budget.compareTo(BigDecimal.valueOf(1000.00)) <= 0) {
    destinationResult = "Balkans";
    if (season.equals("summer")) {
        moneySpent = BigDecimal.valueOf(0.40).multiply(budget);
        holidayInformation = String.format("Camp - %s",
            formatter.format(moneySpent));
    } else {
        moneySpent = BigDecimal.valueOf(0.80).multiply(budget);
        holidayInformation = String.format("Hotel - %s",
            formatter.format(moneySpent));
    }
} else {
```

and finish the conditions with:

```
} else {
    destinationResult = "Europe";
    moneySpent = BigDecimal.valueOf(0.90).multiply(budget);
    holidayInformation = String.format("Hotel - %s",
        formatter.format(moneySpent));
}
```

We always can initialize the variable with a given value and then make only one check. This saves us a logic step.

For example, the following code:

```
if (budget.compareTo(BigDecimal.valueOf(100.00)) <= 0) {
    destinationResult = "Bulgaria";
    if (season.equals("summer")) {
        moneySpent = BigDecimal.valueOf(0.30).multiply(budget);
        holidayInformation = String.format("Camp - %s",
            formatter.format(moneySpent));
    } else {
        moneySpent = BigDecimal.valueOf(0.70).multiply(budget);
        holidayInformation = String.format("Hotel - %s",
            formatter.format(moneySpent));
    }
} else if {
```

can be shortened to this:

```
destinationResult = "Bulgaria";
moneySpent = BigDecimal.valueOf(0.70).multiply(budget);
holidayInformation = String.format("Hotel - %s",
    formatter.format(moneySpent));

if (season.equals("summer")) {
    moneySpent = BigDecimal.valueOf(0.30).multiply(budget);
    holidayInformation = String.format("Camp - %s",
        formatter.format(moneySpent));
}
```

### Printing The Result

It remains to print the obtained result on the console:

```
System.out.println(String.format("Somewhere in %s",
    destinationResult));
System.out.println(holidayInformation);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/654#1>.

## Problem: Operations

Write a program that reads as input data **two integers** (N1 and N2) and an **operator** that performs a particular **mathematical operation** with them. Possible operations are: **addition** (+), **subtraction** (-), **multiplication** (\*), **division** (/) and **modular division** (%). Upon adding, subtracting, and multiplying, print on the console the result and display whether it is **even** or **odd**. In simple division – **only the result**, and in the modular division – **the remainder**.

You need to take into account that the divisor can be equal to zero (= 0), and dividing by zero is not possible. In this case, print a special message.

## Input Data

The input data will be read from the console and will consist of three lines:

- On the first line N1 – integer within the range [0 ... 40 000].
- On the second line N2 – integer within the range [0 ... 40 000].
- On the third line Operator – one character among: "+", "-", "\*", "/", "%".

## Output Data

Print one line on the console:

- If the operation is addition, subtraction or multiplying:
  - "{N1} {operator} {N2} = {output} – {even/odd}".
- If the operation is division:
  - "{N1} / {N2} = {output}" – the result is formatted to the second digit after the decimal point.
- If the operation is modular division:
  - "{N1} % {N2} = {remainder}".
- In case of dividing by 0 (zero):
  - "Cannot divide {N1} by zero".

## Sample Input and Output

| Input        | Output           |
|--------------|------------------|
| 10<br>1<br>- | 10 - 1 = 9 - odd |

| Input        | Output                      |
|--------------|-----------------------------|
| 10<br>0<br>% | Cannot divide<br>10 by zero |

| Input         | Output              |
|---------------|---------------------|
| 10<br>12<br>+ | 10 + 12 = 22 - even |

| Input         | Output                       |
|---------------|------------------------------|
| 112<br>0<br>/ | Cannot divide<br>112 by zero |

| Input | Output           |
|-------|------------------|
| 123   | 123 / 12 = 10.25 |

| Input | Output     |
|-------|------------|
| 10    | 10 % 3 = 1 |



| Input   | Output |
|---------|--------|
| 12<br>/ |        |

| Input  | Output |
|--------|--------|
| 3<br>% |        |

## Hints and Guidelines

The problem is not complicated, but there are many lines of code to write.

### Input Data

After reading the condition, we understand that we expect **three** lines of input data. On the first **two** lines we expect two **integers** (within the specified range), and on the third line - an **arithmetical symbol**.

```
Scanner scanner = new Scanner(System.in);
int N1 = Integer.parseInt(scanner.nextLine());
int N2 = Integer.parseInt(scanner.nextLine());
String nOperator = scanner.nextLine();
```

### Calculations

Let's create and initialize the variables needed for the logic and calculations. In one variable, we will store **the result from the calculations**, and in the other one, we will use it for the **final output** of the program.

```
BigDecimal result = new BigDecimal( val: "0.00");
String output = "";
DecimalFormat intFormatter = new DecimalFormat( pattern: "0");
```

Reading the condition carefully, we understand that there are cases where we don't need to do **any** calculations but simply print the result.

Therefore, we can first check if the second number is **0** (zero), whether the operation is a **division** or a **modular division**, and then initialize the output.

```
if (N2 == 0 && (nOperator.equals("/") || nOperator.equals("%"))) {
    output = String.format("Cannot divide %d by zero", N1);
}
```

Let's put the result as a value upon initializing the **output** parameter. This way we, can make only **one check** - whether it is necessary to **recalculate** and **replace** this result.

Based on the approach that we choose, our next check will be either a simple **else** or a single **if**. In the body of this check, using additional checks, according to the submitted operator, we can divide the logic according to the **structure** of the expected **output**.

From the condition we can see that for **addition (+)**, **subtraction (-)** or **multiplication (\*)** the expected output has the same structure: "{N1} {operator} {N2} = {output} - {even/odd}", whereas for **division (/)** and **modular division (%)** the output has a different structure.

```

} else if (nOperator.equals("/")) {
    result = new BigDecimal(N1).divide(
        new BigDecimal(N2), scale: 2, RoundingMode.HALF_UP);
    DecimalFormat decimalFormatter = new DecimalFormat(pattern: "0.00");
    output = String.format("%d %s %d = %s",
        N1, nOperator, N2, decimalFormatter.format(result));
} else if (nOperator.equals("%")) {
    result = new BigDecimal(N1).remainder(new BigDecimal(N2));
    output = String.format("%d %s %d = %s",
        N1, nOperator, N2, intFormatter.format(result));
} else {

```

We finish the solution by applying conditions for addition, subtraction and multiplication:

```

} else {
    if (nOperator.equals("+")) {
        result = new BigDecimal((N1 + N2));
    } else if (nOperator.equals("-")) {
        result = new BigDecimal((N1 - N2));
    } else if (nOperator.equals("*")) {
        result = new BigDecimal((N1 * N2));
    }

    output = String.format(
        "%d %s %d = %s - %s",
        N1,
        nOperator,
        N2,
        intFormatter.format(result),
        result.intValueExact() % 2 == 0 ? "even" : "odd");
}

```

For short and clear checks, such as the above example for even and odd numbers, you can use a **ternary operator**. Let's look at the possible check **with** and **without** the ternary operator.

Without using a ternary operator, the code is longer but easy to read:

```

String numberIs = "";
if (result.intValueExact() % 2 == 0) {
    numberIs = "even";
} else {
    numberIs = "odd";
}

```

Upon using a ternary operator, the code is much shorter but may require additional efforts to read and understand the logic:

```
String numberIs = result.intValueExact() % 2 == 0 ? "even" : "odd";
```

### Printing The Result

Finally, we need to show the calculated result to the console:

```
System.out.println(output);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/654#2>.

## Problem: Match Tickets

A group of football fans decided to buy tickets for Euro 2016. The ticket price is determined according to **two** categories:

- VIP – 499.99 BGN
- Normal – 249.99 BGN

The fans **have a certain budget**, and the **number of people** in the group determines what percentage of the budget **should be set aside for transport**:

- From 1 to 4 – 75% of the budget
- From 5 to 9 – 60% of the budget
- From 10 to 24 – 50% of the budget
- From 25 to 49 – 40% of the budget
- 50 or more – 25% of the budget

Write a program that **calculates, whether with the rest money from the budget** they can buy tickets for the selected category, as well as how much money they will have **left or will need**.

### Input Data

The input is read from the **console** and contains **exactly 3 lines**:

- On the **first** line is the **budget** – a floating-point number in the range of [1 000.00 ... 1 000 000.00].
- On the **second** line is the **category** – "VIP" or "Normal".
- On the **third** line is the **number of people in the group** – an integer in the range of [1 ... 200].

### Output Data

Print one line on the console:

- If the budget is enough:
  - "Yes! You have {N} leva left." – where N is the remaining amount of the group.
- If the budget is not enough:

- o "Not enough money! You need {M} leva." – where M is the amount that is insufficient.

The amounts must be formatted up to the second digit after the decimal point.

Sample Input and Output

| Input               | Output                           | Explanations                                                                                                                                                                                                                                       |
|---------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1000<br>Normal<br>1 | Yes! You have<br>0.01 leva left. | <b>1 person:</b> 75% of the budget are spent on <b>transport</b> .<br><b>Remaining amount:</b> 1000 – 750 = 250.<br>Category <b>Normal:</b> The ticket <b>price is</b> 249.99 * 1 = 249.99<br>249.99 < 250: they remain to him 250 – 249.99 = 0.01 |

| Input              | Output                                            | Explanations                                                                                                                                                                                                                                                            |
|--------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 30000<br>VIP<br>49 | Not enough<br>money! You<br>need 6499.51<br>leva. | <b>49 people:</b> 40% of the budget are spent on <b>transport</b> .<br>Remaining amount: 30000 – 12000 = 18000.<br>Category <b>VIP:</b> The ticket <b>price is</b> 499.99 49.<br><b>24499.5100000000002</b> < 18000.<br><b>Money needed</b> 24499.51 - 18000 = *6499.51 |

Hints and Guidelines

We will read the input data and do the calculations described in the condition to check if the money will be sufficient.

Input Data

Let's read the condition carefully and look at what we expect to get as **input data**, what we expect to **return, as a result**, and what are the **main steps** when dividing the **logic**.

To begin with, let's process and save the input data in **appropriate variables**:

```
Scanner scanner = new Scanner(System.in);
BigDecimal budget = new BigDecimal(scanner.nextLine());
String ticketType = scanner.nextLine();
int people = Integer.parseInt(scanner.nextLine());
```

Calculations

Let's create and initialize the needed variables for the calculations:

```
BigDecimal transportCharges = new BigDecimal( val: "0.00");
BigDecimal moneyForTickets = new BigDecimal( val: "0.00");
BigDecimal moneyDifference = new BigDecimal( val: "0.00");
```

Let's review the condition again. We need to do **two** different calculations.

The first calculations are about what part of the budget we will have to set aside for **transport**. For the logic of these calculations, we will notice that only **the number of people in the group** matters. Therefore, we will make a logical breakdown by the number of fans.

We will use conditional construction - a series of **if-else** blocks.

```
if (people <= 4) {
    transportCharges = new BigDecimal( val: "0.75").multiply(budget);
} else if (people <= 9) {
    transportCharges = new BigDecimal( val: "0.60").multiply(budget);
} else if (people <= 24) {
    transportCharges = new BigDecimal( val: "0.50").multiply(budget);
} else if (people <= 49) {
    transportCharges = new BigDecimal( val: "0.40").multiply(budget);
} else if (people >= 50) {
    transportCharges = new BigDecimal( val: "0.25").multiply(budget);
}
```

The second calculations are about what amount we will need to buy **tickets for the group**. According to the condition, it depends only on the type of tickets, we have to buy.

Let's use **switch-case** conditional construction.

```
switch (ticketType) {
    case "Normal":
        moneyForTickets =
            new BigDecimal(people).multiply(
                new BigDecimal( val: "249.99"));
        break;
    case "VIP":
        moneyForTickets =
            new BigDecimal(people).multiply(
                new BigDecimal( val: "499.99"));
        break;
    default:
        moneyForTickets =
            new BigDecimal(people).multiply(
                new BigDecimal( val: "249.99"));
        break;
}
```

Once we have calculated the **transport** and the **tickets** costs, what remains is to calculate the final result and see **whether it will succeed** the group of fans to attend the Euro 2016 or **not** with these parameters given.

For printing the result, to skip one **else condition**, we will assume that the group can, by default, attend Euro 2016.

```

DecimalFormat decimalFormatter = new DecimalFormat("0.00");
moneyDifference = budget.subtract(
    transportCharges.add(moneyForTickets));
String result = String.format(
    "Yes! You have %s leva left.",
    decimalFormatter.format(moneyDifference.setScale(
        2, RoundingMode.HALF_EVEN)));

if (moneyDifference.compareTo(BigDecimal.valueOf(0)) < 0) {
    result = String.format(
        "Not enough money! You need %s leva.",
        decimalFormatter.format(moneyDifference.setScale(
            2, RoundingMode.HALF_EVEN).abs()));
}

```

### Printing The Result

Finally, we need to print the calculated result on the console.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/654#3>.

## Problem: Hotel Room

A hotel offers two types of rooms: studio and apartment.

Write a program that calculates the price of the entire stay for a studio and apartment. The prices depend on the month of the accommodation:

| May and October              | June and September              | July and August              |
|------------------------------|---------------------------------|------------------------------|
| Studio – 50 BGN/per night    | Studio – 75.20 BGN/per night    | Studio – 76 BGN/per night    |
| Apartment – 65 BGN/per night | Apartment – 68.70 BGN/per night | Apartment – 77 BGN/per night |

The following discounts are offered:

- For **studio**, in case of more than 7 nights in **May and October**: 5% discount.
- For **studio**, in case of more than 14 nights in **May and October**: 30% discount.
- For **studio**, in case of more than 14 nights in **June and September**: 20% discount.
- For **apartment**, in case of more than 14 nights, no matter the month: 10% discount.

### Input Data

The input data is read from the console and contains exactly two lines:

- On the **first** line is the **month** – May, June, July, August, September, or October.
- On the **second** line is the **number of nights** – integer in the range of [0 ... 200].

## Output Data

Print two lines on the console:

- On the first line: "Apartment: {price for the whole stay} BGN"
- On the second line: "Studio: {price for the whole stay} BGN"

The price for the entire stay must be formatted up to two digits after the decimal point.

## Sample Input and Output

| Input      | Output                                      |
|------------|---------------------------------------------|
| June<br>14 | Apartment: 961.80 lv.<br>Studio: 1052.80 lv |

| Input        | Output                                        |
|--------------|-----------------------------------------------|
| August<br>20 | Apartment: 1386.00 lv.<br>Studio: 1520.00 lv. |

| Input     | Output                                      | Comments                                                                                                                                                                                                                                                                                             |
|-----------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| May<br>15 | Apartment: 877.50 lv.<br>Studio: 525.00 lv. | In <b>may</b> , for more than <b>14 nights</b> , we decrease the price of the <b>studio with 30%</b> ( $50 - 15 = 35$ ), and for the <b>apartment – with 10%</b> ( $65 - 6.5 = 58.5$ ).<br>The entire stay in the <b>apartment – 877.50 BGN</b><br>The entire stay in the <b>studio – 525.00 BGN</b> |

## Hints and Guidelines

First, we will read the input data and then perform the calculations according to the provided price list and the discount rules. Finally, print the result.

### Input Data

According to the condition, we expect to read two lines of input data - on the first line **the month in which the stay is planned**, on the second - **the number of nights**.

Let's process and store the input data in appropriate variables:

```
Scanner scanner = new Scanner(System.in);
String month = scanner.nextLine();
int nights = Integer.parseInt(scanner.nextLine());
```

### Calculations

Let's create and initialize the variables needed for the calculations:

```
BigDecimal studioPrice = new BigDecimal( val: 50.00);
BigDecimal apartmentPrice = new BigDecimal( val: 65.00);
BigDecimal studioRent = new BigDecimal( val: 0.00);
BigDecimal apartmentRent = new BigDecimal( val: 0.00);
```

Looking again at the condition, we notice that our basic logic depends on what **month** we receive as input data and the number of **nights**.

In general, there are different approaches and ways to make the checks in question, but let's focus on the **switch-case** construction, and in the different **case** blocks, we will use conditional statements such as **if** and **if-else**.

Let's start with the first group of months: **May** and **October**. For these two months, the price for the stay is the same for both types of accommodation - **studio** and **apartment**. Therefore, it is necessary to make an internal check against the number of nights to recalculate the corresponding price (if needed).

```
switch (month) {
    case "May":
    case "October":
        studioPrice = new BigDecimal( val: 50.00);
        apartmentPrice = new BigDecimal( val: 65.00);

        studioRent = studioPrice.multiply(new BigDecimal(nights));
        apartmentRent = apartmentPrice.multiply(new BigDecimal(nights));

        if (nights > 14) {
            studioRent = studioRent.multiply(new BigDecimal( val: 0.70));
            apartmentRent = apartmentRent.multiply(new BigDecimal( val: 0.90));
        } else if (nights > 7) {
            studioRent = studioRent.multiply(new BigDecimal( val: 0.95));
        }

        break;
```

For the following months, the logic and the calculations are identical.

```
case "June":
case "September":
    studioPrice = new BigDecimal( val: 75.20);
    apartmentPrice = new BigDecimal( val: 68.70);

    studioRent = studioPrice.multiply(new BigDecimal(nights));
    apartmentRent = apartmentPrice.multiply(new BigDecimal(nights));

    if (nights > 14) {
        studioRent = studioRent.multiply(new BigDecimal( val: 0.80));
        apartmentRent = apartmentRent.multiply(new BigDecimal( val: 0.90));
    }

    break;
```



```

case "July":
case "August":
    studioPrice = new BigDecimal( val: 76.00);
    apartmentPrice = new BigDecimal( val: 77.00);

    studioRent = studioPrice.multiply(new BigDecimal(nights));
    apartmentRent = apartmentPrice.multiply(new BigDecimal(nights));

    if (nights > 14) {
        apartmentRent = apartmentRent.multiply(new BigDecimal( val: 0.90));
    }

    break;
default:
    break;

```

After calculating the relevant prices and the total amount for the stay - let's prepare the formatted result. Before that, we should store it in our output variables - **studioInfo** and **apartmentInfo**.

```

DecimalFormat decimalFormatter = new DecimalFormat("0.00");
String studioInfo = String.format("Studio: %s lv.",
    decimalFormatter.format(
        studioRent.setScale(2, RoundingMode.HALF_EVEN)));
String apartmentInfo = String.format("Apartment: %s lv.",
    decimalFormatter.format(
        apartmentRent.setScale(2, RoundingMode.HALF_EVEN)));

```

To calculate the output parameters, we use the method **java.math.BigDecimal.setScale(int newScale, RoundingMode roundingMode)**. This method rounds a decimal number to a specified number of digits after the decimal point, and we have control of any rounding type - **RoundingMode**. For this purpose, we give the method an integer (**int**) with which we indicate till which sign we want to round the number and value of a given enumeration **RoundingMode** (**UP**, **DOWN**, **CEILING**, **FLOOR**, **HALF\_UP**, **HALF\_DOWN**, **HALF\_EVEN**). In our case, we will round the decimal number up to two digits after the decimal point with rounding type **HALF\_UP**.

### Printing The Result

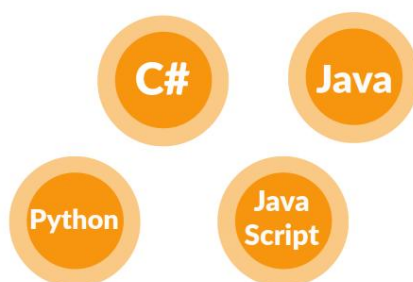
Finally, what remains is to print the calculated results on the console.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/654#4>.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)

# Chapter 5.1. Loops

In the present chapter, we will get familiar with how to **repeat blocks of commands**, known in software development as "**loops**". We will write several loops using the **for** operator in its simplest form. Finally, we will solve some practical problems that require repeating a series of actions, using loops.

## Video

Watch the video lesson on this chapter here:

<https://www.youtube.com/watch?v=VyoaWLnbDiO>.

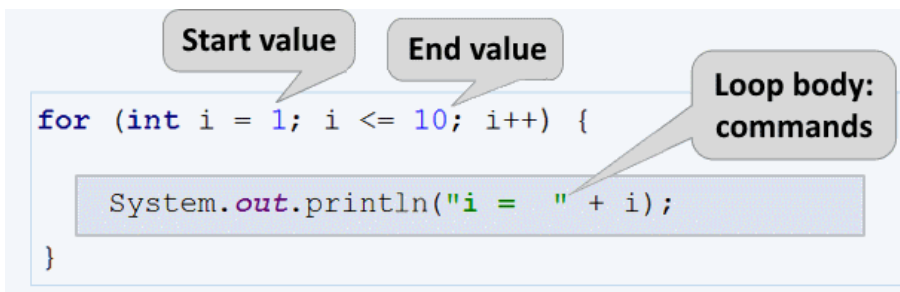
## Repetitions of Blocks of Code (For Loop)

Often in programming, we have to **execute a block of code multiple times**. To do that, we use the so-called **loops**. Let us look at an example of a **for** loop that passes sequentially through the numbers from 1 to 10 and prints them:

```
for (int i = 1; i <= 10; i++) {
    System.out.println("i = " + i);
}
```

The loop starts with the **operator for** and passes through all values for a particular variable in a given range, for example, the numbers from 1 to 10 (included), and for each value, it performs a series of commands.

When declaring the loop, an **initial value** and an **end value** are specified. The **body of the loop** is enclosed in curly brackets **{ }** and represents a block of one or multiple commands. The figure below shows the structure of a **for** loop:



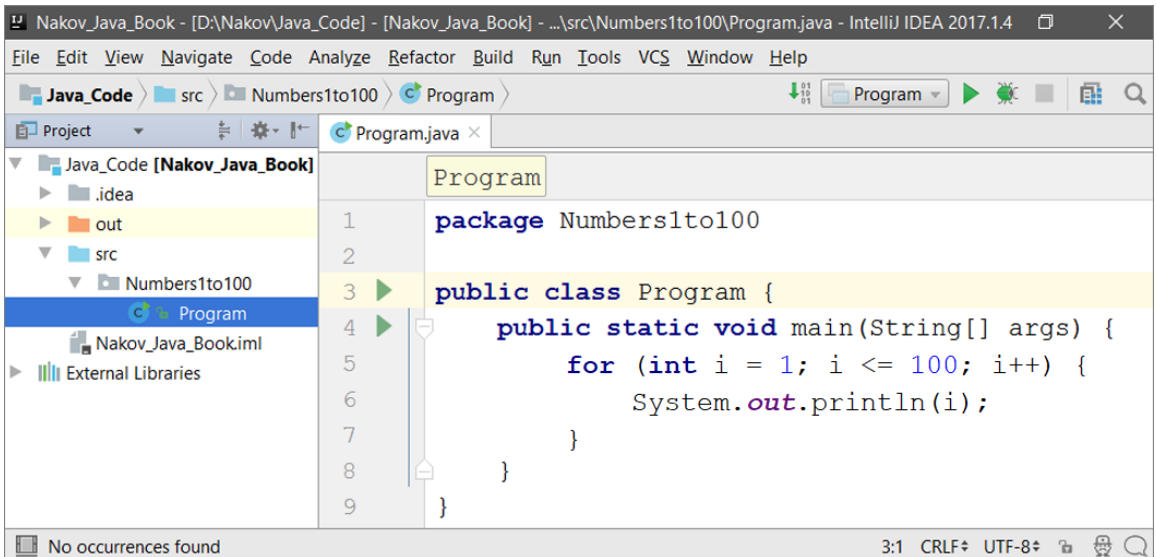
In most cases, a **for** loop is run from **1** to **n** times (for example from 1 to 10). The purpose of the loop is to pass sequentially through the numbers 1, 2, 3, ..., n, and for each number to perform a particular action. In the example above, the **i** variable takes values from 1 to 10, and the current value is printed in the body of the loop. The loop repeats 10 times, and each of these repetitions is called an "**iteration**".

## Problem: Numbers from 1 to 100

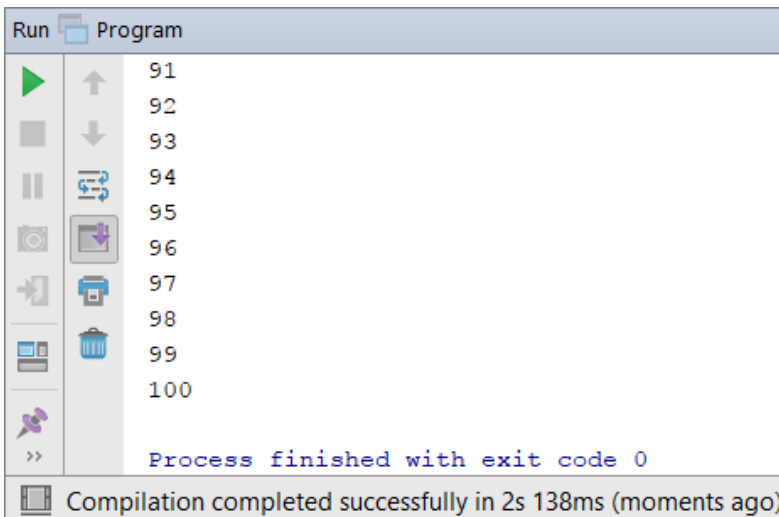
Write a program that **prints the numbers from 1 to 100**. The program does not accept input and prints the numbers from 1 to 100 sequentially, each on a separate line.

## Hints and Guidelines

We can solve the problem using a **for loop**, which will assign values from 1 to 100 to the variable **i** and print the numbers in the body of the loop:



Start the program with [Shift+F10] and test it:



## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#0>.

You should get **100 points** (fully accurate solution).

## Code Snippet for The **for** Loop in IntelliJ IDEA

In software development, we regularly need to write loops dozens of times a day. That is why in most integrated development environments (IDE), there are **code snippets** for

writing loops. One such example is the snippet for the **for** loop in IntelliJ IDEA. Write down **fori** in the Java code editor in IntelliJ IDEA and press [Enter]. IntelliJ IDEA will run the snippet, and you will see the entire **for** loop written down. All you have to do now is to add the values.



Try it yourself to master using the code snippet for the **for** loop in IntelliJ IDEA.

## Problem: Numbers Ending in 7

Write a program that finds all numbers within the range [1 ... 1000] that ends with 7.

### Hints and Guidelines

We can solve the problem by combining a **for** loop, which will go through the numbers from 1 to 1000, and a **condition** to check if they end with 7. There are other solutions too, but let's solve the problem using a **loop + condition**:

```
for (int i = 1; i <= 1000; i++) {
    if (i % 10 == 7) {
        // TODO: Print i
    }
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#1>.

## Problem: Latin Letters

Write a program that prints the letters from the Latin alphabet: a, b, c, ..., z.

### Hints and Guidelines

It is good to know that the **for** loops don't work only with numbers. We can solve the problem by running a **for** loop that passes sequentially through all letters in the Latin alphabet:

```
System.out.print("Latin alphabet: ");
for (char letter = 'a'; letter <= 'z'; letter++) {
    System.out.print(" " + letter);
}
System.out.println();
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#2>.

## Problem: Sum Numbers

Write a program that reads **n** integers from the console and calculates the sum.

- The first line of the input is an integer **n** denoting the number of integers that will be summed.
- The following **n** lines consist of one integer.
- Calculate the sum and print the result.

### Sample Input and Output

| Input         | Output |
|---------------|--------|
| 2<br>10<br>20 | 30     |

| Input                  | Output |
|------------------------|--------|
| 3<br>-10<br>-20<br>-30 | -60    |

| Input                     | Output |
|---------------------------|--------|
| 4<br>45<br>-20<br>7<br>11 | 43     |

| Input    | Output |
|----------|--------|
| 1<br>999 | 999    |

| Input | Output |
|-------|--------|
| 0     | 0      |

### Hints and Guidelines

We can solve the problem by calculating the sum of numbers the following way:

- We read the input number **n**.
- Initially, we start with **sum = 0**.
- We run a loop from 1 to **n**, and on each iteration, we read the input number **num** and add it to the **sum**.
- Finally, we print the calculated result **sum**.

Below you can find the source code for the solution:

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
int sum = 0;

for (int i = 0; i < numbers; i++) {
    int number = Integer.parseInt(scanner.nextLine());
    sum += number;
}

System.out.println(sum);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#3>.

## Problem: Max Number

Write a program that inputs **n integers** ( $n > 0$ ) and finds the **Max Number** (the largest number) among them. The first line of the input specifies the number of integers **n**. The following **n** lines consist of one integer. Examples:

### Sample Input and Output

| Input          | Output |
|----------------|--------|
| 2<br>100<br>99 | 100    |

| Input                 | Output |
|-----------------------|--------|
| 3<br>-10<br>20<br>-30 | 20     |

| Input                     | Output |
|---------------------------|--------|
| 4<br>45<br>-20<br>7<br>99 | 99     |

| Input    | Output |
|----------|--------|
| 1<br>999 | 999    |

| Input         | Output |
|---------------|--------|
| 2<br>-1<br>-2 | -1     |

### Hints and Guidelines

We enter the first number **n** (the number of integers to be entered). We assign the current maximum **max** an initial neutral value, for example **-100000** (or **Integer.MIN\_VALUE**). Using a **for loop** which iterates **n-1** times, we read one integer **num** on each iteration. If the read number **num** is higher than the current maximum **max**, we assign the value of **num** to the variable **max**. Finally, **max** will contain the highest number, and we print it on the console.

```
Scanner scanner = new Scanner(System.in);

System.out.print("n = ");
int n = scanner.nextInt();

System.out.println("Enter the numbers: ");
int max = Integer.MIN_VALUE;

for (int i = 1; i <= n; i++) {
    int num = scanner.nextInt();

    if (num > max) {
        max = num;
    }
}

System.out.println("max = " + max);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#4>.

### Problem: Min Number

Write a program that inputs **n** integers ( $n > 0$ ) and finds the **Min Number** among them. The first line of the input specifies the number of integers **n**. The following **n** lines consist of one integer. Examples:

#### Sample Input and Output

| Input | Output | Input | Output | Input | Output |
|-------|--------|-------|--------|-------|--------|
| 2     | 99     | 3     | -30    | 4     | -20    |
| 100   |        | -10   |        | 45    |        |
| 99    |        | 20    |        | -20   |        |
|       |        | -30   |        | 7     |        |
|       |        |       |        | 99    |        |

### Hints and Guidelines

The problem is identical to the previous one, but this time we will start with another neutral starting value 100000000000000 (or **Integer.MAX\_VALUE**).

```
Scanner scanner = new Scanner(System.in);

System.out.print("n = ");
int n = scanner.nextInt();

System.out.println("Enter the numbers: ");
int min = Integer.MAX_VALUE;

for (int i = 1; i <= n; i++) {
    int num = scanner.nextInt();

    if (num < min) {
        min = num;
    }
}

System.out.println("min = " + min);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#5>.



## Problem: Left and Right Sum

Write a program that inputs  $2 * n$  integers and checks if the sum of the first  $n$  numbers (left sum) is equal to the sum of the second  $n$  numbers (right sum). If the sums are equal, print "Yes" + sum, if not, print "No" + difference. Calculate the difference as a positive number (absolute value). The format of the output must be identical to the examples below.

### Sample Input and Output

| Input | Output         | Input | Output       |
|-------|----------------|-------|--------------|
| 2     | Yes, sum = 100 | 2     | No, diff = 1 |
| 10    |                | 90    |              |
| 90    |                | 9     |              |
| 60    |                | 50    |              |
| 40    |                | 50    |              |

### Hints and Guidelines

Input the number  $n$ , after that the first  $n$  numbers (left half) and sum them up. Input the second  $n$  numbers (right half) and find their sum. We calculate the difference between the sums by absolute value: `Math.abs(leftSum - rightSum)`. If the difference is 0, print "Yes" + sum, otherwise print "No" + difference.

```
Scanner scanner = new Scanner(System.in);

System.out.print("n = ");
int n = scanner.nextInt();
System.out.println("Enter the numbers: ");

int leftSum = 0;
int rightSum = 0;

for (int i = 1; i <= n; i++) {
    leftSum = leftSum + scanner.nextInt();
}

//TODO: Read and calculate the rightSum

if (leftSum == rightSum) {
    System.out.println("Yes, sum = " + leftSum);
} else {
    int difference = Math.abs(leftSum - rightSum);
    System.out.println("No, diff = " + difference);
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#6>.

## Problem: Odd Even Sum

Write a program that inputs **n integers** and checks if the **sum of numbers on even positions** is equal to the **sum of numbers on odd positions**. If the sums are equal, print **"Yes" + sum**, otherwise print **"No" + difference**. Calculate the difference as a positive number (absolute value). The format of the output must be identical to the examples below.

### Sample Input and Output

| Input                     | Output          |
|---------------------------|-----------------|
| 4<br>10<br>50<br>60<br>20 | Yes<br>Sum = 70 |

| Input                  | Output         |
|------------------------|----------------|
| 4<br>3<br>5<br>1<br>-2 | No<br>Diff = 1 |

| Input            | Output         |
|------------------|----------------|
| 3<br>5<br>8<br>1 | No<br>Diff = 2 |

### Hints and Guidelines

We input the numbers one by one and calculate the two **sums** (the numbers on **even** positions and the numbers on **odd** positions). Identically to the previous problem, we calculate the absolute value of the difference and print the result (**"Yes" + sum** if the difference equals 0 or **"No" + difference** in every other case).

```
Scanner scanner = new Scanner(System.in);
System.out.print("n= ");
int n = Integer.parseInt(scanner.nextLine());
System.out.println("Enter the numbers:");

int oddSum = 0;
int evenSum = 0;
for (int i = 1; i <= n; i++) {
    int element = Integer.parseInt(scanner.nextLine());

    if (i % 2 == 0) {
        evenSum += element;
    } else {
        oddSum += element;
    }
}

// TODO: Print the sum/difference
```

**+=** is a **combined assignment operator** that is used frequently. It adds the value of the right to the left operand and then assigns the result to the left operand. **C += A** is equivalent to **C = C + A**.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#7>.

## Problem: Vowels Sum

Write a program that inputs **text** (string), calculates and prints the **sum of the values of vowels** according to the table below:

| a | e | i | o | u |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |

## Sample Input and Output

| Input | Output               |
|-------|----------------------|
| hello | 6<br>(e+o = 2+4 = 6) |
| hi    | 3<br>(i = 3)         |

| Input  | Output                   |
|--------|--------------------------|
| bamboo | 9<br>(a+o+o = 1+4+4 = 9) |
| beer   | 4<br>(e+e = 2+2 = 4)     |

## Hints and Guidelines

We read the input text **s**, after that, we null the sum and run a loop from **0** to **s.length() - 1** (text length -1). We check every letter **s.charAt(i)** if it is a vowel and add its value to the sum.

```
Scanner scanner = new Scanner(System.in);
System.out.print("n= ");
String s = scanner.nextLine();
```

```
int sum = 0;
for (int i = 0; i < s.length(); i++) {
    if (s.charAt(i) == 'a') {
        sum += 1;
    } else if (s.charAt(i) == 'e') {
        sum += 2;
    } else if (s.charAt(i) == 'i') {
        sum += 3;
```

```

    } else if (s.charAt(i) == 'o') {
        sum += 4;
    } else if (s.charAt(i) == 'u') {
        sum += 5;
    }
}

System.out.println("Vowels sum = " + sum);

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#8>.

## What Have We Learned from This Chapter?

We can repeat a block of code using **for** loop:

```

for (int i = 1; i <= 10; i++) {
    System.out.println("i= " + i);
}

```

We can read a series of **n** numbers from the console:

```

Scanner scanner = new Scanner(System.in);
System.out.print("n= ");
int n = Integer.parseInt(scanner.nextLine());
for (int i = 1; i <= n; i++) {
    int num = Integer.parseInt(scanner.nextLine());
}

```

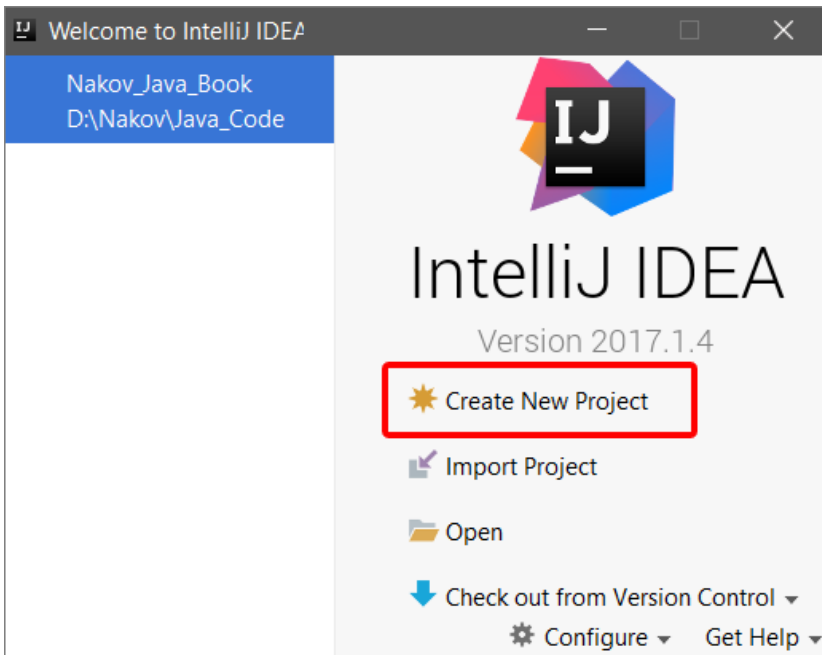
## Problems: Loops

After we had acquainted with the loops, it is time to **solidify what we have learned by practicing it**, by writing a lot of code. Let us solve some problems for exercise.

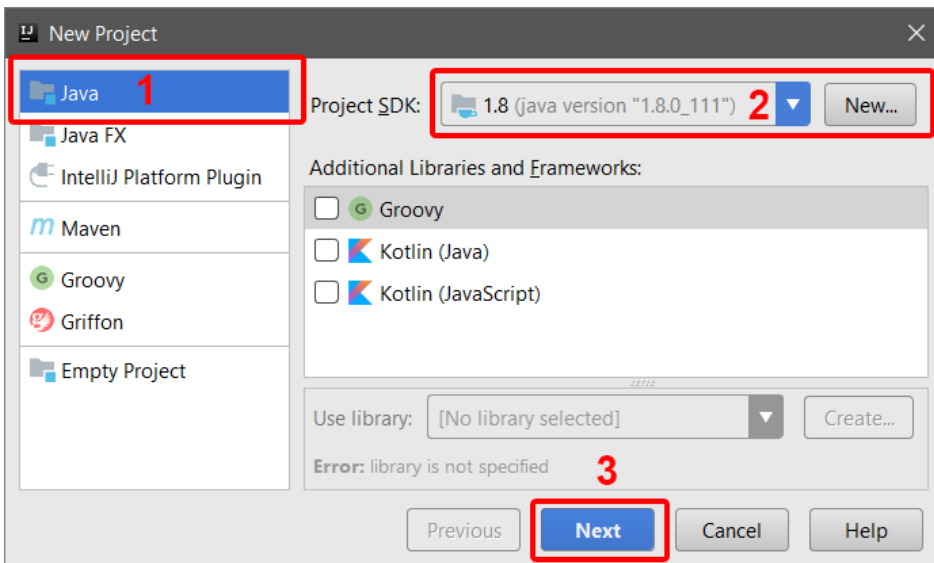
## Creating a new project in IntelliJ IDEA

We start by creating a new project in IntelliJ IDEA to better organize our solutions for the exercise. For each problem, we will create a separate class.

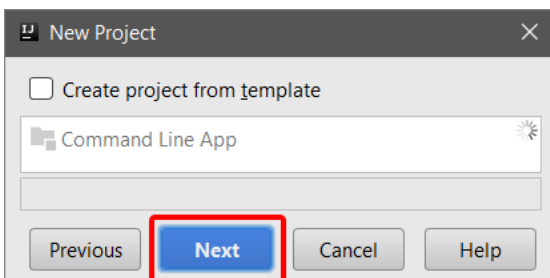
Start IntelliJ IDEA and select: **[Create New Project]**.



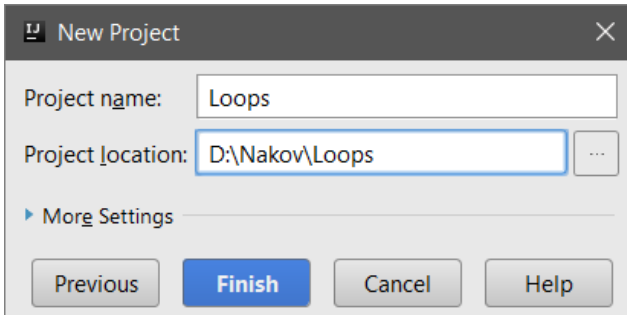
After that, on the left side of the window, select Java. If the **PROJECT SDK:** field is empty, choose the path to the Java **JDK** installation directory and click on the button **[Next]**.



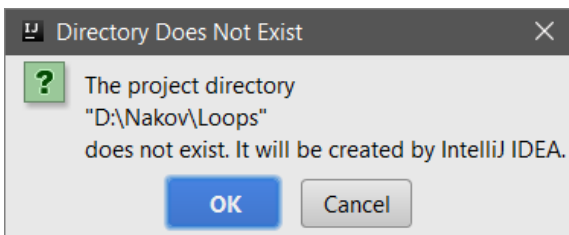
In the next window, we again click on the button **[Next]**.



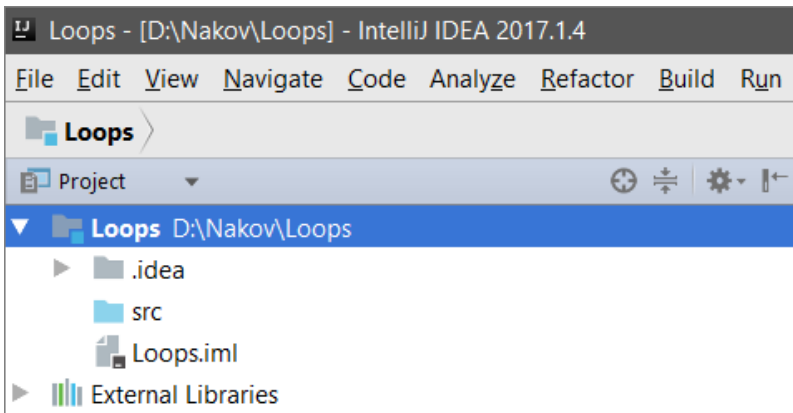
We name the project (Project name:) **Loops** and select the directory where the project will be saved (Project location:). After that, we click on the button **[Finish]**.



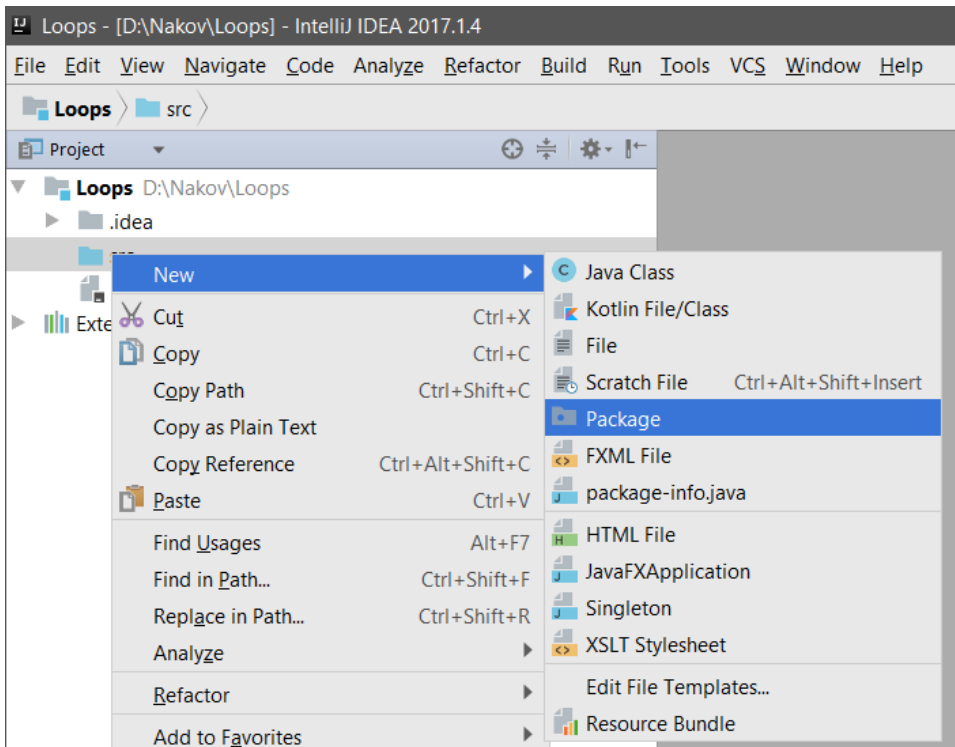
In the next window, we click on the **[OK]** button to create a **Project location** if the directory doesn't exist.



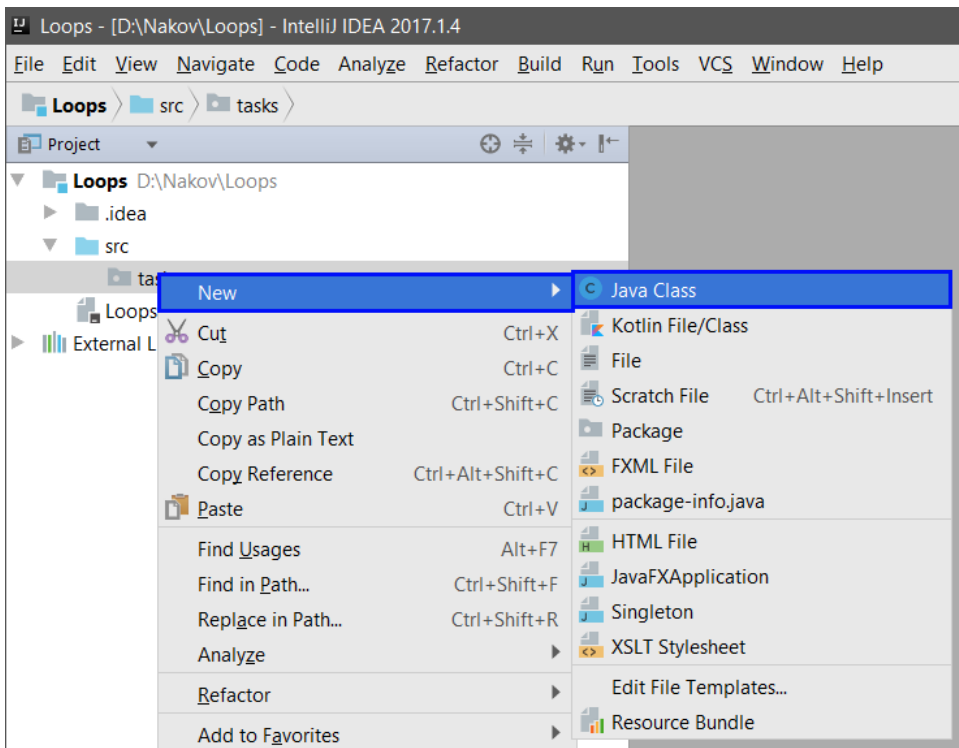
We created a new project. If we click on the triangle in front of **Loops** on the left side of the window, the structure of the project itself will open.

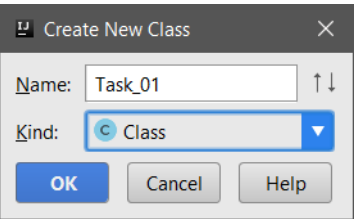


To create a **Package** in our project, which contains all the classes with the solutions from the exercise, right-click with the mouse on **src**, then select **New** -> **Package** and name it **problems** (with a small first letter).

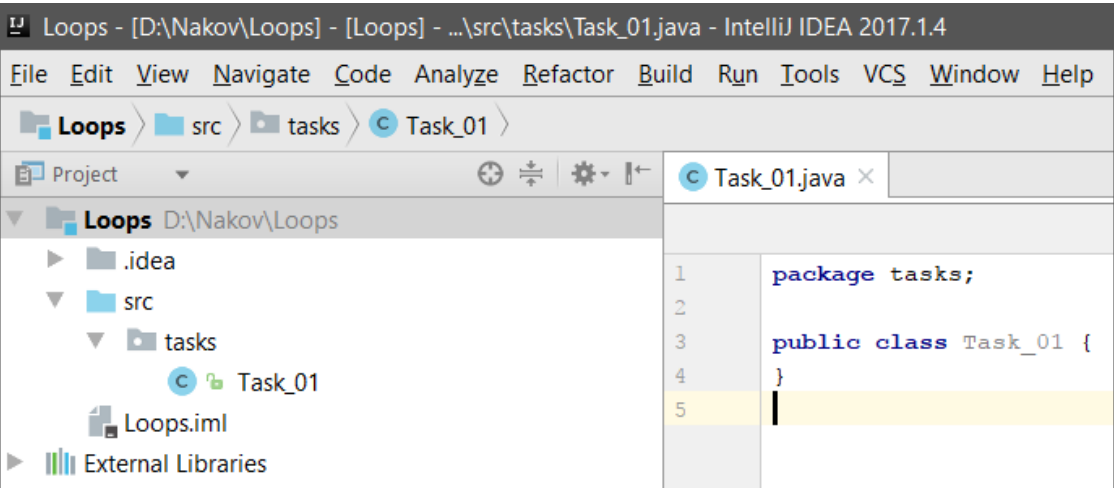


To create a new class for the first problem, right-click with the mouse on **problems**, select **New -> Java Class**, and name it **problem\_01** (with a capital letter).





After these changes, the structure of the project should look like this:



For each subsequent exercise, we will create a new class in the way described above.

**Problem: Half Sum Element**

Write a program that inputs **n integers** and checks if there is a number among them that is equal to the sum of all others. If there is such an element, print **"Yes" + its value**. Otherwise, print **"No" + the difference between the largest element and the sum of the rest**, as an absolute value.

**Sample Input and Output**

| Input                                 | Output          | Comments                      | Input             | Output         | Comments          |
|---------------------------------------|-----------------|-------------------------------|-------------------|----------------|-------------------|
| 7<br>3<br>4<br>1<br>1<br>2<br>12<br>1 | Yes<br>Sum = 12 | 3 + 4 + 1 + 2<br>+ 1 + 1 = 12 | 3<br>1<br>1<br>10 | No<br>Diff = 8 | 10 - (1 + 1)  = 8 |



| Input            | Output         |
|------------------|----------------|
| 3<br>1<br>1<br>1 | No<br>Diff = 1 |

| Input            | Output         |
|------------------|----------------|
| 3<br>5<br>5<br>1 | No<br>Diff = 1 |

| Input                 | Output         |
|-----------------------|----------------|
| 4<br>6<br>1<br>2<br>3 | Yes<br>Sum = 6 |

## Hints and Guidelines

We must calculate the **sum** of all elements, find the **largest** of them and check the condition.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#9>.

## Problem: Odd / Even position

Write a program that reads **n numbers** and calculates the **sum**, the **min**, and **max** values of the numbers on **even** and **odd** positions (counted from 1). If there is no min / max element, print "No".

## Sample Input and Output

| Input                           | Output                                                                       |
|---------------------------------|------------------------------------------------------------------------------|
| 6<br>2<br>3<br>5<br>4<br>2<br>1 | OddSum=9,<br>OddMin=2,<br>OddMax=5,<br>EvenSum=8,<br>EvenMin=1,<br>EvenMax=4 |

| Input            | Output                                                                                      |
|------------------|---------------------------------------------------------------------------------------------|
| 2<br>1.5<br>-2.5 | OddSum=1.5,<br>OddMin=1.5,<br>OddMax=1.5,<br>EvenSum=-2.5,<br>EvenMin=-2.5,<br>EvenMax=-2.5 |

| Input  | Output                                                                         |
|--------|--------------------------------------------------------------------------------|
| 1<br>1 | OddSum=1,<br>OddMin=1,<br>OddMax=1,<br>EvenSum=0,<br>EvenMin=No,<br>EvenMax=No |

| Input               | Output                                                                             |
|---------------------|------------------------------------------------------------------------------------|
| 3<br>-1<br>-2<br>-3 | OddSum=-4,<br>OddMin=-3,<br>OddMax=-1,<br>EvenSum=-2,<br>EvenMin=-2,<br>EvenMax=-2 |

| Input   | Output                                                                            |
|---------|-----------------------------------------------------------------------------------|
| 1<br>-5 | OddSum=-5,<br>OddMin=-5,<br>OddMax=-5,<br>EvenSum=0,<br>EvenMin=No,<br>EvenMax=No |

| Input | Output                                                                           |
|-------|----------------------------------------------------------------------------------|
| 0     | OddSum=0,<br>OddMin=No,<br>OddMax=No,<br>EvenSum=0,<br>EvenMin=No,<br>EvenMax=No |

## Hints and Guidelines

The problem combines several previous problems: finding the **min**, **max**, and the **sum**, as well as processing elements on **even and odd positions**. Check your solutions to the previous problems again.

In the current problem, it is better to work with **fractions** (not integers). The sum, the min, and the max value will also be fractions. We have to use a **neutral starting value** in finding the min / max value, for example **10000.0** and **-10000.0**. If the result is equal to the neutral value, we will print **"No"**.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#10>.

## Problem: Equal Pairs

There are  $2 * n$  numbers. The first and the second form a **pair**, the third and the fourth number also, and so on. Each pair has a **value** – the sum of its numbers. Write a program that checks **if all pairs have equal values**.

If the value is the same, print **"Yes, value=..."** + the **value** otherwise, print the **maximum difference** between two consecutive pairs in the following format - **"No, maxdiff=..."** + the **maximum difference**.

The input consists of number **n**, followed by  $2*n$  integers, one per line.

## Sample Input and Output

| Input                            | Output       | Comments                           | Input                 | Output        | Comments                                                   |
|----------------------------------|--------------|------------------------------------|-----------------------|---------------|------------------------------------------------------------|
| 3<br>1<br>2<br>0<br>3<br>4<br>-1 | Yes, value=3 | values = {3, 3, 3}<br>equal values | 2<br>1<br>2<br>2<br>2 | No, maxdiff=1 | values = {3, 4}<br>differences = {1}<br>max difference = 1 |

| Input                   | Output        | Comments                                                    | Input       | Output        | Comments                                   |
|-------------------------|---------------|-------------------------------------------------------------|-------------|---------------|--------------------------------------------|
| 2<br>-1<br>2<br>0<br>-1 | No, maxdiff=2 | values = {1, -1}<br>differences = {2}<br>max difference = 2 | 1<br>5<br>5 | Yes, value=10 | values = {10}<br>one value<br>equal values |

| Input | Output        | Input | Output        |
|-------|---------------|-------|---------------|
| 2     | Yes, value=-1 | 4     | No, maxdiff=4 |
| -1    |               | 1     |               |
| 0     |               | 1     |               |
| 0     |               | 3     |               |
| -1    |               | 1     |               |
|       |               | 2     |               |
|       |               | 2     |               |
|       |               | 0     |               |
|       |               | 0     |               |

Hints and Guidelines

We read the input numbers **in pairs**. For each pair, we calculate the **sum**. While reading the input pairs, for each pair, except the first one, we have to calculate **the difference with the previous one**. To do that, we need to store the sum of the previous pair in a separate variable. Finally, we find the **largest difference** between the two pairs. If it is 0, print **"Yes"** + the value, otherwise - **"No"** + the difference.

Testing in The Judge System

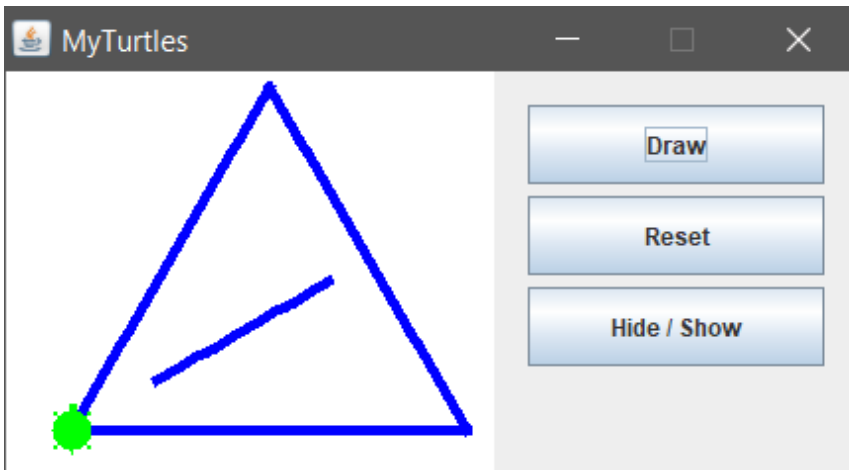
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/655#11>.

Problem: Graphical and Web Applications

In the current chapter, we learned about **loops** as construction in programming that allows us to repeat a given action or a group of actions many times. Now let us play with them. To do that, we will draw some figures that will consist of many repeating graphical elements. This time, we will not do it in the console but in a graphical environment using **"turtle graphics"**. It will be interesting. And it is not at all complicated. Try it!

Problem: Turtle Graphics GUI Application

The purpose of the following exercise is to play with a **drawing library**, also known as **"turtle graphics"**. We will build a graphical application (GUI App) in which we will: **draw different figures**, moving our **"turtle"** across the screen via operations like "move 100 positions forward", "turn 30 degrees to the right", "move 50 more positions forward". The application will look approximately like this:

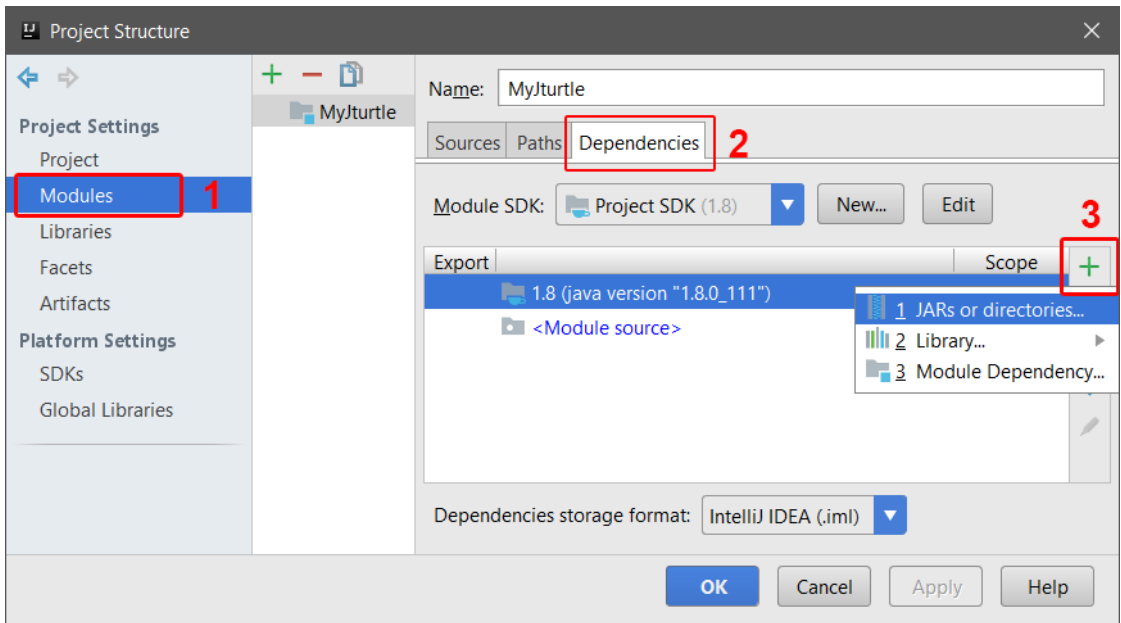


Let us get familiar with the concept of drawing "Turtle Graphics". Take a look at the following sources:

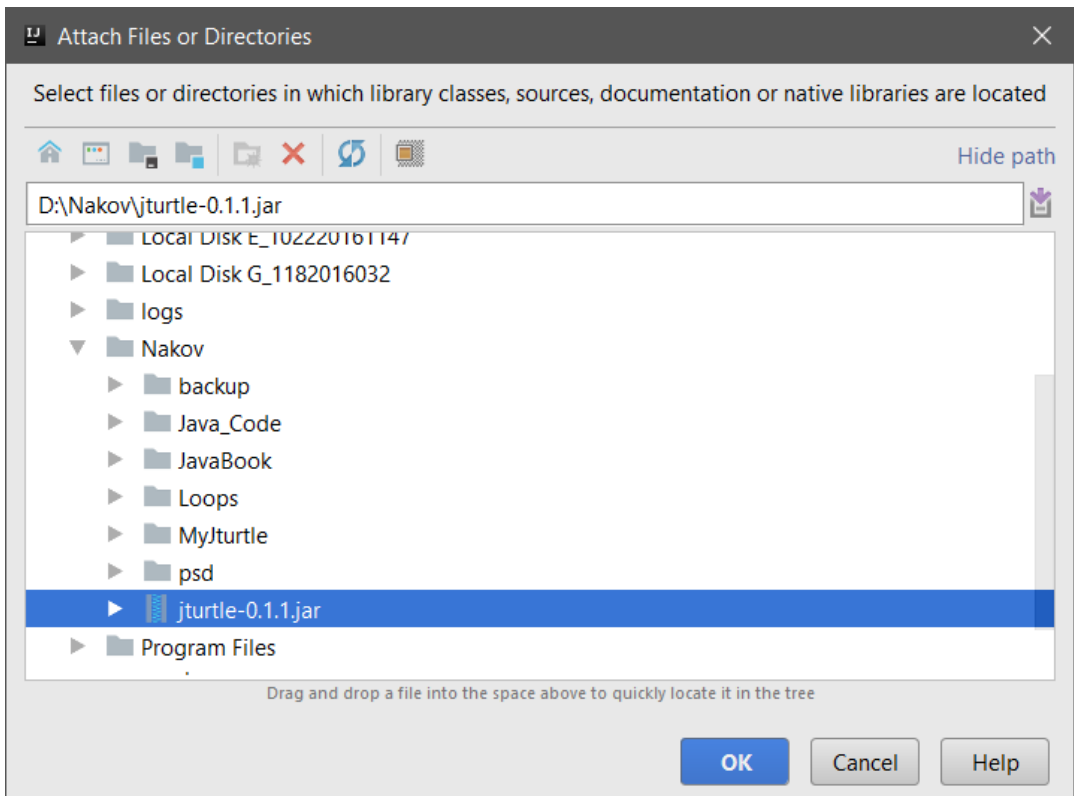
- Definition of "turtle graphics": <https://wiki.c2.com/?TurtleGraphics>
- Article on "turtle graphics" in Wikipedia – [https://en.wikipedia.org/wiki/Turtle\\_graphics](https://en.wikipedia.org/wiki/Turtle_graphics)
- Interactive online tool for drawing with a turtle – <https://blockly-games.appspot.com/turtle>

We will start by creating a new **Java project** in **IntelliJ IDEA**. Add a new package **app** (earlier in this chapter we described how to do it). Download **MyTurtle.java** and **jturtle-0.1.1.jar** from <https://github.com/SoftUni/Programming-Basics-Book-Java-EN/tree/master/assets/chapter-5-1-assets>. **MyTurtle.java** is a pre-written class, which will help us get acquainted faster with the library **jturtle-0.1.1.jar**, which runs the **turtle**. Using file explore, we copy **MyTurtle.java** into the directory **app** of the project. Then we need to add the external library **jturtle-0.1.1.jar** in our project. This is done as follows:

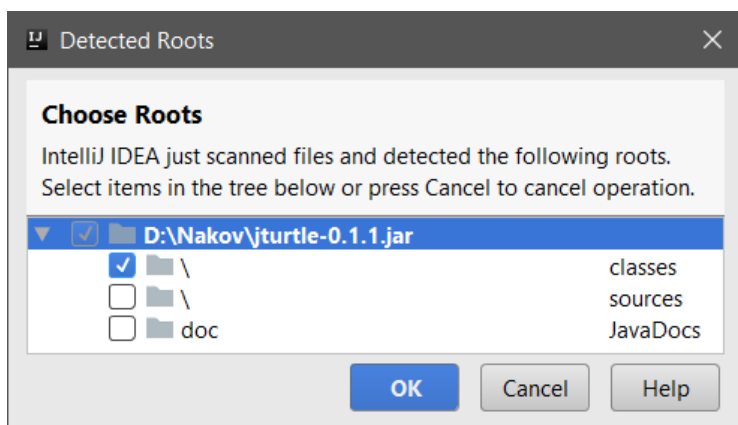
- Select **Project Structure** from the **File** menu (CTRL + SHIFT + ALT + S).
- Click on **Select Modules** in the left pane.
- Click on the **Dependencies** tab.
- Click '+' → **JARs or directories**.



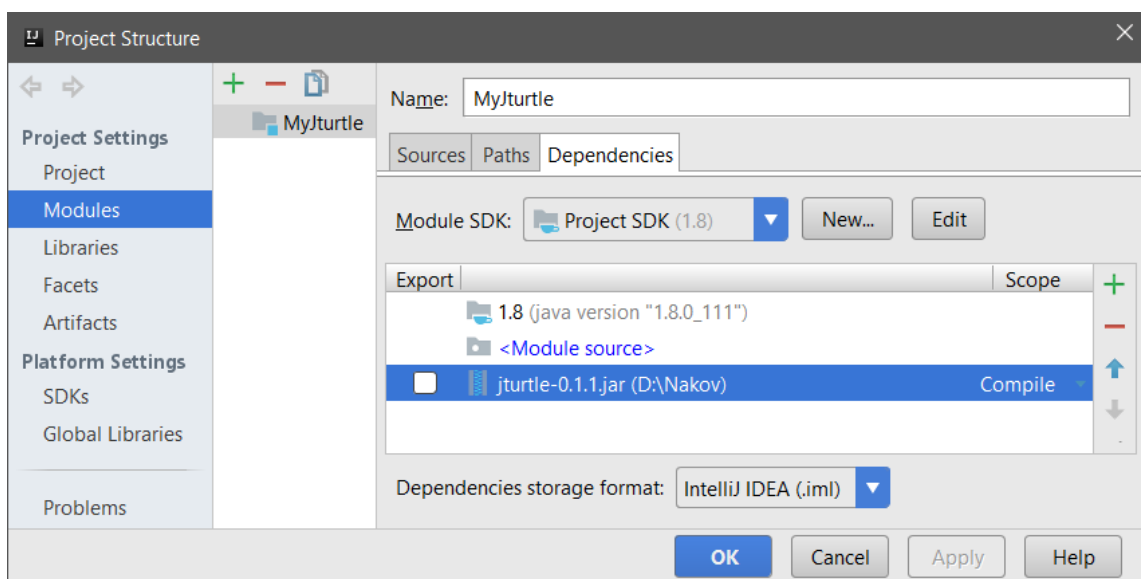
In the next window select the path to `jturtle-0.1.1.jar` and click [OK].



In the next window, mark the box **classes** and click [OK].



In the next window, click [OK], and we are ready to open **MyTurtle.java**.



## Main Methods in **Turtle** Class

These are the main actions in **Turtle** class with which we draw:

| Return value | Method and its description                                              |
|--------------|-------------------------------------------------------------------------|
| Turtle       | <b>back</b> (double distance) – moves the turtle back                   |
| Turtle       | <b>bk</b> (double distance) – works as <b>back</b> (double distance)    |
| Turtle       | <b>forward</b> (double distance) – moves the turtle forward             |
| Turtle       | <b>fd</b> (double distance) – works as <b>forward</b> (double distance) |

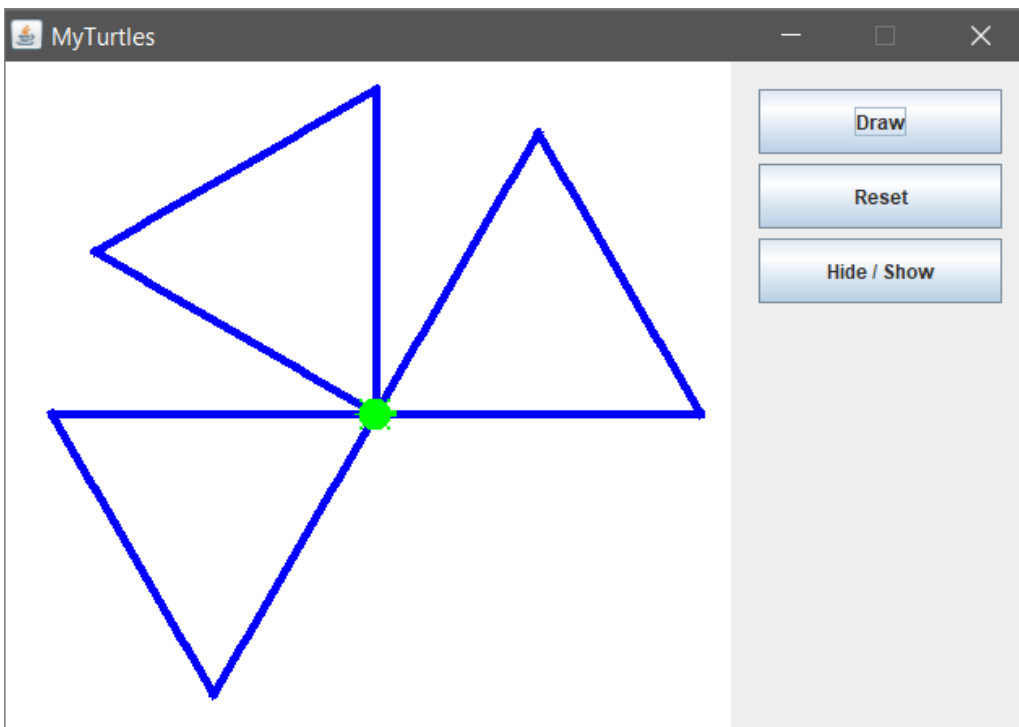
| Return value | Method and its description                                                                    |
|--------------|-----------------------------------------------------------------------------------------------|
| Turtle       | <b>hideTurtle()</b> – hides the turtle                                                        |
| Turtle       | <b>ht()</b> – works as <b>hideTurtle()</b>                                                    |
| Turtle       | <b>home()</b> – moves the turtle back to its original position pointing north                 |
| Turtle       | <b>left(double degrees)</b> – rotates the turtle to the left(in degrees)                      |
| Turtle       | <b>lt(double degrees)</b> – works as <b>left(double degrees)</b>                              |
| Turtle       | <b>penDown()</b> – attaches a pen to the turtle, which leaves a line when the turtle moves    |
| Turtle       | <b>pd()</b> – works as <b>penDown()</b>                                                       |
| Turtle       | <b>penUp()</b> – detaches the pen from the turtle                                             |
| Turtle       | <b>penWidth(int newWidth)</b> – sets the width of the pen                                     |
| Turtle       | <b>right(double degrees)</b> – rotates the turtle to the right(in degrees)                    |
| Turtle       | <b>rt(double degrees)</b> – works as <b>right(double degrees)</b>                             |
| Turtle       | <b>setColor(Color color)</b> – sets the color of the pen                                      |
| Turtle       | <b>setLineWidth(double lineWidth)</b> – sets the thickness of the line                        |
| Turtle       | <b>setPenColor(Color color)</b> – sets the drawing color of the pen                           |
| Turtle       | <b>setPos(double x, double y)</b> – moves the turtle to a new position with coordinates(x, y) |
| Turtle       | <b>showTurtle()</b> – shows the turtle                                                        |
| Turtle       | <b>st()</b> – works as <b>showTurtle()</b>                                                    |
| Turtle       | <b>speed(double newSpeed)</b> – sets the speed of the turtle                                  |

To create a method that draws a triangle in **MyTurtle.java** we find the empty method **drawTriangle()** and write the following code:

```
private void drawTriangle() {  
    turtle.showTurtle();  
    turtle.setPenColor(Color.blue);  
    turtle.pd();  
    turtle.rt(90);  
    turtle.fd(200);  
    turtle.lt(120);  
    turtle.fd(200);  
    turtle.lt(120);  
    turtle.fd(200);  
    turtle.rt(60);  
  
    shape = "";  
}
```

This code moves and rotates the turtle, which is initially in the center of the screen (in the middle of the shape) and draws an equilateral triangle. You can edit and play with it.

We start the application with [Shift + F10]. We test it if it works (press the [Draw] button several times):



Now we can change and make the code of the `turtle` more complicated by adding the following to our method `drawTriangle()`:

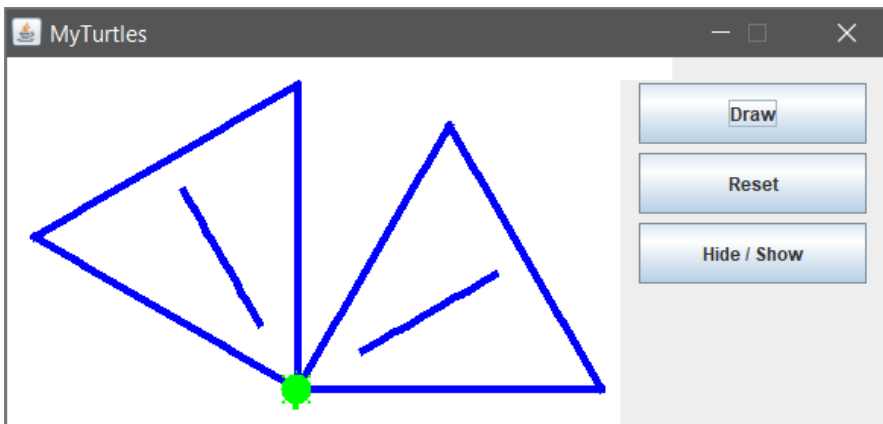


```
private void drawTriangle() {
    ...

    // Draw a line in a triangle
    turtle.rt(150);
    turtle.pu().fd(50);
    turtle.pd().fd(100);
    turtle.pu().bk(150);
    turtle.lt(150);
    turtle.pd();

    shape = "";
}
```

Again, **start** the application with [Shift + F10]. Test if the new program for the turtle works:



Now our turtle is drawing more complex shapes.

Let us write the code for the other two buttons: the purpose of the button [Reset] is to delete the drawn graphic and position the turtle in its starting position. We add the following code to the method **resetTurtle()**:

```
private void resetTurtle() {
    turtle.home();
    myTFrame.getPlayground().clear();
}
```

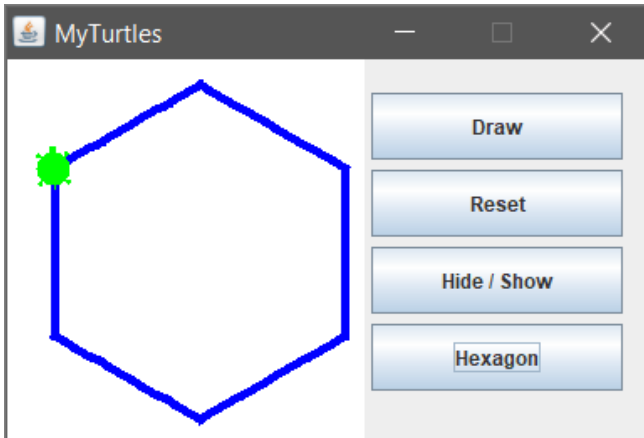
The purpose of the button [Hide / Show Turtle] is to show or hide the turtle. We add the following code to the method **showTurtle()**:

```
private void showTurtle() {
    if (turtle.isHidden()) {
        turtle.showTurtle();
    } else {
        turtle.hideTurtle();
    }
}
```

Again, **start** the application with [Shift + F10] and test whether both buttons work correctly.

## Problem: \* Draw a Hexagon with The Turtle

Add [Hexagon] button which draws a regular hexagon:



Hint:

Repeat 6 times the following in a loop:

- 60 degrees rotation.
- Forward step of 100.

## Adding a New Button for Drawing a Shape

To add a new button for drawing a new shape, we can trace the logic in **MyTurtle.java** for the button [Draw] and create a new button for drawing a Hexagon.

First, we add the new figure in **enum Shape**:

```
public enum Shape {
    Triangle,
    Hexagon
}
```

We create a new button and add **ActionListener**, which assigns to **shape** the new **enum** value:

```
JButton btn4 = new JButton("Hexagon");
btn4.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        shape = Shape.Hexagon.toString();
    }
});
```

Then add the new button in **bList**:

```
List<JButton> bList = new ArrayList<>();
bList.add(btn1);
bList.add(btn2);
bList.add(btn3);
bList.add(btn4);
```

Create a new method **drawHex()** through which the turtle draws a hexagon. Mandatory at the end of the method **shape = ""** because this line will prevent the repeated execution of our method!

```
private void drawHex() {
    // TODO: Create a method in which Turtle to Draw a Hexagon

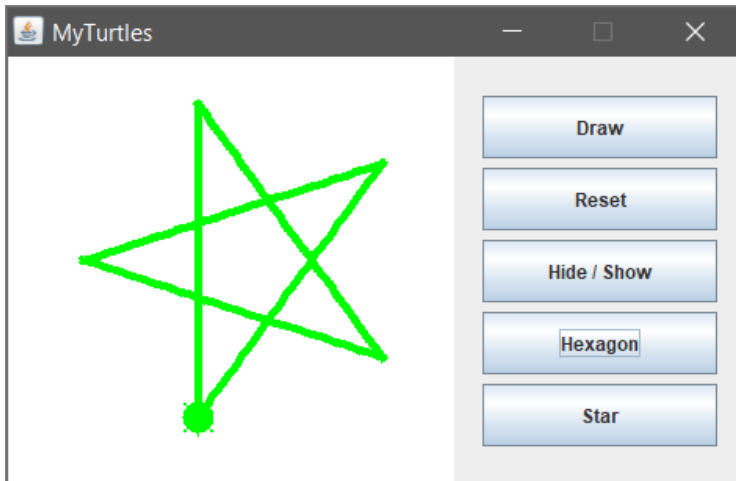
    shape = "";
}
```

In **Thread t** add **case "Hexagon"** to call the method **drawHex()**:

```
private final Thread t;
{
    t = new Thread(new Runnable() {
        public void run() {
            while (!Thread.interrupted()) {
                try {
                    switch (shape) {
                        case "Triangle":
                            drawTriangle();
                            break;
                        case "Hexagon":
                            drawHex();
                            break;
                        default:
                            break;
                    }
                    Thread.sleep(1000);
                } catch (Throwable t) {
                    t.printStackTrace();
                }
            }
        }
    });
    t.setDaemon(true);
    t.setName("Draw");
    t.start();
}
```

## Problem: \* Draw a Star with The Turtle

Add [Star] button, which draws a star with 5 vertices (star pentagon), as shown in the figure below:



Hint:

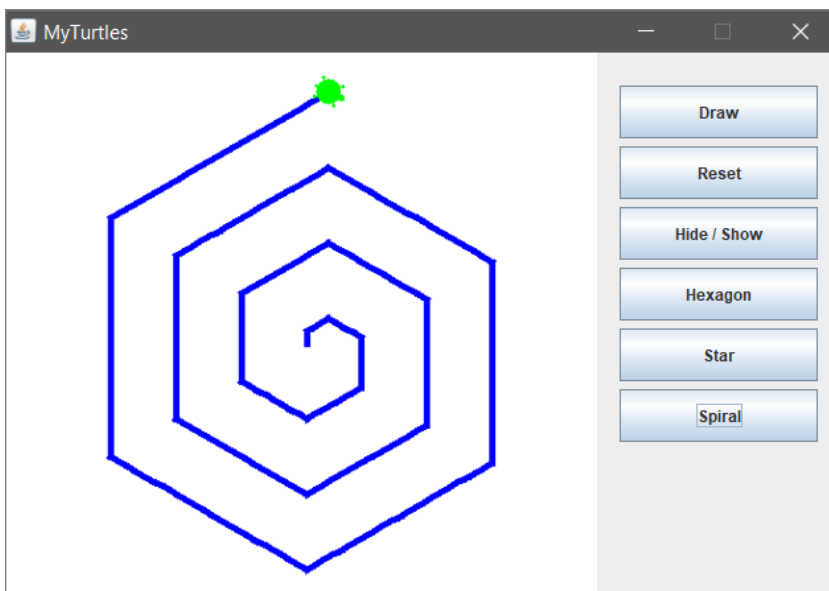
Change the color: `turtle.setPenColor(Color.green);`

Repeat 5 times the following in a loop:

- Forward step of 200.
- 144 degrees rotation.

## Problem: \* Draw a Spiral with The Turtle

Add a [Spiral] button, which draws a spiral with 20 vertices, as shown in the figure below:

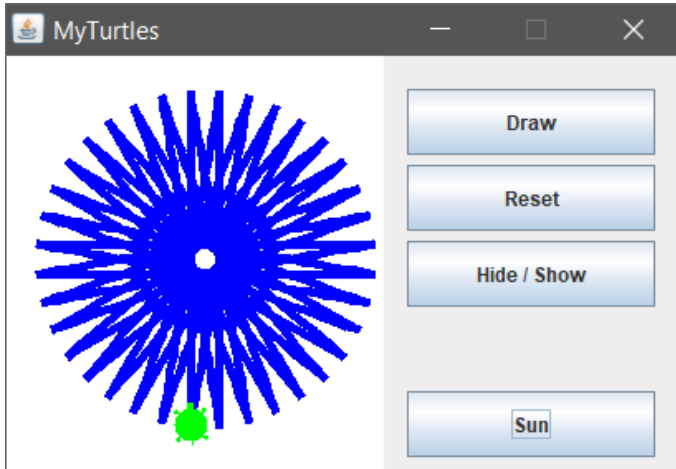


**Hint:**

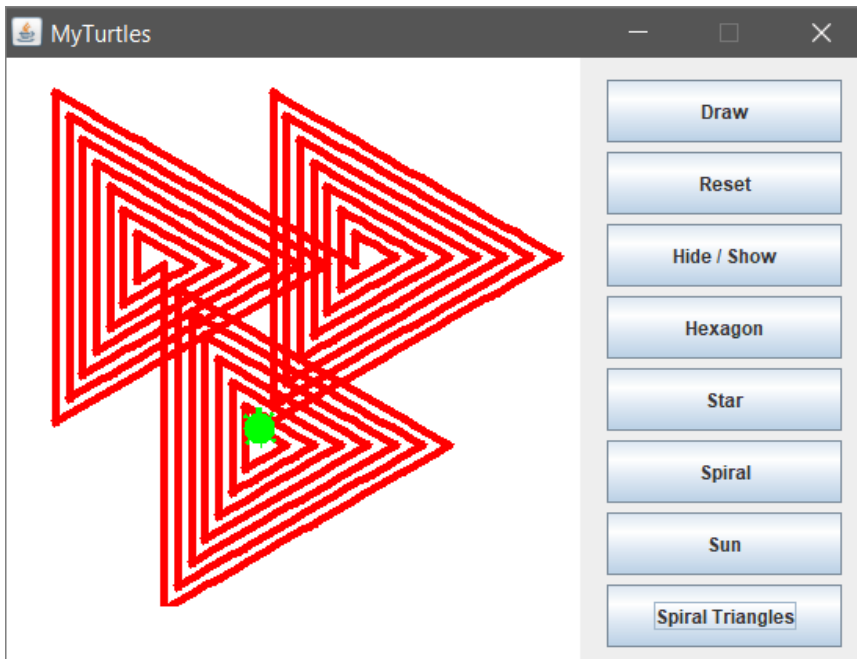
Draw in a loop by moving ahead and rotating. Gradually increase the length of the forward movement with each step and turn it 60 degrees.

**Problem: \* Draw a Sun with The Turtle**

Add [Sun] button, which draws a sun with 36 vertices, as shown in the figure below:

**Problem: \* Draw a Spiral Triangles with The Turtle**

Add [Spiral Triangle] button, which draws three triangles with 22 vertices each, as shown in the figure below:



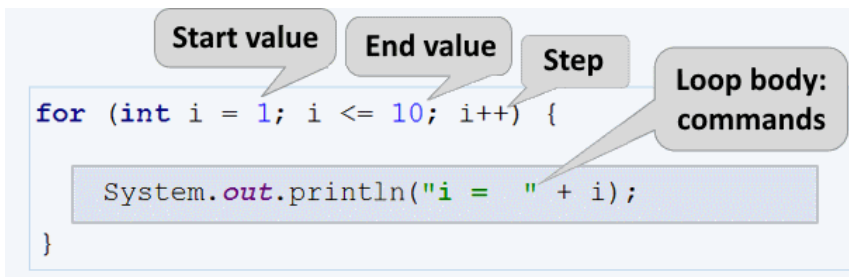
**Hint:**

Draw in a loop by moving forward and rotating. In each step, increase the length of the forward step by 10 and rotate 120 degrees. Repeat 3 times for the three triangles.

If you have a problem with the exercises above, ask for help in the official **SoftUni Reddit Forum**: <https://www.reddit.com/r/softuni/>.

## Chapter 5.2. Loops – Exam Problems

In the previous chapter, we learned how to execute a block of commands **more than once**. That's why we introduced **for loops** and we looked at some of its main use cases. This chapter aims to consolidate our knowledge by solving some more complex problems with loops, given at exams. We will show detailed solutions for some of the examples, and for the others, we will leave guidance only. Before we get to work it's best if we recall the construction of the **for loop**:



For loops consist of:

- Initialization block - executed only once at the beginning before the compiler checks the repeat condition for the first time. In this case, the variable counter (**int i**) is declared and its initial value is set.
- Condition for repeatability (**i <= 10**), ran once before each iteration.
- Counter update (**i++**) – this code is executed after each iteration.
- Loop body - contains an arbitrary block of source code.

## Exam Problems

Let's solve some loop problems from exams in SoftUni.

### Problem: Histogram

An **n** number of integer numbers are given in the range [1 ... 1000]. Some percent of them **p1** are below 200, percent **p2** are from 200 to 399, percent **p3** are from 400 to 599, percent **p4** are from 600 to 799 and the rest are **p5** percent above 800 (inclusive). Write a program that calculates and prints the percentages **p1**, **p2**, **p3**, **p4**, and **p5** on the console.

**Example:** we have **n = 20** and the numbers: 53, 7, 56, 180, 450, 920, 12, 7, 150, 250, 680, 2, 600, 200, 800, 799, 199, 46, 128, 65. We get the following distribution and visualization:

| Group | Numbers                        | Number count | Percent          |
|-------|--------------------------------|--------------|------------------|
| < 200 | 53, 7, 56, 180, 12, 7, 150, 2, | 12           | $p1 = 12 / 20 *$ |

| Group      |     | Numbers          | Number count | Percent                       |
|------------|-----|------------------|--------------|-------------------------------|
|            |     | 199, 46, 128, 65 |              | 100 = 60.00%                  |
| 200<br>399 | ... | 250, 200         | 2            | p2 = 2 / 20 *<br>100 = 10.00% |
| 400<br>599 | ... | 450              | 1            | p3 = 1 / 20 *<br>100 = 5.00%  |
| 600<br>799 | ... | 680, 600, 799    | 3            | p4 = 3 / 20 *<br>100 = 15.00% |
| ≥ 800      |     | 920, 800         | 2            | p5 = 2 / 20 *<br>100 = 10.00% |

Input Data

In the first line of the input, there must be an integer number **n** ( $1 \leq n \leq 1000$ ), which represents how many lines of numbers we should enter. In each of the following **n lines**, we have **one integer number** in the range **[1 ... 1000]** – those are the numbers we use to calculate the histogram.

Output Data

Print on the console **a histogram of 5 lines**, each line should contain a percentage from 0% to 100%, and format it up to two digits after the decimal point (for example 25.00%, 66.67%, 57.14%).

Sample Input and Output

| Input | Output | Input | Output | Input | Output |
|-------|--------|-------|--------|-------|--------|
| 9     | 33.33% | 14    | 57.14% | 7     | 14.29% |
| 367   | 33.33% | 53    | 14.29% | 800   | 28.57% |
| 99    | 11.11% | 7     | 7.14%  | 801   | 14.29% |
| 200   | 11.11% | 56    | 14.29% | 250   | 14.29% |
| 799   | 11.11% | 180   | 7.14%  | 199   | 28.57% |
| 999   |        | 450   |        | 399   |        |
| 333   |        | 920   |        | 599   |        |
| 555   |        | 12    |        | 799   |        |
| 111   |        | 7     |        |       |        |
| 9     |        | 150   |        |       |        |
|       |        | 250   |        |       |        |
|       |        | 680   |        |       |        |
|       |        | 2     |        |       |        |





```
// Variables to keep the count of numbers by groups
```

```
int cntP1 = 0;
int cntP2 = 0;
int cntP3 = 0;
int cntP4 = 0;
int cntP5 = 0;
```

In the variable **n**, we will store the count of numbers, that we are going to read from the console. We choose **int type**, because the condition states, that **n is an integer number** in the range from 1 to 1000. For the variables in which we will store the percentages, we chose the **double type**, because **they are not always expected to be integer numbers**. Additionally, we declare the variables **cntP1**, **cntP2**, etc, in which we will store the count of numbers in the corresponding group, and for them, we choose the **int type** again.

Once we have declared the needed variables, we can proceed with reading the number **n** from the console:

```
Scanner scan = new Scanner(System.in);
n = Integer.parseInt(scan.nextLine());
```

### Processing The Input Data

To read and distribute each number to its corresponding group, we will use a **for** loop from **0** to **n** (count of numbers). Each iteration of the loop will read and distribute **only one** number (**currentNumber**) to its corresponding group. For the compiler to determine if a number belongs to a group, **we perform a check-in its range**. If so- we add to the count of its corresponding group (**cntP1**, **cntP2**, etc.) by one.

```
for (int i = 0; i < n; i++) {
    int currentNumber = Integer.parseInt(scan.nextLine());

    if (currentNumber < 200) {
        cntP1++;
    }else if (currentNumber >= 200 && currentNumber <= 399) {
        cntP2++;
    }else if (currentNumber >= 400 && currentNumber <= 599) {
        cntP3++;
    }else if (currentNumber >= 600 && currentNumber <= 799) {
        cntP4++;
    }else {
        cntP5++;
    }
}
```

After we determined how many numbers each group has, we can move on to calculating the percentage, which is the main point of the problem. For this, we'll use the following formula:

$(\text{group percentage}) = (\text{count of numbers in the group}) * 100 / (\text{count of all numbers})$

This formula is the program code looks like the following:

```
p1Percentage = cntP1 * 100 / n;
p2Percentage = cntP2 * 100 / n;
p3Percentage = cntP3 * 100 / n;
p4Percentage = cntP4 * 100 / n;
p5Percentage = cntP5 * 100 / n;
```

If we divide by **100** (**int** number type) instead of **100.0** (**double** number type), the so-called **integer division** will be performed and only the whole part of the division will be stored in the variable, which is not the desired result. For example:  $5 / 2 = 2$ , but  $5 / 2.0 = 2.5$ . Taking this in mind, the formula for the first number should look like this:

```
p1Percentage = cntP1 * 100.0 / n;
// TODO: calculate the other percentages
```

To make things even clearer, let's take a look at this example:

| Input | Output |
|-------|--------|
| 3     | 66.67% |
| 1     | 0.00%  |
| 2     | 0.00%  |
| 999   | 0.00%  |
|       | 33.33% |

In the case of **n = 3**. For the loop we have:

- **i = 0** - we read the number 1, which is less than 200 and falls in the first group (**p1**), and we increment the group counter (**cntP1**) by 1.
- **i = 1** - we read the number 2, which again falls in the first group (**p1**) and we increment the counter (**cntP1**) again by 1.
- **i = 2** - we read the number 999, which falls in the last group (**p5**), because it is greater than 800, and we increment the group counter (**cntP5**) by 1.

After reading the numbers in group **p1** we have 2 numbers, and in **p5** we have 1 number. We have **no numbers** in the other groups. Applying the above formula, we calculate the percentages of each group. If we multiply within the formula by **100**, instead of **100.0** we'll attain 66% for group **p1**, and 33% for group **p5** (there will be no fractional part).

## Final Output

All that remains is to print the results. The condition states, that the percentages must be accurate to **two digits after the decimal point**. We can achieve this by setting the format to **"%.2f%"** in the **String.format(...)** method:

```
System.out.println(String.format("%.2f%%", p1Percentage));
System.out.println(String.format("%.2f%%", p2Percentage));
System.out.println(String.format("%.2f%%", p3Percentage));
System.out.println(String.format("%.2f%%", p4Percentage));
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#0>.

## Problem: Smart Lilly

Lilly is **N** years old. For every **birthday** she receives a present. For each **odd** birthday (1, 3, 5, ..., n), she receives **toys**, and for every **even** birthday (2, 4, 6, ..., n), she receives **money**. For her **second birthday**, she received **10.00 USD**., and the number of money increases by **10.00 USD** for each following even birthday (2 -> 10, 4 -> 20, 6 -> 30, etc.). Over the years Lilly secretly saved her money. Her brother, in the years when Lilly received money, takes **1.00 USD** of it. Lilly has sold the toys, received over the years, each for **P USD**, and added the money to her savings. With the money, she wanted to buy a washing machine for **X USD**. Write a program that calculates how much money she has saved and if the amount is enough to buy a washing machine.

### Input Data

The console reads **3 numbers**, each on a separate line:

- Lilly's age – an integer number in the range of [1 ... 77].
- Price of the washing machine – a double number in the range of [1.00 ... 10 000.00].
- Unit price per toy – an integer number in the range of [0 ... 40].

### Output Data

Print on the console a single line:

- If Lilly's money is enough:
  - "Yes! {N}" – where **N** is the remaining money after the purchase.
- If the money is not enough:
  - "No! {M}" – where **M** is the insufficient amount.
- The numbers **N** and **M** must be formatted up to the second digit after the decimal point.

## Sample Input and Output

| Input              | Output        | Comments                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10<br>170.00<br>6  | Yes!<br>5.00  | For her first birthday, she receives a toy; 2nd -> 10 USD; 3rd -> toy; 4th -> 10 + 10 = 20 USD; 5th -> toy; 6th -> 20 + 10 = 30 USD; 7th -> toy; 8th -> 30 + 10 = 40 USD; 9th -> toy; 10th -> 40 + 10 = 50 USD.<br>She has saved -> 10 + 20 + 30 + 40 + 50 = 150 USD. She sold 5 toys for 6 USD each = 30 USD.<br>Her brother took 1 USD 5 times = 5 USD. Remaining amount -> 150 + 30 - 5 = 175 USD. 175 >= 170 (price of the washing machine) She was able to buy it and has a remaining amount of money 175-170 = 5 USD. |
| 21<br>1570.98<br>3 | No!<br>997.98 | She has saved 550 USD. She sold 11 toys 3 USD each = 33 USD. Her brother has taken for 10 years 1 USD each year = 10USD. The remainder 550 + 33 - 10 = 573 USD.<br>573 < 1570.98 - she did not manage to buy a washing machine. The insufficiency amount is 1570.98-573 = 997.98 USD.                                                                                                                                                                                                                                       |

## Hints and Guidelines

The solution to this problem, like the previous one can be split into three parts – reading the input data, processing it, and printing the output.

```
Scanner scan = new Scanner(System.in);
```

```
int age = Integer.parseInt(scan.nextLine());
```

```
double priceOfWashingMachine = Double.parseDouble(scan.nextLine());
```

```
int presentPrice = Integer.parseInt(scan.nextLine());
```

We begin again by choosing the appropriate **data types** and names of the variables. Lilly's years (**age**) and the toy's unit price (**presentPrice**) are conditionally given to be **whole numbers**. That's why we'll use the **int** type. For the washing machine price (**priceOfWashingMachine**) we know, that it is a **real number** and we choose **double**. In the code above we **declare** and **initialize** (assign a value to) the variables.

```
int numberOfToys = 0;
```

```
int savedMoney = 0;
```

```
int moneyForBirthday = 10;
```

To solve the problem, we will need several auxiliary variables – for the **toy count** (**numberOfToys**), for the money saved (**savedMoney**), and for the money she received each birthday (**moneyForBirthday**). The initial value of **moneyForBirthday** is 10 because the description states the first sum Lilly got is 10 USD.

```

for (int currentYear = 1; currentYear <= age; currentYear++) {
    if (currentYear % 2 == 0) {
        savedMoney += (moneyForBirthday - 1);
        moneyForBirthday += 10;
    } else {
        numberOfToys++;
    }
}

```

Using a **for loop** we iterate through each of Lilly's birthdays. When the counter in our loop (which equates to Lilly's age) is an **even number**, this means that Lilly **receives money** and we add this money to her total savings accordingly. Simultaneously we **subtract 1 USD** - the money her brother takes. Then we **increase** the variable value **moneyForBirthday**, i.e. we increase by 10 the sum she will get on her next birthday. The opposite of that, when the counter (Lilly's age) is an **odd number**, we increase the **toy counter**. We do the parity check by **division with a remainder (%) by 2** - when the remainder is 0, the number is even, when we have a remainder of 1 - it is odd.

We also add the money from the sold toys to Lilly's savings:

```

savedMoney += numberOfToys * presentPrice;

```

Finally, we need to print the results, following the formatting specified in the condition, i.e. the sum must be **rounded up to 2 digits after the decimal point**:

```

System.out.println((savedMoney >= priceOfWashingMachine ?
    String.format("Yes! %.2f", (savedMoney - priceOfWashingMachine))
    : String.format("No! %.2f", (priceOfWashingMachine - savedMoney))));

```

In this case, we chose to use the **conditional operator (?:)** (also called a ternary operator), because it's shorter to write. The syntax is as follows: **operand1 ? operand2 : operand3**. The first operand needs to be **boolean type** (i.e. to return **true/false**). If **operand1** returns **true**, **operand2** will be executed, and if it returns **false** - **operand3** will be executed. In our case, we check if the **saved money** by Lilly is enough for the washing machine. If they are more or equal to the price of the washing machine, the check **savedMoney >= priceOfWashingMachine** will return **true** and will print out „Yes! ...“, and if they are less the result will be **false** and will print „No! ...“. Of course, instead of the ternary operand, we can use **if-else** checks.

More about the ternary operator: <https://www.programiz.com/java-programming/ternary-operator>

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#1>.

## Problem: Back to The Past

Ivan is **18 years old** and receives an inheritance, that consists of **X sum of money** and a time machine. He decides to **go back to the year 1800**, but he doesn't know if the money

is enough to live without working. Write a program that calculates if Ivan is going to have enough money, so he won't need to work until a particular year (inclusive). Assuming that for every even (1800, 1802, etc.) year he will spend 12 000 dollars. For every odd (1801, 1803 etc.) year he will spend  $12\,000 + 50 \cdot [\text{the age he reached in the given year}]$ .

### Input Data

The input is read from the console and contains exactly 2 lines:

- Inheritance money – a real number in the range of [1.00 ... 1 000 000.00].
- The year he needs to live to (inclusive) – a whole number in the range of [1801 ... 1900].

### Output Data

Print on the console 1 line. The sum must be formatted up to two symbols after the decimal point:

- If the money is enough:
  - „Yes! He will live a carefree life and will have {N} dollars left.“ – where N is the money, which he will remain with.
- If the money is NOT enough:
  - „He will need {M} dollars to survive.“ – where M is the amount that is NOT enough.

### Sample Input and Output

| Input             | Output                                                                 | Explanations                                                                                                                                                                                                                                                     |
|-------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 50000<br>1802     | Yes! He will live a carefree life and will have 13050.00 dollars left. | 1800 → even<br>→ Spends 12000 dollars<br>→ Remain $50000 - 12000 = 38000$<br>1801 → odd<br>→ Spends $12000 + 19 \cdot 50 = 12950$ dollars<br>→ Remaining $38000 - 12950 = 25050$<br>1802 → even<br>→ Spends 12000 dollars<br>→ Remaining $25050 - 12000 = 13050$ |
| 100000.15<br>1808 | He will need 12399.85 dollars to survive.                              | 1800 → even<br>→ Remaining $100000.15 - 12000 = 88000.15$<br>1801 → odd<br>→ Remaining $88000.15 - 12950 = 75050.15$                                                                                                                                             |

| Input | Output | Explanations                                                                              |
|-------|--------|-------------------------------------------------------------------------------------------|
|       |        | ...<br>1808 → <b>even</b> → $-399.85 - 12000 = -12399.85$<br>12399.85 insufficient amount |

## Hints and Guidelines

The method for solving this problem isn't much different from the last one, we begin by **declaring and initializing** the needed variables:

```
Scanner scan = new Scanner(System.in);
```

```
double heritage = Double.parseDouble(scan.nextLine());
int yearToLive = Integer.parseInt(scan.nextLine());
int years = 18;
```

The condition states, that Ivan is 18 years old, so when we declare the variable **years** we assign its initial value of **18**. The rest of the variables are read from the console.

```
for (int currentYear = 1800; currentYear <= yearToLive; currentYear++) {
    if (currentYear % 2 == 0) {
        heritage -= 12000;
    } else {
        heritage -= (12000 + 50 * years);
    }

    years++;
}
```

With the help from the **for** loop, we'll iterate through all the years. We begin with the year **1800** – the year that Ivan goes back to, and we get to the year he has to live to. In the loop, we check if the current year is **even** or **odd**. We achieve this by **division with a remainder (%)** by 2. If the year is **even**, we subtract from the inheritance (**heritage**) 12000, and if it's **odd**, we subtract from the inheritance (**heritage**)  $12000 + 50 * (\text{Ivan's age})$ .

Finally, it remains to print the results. By performing a **check if the inheritance (heritage)** was enough to live without working or not. If the (**heritage**) is a **positive** number, we print: „**Yes! He will live a carefree life and will have {N} dollars left.**“, and if it's a **negative** number: „**He will need {M} dollars to survive.**“. Do not forget to format the sum up to the second symbol after the decimal point.

**Hint:** Consider using the **Math.abs(...)** function when printing the output if the inheritance isn't enough.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#2>.



## Problem: Hospital

For a given period, everyday patients arrive at the hospital for an examination. The hospital initially has 7 doctors. Each doctor can treat only one patient per day, but sometimes there is a shortage of doctors, so the untreated patients are sent to other hospitals. Every third day the hospital makes calculations and if the number of untreated patients is greater than the treated patients, they hire one more doctor. The hire happens before the daily patient's admission begins.

Write a program, that calculates for a given period the number of treated and untreated patients.

### Input Data

The input is read from the console and contains:

- On the first line – the period, for which you need to make the calculations. Integer number in the range of [1 ... 1000].
- On the next lines (equal to the number of days) – patient count, who arrive for treatment at the current day. Integer number in the range of [0 ... 10 000].

### Output Data

Print on the console 2 lines:

- On the first line: "Treated patients: {number of treated patients}."
- On the second line: "Untreated patients: {number of untreated patients}."

### Sample Input and Output

| Input                  | Output                                           | Explanations                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4<br>7<br>27<br>9<br>1 | Treated patients: 23.<br>Untreated patients: 21. | <p><b>Day 1:</b> 7 treated and 0 untreated patients for the day</p> <p><b>Day 2:</b> 7 treated and 20 untreated patients for the day</p> <p><b>Day 3:</b> To this moment the treated patients are 14, and the untreated – 20 -&gt; New doctor is hired<br/>-&gt; 8 treated and 1 untreated patient for the day</p> <p><b>Day 4:</b> 1 treated and 0 untreated patients for the day</p> <p><b>In total:</b> 23 treated and 21 untreated patients.</p> |

| Input | Output                  |
|-------|-------------------------|
| 6     | Treated patients: 40.   |
| 25    | Untreated patients: 87. |
| 25    |                         |
| 25    |                         |
| 25    |                         |
| 25    |                         |
| 2     |                         |

| Input | Output                 |
|-------|------------------------|
| 3     | Treated patients: 21.  |
| 7     | Untreated patients: 0. |
| 7     |                        |
| 7     |                        |

Hints and Guidelines

We start again, by **declaring and initializing** the necessary variables:

```
int period = Integer.parseInt(scanner.nextLine());
```

```
int treatedPatients = 0;
int untreatedPatients = 0;
int countOfDoctors = 7;
```

The period which we have to make the calculations is read from the console and we store it in the variable **period**. We are going to need also some auxiliary variables: number of treated patients (**treatedPatients**), number of untreated patients (**untreatedPatients**), and the number of doctors (**countOfDoctors**), which by default is 7.

```
for (int day = 1; day <= period; day++) {
    int currentPatients = Integer.parseInt(scanner.nextLine());

    if ((day % 3 == 0) && (untreatedPatients > treatedPatients)){
        countOfDoctors++;
    }

    if (currentPatients > countOfDoctors) {
        treatedPatients += countOfDoctors;
        untreatedPatients += currentPatients - countOfDoctors;
    }else {
        treatedPatients += currentPatients;
    }
}
```

Using a **for** loop, we iterate through all days in the given period (**period**). For each day, we read from the console the number of patients (**currentPatients**). Incrementing the number of doctors by the condition can be done **every third day**, BUT if only the number of the untreated patients is **greater** than the treated ones. For this purpose, we check if

the day is the third one, using the arithmetical operator for division with the remainder (%): **day % 3 == 0**.

For example:

- If it's the **third one**, the remainder from the division by 3 will be 0 (**3 % 3 = 0**) and the check **day % 3 == 0** will return **true**.
- If it's the **second one**, the remainder from the division by 3 will be 2 (**2 % 3 = 2**) and the check will return **false**.
- If it's the **fourth one**, the remainder from the division will be 1 (**4 % 3 = 1**) and again the check will return **false**.

If the check **day % 3 == 0** returns **true**, the program will check if the untreated patient's count is greater than the treated ones: **untreatedPatients > treatedPatients**. If again the result is **true**, then we'll increase the number of doctors (**countOfDoctors**).

After that, we check the number of patients for the day (**currentPatients**) whether it's greater than the number of doctors (**countOfDoctors**). If the patient number is **greater**:

- We increase the value of the variable **treatedPatients** with the number of doctors (**countOfDoctors**).
- We increase the value of the variable **untreatdPatients** with the number of the rest of the patients, which we calculate, by subtracting the number of the doctors from all the patients (**currentPatients - countOfDoctors**).

If the patient count is **less**, we increase only the variable **treatedPatients** with the number of patients for the day (**currentPatients**).

Finally, we only need to print the number of treated and untreated patients.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#3>.

## Problem: Division

There are **n** integer numbers given in the range of [1 ... 1000]. From them, some percentage **p1** is divisible without remainder by 2, percentage **p2** is divisible by 3 without remainder, percentage **p3** is divisible by 4 without remainder. Write a program that calculates the percentages of **p1**, **p2** and **p3**, and prints them out. **Example:** We have **n = 10** numbers: 680, 2, 600, 200, 800, 799, 199, 46, 128, 65. We get the following distribution and visualization:

| Division without remainder by: | Numbers                        | Count | Percent                         |
|--------------------------------|--------------------------------|-------|---------------------------------|
| 2                              | 680, 2, 600, 200, 800, 46, 128 | 7     | $p1 = (7 / 10) * 100 = 70.00\%$ |
| 3                              | 600                            | 1     | $p2 = (1 / 10) * 100 = 10.00\%$ |
| 4                              | 680, 600, 200, 800, 128        | 5     | $p3 = (5 / 10) * 100 = 50.00\%$ |

Input Data

On the first line of input is the integer number **n** ( $1 \leq n \leq 1000$ ) – count of numbers. On the next **n lines**, we have **one integer number** in the range of **[1 ... 1000]** – the numbers that have to be checked for division.

Output Data

Print on the console **3 lines**, each one containing a percentage between 0% and 100%, with accuracy up to two symbols after the decimal point, for example 25.00%, 66.67%, 57.14%.

- On the **first line** – the percentage of numbers that **are divisible by 2**.
- On the **second line** – the percentage of numbers that **are divisible by 3**.
- On the **third line** – the percentage of numbers that **are divisible by 4**.

Sample Input and Output

| Input | Output | Input | Output  | Input | Output  |
|-------|--------|-------|---------|-------|---------|
| 10    | 70.00% | 3     | 33.33%  | 1     | 100.00% |
| 680   | 10.00% | 3     | 100.00% | 12    | 100.00% |
| 2     | 50.00% | 6     | 0.00%   |       | 100.00% |
| 600   |        | 9     |         |       |         |
| 200   |        |       |         |       |         |
| 800   |        |       |         |       |         |
| 799   |        |       |         |       |         |
| 199   |        |       |         |       |         |
| 46    |        |       |         |       |         |
| 128   |        |       |         |       |         |
| 65    |        |       |         |       |         |

## Hints and Guidelines

For the current problem and the next one, you'll need to write the code by yourself, following the given guidelines.

The program that solves the current problem is analogous to the **Histogram** problem, which we reviewed earlier. Therefore, we can start with declaring the needed variables: Exemplary names for the variables can be **n** – count of the numbers (that we have to read from the console) and **divisibleBy2**, **divisibleBy3**, **divisibleBy4** – auxiliary variables, containing the count of numbers in the corresponding group.

To read and assign every number in its corresponding group, we should iterate the **for** loop from **0** to **n** (count of numbers). Each iteration of the loop must read and allocate **only one number**. The difference here is that **a number can end up in multiple groups simultaneously**, that's why we should create **three separate if statements for each number** – respectively whether it is divisible by 2, 3, and 4 (the **if-else** statement, in this case, won't be useful because after finding a match it stops further checking of conditions) and increase the value of the variable that keeps the count of numbers for the corresponding group.

Finally, we have to print out the obtained results, by following the specified formatting

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#4>.

## Problem: Logistics

It's your responsibility for the logistics of various types of cargo. Depending on the weight of each load, a **different vehicle type** is needed, and **the costs will vary per ton**:

- Up to **3 tons** – **van** (200 USD per ton).
- Upward of 3 and up to **11 tons** – **lorry** (175 USD per ton).
- Over **11 tons** – **train** (120 USD per ton).

It's your problem to calculate, **the average price per ton of transported cargo**, and also **what percentage of cargo** is transported by the various vehicle types.

## Input Data

From the console, we must read a **sequence of numbers**, each on a separate line:

- **First line:** **cargo count** for transport – an **integer number** in the range of [1 ... 1000].
- On each of the next lines, we input **the weight of the current cargo** – an **integer number** in the range of [1 ... 1000].

## Output Data

Print out on the console **4 lines**, as follows:

- **Line #1** – an **average price per ton of cargo** (rounded up to the second symbol after the decimal point).

- **Line #2** – a **percentage** of cargo, transported by **van** (between 0.00% and 100.00%, rounded up to the second symbol after the decimal point).
- **Line #3** – a **percentage** of cargo, transported by **lorry** (between 0.00% and 100.00%).
- **Line #4** – a **percentage** of cargo, transported by **train** (between 0.00% and 100.00%).

Sample Input and Output

| Input                  | Output                               | Explanations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4<br>1<br>5<br>16<br>3 | 143.80<br>16.00%<br>20.00%<br>64.00% | Two of the cargoes are transported by <b>van</b> <b>1 + 3</b> , in total <b>4</b> tons.<br>One of the shipments is transported by <b>lorry</b> : <b>5</b> tons.<br>One of the shipments is transported by <b>train</b> : <b>16</b> tons.<br><b>The sum</b> of all the cargo is: <b>1 + 5 + 16 + 3 = 25</b> tons.<br>Percentage cargo <b>by van</b> : <b>4/25*100 = 16.00%</b><br>Percentage cargo <b>by lorry</b> : <b>5/25*100 = 20.00%</b><br>Percentage cargo <b>by train</b> : <b>16/25*100 = 64.00%</b><br><b>Average price</b> per ton of transported cargo: <b>(4 * 200 + 5 * 175 + 16 * 120) / 25 = 143.80</b> |

| Input                        | Output                              | Input                     | Output                             |
|------------------------------|-------------------------------------|---------------------------|------------------------------------|
| 5<br>2<br>10<br>20<br>1<br>7 | 149.38<br>7.50%<br>42.50%<br>50.00% | 4<br>53<br>7<br>56<br>999 | 120.35<br>0.00%<br>0.63%<br>99.37% |

Hints and Guidelines

First, we have to read the **weight of each cargo** and will **sum up** how many tons are transported by **van**, **lorry**, and **train** and we are going to calculate **the total tons** of transported cargo. We will calculate **the prices for each type of transport** determined by the total transported tonnage and **the total price**. In the end, we'll calculate and print **the total average price per ton** and **how much cargo is being transported by each transport type**.

We declare the needed variables, for example: **countOfLoads** – the number of loads to be transported (we read them from the console), **sumOfTons** – the sum of the total weight of all cargo, **vanTons**, **truckTons**, **trainTons** – variables that keep the sum of the weight, transported by van, lorry, and train.

We are going to need a **for** loop from **0** to **countOfLoads-1** to iterate through all the shipments. For each shipment, we read its weight (in tons) from the console and we save it to a variable, for example, **tons**. We add up to the tonnage sum of all cargos (**sumOfTons**) the weight of the current load (**tons**). After we've read the weight of the current cargo, we have to determine which transportation type we are going to use for it (van, lorry, or train). For this problem, we're going to need **if-else** statements:

- If the value of the variable **tons** is less than 3, increase the value of **vanTons** by the value of **tons**:
- **vanTons += tons;**
- Otherwise, if the value of **tons** is up to 11, we increase **truckTons** by **tons**.
- If **tons** are more than 11, we increase **trainTons** by **tons**.

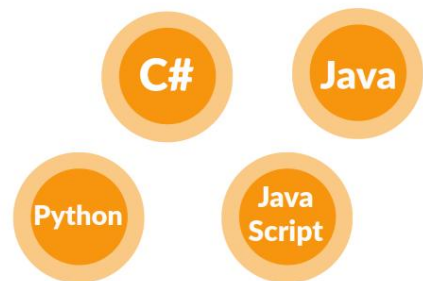
Before we print out the output, we have to calculate the tonnage percentage of each transportation type and the average price per ton. For the average price, we are going to need one more auxiliary variable **totalPrice**, in which we are going to add up the total price of all transported cargo (by van, lorry, and train). For us to get the average price, we have to divide **totalPrice** by **sumOfTons**. It remains to calculate the percentages of the tonnage, transported by each transport type, and print out the results, adhering to the formatting specified in the description.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/656#5>.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**

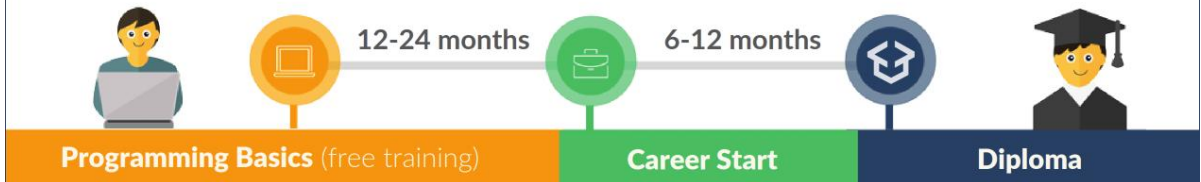


**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)



# Chapter 6.1. Nested Loops

This chapter will discuss **nested loops** in the Java language. We will use **for** loops to **draw** different **figures** containing symbols and signs arranged in rows and columns **on the console**. We will use **single loops** and **nested loops** (loop statement inside another loop statement), **calculations** and **checks**, to print on the console simple and not so simple figures in given sizes.

## Video

Watch the video lesson on this chapter here: <https://youtu.be/FU90zeFZZFs>.

## Problem: Rectangle of 10 x 10 Stars

Write a program that draws a rectangle of **10 x 10** asterisks in the console.

| Input      | Output                                                                   |
|------------|--------------------------------------------------------------------------|
| (no input) | <pre> ***** ***** ***** ***** ***** ***** ***** ***** ***** ***** </pre> |

## Hints and Guidelines

To solve the problem, we will use a method. In Chapter 10, we will look in detail at what methods are and how to use them. The method will allow us to execute the same code more than once and in more than one place in a program. At this stage, we will not reveal more than the concept of methods.

```

public class Main {
    public static void main(String[] args) {
        String symbol = "*";
        for (int i = 0; i < 10; i++) {
            System.out.println(
                generateFrom( numberOfStars: 10, symbol));
        }
    }
}

```

```

public static String generateFrom(
    int numberOfStars, String symbol ) {
    StringBuffer builder = new StringBuffer();
    for (int i = 0; i < 10; i++) {
        builder.append(symbol);
    }
    return builder.toString();
}
}

```

How does the example work? The control variable of the loop (**i = 0**) is initialized. The variable is incremented on each iteration, as long as it is **less than 10**. After each execution of the code in the loop body, and after the iteration, a check about the state of the variable is performed. Thus, the code in the loop body is executed exactly **ten times** i.e. for each row of the rectangle. In the loop body, we call the method **generateFrom**. The method uses the class **StringBuffer** and another **for** loop. Each iteration of a **for** loop (the one in the method) will append one asterisk, thus creating a row of ten asterisks. When the whole execution of the loop (the one in the method) finishes, we have a string of asterisks as a result. Then, the execution of the code returns to the point where we call the method. Then, prints the resulting string on the console.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#0>.

## Problem: Rectangle of N x N Stars

Write a program that reads an integer **n** from the console, and as output, draws a rectangle with a size of **n x n** asterisks.

| Input | Output             | Input | Output                   | Input | Output                           |
|-------|--------------------|-------|--------------------------|-------|----------------------------------|
| 2     | <pre> ** ** </pre> | 3     | <pre> *** *** *** </pre> | 4     | <pre> **** **** **** **** </pre> |

## Hints and Guidelines

To perform the problem, we will use the **Scanner** class, which allows us to read the input size of the figure from the console.

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = Integer.parseInt(scanner.nextLine());
        String symbol = "*";
        for (int i = 0; i < n; i++) {
            System.out.println(generateFrom(n, symbol));
        }
    }

    public static String generateFrom(
        int numberOfStars, String symbol) {
        StringBuffer builder = new StringBuffer();
        for (int i = 0; i < numberOfStars; i++) {
            builder.append(symbol);
        }
        return builder.toString();
    }
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#1>.

## Nested Loops

Nested loops are construction in which **in the body of one loop** statement (called outer), **another loop statement is performed** (called inner). For each iteration of the outer loop statement, the inner one is executed **completely**. This happens as follows:

- When the execution of nested loops begins, **the outer loop starts** first which includes the following steps: the control variable is **initialized**, then checks the condition for the end of the loop, and then the code in the body starts to execute.
- Then **the inner loop is executed**, which includes the same steps: the control variable is initialized, then checks the condition for the end of the loop, and then the code in the body starts to execute.
- When the condition for **the end of the inner loop** is met, the program returns one step up, and the started execution of the outer loop continues. This results that the variable of the outer loop is changing (incremented), then checks whether the end condition of the outer loop is satisfied. If not, **a new execution of the nested (inner) loop starts**.
- This is repeated until the outer loop variable reaches the **end of the loop** condition.

Here is an **example** that illustrates the nested loops. The goal is to print a rectangle of **n** x **n** asterisks, where for each row, the outer loop iterates from **1** to **n**, and for each column iterates an inner loop from **1** to **n**:

```

Scanner scanner = new Scanner(System.in);
int numberOfStars = Integer.parseInt(scanner.nextLine());

for (int outer = 1; outer <= numberOfStars; outer++) {
    for (int inner = 1; inner <= numberOfStars; inner++) {
        System.out.print("*");
    }
    System.out.println();
}

```

How does the above example work? After the initialization of the **outer** loop, its **body** begins to execute, which contains **another (inner) loop**. The inner loop returns **numberOfStars** as a string of asterisks. After the **inner loop completes** its execution, the program control returns at the first iteration of **the outer loop**, and the code within it **continues to execute**. A statement (**System.out.println ()**) exists in the body of the outer loop. This statement will move the cursor to the following line when the inner loop completes its execution. Without this code, we will print all asterisks on one line. Usage of **println()** method in the inner loop instead of the **print()** method will print all asterisks on separate lines. You can try and see for yourself. Then **increment** of the variable (in our case by 1) of **the outer loop** follows, and the whole **inner loop** starts execution again. The inner loop executes as many times as the body of the outer loop is executed (in our case **numberOfStars** times).

## Problem: Square of Stars

Write a program that draws on the console a square of **N x N** asterisks:

| Input | Output               | Input | Output                         | Input | Output                                       |
|-------|----------------------|-------|--------------------------------|-------|----------------------------------------------|
| 2     | <pre> * * * * </pre> | 3     | <pre> * * * * * * * * * </pre> | 4     | <pre> * * * * * * * * * * * * * * * * </pre> |

## Hints and Guidelines

The problem is similar to the previous one. Here, it is necessary to consider how to print a space after the asterisks so that there are no unnecessary spaces at the beginning of the end.

```

Scanner scanner = new Scanner(System.in);
int numberOfStars = Integer.parseInt(scanner.nextLine());

for (int outer = 1; outer <= numberOfStars; outer++) {
    System.out.print("*");
    for (int inner = 1; inner <= numberOfStars; inner++) {
        System.out.print(" *");
    }
    System.out.println();
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#2>.

### Problem: Triangle of Dollars

Write a program that reads an integer number **n** from the console and returns as output a triangle of dollar signs.

| Input | Output                  | Input | Output                                 |
|-------|-------------------------|-------|----------------------------------------|
| 3     | \$<br>\$ \$<br>\$ \$ \$ | 4     | \$<br>\$ \$<br>\$ \$ \$<br>\$ \$ \$ \$ |

### Hints and Guidelines

The problem is **similar** to those of drawing **rectangles** and **squares**. We will use **nested loops** again, but there is a **trick** here. The difference is that the **number of columns** we need to print as output depends on the **row** we are on, not the input integer **n**. From the sample input and output data, we notice that **the number of dollars depends on which line** we are at the time of printing, i.e. one dollar sign means at the first line, two dollar signs at the second line, etc. Let's look at the example below in more detail. We see that the **variable** of the **nested** loop is bound to the variable of the **outer** loop. In this way, our program prints the desired triangle.

```
Scanner scanner = new Scanner(System.in);
int numberOfSymbols = Integer.parseInt(scanner.nextLine());

for (int row = 1; row <= numberOfSymbols; row++) {
    System.out.print("$");
    for (int col = 1; col < row; col++) {
        System.out.print("$");
    }
    System.out.println();
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#3>.

### Problem: Square Frame

Write a program that reads an integer number **n** from the console and returns as output a square frame with a size of **n \* n**.

| Input | Output                                       | Input | Output                                                                   | Input | Output                                                                                           |
|-------|----------------------------------------------|-------|--------------------------------------------------------------------------|-------|--------------------------------------------------------------------------------------------------|
| 4     | <pre> + - - +   - -     - -   + - - + </pre> | 5     | <pre> + - - - +   - - -     - - -     - - -     - - -   + - - - + </pre> | 6     | <pre> + - - - - +   - - - -     - - - -     - - - -     - - - -     - - - -   + - - - - + </pre> |

### Hints and Guidelines

We can solve the problem as follows:

- We read an integer **n** from the console.
- We print **the upper part** of the frame: first the sign **+**, then **n-2** times the sign **-**, and finally the sign **+**.
- We print **the middle part** of the frame: we print **n-2** lines as first print the sign **|**, then **n-2** times the sign **-** and finally again the sign **|**. We can achieve this with nested loops.
- We print **the lower part** of the frame: first print the sign **+**, then **n-2** times the sign **-** and finally the sign **+**.

Here is an example implementation of the described logic above, with nested loops:

```

Scanner scanner = new Scanner(System.in);

// Print the top row: + - - - +
int numberOfRows = Integer.parseInt(scanner.nextLine());
System.out.print("+");
for (int i = 0; i < numberOfRows - 2; i++) {
    System.out.print(" -");
}
System.out.println(" +");

// Print the middle rows: | - - - |
for (int row = 0; row < numberOfRows - 2; row++) {
    System.out.print("|");
    for (int i = 0; i < numberOfRows - 2; i++) {
        System.out.print(" -");
    }
    System.out.println(" |");
}

// Print the bottom row: + - - - +
System.out.print("+");
for (int i = 0; i < numberOfRows - 2; i++) {
    System.out.print(" -");
}
System.out.println(" +");

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#4>.

## Problem: Rhombus of Stars

Write a program that reads an integer number **n** from the console and returns as output a diamond of asterisks with a size of **n**.

| Input | Output | Input | Output                              | Input | Output                                                   | Input | Output                                                                                         |
|-------|--------|-------|-------------------------------------|-------|----------------------------------------------------------|-------|------------------------------------------------------------------------------------------------|
| 1     | *      | 2     | <pre>   *  * *   *           </pre> | 3     | <pre>     *    * *   * * *    * *     *           </pre> | 4     | <pre>       *      * *     * * *    * * * *   * * * *    * * *     * *      *           </pre> |

## Hints and Guidelines

To solve the problem, we divide (imaginary) the rhombus into **two parts**: **upper** (includes and the middle row) and **lower**. We will use **two** loops for each part of the rhombus to print the **output** in the console. The reader must find the relationship between **n** and the variables in the loops.

For the upper part of the rhombus (first loop) we can use the following guidelines:

- We print **n-row** intervals.
- We print **\***.
- We print **row-1** times **\***.

We will use the **same** way to output the **lower** part of the rhombus, but we leave the reader to try to do it himself.

```

for (int row = 1; row <= n; row++) {
    for (int col = 1; col <= n - row; col++) {
        System.out.print(" ");
    }
    System.out.print("*");
    for (int col = 1; col < row; col++) {
        System.out.print(" *");
    }
    System.out.println();
}
//TODO: Print the rest of the rhombus

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#5>.

## Problem: Christmas Tree

Write a program that reads an integer number **n** ( $1 \leq n \leq 100$ ) from the console and returns as output **Christmas tree** with a height of **n+1**.

| Input | Output           | Input | Output                      | Input | Output                                      | Input | Output                                                           |
|-------|------------------|-------|-----------------------------|-------|---------------------------------------------|-------|------------------------------------------------------------------|
| 1     | <pre>*   *</pre> | 2     | <pre>  *   *  **   **</pre> | 3     | <pre>    *   *    **   **   ***   ***</pre> | 4     | <pre>      *   *      **   **     ***   ***    ****   ****</pre> |

## Hints and Guidelines

As we saw in the previous problems, we can apply a similar principle and **divide** the **Christmas tree** into **three** logical parts. The first part is the **asterisks (\*)** and the **spaces** before and after them, the **middle** part is **|**, and the last part are again **asterisks (\*)**, and this time **empty** spaces are only **in front of** them. Printing the output can be done with **one loop**, and again we will use the method as we have done at the beginning of this chapter.

```
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = Integer.parseInt(scanner.nextLine());
        for (int i = 0; i <= n; i++) {
            String stars = generateFrom( symbol: "*", i);
            String spaces = generateFrom( symbol: " ",
                                         numberOfStars: n - i);
            System.out.print(spaces);
            System.out.print(stars);
            System.out.print(" | ");
            System.out.print(stars);
            System.out.print(spaces);
            System.out.print();
        }
    }
    public static String generateFrom(
        String symbol, int numberOfStars) {
        StringBuffer builder = new StringBuffer();
        for (int i = 0; i < numberOfStars; i++) {
            builder.append(symbol);
        }
        return builder.toString();
    }
}
```



### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#6>.

## Drawing More Complex Figures

Let's look at a few exercises about how we can draw figures on the console with more complex logic. To solve these problems, we need to think more about the program logic before we begin writing.

### Problem: Sunglasses

Write a program that reads an integer number  $n$  ( $3 \leq n \leq 100$ ) from the console and returns as output **sunglasses** with a size of  $5 \cdot n \times n$ , just like the examples below.

| Input | Output                                                |
|-------|-------------------------------------------------------|
| 3     | <pre>*****  ***** */////*  */////* *****  *****</pre> |

| Input | Output                                                                                   |
|-------|------------------------------------------------------------------------------------------|
| 4     | <pre>*****  ***** */////*   */////* */////*  */////* */////*  */////* *****  *****</pre> |

| Input | Output                                                                                                                            |
|-------|-----------------------------------------------------------------------------------------------------------------------------------|
| 5     | <pre>*****  ***** *////////*  *////////* *////////*   *////////* *////////*  *////////* *////////*  *////////* *****  *****</pre> |

### Hints and Guidelines

As we see in the examples below, we can apply a similar principle and divide the sunglasses into **three** logical parts - upper, middle, and lower. Below is part of the code with which we can solve the problem.

When drawing the top and bottom rows, use **2 \* n** asterisks, **n** spaces, and **2 \* n** asterisks as output.

```
//Print the top part
System.out.print(generateFrom( symbol: "**", numberOfStars: 2 * n));
System.out.print(generateFrom( symbol: " ", n));
System.out.println(generateFrom( symbol: "**", numberOfStars: 2 * n));
for (int i = 0; i < n - 2; i++) {
    // TODO: Print the middle part
}
//Print the bottom part
System.out.print(generateFrom( symbol: "**", numberOfStars: 2 * n));
System.out.print(generateFrom( symbol: " ", n));
System.out.println(generateFrom( symbol: "**", numberOfStars: 2 * n));
```

When printing the **middle** part, we need to **check** whether the line is  $(n-1) / 2 - 1$ , because as the example shows on [this line](#), we need to print **vertical dashes** instead of spaces.

```
// Print the middle part
for (int i = 0; i < n - 2; i++) {
    // TODO: Print *////////*
    if (i == (n - 1 / 2 - 1)) {
        System.out.print(generateFrom( symbol: "|", n));
    } else {
        System.out.print(generateFrom( symbol: " ", n));
    }
    // TODO: Print *////////*
    System.out.println();
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#7>.

## Problem: House

Write a program that reads an integer number  $n$  ( $2 \leq n \leq 100$ ) from the console and returns as output a **house** with a size of  $n \times n$ , just like the examples below.

| Input | Output           | Input | Output                 | Input | Output                         | Input | Output                                   |
|-------|------------------|-------|------------------------|-------|--------------------------------|-------|------------------------------------------|
| 2     | <pre>**   </pre> | 3     | <pre>-*- ***  * </pre> | 4     | <pre>-**- ****  **   ** </pre> | 5     | <pre>--*-- -***- *****  ***   *** </pre> |

## Hints and Guidelines

According to the condition of the problem, the house has dimensions of  $n \times n$ , but from the input and output data, we can see the following:

- The house consists of two parts: **roof** and **base**.

```

--**--
--***--
--****--
*****

```

**Roof**

```

*****|
*****|
*****|
*****|

```

**Foundation**

- When  $n$  is an even number, the roof of the house is obtuse.
- When  $n$  is odd, the roof is one row larger than the base.

### The Roof

- It consists of **asterisks** (\*) and **dashes** (-).
- At the top part of the roof, we have one or two stars depending on whether  $n$  is even or odd (also related to the dashes).
- In the lowest part, we have many stars and a few or no dashes.
- At each row of the roof (looking from top to bottom), the stars are increased by two, and the dashes are decreased by two.

### The Base

- The height is  $n$  rows.
- It consists of **asterisks** (\*) and **pipes** (|).
- Each row consists of two **pipes** (one at the beginning and one at the end of the row) and a sequence of  $n - 2$  **asterisks** between the pipes.

We read an integer  $n$  from the console and assign the value of an integer to a variable of type **int**.

```
int n = Integer.parseInt(scanner.nextLine());
```



It is of high importance to perform checks on whether the input data is **correct**! In the current problems, it is not a problem to convert directly the data from the console into **int** type because it is said that we will read valid integers. If you are making more complex programs, it is a good practice to check the input data.

To draw the **roof**, we write down how many **asterisks** we start within a variable called **stars**:

- If **n** is even, they are two.
- If **n** is odd it is one

```
int stars = 1;
if (n % 2 == 0) {
    stars++;
}
```

We calculate the length of the roof, and it equals half of **n**. Assign the result to the variable *\*roofLength*.

```
double roofLength = (int) Math.ceil(n / 2);
```

It is important to note that when **n** is an odd number, the length of the roof is one row bigger than the base one. In Java, when you divide two numbers of integer type with a remainder, the result will be a number without a remainder.

Example:

```
int result = 3 / 2; // result: 1
```

If we want to round up the result, we need to use the method **Math.ceil(...)**:

```
int result = (int) Math.ceil(3 / 2f);
```

In this example, the division is not between two integers. The letter "f" after a number shows that this number is of **float** type (a floating-point number). The result of **3 / 2f** is **1.5f**. **Math.ceil(...)** method rounds up the result and, in this case, **1.5f** will become 2. We write **(int)** before **Math.ceil(...)**, so we can transfer the result back to **int** type and assign it to the variable **result**.

After we have calculated the length of the roof, we use a loop from 0 to **roofLength**. On each iteration we will:

- Calculate the number of dashes we need to draw. The number will be equal to **(n - stars) / 2**. Assign it in variable **padding**.

```
int padding = (n - stars) / 2;
```

- Print on the console: "dashes" (*padding times*) + "asterisks" (*stars times*) + "dashes" (*padding\* times*).

```
String line = generateFrom( symbol: "-", padding)
    + generateFrom( symbol: "*", stars)
    + generateFrom( symbol: "-", padding);
System.out.println(line);
```

- Before the iteration is over, we add 2 to the **stars** (the number of the asterisks).

```
stars += 2;
```



It is not a good practice to concatenate many character strings as is shown above, because this leads to **performance** issues. Learn more at: <https://docs.oracle.com/javase/7/docs/api/java/lang/StringBuilder.html>

After we have finished with the roof, it is time for the base. It is easier to print:

- We use a loop from 0 to  $n$  (not inclusive).
- We print on the console: `| + *(n - 2 times) + |`.

```
for (int i = 0; i < n / 2; i++) {
    String line = "|" +
        generateFrom( symbol: "*", number: n - 2)
        + "|";
    System.out.println(line);
}
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#8>.

Problem: Diamond

Write a program that reads an integer number  $n$  ( $1 \leq n \leq 100$ ) from the console and returns as output a **diamond** with a size of  $n$ , just like the examples below.

| Input | Output | Input | Output | Input | Output                 | Input | Output                             | Input | Output                                  |
|-------|--------|-------|--------|-------|------------------------|-------|------------------------------------|-------|-----------------------------------------|
| 1     | *      | 2     | **     | 3     | <pre>-*- *-* -*-</pre> | 4     | <pre>-***- *-**- *--*- -***-</pre> | 5     | <pre>--*-- -**- *-**- *--*- --*--</pre> |

Hints and Guidelines

According to the requirements of the problem, have  $n$  rows and  $n$  columns to draw a diamond, but from the input and output data, we can see the following:

- All lines contain exactly  $n$  characters, and all lines, except the upper and the bottom ones, have **two asterisks** (\*).

We can divide (imaginary) the diamond into **two parts**:

- **Upper** part. It starts from the top to the middle of the diamond (includes **also** the middle row).
- **Lower** part. It starts from the row after the middle up to the bottom tip (inclusive) of the diamond.

Upper Part

- If  $n$  is **odd**, begins with **1 asterisk** (\*).

- If **n** is **even**, begins with 2 asterisks (```**`).
- With each row down, the asterisks move away from each other.
- The space in front, between, and after the **asterisks** (**\***) is filled with **dashes** (**-**).

### Lower Part

- With each row down, the asterisks come together. It means that the space (**dashes**) between them decreases, and the space (**dashes**) on the left and right increases.
- At its bottom, it has 1 or 2 **asterisks**, depending on whether **n** is odd or even.

### Upper and Lower Part

- On each row, the asterisks are surrounded from the outer side (the left or the right one) by **dashes**, except the middle row.
- Each row has a space between the two **asterisks**, except the first and last row (sometimes **the asterisk is one**).

### Reading The Input Data

We read an integer **n** from the console and assign the value of an integer to a variable of type **int**.

```
int n = Integer.parseInt(scanner.nextLine());
```

### Printing The Upper Part of The Diamond

We begin drawing the upper part of the diamond. The first thing we need to do is to calculate the number of the outer **dashes leftRight** (the dashes on the outer side of the **asterisks**). It is equal to  $(n - 1) / 2$ , rounded downward.

```
int leftRight = (n - 1) / 2;
```

After we have calculated **leftRight**, we start drawing the **upper part** of the diamond. We can use a loop from `0*` to `n / 2 + 1**` (rounded downward).

At each iteration of the loop the following steps are performed:

- We draw as an output on the console **dashes** on the left side of the diamond (with length equal to **leftRight**) and right after them the first **asterisk**.

```
System.out.print(generateFrom( symbol: "-", leftRight));  
System.out.print("*");
```

- We will calculate the distance between the two **asterisks**. We can do this by subtracting from **n** the number of the outer **dashes** and the number 2 (the number of the **asterisks**, i.e. the diamond's outline). The result of the subtraction is assigned to a variable **mid**.

```
int mid = n - 2 * leftRight - 2;
```

- If the **mid** is lower than 0, we know that on the row should be only 1 asterisk. If it is higher or equal to 0 then we have to print **dashes** with length equal to **mid** and one **asterisk** after them.
- We draw as an output on the console **dashes** on the right side of the diamond with a length equal to **leftRight**.

```
System.out.println(generateFrom( symbol: "-", leftRight));
```

- At the end of the loop, we decrease **leftRight** by 1 (the asterisks **\*** are moving away from each other).

We are ready with the upper part.

### Printing The Lower Part of The Diamond

Printing the lower part is similar to that of the upper part. The differences are:

- Increase instead of decrease **leftRight** with one, but at the beginning of the loop
- The loop will be from 0 to  $(n - 1) / 2$ .

```
for (int i = 0; i < (n - 1) / 2; i++) {
    leftRight++;
    System.out.print(generateFrom( symbol: "-", leftRight));
    System.out.print("*");
    int mid = n - 2 * leftRight - 2;
    if (mid >= 0) {
        System.out.print(generateFrom( symbol: "-", mid));
        System.out.print("*");
    }
    System.out.println(generateFrom( symbol: "-", leftRight));
}
```



Repeating a code is considered **bad practice** because the code becomes very hard to maintain. Let's imagine that we have a piece of code (e.g. the logic for drawing a row from the diamond example) at a few more places and we decide to change it. For this, we will have to go through all the places and change it everywhere. Now let's imagine that you need to reuse a piece of code not 1, 2, or 3 times but tens of times. A way to overcome this problem is to use **methods**. You can look for additional information for methods on the Internet or look at [Chapter "10" \(Methods\)](#).

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/657#9>.

## What Have We Learned from This Chapter?

We learned to draw figures with nested **for** loops:

```
for (int r = 1; r <= 5; r++)
{
    System.out.println("*");
    for (int c = 1; c < 5; c++)
        System.out.println(" *");
    System.out.println();
}
```

We also learned that we can **use methods** to avoid repeating the code multiple times.



## Chapter 6.2. Nested Loops – Exam Problems

In the previous chapter we got familiar with **nested loops** and to use them **drawing** different kinds of **figures on the console**. We learned how to print different size figures, inventing an appropriate logic to construct them using **single and nested for** loops in combination with various calculations and program logic:

```
for (int row = 1; row <= 5; row++) {
    System.out.print("*");
    for (int col = 1; col < 5; col++) {
        System.out.print(" *");
    }
    System.out.println();
}
```

We also declared our method **repeatStr(...)** which helps us to print a given symbol (or a series of characters) a **certain** number of times:

```
static String repeatStr(String strToRepeat, int count) {
    String text = "";
    for (int i = 0; i < count; i++) {
        text = text + strToRepeat;
    }
    return text;
}
```

### Exam Problems

Let's solve several nested loops related exam problems to practice what we have learned so far and to develop our further algorithmic thinking.

#### Problem: Draw a Fort

Write a program that reads from the console an **integer n** and draws a **fortress** with width **2 \* n columns** and height **n rows** like the examples below. The left and right columns on the inside are **n / 2** wide.

#### Input Data

The input is an **integer n** in the range of [3 ... 1000].

#### Output Data

Print on the console **n** text rows, depicting **the fortress**, exactly like the examples.

Sample Input and Output

| Input | Output                                                        | Input | Output                                                                | Input | Output                                                                        |
|-------|---------------------------------------------------------------|-------|-----------------------------------------------------------------------|-------|-------------------------------------------------------------------------------|
| 3     | <pre> / ^ \ / ^ \<br/>         <br/> \ ^ \ / ^ \<br/>  </pre> | 4     | <pre> / ^ ^ \ / ^ ^ \<br/>         <br/> \ ^ ^ \ / ^ ^ \<br/>  </pre> | 5     | <pre> / ^ ^ ^ \ / ^ ^ ^ \<br/>         <br/> \ ^ ^ ^ \ / ^ ^ ^ \<br/>  </pre> |

Hints and Guidelines

From the problem statement, we see that the **input data** will be one line only containing an **integer** within the range [3 ... 1000]. Therefore, we will use a **variable** of type **int**.

```
Scanner scan = new Scanner(System.in);  
int n = Integer.parseInt(scan.nextLine());
```

Once we have declared and initialized the input data, we need to divide the **fort** into three parts:

- roof
- body
- base

We can see from the examples that the **roof** is made of **two towers** and a **middle part**. Each tower has a beginning **/**, a middle part **^**, and an end **\**.



`\` is a special symbol in Java. Using it in the `System.out.println(...)` method, the console will not print it. That's why using `\\` we instruct the console that we want to print exactly this symbol, without being interpreted as a special one (it is **shielded**- this is called “**character escaping**”).

The size of the middle part is `n / 2`, therefore we can write this value in a new **variable**. It will store the **size** of the **middle part of the tower**.

```
int colSize = n / 2;
```

Now we declare a second **variable**, where we will store the **value** of **space between the two towers**. The middle roof part has size of `2 * n - 2 * colSize - 4`.

```
int midSize = 2 * n - 2 * colSize - 4;
```

To print the **roof** on the console, we will use our `repeatStr(...)` method, which accepts two parameters (**string**, **int**) and concatenate a certain symbol (or series of characters) **n** times.

```
String underscores = repeatStr("_", midSize);
String arrows = repeatStr("^", colSize);

System.out.println(String.format("/%s\\%s/%s\\",
    arrows,
    underscores,
    arrows));
```

The fort body contains a beginning `|`, a middle part (**white space**), and an end `|`. The middle part is a blank space with a size of  $2 * n - 2$ . The number of rows used for walls could be found by the given parameters -  $n - 3$ .

```
for (int row = 1; row <= n - 3 ; row++) {
    System.out.println(String.format("|%s|",
        repeatStr(" ", 2 * n - 2)));
}
```

To draw the last row, which is a part of the base, we need to print the beginning `|`, the middle part (**white space**)\_(**white space**), and an end `|`. To do this, we can use the declared `colSize` and `midSize` variables, because we can see from the examples that they are equal to the `_` symbol in the roof.

```
String sideSpace = repeatStr(" ", colSize + 1);
System.out.println(String.format("|%s%s%s|",
    sideSpace,
    underscores,
    sideSpace));
```

We add to the values of the **blank spaces + 1** because we have **one** blank space more in the examples.

The fort's base structure is the same as the roof one. It is made of **two towers** and a middle part. Each tower has a beginning `\`, a middle part `_`, and an end `/`.

```
String middleSpaces = repeatStr(" ", midSize);
underscores = repeatStr("_", midSize);
System.out.println(String.format("\\%s/%s\\%s/",
    underscores,
    middleSpaces,
    underscores));
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/658#0>.

## Problem: Butterfly

Write a program that reads an integer `n` from the console and draws a butterfly with  $2 * n - 1$  columns width and  $2 * (n - 2) + 1$  rows height as in the examples below. Its left and right sides are  $n - 1$  wide.

Input Data

The input is an integer `n` in the range of `[3 ... 1000]`.

Output Data

Print  $2 * (n - 2) + 1$  text lines, representing the butterfly on the console, just like in the examples.

Sample Input and Output

| Input | Output                     | Input | Output                                                                          | Input | Output                                                                                                                                                              |
|-------|----------------------------|-------|---------------------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3     | <pre>* \ /*  @ */ \*</pre> | 5     | <pre>*** \ /*** --- \ /--- *** \ /***  @ *** / \*** --- / \--- *** / \***</pre> | 7     | <pre>***** \ /***** ----- \ /----- ***** \ /***** ----- \ /----- ***** \ /*****  @ ***** / \***** ----- / \----- ***** / \***** ----- / \----- ***** / \*****</pre> |

Hints and Guidelines

From the problem statement, we see that the **input data** will be read from only one line, which will contain an **integer** in the range `[3 ... 1000]`. For this reason, we will use a **variable** of type **int**.

```
Scanner scan = new Scanner(System.in);
int n = Integer.parseInt(scan.nextLine());
```

We can divide the figure into 3 parts - **upper wing**, **body**, and **lower wing**. To draw the upper wing, we need to divide it into parts - a beginning `*`, a middle part `\ /` and an end `*`. After looking at the examples we can say that the beginning has size `n - 2`.

```
int halfRowSize = n - 2;
```

We also see that the upper wing of the butterfly is of size `n - 2`, so we can make a loop that iterates **halfRowSize** times.

```
for (int i = 1; i <= halfRowSize; i++) {
    // ...
}
```

From the examples, we can notice that on an **even** row we have a beginning `-`, a middle part `\ /` and an end `*`, and on an **odd** line - a beginning `*`, a middle part `\ /` and an end `-`.

Therefore, we need to do an **if-else** condition to check whether the line is even or odd and print one of the two types of lines accordingly.

```
for (int i = 1; i <= halfRowSize; i++) {
    if (i % 2 != 0) {
        System.out.println(String.format("%s\\ /%s",
            repeatStr("*", halfRowSize),
            repeatStr("*", halfRowSize)));
    } else {
        System.out.println(String.format("%s\\ /%s",
            repeatStr(" ", halfRowSize),
            repeatStr(" ", halfRowSize)));
    }
}
```

To create the **body of the butterfly**, we can use again the **halfRowSize** variable and print exactly **one** line on the console. The body structure has a beginning (**white space**), a middle **@**, and an end (**white space**).

```
System.out.println(String.format("%s @ %s",
    repeatStr(" ", halfRowSize),
    repeatStr(" ", halfRowSize)));
```

It remains to print the lower wing on the console, which is identical to the upper wing.

```
for (int i = 1; i <= halfRowSize; i++) {
    if (i % 2 != 0) {
        System.out.println(String.format("%s\\ /%s",
            repeatStr("*", halfRowSize),
            repeatStr("*", halfRowSize)));
    } else {
        System.out.println(String.format("%s\\ /%s",
            repeatStr(" ", halfRowSize),
            repeatStr(" ", halfRowSize)));
    }
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/658#1>.

## Problem: Stop

Write a program that reads an **integer** **n** from the console and draws a **STOP** warning sign with size as in the example below.

Input Data

The input is an integer N in the range of [3 ... 1000].

Output Data

Print text lines on the console depicting the STOP warning sign, just as in the examples below.

Sample Input and Output

| Input | Output                                                                                 | Input | Output                                                                                                                                                                          |
|-------|----------------------------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3     | <pre>...._..... ...//_\\... ..//_\\.. .//_\\ //_STOP!_\\ \\_// .\\_//. ..\\_//..</pre> | 6     | <pre>....._..... .....//_\\..... .....//_\\..... ....//_\\.... ...//_\\... ..//_\\.. .//_\\ //_STOP!_\\ \\_// .\\_//. ..\\_//.. ...\\_//... ....\\_//.... .....\\_//.....</pre> |

Hints and Guidelines

From the problem statement, we see that the input data will be read from only one line which contains an integer in the range of [3 ... 1000]. Therefore, we will use a variable of type int.

```
Scanner scanner = new Scanner(System.in);
int n = scanner.nextInt();
```

We can divide the figure into 3 parts - upper, middle, and lower. The upper part consists of two sub-parts - an initial line and lines in which the character expands. The start row is composed of a beginning ., a middle part \_ and an end . . After looking at the examples we can say that the beginning is of size n + 1 and it is better to separate this value into a separate variable.

```
int dots = n + 1;
```

We need to create a second variable in which we will store the value of the middle of the initial row and is of size 2 \* n + 1.

```
int underscores = 2 * n + 1;
```

Once we have declared and initialized the two variables, we can print the start line to the console. Let's not forget to pass a new line to the console with `System.out.println(...)`.

```
System.out.printf("%s%s%s",
```

```
System.out.println();
```

To draw the lines where the sign is getting "wider", we have to create a **loop**, that iterates **n** number of times. The structure of a line consists of a beginning `.`, `//` + middle part `_` + `\\`, and an end `.`. To be able to reuse the created **variables**, we need to decrease the **dots** by 1 and the **underscores** by 2, because we have already **printed** the first row and the dots and underscores in the next line of the figure **decrease**.

```
underscores -= 2;
```

```
dots--;
```

At each subsequent iteration the **beginning** and the **end** decrease with 1, and the **middle part** increases by 2.

```
for (int i = 0; i < n; i++) {
    System.out.printf("%s//%s\\\\\\\\%s",
        repeatStr( strToRepeat: ".", dots ),
        repeatStr( strToRepeat: "_", underscores ),
        repeatStr( strToRepeat: ".", dots ) );
    System.out.println();

    underscores += 2;
    dots--;
}
```

The middle part of the figure has a beginning `//` + `_`, a middle part **STOP!** and an end `_` + `\\`. The count of the underscores `_` is  $(\text{underscores} - 5) / 2$ .

```
System.out.printf("//%sSTOP!%s\\\\\\\\",
    repeatStr( strToRepeat: "_", (underscores - 5) / 2 ),
    repeatStr( strToRepeat: "_", (underscores - 5) / 2 );
System.out.println();
```

The lower part of the figure where the sign **decreases** can be done by creating a **loop** again, which iterates **n** number of times. The structure of a single line is a beginning `.` + `\\`, a middle part `_` and an end `//` + `.`. The number of the **dots** in the first iteration of the loop should be 0 and **increase** by one on each subsequent iteration. Therefore, we can say that the size of the **dots in the lower part of the figure** is equal to **i**.

```
for (int i = 0; i < n; i++){
    System.out.printf("%s\\\\" + strToRepeat + "%s",
        repeatStr(strToRepeat, "."),
        repeatStr(strToRepeat, "_"),
        repeatStr(strToRepeat, "."));
    System.out.println();
}
```

For our program to work properly, we need to **reduce** the number of **i** by 2 at each iteration of the **loop**.

```
for (int i = 0; i < n; i++){
    System.out.printf("%s\\\\" + strToRepeat + "%s",
        repeatStr(strToRepeat, "."),
        repeatStr(strToRepeat, "_"),
        repeatStr(strToRepeat, "."));
    System.out.println();
    underscores -= 2;
}
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/658#2>.

Problem: Arrow

Write a program that reads an **odd integer n** from the console and draws a **vertical arrow** pointing down, with dimensions as in the examples below.

Input Data

The input is an **odd integer number n** in the range of [3 ... 79].

Output Data

Print a vertical arrow on the console with **"#" (hash)** outlining the arrow and **"." (dot)** outlining the rest.

Sample Input and Output

| Input | Output                                    | Input | Output                                                                     |
|-------|-------------------------------------------|-------|----------------------------------------------------------------------------|
| 3     | .###.<br>.#.#.<br>##.##<br>.#.#.<br>..#.. | 5     | ..#####..<br>..#...#..<br>..#...#..<br>..#...#..<br>..#...#..<br>###...### |



| Input | Output |
|-------|--------|
|       |        |

| Input | Output                                           |
|-------|--------------------------------------------------|
|       | .#.....#.<br>..#...#..<br>...#.#...<br>....#.... |

## Hints and Guidelines

From the problem condition, we see that the **input data** will be read from only one line which contains an **integer** in the range of [3 ... 79]. Therefore, we will use a **variable** of type **int**.

```

// Example: Input: 5, Output:
int n = 5;

```

We can divide the figure into **3 parts** - upper, middle, and lower. The **upper part** consists of two subparts – the initial row and the arrow body. From the examples, we see that the number of the **outer dots** in the initial row and in the arrow body are  $(n - 1) / 2$ . We can store this value in an **outerDots** variable.

```
int outerDots = (n - 1) / 2;
```

The number of the **internal dots** in the arrow body is  $(n - 2)$ . We need to create a **variable** named **innerDots**, which will store this value.

```
int innerDots = n - 2;
```

We can see from the examples the structure of the initial line. We must use the **outerDots** and **n** variables declared and initialized by us to print the **starting row**. We must remember that we have to move to a new line with **System.out.println()**.

```

System.out.printf("%s%s%s",
    repeatStr(".", outerDots),
    repeatStr("#", 1),
    repeatStr(".", outerDots));
System.out.println();

```

To draw on the **arrow's body** on the console, we have to create a **loop**, which iterates  $n - 2$  times.

```

for (int i = 0; i < n - 2; i++) {
    System.out.printf("%s#%s#%s",
        repeatStr(".", outerDots),
        repeatStr(".", innerDots),
        repeatStr(".", outerDots));
    System.out.println();
}

```

The middle part of the figure consists of a beginning `#`, a middle part `.` and an end `#`. We see from the examples that the number of the `#` is equal to `outerDots` increased by 1 and that is why we can use the same variable again.

```
System.out.printf(" %s",
    "      .      ",
    "      .      ",
    "      .      " );

System.out.println();
```

To draw the arrow's lower part, we need to assign new values to the variables `outerDots` and `innerDots`.

```
outerDots = 1;
innerDots = 2 * n - 5;
```

The loop, we are going to make must iterates `n - 2` times and we need to print the last row of the figure separately. On each iteration `outerDots` increase by 1, and `innerDots` decrease by 2.

```
for (int i = 0; i < n - 2; i++) {
    System.out.printf(" %s",
        "      .      ",
        "      .      ",
        "      .      " );

    System.out.println();

    outerDots++;
    innerDots -= 2;
}
```

The last row of our figure is made of a beginning `.`, a middle part `#`, and an end `.`. The number of the `.` is equal to `outerDots`.

```
System.out.printf(" %s",
    "      .      ",
    "      .      " );

System.out.println();
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/658#3>.

## Problem: Axe

Write a program that takes as an input an integer `n` and draws an axe with size as shown below. The axe is `5 * N` columns wide.

Input Data

The input is an integer `n` in the range of `[2...42]`.

Output Data

Print on the console an **axe**, exactly like the examples.

Sample Input and Output

| Input | Output                                                                                                                                                                                                                                                |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8     | <pre>-----**----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- *****_*----- *****_*----- *****_*----- *****_*----- -----*_*----- -----*_*----- -----*_*----- -----*****-----</pre> |

| Input | Output                                                           |
|-------|------------------------------------------------------------------|
| 2     | <pre>-----**----- -----*_*----- *****_*----- -----***-----</pre> |

| Input | Output                                                                                                                                  |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 5     | <pre>-----**----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- *****_*----- *****_*----- -----*_*----- -----*****-----</pre> |

Hints and Guidelines

To solve the problem, we first need to calculate the **dashes on the left**, the **middle dashes**, the **dashes on the right**, and the **whole length of the figure**.

```

width = 5 * n;
leftDashes = 3 * n;
middleDashes = 0;
rightDashes =
    width - leftDashes - middleDashes - 2;

```

Once we have declared and initialized the **variables**, we can begin drawing the figure by starting with the **upper part**. From the examples, we can figure out the structure of the **first row**, and create a loop that iterates **n** number of times. At each iteration, of the loop the **middle dashes** increase by 1, and the **dashes on the right** decrease by 1.

```

for (int i = 0; i < n; i++) {
    System.out.printf("%s*s*s*s",
        repeatStr(" ", leftDashes),
        repeatStr(" ", middleDashes),
        repeatStr(" ", rightDashes));
    System.out.println();

    middleDashes++;
    rightDashes--;
}

```

To be able to reuse the created **variables** when drawing the axe's handle, we need to decrease the **middle dashes** by 1 and increase the **dashes on the right** by 1.

```

middleDashes--;
rightDashes++;

```

We can draw the **handle of the axe** by iterating the loop **n - 2** times. From the examples we can understand what its structure is.

```

for (int i = 0; i < n - 2; i++) {
    System.out.printf("%s*s*s",
        repeatStr(" ", leftDashes),
        repeatStr(" ", middleDashes),
        repeatStr(" ", rightDashes));
    System.out.println();

    middleDashes++;
    rightDashes--;
}

```

The **lower part** of the figure we have to divide into 2 subparts - the **head of the axe** and the **last row** of the figure. We will print the **axe's head** on the console by creating a loop that repeats **n / 2 - 1** number of times. At each iteration the **dashes on the left** and the **dashes on the right** decrease by 1, and the **middle dashes** increase by 2.

```

for (int i = 0; i < 10; i++) {
    System.out.printf("%10s",
        repeatStr(10, leftDashes),
        repeatStr(10, middleDashes),
        repeatStr(10, rightDashes));
    System.out.println();

    leftDashes++;
    middleDashes++;
    rightDashes += 2;
}

```

For the **last row** of the figure, we can use the three declared and initialized variables **leftDashes**, **middleDashes**, **rightDashes** again.

```

System.out.printf("%10s",
    repeatStr(10, leftDashes),
    repeatStr(10, middleDashes),
    repeatStr(10, rightDashes));
System.out.println();

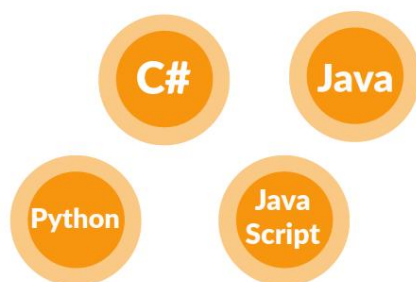
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/658#4>.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)

# Chapter 7.1. Complex Loops

Having learned what **for** loops are and what they serve for, it is now time to get to know other types of loops and some more complex loops constructs. They will expand our knowledge and help us solve more challenging problems. In particular, we will have a look at how the following constructs are used:

- loops with a step
- **while** loops
- **do-while** loops
- infinite loops

In the current, chapter we will also get acquainted with the **break** statement and how to terminate a loop using it. We will also learn how to keep track of errors during the execution of our program, using a **try-catch** block.

## Video

Watch a video lesson about While Loops: <https://youtu.be/dG2Mcibyw3E>.

## Loops With a Step

In chapter "Loops", we learned how the **for** loop works, and we already know when and for what purpose to use it. In this topic, we will pay attention to a specific and very important part of its construction, namely the **step**.

### What is a Step?

The **step** is that part of the **for** loop construction that states how much to increase or decrease the value of its leading variable. It is declared last in the **for** loop block.

It most often has a **size 1** and in that case instead of writing **i += 1** or **i -= 1**, we can use the **i++** or **i--** operators. If we want the step to be different than 1, we use the operator **i += (step size)** to increment and **i -= (step size)** to decrement. For a step size of 10, the loop would look like this:

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());

for (int i = 1; i <= n; i += 10) {
    System.out.println(i);
}
```

Specify  
a step

The following is a series of examples, the solution of which will help us better understand the use of the **step** in the **for** loop.

## Problem: Numbers 1...N with Step 3

Write a program that prints the numbers from 1 to  $n$  with a step 3. For example, if  $n = 100$ , the result will be 1, 4, 7, 10, ..., 94, 97, 100.

We can solve the problem through the following sequence of actions (algorithm):

- We read the number  $n$  from the console input.
- We run a **for** loop from 1 to  $n$  with a step size of 3.
- We print the value of the current step in the body of the loop.

```
Scanner scan = new Scanner(System.in);
int n = Integer.parseInt(scan.nextLine());

for(int i = 1; i <= n; i+=3) {
    System.out.println(i);
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#0>.

## Problem: Numbers N...1

Write a program that prints the numbers from  $n$  to 1 in reverse order (step -1). For example, if  $n = 100$ , the result will be 100, 99, 98, ..., 3, 2, 1.

We can solve the problem in the following way:

- We read the number  $n$  from the console input.
- We run a **for** loop in which we assign **int i = n**.
- We reverse the loop condition: **i >= 1**.
- We define the step size as -1.
- We print the value of the current step in the body of the loop.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());

for (int i = n; i >= 1; i--) {
    System.out.println(i);
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#1>.



## Problem: Powers of Two

In the following example, we will have a look at using the usual step with size 1 but with a slightly different logic in the body of the loop.

Write a program that prints the numbers from 1 to  $2^n$  (two to the power of  $n$ ). For example, if  $n = 10$ , the result will be 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.next());
int num = 1;

for (int i = 0; i <= n; i = i + 1) {
    System.out.println(num);
    num = num * 2;
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#2>.

## Problem: Even Powers of 2

Print the even powers of 2 to  $2^n$ :  $2^0$ ,  $2^2$ ,  $2^4$ ,  $2^8$ , ...,  $2^n$ . For example, if  $n = 10$ , the result will be 1, 4, 16, 64, 256, 1024.

Here is a way to solve the problem:

- We create a variable **num** for the current number to which we assign an initial value of 1.
- We set a value of 2 for the **step** of the loop.
- In the body of the loop, we print the value of the current step and increase the current number **num** 4 times (as per the condition of the problem).

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.next());
int num = 1;

for (int i = 0; i <= n; i += 2) {
    System.out.println(num);
    num = num * 2 * 2;
}
```

## Testing in The Judge System

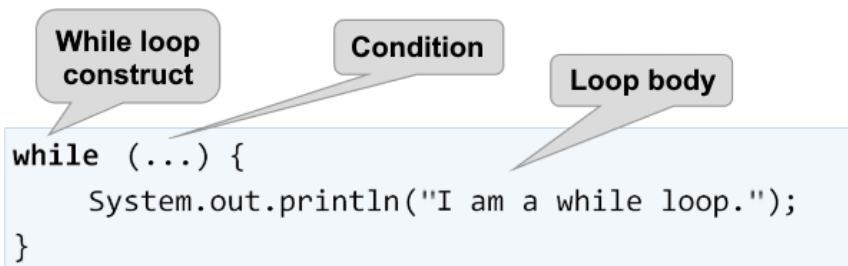
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#3>.

## While Loop

The next type of loops we will get acquainted with are called '**while**' loops. What is specific about them is that they repeat a block of commands, **as long as a given condition is true**. Their structure differs from the **for** loops, and they even have a simplified syntax.

### What is a While Loop?

In programming, the **while** loop is used when we want to **repeat** the execution of a certain logic while **a given condition is met**. We define '**condition**' as any **expression** that returns '**true**' or '**false**'. When the **condition** is **false**, the **while** loop terminates and the program **continues** with the execution of the code following the loop. The **while** loop construction looks like this:



The following is a series of examples, the solution of which will help us better understand the use of the **while** loop.

### Problem: Sequence $2k+1$

Write a program that prints all numbers  $\leq n$  from the sequence: 1, 3, 7, 15, 31, ..., having in mind that each next number = previous number \* 2 + 1.

Here is how we can solve the problem:

- We create a variable **num** for the current number to which we assign an initial value of 1.
- We put the **current number  $\leq n$**  as the loop condition.
- In the **body of the loop**, we print the value of the current number and increase the current number, using the formula from the problem.

Here is a sample representation of the described steps:

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
int k = 1;
while (k <= n) {
    System.out.println(k);
    k = 2 * k + 1;
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#4>.

### Problem: Numbers in Range [1...100]

Enter an integer in the range [1 ... 100]. If it is invalid, input an integer again. In this case, we will consider any number that **is not** in the specified range as invalid.

We can solve the problem through the following algorithm:

- We create a variable **num**, to which we assign the integer value obtained from the console input.
- For a loop condition, we put an expression, that is **true** if the number of the input **is not** in the range specified in the problem condition.
- In **the body of the loop**: we print a message "Invalid number!" on the console, and then we assign a new value to **num** from the console input.
- Having validated the input number, we print its value outside the body of the loop.

Here is a sample representation of the algorithm, using a **while** loop:

```
Scanner scanner = new Scanner(System.in);
int num = Integer.parseInt(scanner.nextLine());

while (num < 1 || num > 100) {
    System.out.println("Invalid number!");
    num = Integer.parseInt(scanner.nextLine());
}

System.out.println("The number is: " + num);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#5>.

## Greatest Common Divisor (GCD)

Before proceeding to the next problem, it is necessary to get acquainted with the definition of the **greatest common divisor** (GCD).

**Definition of GCD:** the greatest common divisor of two **natural** numbers **a** and **b** is the largest number that divides **simultaneously** both **a** and **b** without a remainder. For example:

| a  | b  | GCD |
|----|----|-----|
| 24 | 16 | 8   |

| a  | b | GCD |
|----|---|-----|
| 15 | 9 | 3   |

| a  | b  | GCD |
|----|----|-----|
| 67 | 18 | 1   |
| 12 | 24 | 12  |

| a   | b  | GCD |
|-----|----|-----|
| 10  | 10 | 10  |
| 100 | 88 | 4   |

## The Euclidean Algorithm

In the next problem, we will use **the Euclidean algorithm**, one of the first published algorithms for finding the GCD:

**Until** we reach a remainder of 0:

- We divide the larger number by the smaller one.
- We take the remainder of the division.

**Pseudocode** for the Euclidean algorithm:

```
while b ≠ 0
    var oldB = b;
    b = a % b;
    a = oldB;
print a;
```

## Problem: Greatest Common Divisor (GCD)

Enter **integers a** and **b** and find **GCD(a, b)**.

We will solve the problem using **the Euclidean algorithm**:

- We create **a** and **b** variables, to which we assign the **integer** values obtained from the console input.
- For a loop condition, we put an expression that is **true** if the **b** number is different than 0.
- In **the body of the loop**: we follow the conditions of the pseudocode:
  - We create a temporary variable, to which we assign the **current** value of **b**.
  - We assign a new value to **b**, which is the remainder of the division of **a** and **b**.
  - We assign the **previous** value of the **b** variable to **a**.
- Once the loop is complete and, we have found the GCD, we print it on the screen.

```

Scanner scanner = new Scanner(System.in);
int a = Integer.parseInt(scanner.nextLine());
int b = Integer.parseInt(scanner.nextLine());

while (b != 0) {
    int oldB = b;
    b = a % b;
    a = oldB;
}

System.out.println("GCD = " + a);

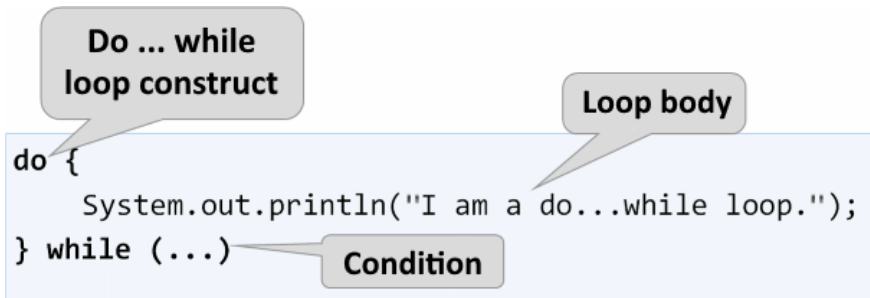
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#6>.

## Do-While Loop

The next type of loop we will get acquainted with is the **do-while**. It resembles the **while** loop by structure, but there is a significant difference between the two. It consists in the fact that **do-while** will execute its body **at least once**. Why is this happening? At **do-while** loop construction, the **condition** is always checked **after** its body, which ensures that the code will be **executed** after the **first loop iteration**, and the **check for the loop termination** will be applied to each subsequent iteration of the **do-while**.



The following is the usual series of sample problems, the solution of which will help us better understand the use of the **do-while** loop.

## Problem: Factorial

Calculate  $n! = 1 * 2 * 3 * \dots * n$ , where  $n$  is a natural number. For example, if  $n = 5$ , the result will be:  $5! = 1 * 2 * 3 * 4 * 5 = 120$ .

Here is how we can calculate the factorial:

- We create the variable **n**, to which we assign the **integer** value obtained from the console input.
- We create another variable - **fact**, with the initial value of 1. We will use it to calculate and store the factorial.

- For a loop condition, we will use `n > 1`, since each time we perform the calculations in the body of the loop, we will reduce the value of `n` by 1.
- In the body of the loop:
  - We assign a new value to the `fact`, which is the result of multiplying the current values of `fact` and `n`.
  - We decrease the value of `n` by -1.
- We print the final value of the factorial outside the body of the loop.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
int fact = 1;

do {
    fact = fact * n;
    n--;
} while (n > 1);

System.out.println(fact);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#7>.

## Problem: Sum Digits

Sum the digits of a **positive** integer `n`. For example, if `n = 5634`, the result will be:  $5 + 6 + 3 + 4 = 18$ .

We can use the following idea to solve the problem:

- We create the variable `n` to which we assign a value equal to the number entered by the user.
- We create a second variable - `sum`, with the initial value of 0. We will use it to calculate and store the result.
- For a loop condition, we will use `n > 0`, since we will remove the last digit of `n` after each calculation of the result in the body of the loop.
- In the body of the loop:
  - We assign a new value to `sum`, equals to the sum of the current value of `sum` and the last digit of `n`.
  - We assign a new value to `n`, which is the result of removing the last digit of `n`.
- We print the final value of the sum outside the body of the loop.

```

Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
int sum = 0;

do {
    sum = sum + (n % 10);
    n = n / 10;
} while (n > 0);

System.out.println("Sum of digits: " + sum);

```



`n % 10`: returns the last digit of the number `n`.  
`n / 10`: deletes the last digit of `n`.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#8>.

## Infinite Loops and `break` Operator

So far, we got acquainted with different types of loops and learned about their constructions and how they are applied. Now, we need to understand what an **infinite loop** is, when it occurs and how we can **terminate** its execution through the **break** operator.

### What is an Infinite Loop?

An infinite loop **repeats** infinitely the execution of its body. For **while** and **do-while** loops the check for the end of a loop is a conditional expression that **always** returns **true**. An infinite **for** occurs when there is **no condition** to end the loop.

Here is what an **infinite while** loop looks like:

```

while (true) {
    System.out.println("Infinite loop");
}

```

And here is what an **infinite for** loop looks like:

```

for (; ; ) {
    System.out.println("Infinite loop");
}

```

### The `break` Operator

We already know that the infinite loop executes a specific code to infinity, but what if we want to forcibly exit the loop under a given condition at some point? In this situation, the **break** operator comes in handy.



The **break** operator stops the loop execution at the moment it is called and continues from the first line after the end of the loop. This means that the current iteration of the loop will not be completed and, accordingly the rest of the code in the body of the loop will not be executed.

## Problem: Prime Number Checking

The next problem we are going to solve requires conducting a **prime number check**. Before proceeding with it, let's remember what prime numbers are.

**Definition:** an integer is **prime** when it is divisible without remainder only to itself and 1. By definition, prime numbers are positive and greater than 1. The smallest prime number is 2.

We can assume that an integer  $n$  is prime if  $n > 1$  and  $n$  is not divisible by any of the numbers between 2 and  $n-1$ .

The first few prime numbers are: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, ...

In contrast, **non-prime (composite) numbers** are such numbers that are composed of the product of prime numbers.

Here are a few examples of composite numbers:

- $10 = 2 * 5$
- $42 = 2 * 3 * 7$
- $143 = 11 * 13$

**Verification algorithm** whether an integer is **prime**: we check if  $n > 1$  and  $n$  are divisible by 2, 3, ...,  $n-1$  without remainder.

- If it is divisible by any of the numbers, it is **composite**.
- If it is not divisible by any of the numbers, then it is **prime**.



We can optimize the algorithm by checking the divisors to  $\sqrt{n}$  instead of  $n-1$ . Think about the reason why.

## Problem: Check Prime

Check if a number  $n$  is prime. To do so, we will check if  $n$  is divisible by the numbers between 2 and  $\sqrt{n}$ .

Here is the algorithm for prime number verification, described in steps:

- We create the variable  $n$ , to which we assign the **integer** value obtained from the console input.



- We create a boolean variable - **isPrime** with an initial value **true** if the number is equal or greater than two (since by definition, numbers such as 0, 1, -1, and -2 are non-prime). We assume that a number is prime until proven otherwise.
- We create a **for** loop, and we assign 2 as an initial value of the loop variable and its current value **<= √n** as a condition. The step of the loop is 1.
- In the body of the loop, we check if **n** divided by the current value has a remainder. If there is no remainder, we change the value of **isPrime** to **false** and forcibly terminate the loop with a **break** operator.
- Depending on the **isPrime** value, we print whether the number is prime (**true**) or composite (**false**).

Here is a sample implementation of the described algorithm:

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
boolean prime = n >= 2;

for (int i = 2; i <= Math.sqrt(n); i++) {
    if (n % i == 0) {
        prime = false;
        break;
    }
}

if (prime) {
    System.out.println("Prime");
} else {
    System.out.println("Not prime");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#9>.

## Problem: Enter Even Number

Write a program that checks if a number **n** is even – if it is, prints it on the screen. An even number is each number that is divisible by 2 without a remainder. In the case of an invalid number, it should be re-input, and a message stating that the number is not even should be displayed.

Here is an idea of how to solve the problem:

- We create a variable **n**, to which we assign an initial value of **0**.
- We create an infinite **while** loop and put **true** as a condition.

- In the body of the loop:
  - We take the integer obtained by the console input and assign it to `n`.
  - If **the number is even**, we exit the loop with a **break**.
  - Otherwise, we display a message stating that **the number is not even**. The iterations continue until an even number is entered.
- We print the even number on the screen.

Here is a sample implementation of the idea:

```
Scanner scanner = new Scanner(System.in);
int n = 0;
while (true) {
    System.out.print("Enter even number: ");
    n = Integer.parseInt(scanner.nextLine());
    if (n % 2 == 0) {
        break;
    }
    System.out.println("The number is not even.");
}
System.out.printf("Even number entered: %d.", n);
```

Note: although the code above is correct, it will not work if the user enters text instead of numbers, for example, **Invalid number**. Then the parsing of the input text to a number will break, and the program will display an **error message (exception)**. Later in this chapter, we will learn how to deal with this problem and how to catch and handle exceptions using the **try-catch** construct.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#10>.

## Nested Loops and the `break` Operator

Having learned what **nested loops** are and how the **break** operator performs, it is time to understand how they both work together. For a better understanding, let's write a step-by-step **program** that should make all possible combinations of **pairs of numbers**. The first number of the combination is increasing from 1 to 3, and the second one is decreasing from 3 to 1. The problem must continue running until **`i + j` is not equal to 2** (i.e., **`i = 1` and `j = 1`**).

The desired result is:

```
"C:\Program Files\Java\jdk1.8.0_131\bin\java" ...
1 3
1 2
Process finished with exit code 0
```

Here is one **wrong solution**, which seems correct at first glance:

```
for (int i = 1; i <= 3; i++) {
    for (int j = 3; j >= 1; j--) {
        if (i + j == 2) {
            break;
        }
        System.out.println(i + " " + j);
    }
}
```

If we leave our program this way, the result will be the following:

```
"C:\Program Files\Java\jdk1.8.0_131\bin\java" ...
1 3
1 2
2 3
2 2
2 1
3 3
3 2
3 1
Process finished with exit code 0
```

Why does this happen? As we can see, the result **is missing 1 1**. When the program reaches the point that **i 1** and **j 1**, it enters the **if** check and performs the **break** operation. Thus, we **exit the inner loop**, but then the execution of the outer loop continues. **i** increases, the program enters the inner loop and prints the result.



When we use the **break** operator in a **nested loop**, it interrupts the execution of the inner loop **only**.

What is **the correct solution**? One way of solving this problem is by declaring a **bool variable**, which keeps track of whether the loop should continue. In case of need to exit (from all nested loops), the variable is set to **'true'** and exits the internal cycle with a **'break'**, and at the next check, will exit from the outer loop. Here is a sample implementation of this idea:

```

boolean hasToEnd = false;
for (int i = 1; i <= 3; i++) {
    if (hasToEnd == false) {
        for (int j = 3; j >= 1; j--) {
            if (i + j == 2) {
                hasToEnd = true;
                break;
            }
            System.out.println(i + " " + j);
        }
    }
}

```

Thus, when  $i + j = 2$ , the program will set the **hasToEnd = true** and exit the inner loop. Upon the next iteration of the outer loop due to the **if** check, the program will not reach the inner loop and interrupt its execution.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#11>.

## Error Handling: try-catch Construction

The last thing we will get familiar with within this chapter is how to "catch" errors with the **try-catch** construction.

### What is a try-catch?

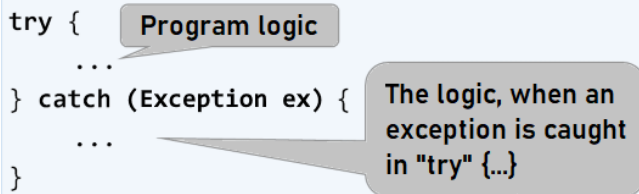
The **try-catch** program construction is used to intercept and handle exceptions (errors) during the program execution.

In programming, **exceptions** are notifications of an event that disrupts the normal operation of a program. Such exceptional events **interrupt the execution** of our program, and it is looking for something to handle the situation. If it does not find it, the exception is printed on the console (the program "fails"). If it finds, **the exception is processed** and the program continues its normal execution without "failing". In a bit, we will see how this happens.

When an exception occurs, the exception is said to have been **"thrown"** (throw exception). Hence the expression **"catch exception"**.

### The try-catch Construction

The **try-catch** construction has different types, but for now, we will get acquainted only with the most basic of them, where the **catch** block will intercept any error in a variable named **ex**



```
try {
    ...
} catch (Exception ex) {
    ...
}
```

**Program logic**

**The logic, when an exception is caught in "try" {...}**

In the next problem, we will see how to handle a situation where the user enters an input other than a number (for example, **string** instead of **int**) using **try-catch**.

## Problem: Enter Even Number (with Text Input)

Write a program that checks if a number **n** is even – if it is, prints it on the screen. Upon entering an **invalid number**, a message appears stating that the input data is not a valid number, and another input data must be made.

Here is how we can solve the problem:

- We create an infinite **while** loop, for which we set **true** as a condition.
- In the body of the loop:
  - We create a **try-catch** construction.
  - In the **try** block, we write the programming logic for reading the user input, parsing it to a number, and then checking for an even number.
  - If **the number is even**, we print it and exit the loop (with **break**). The program is done, and it ends.
  - If **the number is odd**, we print a message that an even number is required, without exiting the loop (since we want to repeat the step by entering input data).
  - If we **catch an exception** upon executing the **try** block, we write a message for an invalid input number, and the loop is repeated because we do not explicitly exit it.

Here is a sample implementation of the described idea:

```
Scanner scanner = new Scanner(System.in);
int n = 0;
while (true) {
    try {
        System.out.print("Enter even number: ");
        n = Integer.parseInt(scanner.nextLine());
        if (n % 2 == 0) {
            System.out.printf("Even number entered: %d.", n);
            break;
        }
        System.out.println("The number is not even.");
    } catch (Exception ex) {
        System.out.println("Invalid number.");
    }
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#12>.

Now the solution should always work whether we enter integers, invalid numbers (for example, too many digits), or texts that do not contain numbers.

## Problems with Loops

In this chapter, we have introduced several new types of loops that we can use to make repetitions with more complex program logic. Let's solve several problems using the new knowledge.

### Problem: Fibonacci

In mathematics, Fibonacci numbers form a series that looks like this: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

The formula to form the sequence is:

```
F0 = 1
F1 = 1
Fn = Fn-1 + Fn-2
```

### Sample Input and Output

| Input (n) | Output | Comment                 |
|-----------|--------|-------------------------|
| 10        | 89     | $F(11) = F(9) + F(8)$   |
| 5         | 8      | $F(5) = F(4) + F(3)$    |
| 20        | 10946  | $F(20) = F(19) + F(18)$ |
| 0         | 1      |                         |
| 1         | 1      |                         |

Write a program in which we enter an **integer** number **n** and calculate the **n-number** of Fibonacci.

### Hints and Guidelines

An idea to solve the problem:

- We create a **n variable**, to which we assign the **integer** value obtained from the console input.

- We create **f0** and **f1** variables to which we assign value **1** since this is how the sequence begins.
- We create a **for** loop with a condition current value **i < n - 1**.
- In the **body of the loop**:
  - We create a **temporary** variable **fNext** to which we assign the next number of the Fibonacci sequence.
  - We assign the current value of **f1** to **f0**.
  - We assign the value of the temporary variable **fNext** to **f1**.
- We print the n-number of Fibonacci outside the loop.

Sample implementation:

```
Scanner s = new Scanner(System.in);
int n = Integer.parseInt(s.nextLine());

int f0 = 1;
int f1 = 1;

for(int i = 0; i<n-1; i++) {
    int nF = f0 + f1;
    f0 = f1;
    f1 = nF;
}

System.out.println(f1);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#13>.

## Problem: Number Pyramid

Print the numbers 1... n in a pyramid as in the examples below. On the first line, we print one number on the second line, we print two numbers, on the third line, we print three numbers, etc., until the numbers run out. On the last line, we print as many numbers as we have left until we get to n.

### Sample Input and Output

| Input | Output                 |
|-------|------------------------|
| 7     | 1<br>2 3<br>4 5 6<br>7 |

| Input | Output          |
|-------|-----------------|
| 5     | 1<br>2 3<br>4 5 |

| Input | Output                        |
|-------|-------------------------------|
| 10    | 1<br>2 3<br>4 5 6<br>7 8 9 10 |

## Hints and Guidelines

We can solve the problem with **two nested loops** (by rows and columns) by printing in them and leaving when reaching the last number. Here is the idea, described in more detail:

- We create a **n** variable, to which we assign the integer value obtained from the console input.
- We create a **num** variable with an initial value of 1. It will keep the number of printed numbers. At each iteration, we will **increase** it by **1** and print it.
- We create an **outer for** loop, which will be responsible for the **rows** in the table. We name the loop variable as a **row** and assign it an initial value of 0. We set **row < n** as a condition. The size of the step is 1.
- In the body of the loop, we create an **inner for** loop, which will be responsible for the **columns** in the table. We name the loop variable as **col** and assign it an initial value of 0. We set **col < row** (**row** = number of digits per line) as a condition. The size of the step is 1.
- In the body of the loop:
  - We check if **col > 1** and if yes – we print space. If we don't perform this check and directly print space, we will have an unnecessary one at the beginning of each line.
  - We **print** the number **num** in the current cell of the table and **increase it by 1**.
  - We perform a check for **num > n**. If **num** is greater than **n**, we **break** the inner loop.
- We print a **blank line** to proceed to the next one.
- We check again if **num > n**. If it's greater, we **interrupt** the execution of our **program** with a **break**.

Here is a sample implementation:

```
int n = Integer.parseInt(scanner.nextLine());
int num = 1;

for (int row = 1; row <= n; row++) {
    for (int col = 1; col <= row; col++) {
        if (col > 1) {
            System.out.print(" ");
        }
        System.out.print(num);
        num++;

        if (num > n) {
            break;
        }
    }
}
```



```

        System.out.println();

        if (num > n) {
            break;
        }
    }
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#14>.

## Problem: Number Table

Print the numbers 1 ... n in a table as per the examples below.

### Sample Input and Output

| Input | Output                         | Input | Output                                       |
|-------|--------------------------------|-------|----------------------------------------------|
| 3     | <pre> 1 2 3 2 3 2 3 2 1 </pre> | 4     | <pre> 1 2 3 4 2 3 4 3 3 4 3 2 4 3 2 1 </pre> |

## Hints and Guidelines

We can solve the problem with **two nested loops** and a few calculations inside:

- We read from the console the table size in an integer variable **n**.
- We create a **for** loop, which is responsible for the table rows. We name the loop variable as a **row** and assign it an initial **value 0**. We set **row < n** as a condition. The size of the step is 1.
- In **the body of the loop**, we create a nested **for** loop, which is responsible for the table columns. We name the loop variable as **col** and assign it an initial **value of 0**. We set **col < n** as a condition. The size of the step is 1.
- In **the body of the nested loop**:
  - We create a **num** variable, to which we assign the result of **current row + current column + 1** (+1, since we start counting from 0).
  - We perform a check if **num > n**. If **num** is **greater** than n, we assign a new value of **two times n - current value of num** to **num**. We do this to not exceed **n** in any of the table cells.
    - We print the number from the current table cell.
- We print a **blank line** in the outer loop to proceed to the next one.

```

Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());

for (int row = 0; row < n; row++) {
    for (int col = 0; col < n; col++) {
        int num = row + col + 1;
        if (num > n) {
            num = 2 * n - num;
        }

        System.out.print(num + " ");
    }
    System.out.println();
}

```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/659#15>.

## What Have We Learned from This Chapter?

We can use the **for** loops with a **step**:

```

for (int i = 1; i <= n; i+=3) {
    System.out.println(i);
}

```

The **while** / **do-while** loops are repeated as long as a given **condition** is true:

```

int num = 1;
while (num <= n) {
    System.out.println(num++);
}

```

If we need to **interrupt** the loop execution, we use the **break** operator:

```

int n = 0;
while (true) {
    n = Integer.parseInt(scanner.nextLine());
    if (n % 2 == 0) {
        break; // even number -> exit from the loop
    }
    System.out.println("The number is not even.");
}

```

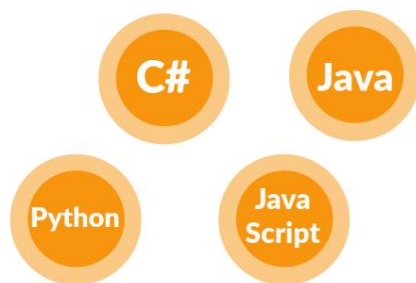
```
System.out.println("Even number entered: {0}", n);
```

We already know how to catch **errors** during the execution of our program:

```
try {
    System.out.println("Enter even number: ");
    int n = Integer.parseInt(scanner.nextLine());
} catch (Exception ex) {
    System.out.println("Invalid number.");
}
// If Integer.parseInt(...) fails, the catch { ... } block will be
// executed
```

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



[Apply now](#)

[softuni.org/apply](https://softuni.org/apply)

## Chapter 7.2. Complex Loops – Exam Problems

We have already **learned** how to execute a particular **command block more than once** by using a **for** loop. In the previous chapter we **reviewed** several other **loop constructions**, which could help us solve more complex problems:

- loops with a **step**
- nested loops
- **while** loops
- **do-while** loops
- endless loops and breaking out of loops (**break** operator)
- **try-catch** construction

### Exam Problems

Let's consolidate our knowledge by solving several more complex loop problems from practical exams.

#### Problem: Stupid Password Generator

Write a program that enters two integers **n** and **l** and generates, in alphabetical order, all possible **"stupid" passwords** that consist of the following **5 characters**:

- Character 1: a digit from **1** to **n**.
- Character 2: a digit from **1** to **n**.
- Character 3: a small letter from the first **l** letters of the Latin alphabet.
- Character 4: a small letter from the first **l** letters of the Latin alphabet.
- Character 5: a digit from **1** to **n**, **greater than the first 2 digits**.

#### Input Data

The input is read from the console and consists of **two integers**: **n** and **l** within the range **[1 ... 9]**, each on a single line.

#### Output Data

Print on the console **all "stupid" passwords in alphabetical order**, separated by **space**.

#### Sample Input and Output

| Input | Output                              | Input | Output            |
|-------|-------------------------------------|-------|-------------------|
| 2     | 11aa2 11ab2 11ac2 11ad2 11ba2 11bb2 | 3     | 11aa2 11aa3 12aa3 |
| 4     | 11bc2 11bd2 11ca2 11cb2 11cc2 11cd2 | 1     | 21aa3 22aa3       |

| Input | Output                  | Input | Output |
|-------|-------------------------|-------|--------|
|       | 11da2 11db2 11dc2 11dd2 |       |        |

| Input | Output                                                                                                                                                                  | Input | Output                                                                  |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------|
|       | 11aa2 11aa3 11aa4 11ab2 11ab3 11ab4<br>11ba2 11ba3 11ba4 11bb2 11bb3 11bb4<br>12aa3 12aa4 12ab3 12ab4 12ba3 12ba4<br>12bb3 12bb4 13aa4 13ab4 13ba4 13bb4                |       | 11aa2 11aa3<br>11ab2 11ab3<br>11ba2 11ba3<br>11bb2 11bb3                |
| 4     | 21aa3 21aa4 21ab3 21ab4 21ba3 21ba4                                                                                                                                     | 3     | 12aa3 12ab3                                                             |
| 2     | 21bb3 21bb4 22aa3 22aa4 22ab3 22ab4<br>22ba3 22ba4 22bb3 22bb4 23aa4 23ab4<br>23ba4 23bb4 31aa4 31ab4 31ba4 31bb4<br>32aa4 32ab4 32ba4 32bb4 33aa4 33ab4<br>33ba4 33bb4 | 2     | 12ba3 12bb3<br>21aa3 21ab3<br>21ba3 21bb3<br>22aa3 22ab3<br>22ba3 22bb3 |

Hints and Guidelines

We can split the solution into three parts:

- **Read the input** – for the current problem this consists of reading two numbers **n** and **l**, each on a single line.
- **Process the input** – use nested loops to iterate through every possible character for each of the five password characters.
- **Print the result** – print each "stupid" password that meets the requirements.

Read The Input

To **read** the **input** we will declare two integer variables **int: n** and **l**.

```
Scanner scan = new Scanner(System.in);  
  
int n = Integer.parseInt(scan.nextLine());  
int l = Integer.parseInt(scan.nextLine());
```

Let's declare and initialize the **variables**, which will **store the characters** of the password: for the **numeric** characters - **int** - **d1, d2, d3**, and for the **letters** - **char** - **l1, l2**.

Processing The Input Data and Printing Output

We have to create **five** nested **for** loops, one for each variable. To ensure that the last digit **d3** is **greater** than the first two, we will use the built-in function **Math.max(...)**.

```

for (int d1 = 1; d1 <= n; d1++) {
    for (int d2 = 1; d2 <= n; d2++) {
        for (char l1 = 'a'; l1 < 'a' + 1 ; l1++) {
            for (char l2 = 'a'; l2 < 'a' + 1; l2++) {
                for (int d3 = Math.max(d1, d2) + 1; d3 <= n; d3++) {
                    System.out.print(String.format("%d%d%c%c%d ",
                                                    d1, d2, l1, l2, d3));
                }
            }
        }
    }
}

```

Do You Know That...?

- We can define a **for** loop with a variable of type **char**:

```
for (char ch = 'a'; ch < 'z'; ch++)
```

- We can read a variable of type **char** from the console by using the following structure:

```
char ch = (char)System.in.read();
```

- We can convert an **uppercase** character to **lowercase** by using the built-in function in Java:

```
Character.toLowerCase(ch);
```

- When reading characters from the console, we can directly convert uppercase letters to lowercase letters by combining the two lines above:

```
char ch = Character.toLowerCase((char)System.in.read());
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/660#0>.

## Problem: Magic Combination

Write a program that reads a **magic** integer number and generates all possible **6-digit** numbers, for which the product of their digits is equal to the magic number.

Example: "Magic number" → 2

- 111112 →  $1 * 1 * 1 * 1 * 1 * 2 = 2$
- 111121 →  $1 * 1 * 1 * 1 * 2 * 1 = 2$
- 111211 →  $1 * 1 * 1 * 2 * 1 * 1 = 2$
- 112111 →  $1 * 1 * 2 * 1 * 1 * 1 = 2$
- 121111 →  $1 * 2 * 1 * 1 * 1 * 1 = 2$
- 211111 →  $2 * 1 * 1 * 1 * 1 * 1 = 2$

Input Data

The input is read from the console and consists of **one integer** within the range [1 ... 600 000].

Output Data

Print on the console **all magic numbers**, separated by **space**.

Sample Input and Output

| Input | Output                                                   | Input | Output                                                                                                                                                                                                                                                                                                                                                                                                                                   | Input          | Output |
|-------|----------------------------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|--------|
| 2     | 111112<br>111121<br>111211<br>112111<br>121111<br>211111 | 8     | 111118 111124 111142 111181 111214<br>111222 111241 111412 111421 111811<br>112114 112122 112141 112212 112221<br>112411 114112 114121 114211 118111<br>121114 121122 121141 121212 121221<br>121411 122112 122121 122211 124111<br>141112 141121 141211 142111 181111<br>211114 211122 211141 211212 211221<br>211411 212112 212121 212211 214111<br>221112 221121 221211 222111 241111<br>411112 411121 411211 412111 421111<br>811111 | 53<br>14<br>41 | 999999 |

Hints and Guidelines

The **solution** of the magic numbers problem follows **the same** principle (again we have to generate all combinations for several elements). Try to solve the problem yourself by following these steps.

- Declare and initialize a **variable** of type **int** and read the **input** from the console.
- Nest **six for loops** into each other, one for each digit of the required 6-digit numbers.
- In the last loop, use an **if** construction to check if the **product** of the six digits is **equal** to the **magic** number.



```

Scanner scan = new Scanner(System.in);

int magic = Integer.parseInt(scan.nextLine());

for (int d1 = 0; d1 <= 9; d1++) {
    for (int d2 = 0; d2 <= 9; d2++) {
        for (int d3 = 0; d3 <= 9; d3++) {
            for (int d4 = 0; d4 <= 9; d4++) {
                for (int d5 = 0; d5 <= 9; d5++) {
                    for (int d6 = 0; d6 <= 9; d6++) {
                        if (d1 * d2 * d3 * d4 * d5 * d6 == magic) {
                            System.out.println("Finding " + magic + ":",
                                d1, d2, d3, d4, d5, d6);
                        }
                    }
                }
            }
        }
    }
}

```

In the previous chapter, we reviewed other loop constructions. Let's check a sample solution of the same problem, where we use a **while** cycle. First, we have to store the **entered magic number** in a suitable variable. After that, we initialize 6 variables - one for each of the six digits of the numbers we have to output as a **result**.

```

Scanner scanner = new Scanner(System.in);
int magic = Integer.parseInt(scanner.nextLine());

int d1 = 0;
int d2;
int d3;
int d4;
int d5;
int d6;

```

After that, we start constructing the **while** loops.

- We initialize the **first digit**: **d1 = 0**.
- We set a **condition for each loop**: the digit has to be lower than or equal to 9.
- At the **start** of each loop, we set the value of the **next** digit, in this case: **d2 = 0**. In the nested **for** loops, we initialize the variables in the inner loops with each incrementation of the outer loops. We want to achieve the same behavior here as well.
- At the **end** of each loop, we **increase** the digit by one: **d++**.
- In the **innermost** loop we do the **check** and, if necessary, print on the console.

```

while (d1 <= 9) {
    d2 = 0;
    while (d2 <= 9) {
        d3 = 0;
        while (d3 <= 9) {
            d4 = 0;
            while (d4 <= 9) {
                d5 = 0;
                while (d5 <= 9) {
                    d6 = 0;
                    while (d6 <= 9) {
                        if (d1 * d2 * d3 * d4 * d5 * d6 == magic) {
                            System.out.println(String.format("%d%d%d%d%d%d",
                                d1, d2, d3, d4, d5, d6));
                        }
                        d6++;
                    }
                    d5++;
                }
                d4++;
            }
            d3++;
        }
        d2++;
    }
    d1++;
}

```

Let's remove the **if** check from the innermost loop. Now, let's initialize each variable outside the loops and delete the lines **dx = 0**. After we start the program, we get only 10 results. Why? And what if we use a **do-while** loop? In our case this loop doesn't seem suitable, does it? Think about why. Of course, you can also solve the problem by using an endless loop.

```

int d1 = 0;
while (true) {
    int d2 = 0;
    while (true) {
        System.out.println(String.format("%d%d", d1, d2));
        d2++;
        if (d2 == 10) {
            break;
        }
    }
    d1++;
    if (d1 == 10) {
        break;
    }
}

```

As we can see, we can solve the same problem by using different types of loops. Of course, there is a most suitable choice for each problem. To practice each type of loop, try to solve each of the following problems by using every type of loop we've learned so far.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/660#1>.

## Problem: Stop Number

Write a program that prints on the console all numbers from **N** to **M** that are **divisible by 2 and 3 without remainder**, in **reversed order**. We will read from the console **one** additional "stop" number **S**. If any of the numbers divisible by 2 and 3 **is equal to the stop number**, it **should not be printed** and the program should end. **Otherwise**, print all numbers up to **N** that meet the condition.

### Input Data

Read from the console 3 numbers, each on a single line:

- **N** - integer number:  $0 \leq N < M$ .
- **M** - integer number:  $N < M \leq 10000$ .
- **S** - integer number:  $N \leq S \leq M$ .

### Output Data

Print on the console, on a single line, all numbers that meet the condition, separated by space.

### Sample Input and Output

| Input         | Output        | Comments                                                                                                                                                                                      |
|---------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1<br>30<br>15 | 30 24 18 12 6 | The numbers from 30 to 1 that are divisible by both 2 and 3 without remainder are: 30, 24, 18, 12 and 6. The number 15 <b>is not equal</b> to any of them, so the sequence <b>continues</b> . |

| Input         | Output      | Comments                                                                                                                                                                                 |
|---------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1<br>36<br>12 | 36 30 24 18 | The numbers from 36 to 1 that are divisible by both 2 and 3 without remainder are: 36, 30, 24, 18, 12 and 6. The number 12 <b>is equal</b> to the stop number, so we <b>stop at 18</b> . |

### Hints and Guidelines

The problem can be divided into **four** logical parts:

- **Read** the input from the console.

- **Check** all numbers in the given range by running a **loop**.
- **Check** if each of the numbers within the given range meets the required conditions.
- **Print** the numbers.

The **first** part is easy - we read **three** integer numbers from the console, so we use **int**.

We're also familiar with the **second** part - the initialization of a **for** loop. There's a little **catch** here - the problem requires us to print the numbers in **reversed order**. This means that the **initial** value of the variable **i** will be the **greater number**, and from the examples, we can see that this is **M**. Thus, the **final** value of **i** should be **N**. The fact that we will print the results in reversed order and the values of **i** suggest that the step will **decrease i** by 1.

```
for (int i = m; i >= n ; i--)
```

After we have initialized the **for** loop, it is time for the **third** part of the problem - to **check** the condition if the given **number is divisible by both 2 and 3 without remainder**. We will do this by using one simple **if** condition, that we will let the reader construct.

The other **tricky** part in this problem is that, apart from the check above, we need an **additional** one - if the **number is equal to the "stop" number** entered from the console on the third line. To reach this check, the number we're checking has to pass the check above. That's why we add one more **if** statement **nested in the previous one**. If the condition is **true**, we have to stop printing. We can achieve this with the **break** operator which will lead us **out** of the **for** loop.

Accordingly, if the **condition** that checks whether the number is equal to the "stop" number returns **false**, our program should **continue printing**. This covers the **fourth and last** part of our program.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/660#2>.

## Problem: Special Numbers

Write a program that **reads an integer N** and generates all possible "**special**" numbers from **1111 to 9999**. To be considered "**special**", a number must satisfy the **following condition**:

- **N must be divisible by each of its digits without remainder.**

**Example:** if **N = 16**, 2418 is a special number:

- $16 / 2 = 8$  without remainder
- $16 / 4 = 4$  without remainder
- $16 / 1 = 16$  without remainder
- $16 / 8 = 2$  without remainder

Input Data

The input is read from the console and consists of **one integer** within the range [1 ... 600 000].

Output Data

Print on the console **all special numbers**, separated by **space**.

Sample Input and Output

| Input | Output                                                                                | Comments                                                                                                                             |
|-------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 3     | 1111 1113 1131 1133 1311 1313 1331<br>1333 3111 3113 3131 3133 3311 3313<br>3331 3333 | 3 / 1 = 3 without<br>remainder<br>3 / 3 = 1 without<br>remainder<br>3 / 3 = 1 without<br>remainder<br>3 / 3 = 1 without<br>remainder |

| Input | Output | Input | Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11    | 1111   | 16    | 1111 1112 1114 1118 1121 1122 1124 1128 1141<br>1142 1144 1148 1181 1182 1184 1188 1211 1212<br>1214 1218 1221 1222 1224 1228 1241 1242 1244<br>1248 1281 1282 1284 1288 1411 1412 1414 1418<br>1421 1422 1424 1428 1441 1442 1444 1448 1481<br>1482 1484 1488 1811 1812 1814 1818 1821 1822<br>1824 1828 1841 1842 1844 1848 1881 1882 1884<br>1888 2111 2112 2114 2118 2121 2122 2124 2128<br>2141 2142 2144 2148 2181 2182 2184 2188 2211<br>2212 2214 2218 2221 2222 2224 2228 2241 2242<br>2244 2248 2281 2282 2284 2288 2411 2412 2414<br>2418 2421 2422 2424 2428 2441 2442 2444 2448<br>2481 2482 2484 2488 2811 2812 2814 2818 2821<br>2822 2824 2828 2841 2842 2844 2848 2881 2882<br>2884 2888 4111 4112 4114 4118 4121 4122 4124<br>4128 4141 4142 4144 4148 4181 4182 4184 4188<br>4211 4212 4214 4218 4221 4222 4224 4228 4241<br>4242 4244 4248 4281 4282 4284 4288 4411 4412<br>4414 4418 4421 4422 4424 4428 4441 4442 4444<br>4448 4481 4482 4484 4488 4811 4812 4814 4818<br>4821 4822 4824 4828 4841 4842 4844 4848 4881 |

| Input | Output | Input | Output                                                                                                                                                                                                                                                                                                                                                              |
|-------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        |       | 4882 4884 4888 8111 8112 8114 8118 8121 8122<br>8124 8128 8141 8142 8144 8148 8181 8182 8184<br>8188 8211 8212 8214 8218 8221 8222 8224 8228<br>8241 8242 8244 8248 8281 8282 8284 8288 8411<br>8412 8414 8418 8421 8422 8424 8428 8441 8442<br>8444 8448 8481 8482 8484 8488 8811 8812 8814<br>8818 8821 8822 8824 8828 8841 8842 8844 8848<br>8881 8882 8884 8888 |

Hints and Guidelines

Solve the problem on your own using what you have learned from the previous two problems. Keep in mind the difference between operators for **integer division ( / )** and **division with remainder ( % )** in Java.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/660#3>.

Problem: Digits

Write a program that reads from the console 1 integer within the range [100 ... 999], and then prints it a predefined number of times — modifying it before each print, as follows:

- If the number is divisible by 5 without remainder, **subtract** from it **its first digit**.
- If the number is divisible by 3 without remainder, **subtract** from it **its second digit**.
- If none of the conditions above is true, **add** to it **its third digit**.

Print on the console **N lines**, each containing **M numbers** that are the result of the actions above, where:

- N = sum of the first and second digits of the number.
- M = sum of the first and third digits of the number.

Input Data

The input is read from the **console** and is an integer within the range [100 ... 999].

Output Data

Print on the console **all integers** that are the result of the calculations above in the respective number of rows and columns as in the examples.

## Sample Input and Output

| Input | Output                                                                                                                                                                                                                                                                                                                                                                                             | Comments                                                                                                                                                                                                                                           |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 376   | 382 388 394 400 397 403 409 415 412<br>418 424 430 427 433 439 445 442 448<br>454 460 457 463 469 475 472 478 484<br>490 487 493 499 505 502 508 514 520<br>517 523 529 535 532 538 544 550 547<br>553 559 565 562 568 574 580 577 583<br>589 595 592 598 604 610 607 613 619<br>625 622 628 634 640 637 643 649 655<br>652 658 664 670 667 673 679 685 682<br>688 694 700 697 703 709 715 712 718 | 10 lines with 9 numbers on each<br>Input number 376 → neither by 5, nor by 3 → $376 + 6 \rightarrow =$<br>$= 382 \rightarrow$ neither by 5, nor by 3 → $382 + 6 =$<br>$= 388 + 6 = 394 + 6 =$<br>$400 \rightarrow$ division by 5 → $400 - 3 = 397$ |

| Input | Output                                                   | Comments                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 132   | 129 126 123<br>120 119 121<br>123 120 119<br>121 123 120 | $(1 + 3) = 4$ and $(1 + 2) = 3 \rightarrow$ lines with 3 numbers on each<br>Input number 132<br>$132 \rightarrow$ division by 3 → $132 - 3 =$<br>$= 129 \rightarrow$ division by 3 → $129 - 3 =$<br>$= 126 \rightarrow$ division by 3 → $126 - 3 =$<br>$= 123 \rightarrow$ division by 3 → $123 - 3 =$<br>$= 120 \rightarrow$ division by 5 → $120 - 1 =$<br>..... $121 \rightarrow$ neither by 5, nor by 3 → $121 + 2 = 123$ |

## Hints and Guidelines

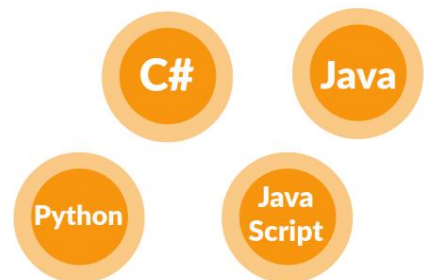
Solve the problem **on your own**, using what you have learned from the previous ones. Remember that you have to define a **separate** variable for each digit of the input number.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/660#4>.

# High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



**SoftUni** provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

## The path of the student at SoftUni



**Apply now**

[softuni.org/apply](https://softuni.org/apply)



# Chapter 8.1. Practical Exam Preparation – Part I

In the current chapter, we will examine a few **problems** with a level of **difficulty** that can be expected in **the problems** of the practical **exam** at SoftUni.

## Practical Exam

The **practical exam** includes **6** problems, for which you will have **4 hours**, to solve. **Each** of the exam problems will **cover** one of the **topics** studied in this book. The themes of the problems are as follows:

- problem with simple calculations (no conditions)
- problem with simple condition
- problem with more complex conditions
- problem with a simple loop
- problem with nested loops (drawing a figure on the console)
- problem with nested loops and more complex logic

## The Online Evaluation System (Judge)

All exams and exercises from this book are automatically **tested** through the online **Judge** system: <https://judge.softuni.org>. For **each** of the problems, there are **visible** (zero point) tests to help you understand what is expected of the problem and fix your mistakes, as well as **competition** tests that are **hidden** and check if your solution is working properly.

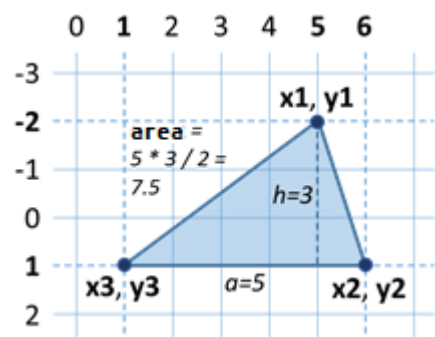
**How** does the testing in **Judge** work? **You upload** the source code and from the menu below you choose to compile it as a **Java** program. The program is being **tested** with a series of tests, giving **points** for each **successful** test.

## Simple Calculations – Problems

The first problem of the Practical Exam covers **simple calculations without checks and loops**. Here are a few problems:

Problem: Triangle Area

A triangle in the plane is defined by the coordinates of its three vertices. First, the vertex  $(x_1, y_1)$  is set. Then the other two vertices are set:  $(x_2, y_2)$  and  $(x_3, y_3)$ , which lie on a common horizontal line (i.e. they have the same Y coordinates). Write a program that calculates the area of the triangle by the coordinates of its three vertices.



Input Data

The console provides 6 integers (one per line):  $x_1, y_1, x_2, y_2, x_3, y_3$ .

- All input numbers are in the range  $[-1000 \dots 1000]$ .
- It is guaranteed that  $y_2 = y_3$ .

Output Data

Print on the console the area of the triangle.

Sample Input and Output

| Input                       | Output | Visualization | Comments                                                                                                                                                                |
|-----------------------------|--------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5<br>-2<br>6<br>1<br>1<br>1 | 7.5    |               | The side of the triangle $a$<br>$= 6 - 1 = 5$<br>The height of the<br>triangle $h = 1 - (-2) = 3$<br>The area of the triangle<br>$S = a * h / 2 = 5 * 3 / 2$<br>$= 7.5$ |

| Input                         | Output | Visualization | Comments                                                                                                                                                 |
|-------------------------------|--------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4<br>1<br>-1<br>-3<br>3<br>-3 | 8      |               | The side of the triangle $a = 3 - (-1) = 4$<br>The height of the triangle $h = 1 - (-3) = 4$<br>The area of the triangle $S = a * h / 2 = 4 * 4 / 2 = 8$ |

## Hints and Guidelines

It is very important when facing such types of problems where some coordinates are given to pay attention to **the order** in which they are submitted and to properly understand which of the coordinates we will use and how. In this case, the input is in order  $x_1, y_1, x_2, y_2, x_3, y_3$ . If we do not follow this sequence, the solution becomes incorrect. First, we write the code that reads the input data:

```
Scanner scanner = new Scanner(System.in);
int x1 = Integer.parseInt(scanner.nextLine());
int y1 = Integer.parseInt(scanner.nextLine());
int x2 = Integer.parseInt(scanner.nextLine());
int y2 = Integer.parseInt(scanner.nextLine());
int x3 = Integer.parseInt(scanner.nextLine());
int y3 = Integer.parseInt(scanner.nextLine());
```

We have to calculate **the side** and **the height** of the triangle. From the pictures, as well as the condition  $y_2 = y_3$ , we notice that the one **side** is always parallel to the horizontal axis. This means that its **length** is equal to the length of the segment between its coordinates  $x_2$  and  $x_3$ , which is equal to the difference between the larger and the smaller coordinates. Analogically, we can calculate **the height**. It will always be equal to the difference between  $y_1$  and  $y_2$  (or  $y_3$ , as they are equal). Since we do not know if  $x_2$  is greater than  $x_3$ , or  $y_1$  will be below or above the triangle side, we will use **the absolute values** of the difference to always get positive numbers because one segment cannot have a negative length.

```
int a = Math.abs(x2 - x3);
int h = Math.abs(y2 - y1);
```

we will calculate it using our familiar formula for finding an **area of a triangle**. An important thing to consider is that although we get only integers at an input, **the area** will not always be an integer. That's why we use a variable of **double** type for the area. We have to convert the right side of the equation because if we give whole numbers as equation parameters, our result will also be an integer.

```
double s = (double)a * h / 2;
```

The only thing left is to print the area on the console.

```
System.out.print(s);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#0>.

## Problem: Bricks

Construction workers have to transfer a total of **x** bricks. **Workers** are **w** in numbers and work simultaneously. They transport the bricks in trolleys, each with a **capacity** of **m** bricks. Write a program that reads the integers **x**, **w**, and **m**, and calculates **what is the minimum number of courses** the workers need to do to transport the bricks.

### Input Data

On the console **3 integers** are given, one per line:

- The number of bricks **x** is read from the first line.
- The number of workers **w** is read from the second line.
- The capacity of the trolley **m** is read from the third line.

All input numbers are integers in the range [**1 ... 1000**].

### Output Data

Print on the console **the minimum number of courses** needed to transport the bricks.

### Sample Input and Output

| Input          | Output | Comments                                                                                                                                                                                                      |
|----------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 120<br>2<br>30 | 2      | We have <b>2</b> workers, each transporting <b>30</b> bricks per course. In total, workers are transporting <b>60</b> bricks per course. To transport <b>120</b> bricks, exactly <b>2</b> courses are needed. |

| Input          | Output | Comments                                                                                                                                                                                                                                                                                                                         |
|----------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 355<br>3<br>10 | 12     | We have <b>3</b> workers, each transporting <b>10</b> bricks per course. In total, workers are transporting <b>30</b> bricks per course. To transport <b>355</b> bricks, exactly <b>12</b> courses are needed: <b>11</b> complete courses carry <b>330</b> bricks and the last twelfth course carries the last <b>25</b> bricks. |

| Input        | Output | Comments                                                                    |
|--------------|--------|-----------------------------------------------------------------------------|
| 5<br>5<br>30 | 1      | We have <b>5</b> workers, each transporting <b>30</b> bricks per course. In |

| Input    | Output | Comments                                                                                                                                                                  |
|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12<br>30 |        | total, workers are transporting <b>150</b> bricks per course. To transport <b>5</b> bricks, only <b>1</b> course is sufficient (although incomplete, with only 5 bricks). |

## Hints and Guidelines

The input is standard, and we only need to be careful about the sequence in which we read the data.

```
Scanner scanner = new Scanner(System.in);
int x = Integer.parseInt(scanner.nextLine());
int w = Integer.parseInt(scanner.nextLine());
int m = Integer.parseInt(scanner.nextLine());
```

We calculate how many **bricks** the workers transport in a single course.

```
int bricksInOneCourse = w * m;
```

By dividing the total number of **bricks transported for 1 course**, we will obtain the number of **courses** required to carry them. We have to consider that when dividing whole numbers, the remainder is ignored and always rounded down. To avoid this, we will convert the right side of the equation to **double** and use the **Math.ceil(...)** function to round the result always up. When the bricks can be transferred with **an exact number of courses**, the division will return a whole number and there will be nothing to round. Accordingly, if not, the result of the division will be **the number of exact courses** but a decimal fraction. The decimal part will be rounded up and we will get the required **1 course** for the remaining bricks.

```
double totalCourses = Math.ceil((double) x / bricksInOneCourse);
```

Finally, we print the result on the console.

```
System.out.print(totalCourses);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#1>.

## Simple Conditions – Problems

The second problem of the Practical Exam covers **conditional statements** and **simple calculations**. Here are a few problems:

### Problem: Point on a Segment

A **horizontal segment** is placed on a horizontal line, set with the **x** coordinates of both ends: **first** and **second**. A **point** is located **on** the same horizontal line and is set with its **x** coordinate. Write a program that checks whether the point is **inside** or **outside** the segment and calculates **the distance to the nearest end** of the segment.

### Input Data

The console reads **3 integer numbers** (one per line):

- On the first line, the number "first" is read – **one end of the segment**.
- On the second line, the number "second" is read – **the other end of the segment**.
- On the third line, the number "point" is read – **the location of the point**.

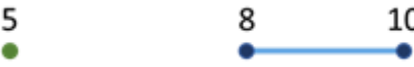
All input numbers are integers in the range [-1000 ... 1000].

### Output Data

Print the result on the console:

- On the first line, print **"in"** or **"out"** – whether the point is inside or outside the segment.
- On the second line, print the distance from the point to the nearest end of the segment.

### Sample Input and Output

| Input        | Output   | Visualization                                                                       |
|--------------|----------|-------------------------------------------------------------------------------------|
| 10<br>5<br>7 | in<br>2  |    |
| 8<br>10<br>5 | out<br>3 |  |
| 1<br>-2<br>3 | out<br>2 |  |

### Hints and Guidelines

We read the input from the console.

```
Scanner scanner = new Scanner(System.in);
int first = Integer.parseInt(scanner.nextLine());
int second = Integer.parseInt(scanner.nextLine());
int point = Integer.parseInt(scanner.nextLine());
```

Since we do not know which **point** is on the left and which is on the right, we will create two variables to mark this. Since the **left point** is always the one with the smaller **x coordinate**, we will use **Math.min(...)** to find it. Accordingly, **the right one** is always the one with a larger **x coordinate** and we will use **Math.max(...)**. We will also find the distance from **point x** to the two points. Because we do not know their position relative to each other, we will use **Math.abs(...)** to get a positive result.

```
int left = Math.min(first, second);
int right = Math.max(first, second);
int distanceLeft = Math.abs(left - point);
int distanceRight = Math.abs(right - point);
```

The shorter of the two **distances** we can find using **Math.min(...)**.

```
int minDistance = Math.min(distanceLeft, distanceRight);
```

What remains is to find whether **the point** is on or out of the line. The point will be **on the line** whenever it **matches** one of the other two points or its **x coordinate** lies **between** them. Otherwise, the point is **outside the line**. After checking, we display one of the two messages, depending on which condition is satisfied.

```
if (point >= left && point <= right) {
    System.out.println("in");
} else {
    System.out.println("out");
}
```

Finally, we print **the distance** previously found.

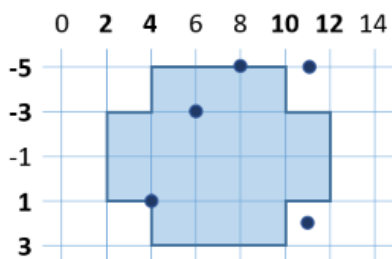
```
System.out.println(minDistance);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#2>.

## Problem: Point in a Figure

Write a program that checks whether a point (with coordinates **x** and **y**) is **inside** or **outside** the following figure:



## Input Data

The console reads **two integers** (one per line): **x** and **y**.

All input numbers are integers in the range **[-1000 ... 1000]**.

## Output Data

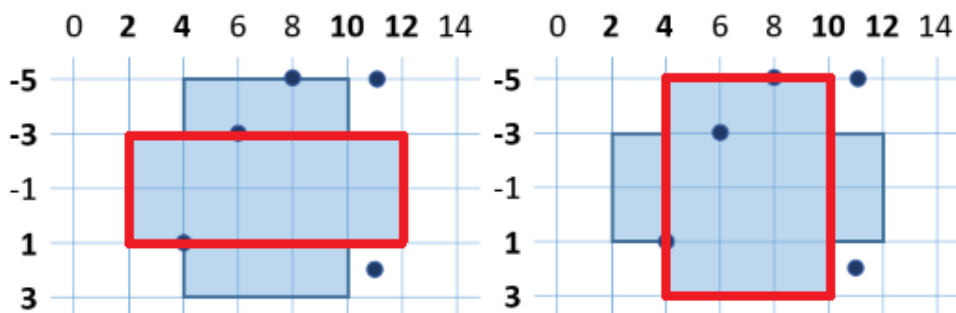
Print on the console "in" or "out" – whether the point is **inside** or **outside** the figure (the outline counts as inside).

## Sample Input and Output

| Input   | Output | Input   | Output | Input    | Output | Input   | Output |
|---------|--------|---------|--------|----------|--------|---------|--------|
| 8<br>-5 | in     | 6<br>-3 | in     | 11<br>-5 | out    | 11<br>2 | out    |

## Hints and Guidelines

To find whether our **point** is in the figure, we will divide **the figure** into 2 rectangles:



A sufficient condition is **a point** to be located in one of them, to be in **the figure**.

We read the input data from the console:

```
Scanner scanner = new Scanner(System.in);
int x = Integer.parseInt(scanner.nextLine());
int y = Integer.parseInt(scanner.nextLine());
```

We will initialize two variables that will mark whether **the point** is in one of the rectangles. When printing the message, we will check whether any of the variables has accepted a value of **true**. It is enough for **only one** of them to be **true** so that the point is in the figure.

```
if ((x >= 2 && x <= 12 && y >= -3 && y <= 1)
    || (x >= 4 && x <= 10 && y >= -5 && y <= 3)) {
    System.out.print("in");
} else {
    System.out.print("out");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#3>.



## Complex Conditions – Problems

The third problem of the Practical Exam includes **several nested checks combined with simple calculations**. Here are a few problems:

### Problem: Date After 5 days

There are two numbers given - **d** (day) and **m** (month) that form a **date**. Write a program that prints the date that will be **5 days after particular date**. For example, 5 days after **28.03** is the date **2.04**, we assume that the months: April, June, September, and November have 30 days, February has 28 days, and the rest have 31 days. Months to be printed with **leading zero** when they contain a single digit (e.g. 01, 08).

#### Input Data

The input is read from the console and consists of two lines:

- On the first line, we read an integer **d** in the range **[1 ... 31]** – day. The number of days does not exceed the number of days in that month (e.g. 28 for February).
- On the second line, we read an integer **m** in the range **[1 ... 12]** – month. Month 1 is January, month 2 is February, ..., month 12 is December. The month may contain a leading zero (e.g. April may be written as 4 or 04).

#### Output Data

Print a single line containing the date after 5 days in the format **day.month** on the console. The month must be a 2-digit number with a leading zero, if necessary. The day must be written without leading zero.

#### Sample Input and Output

| Input    | Output | Input    | Output | Input   | Output | Input    | Output |
|----------|--------|----------|--------|---------|--------|----------|--------|
| 28<br>03 | 2.04   | 27<br>12 | 1.01   | 25<br>1 | 30.01  | 26<br>02 | 3.03   |

#### Hints and Guidelines

We take the input from the console.

```
Scanner scanner = new Scanner(System.in);
int d = Integer.parseInt(scanner.nextLine());
int m = Integer.parseInt(scanner.nextLine());
```

To make our checks easier, we'll create a variable that will contain the **number of days** that we have in the month we set.

```

int daysInMonth = 31;
if (m == 2) {
    daysInMonth = 28;
}

if (m == 4 || m == 6 || m == 9 || m == 11) {
    daysInMonth = 30;
}

```

We increase the day by 5.

```
d += 5;
```

We check if the day has not exceeded the number of days in the month. If so, we must deduct the days of the month from the obtained day to calculate which day of the next month our day corresponds to.

```

if (d > daysInMonth) {
    d -= daysInMonth;
}

```

After we have passed to the next month, this should be noted by increasing the initial one by 1. We need to check if it has not become greater than 12 and if it has, to adjust it. Because we cannot skip more than one month when we increase by 5 days, the following check is enough.

```

if (d > daysInMonth) {
    d -= daysInMonth;
    m++;

    if (m > 12) {
        m = 1;
    }
}

```

The only thing that remains is to print the result on the console. It is important to format the output correctly to display the leading zero in the first 9 months. This is done by adding a formatting string `%02d` at the second element. `0` shows what number must be used in that place, and `2` shows the number of digits that must be filled with the corresponding number.

```
System.out.printf("%d.%02d", d, m);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#4>.

## Problem: Sums 3 Numbers

There are 3 integers given. Write a program that checks if the sum of two of the numbers is equal to the third one. For example, if the numbers are 3, 5, and 2, the sum of two of the numbers is equal to the third one:  $2 + 3 = 5$ .

## Input Data

The console reads **three integers**, one per line. The numbers are in the range [1 ... 1000].

## Output Data

- Print a text line on the console containing the solution of the problem in the format "**a + b = c**", where **a**, **b** and **c** are among the three input numbers and **a ≤ b**.
- If the problem has no solution, print "**No**" on the console.

## Sample Input and Output

| Input | Output    | Input | Output    | Input | Output | Input | Output |
|-------|-----------|-------|-----------|-------|--------|-------|--------|
| 3     | 2 + 3 = 5 | 2     | 2 + 2 = 4 | 1     | No     | 2     | No     |
| 5     |           | 2     |           | 1     |        | 6     |        |
| 2     |           | 4     |           | 5     |        | 3     |        |

## Hints and Guidelines

We take the input from the console.

```
Scanner scanner = new Scanner(System.in);
int a = Integer.parseInt(scanner.nextLine());
int b = Integer.parseInt(scanner.nextLine());
int c = Integer.parseInt(scanner.nextLine());
```

We have to check if the **sum** of a pair of numbers is equal to the third number. We have three possible cases:

- $a + b = c$
- $a + c = b$
- $b + c = a$

We will write a **template**, which will later be complemented by the required code. If none of the above three conditions is met, we will make our program print "**No**".

```
if (a + b == c) {
    // TODO
} else if (a + c == b) {
    // TODO
} else if (b + c == a) {
    // TODO
} else {
    System.out.print("No");
}
```

We now have to understand the order in which the **two addends** will be written in the output of the program. For this purpose, we will create **a nested condition** that checks which one of the two numbers is the larger one. In the first case, it will look like this:

```

if (a + b == c) {
    if (a > b) {
        System.out.printf("%d + %d = %d", b, a, c);
    } else {
        System.out.printf("%d + %d = %d", a, b, c);
    }
}

```

Similarly, we will supplement the other two cases. The full code of the program will look like this:

```

if (a + b == c) {
    if (a > b) {
        System.out.printf("%d + %d = %d", b, a, c);
    } else {
        System.out.printf("%d + %d = %d", a, b, c);
    }
} else if (a + c == b) {
    if (a < c) {
        System.out.printf("%d + %d = %d", a, c, b);
    } else {
        System.out.printf("%d + %d = %d", c, a, b);
    }
} else if (b + c == a) {
    if (b < c) {
        System.out.printf("%d + %d = %d", b, c, a);
    } else {
        System.out.printf("%d + %d = %d", c, b, a);
    }
} else {
    System.out.print("No");
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#5>.

## Simple Loops – Problems

The fourth problem of the Practical Exam includes a single loop with simple logic in it. Here are a few examples:

### Problem: Sums Step 3

We are given  $n$  integers:  $a_1, a_2, \dots, a_n$ . Calculate the sums:

- $\text{sum1} = a_1 + a_4 + a_7 + \dots$  (the numbers are summed, starting from the first one with step of 3).
- $\text{sum2} = a_2 + a_5 + a_8 + \dots$  (the numbers are summed, starting from the second one with step of 3).

- $\text{sum3} = a_3 + a_6 + a_9 + \dots$  (the numbers are summed, starting from the third one with step of 3).

## Input Data

The input data is read from the console. The first line holds an integer  $n$  ( $0 \leq n \leq 1000$ ). On the next  $n$  lines, we are given  $n$  integers in the range  $[-1000 \dots 1000]$ :  $a_1, a_2, \dots, a_n$ .

## Output Data

On the console, we should print 3 lines containing the 3 sums in a format such as in the example.

## Sample Input and Output

| Input | Output   | Input | Output    | Input | Output    |
|-------|----------|-------|-----------|-------|-----------|
| 2     | sum1 = 3 | 4     |           | 5     |           |
| 3     | sum2 = 5 | 7     | sum1 = 19 | 3     | sum1 = 10 |
| 5     | sum3 = 0 | -2    | sum2 = -2 | 5     | sum2 = 13 |
|       |          | 6     | sum3 = 6  | 2     | sum3 = 2  |
|       |          | 12    |           | 7     |           |
|       |          |       |           | 8     |           |

## Hints and Guidelines

We will take the count of numbers from the console and declare starting values of the three sums.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());

int sum1 = 0;
int sum2 = 0;
int sum3 = 0;
```

Since we do not know in advance how many numbers we will process, we will take them one at a time in a loop which will be repeated  $n$  times and we will process them in the body of the loop.

```
for (int i = 0; i < n; i++) {
    int a = Integer.parseInt(scanner.nextLine());
    // TODO
}
```

To find out in which of the three sums we need to add the number, we will divide its sequence number into three and we will use the remainder. We will use the variable  $i$ , which tracks the number of runs of the loop, to find out which sequence number we are at. When the remainder of  $i/3$  is zero, it means we will add this number to the first sum, when it is 1 to the second one, and when it is 2 to the third one.

```
for (int i = 0; i < n; i++) {
    int a = Integer.parseInt(scanner.nextLine());
    if (i % 3 == 0) {
        sum1 += a;
    }
    if (i % 3 == 1) {
        sum2 += a;
    }
    if (i % 3 == 2) {
        sum3 += a;
    }
}
```

Finally, we will print the result on the console in the required **format**.

```
System.out.printf("sum1 = %d%n", sum1);
System.out.printf("sum2 = %d%n", sum2);
System.out.printf("sum3 = %d%n", sum3);
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#6>.

Problem: Increasing Elements

A series of **n** numbers is given: **a1, a2, ..., an**. Calculate the **length** of the longest increasing **sequence** of consecutive elements in the series of numbers.

Input Data

The input data is read from the console. The first line holds an integer **n** ( $0 \leq n \leq 1000$ ). On the following **n** lines, we are given **n** integers in the range [-1000 ... 1000]: **a1, a2, ..., an**.

Output Data

On the console, we must print one number – the **length** of the longest increasing sequence.

Sample Input and Output

| Input            | Output | Input                 | Output | Input                 | Output | Input                 | Output |
|------------------|--------|-----------------------|--------|-----------------------|--------|-----------------------|--------|
| 3<br>5<br>2<br>4 | 2      | 4<br>2<br>8<br>7<br>6 | 2      | 4<br>1<br>2<br>4<br>4 | 3      | 4<br>5<br>6<br>7<br>8 | 4      |

## Hints and Guidelines

To solve this problem, we need to think in a bit **more algorithmic way**. A **sequence of numbers** is given to us, and we need to check whether each **subsequent** one will be **larger than the previous one** and if so, we count how long is the sequence in which this condition is fulfilled. Then we have to find **which sequence** of these is **the longest** one. To do this, let's create some variables that we will use during solving the problem.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
int countCurrentLongest = 0;
int countLongest = 0;
int aPrev = 0;
int a = 0;
```

The variable **n** is the count of numbers we get from the console. In **countCurrentLongest** we will keep the number of elements in the increasing sequence we are currently counting. For example, in the sequence: 5, 6, 1, 2, 3 **countCurrentLongest** will be 2 when we reach the **second element** of the counting (5, 6, 1, 2, 3) and will become 3 when we reach the **last element** (5, 6, 1, 2, 3), because the increasing row 1, 2, 3 has 3 elements. We will use **countLongest** to keep the **longest** increasing sequence. The other variables are **a** – the number we are **currently in**, and **aPrev** – the **previous number** which we will compare with **a** to see if the row is **growing**.

We begin to run the numbers and check if the present number **a** is larger than the previous **aPrev** one. If this is true, then the row **is growing**, and we need to increase its number by **1**. This is stored in the variable that tracks the length of the sequence we are currently in – **countCurrentLongest**. If the number **a** is **not greater** than the previous one, it means that **a new sequence** starts, and we have to start the count from **1**. Finally, after all the checks are done, **aPrev** becomes **the number** we are **currently** using, and we start the loop from the beginning with **the next** entered **a**.

Here is a sample implementation of the algorithm described:

```
for (int i = 0; i < n; i++){
    a = Integer.parseInt(scanner.nextLine());

    if (a > aPrev){
        countCurrentLongest++;
    }
    else{
        countCurrentLongest = 1;
    }

    aPrev = a;
}
```

What remains is to see which of all sequences is **the longest** one. We will do this by checking in the loop if **the sequence** we are **currently** in has become longer than the **longest one by now**. The whole loop will look like this:

```
for (int i = 0; i < n; i++){
    a = Integer.parseInt(scanner.nextLine());

    if (a > aPrev){
        countCurrentLongest++;
    }
    else{
        countCurrentLongest = 1;
    }

    if (countCurrentLongest > countLongest){
        countLongest = countCurrentLongest;
    }

    aPrev = a;
}
```

Finally, we print the length of **the longest** sequence found.

```
System.out.print(countLongest);
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#7>.

## Drawing Figures on The Console – Problems

The **fifth** problem of the Practical Exam requires **using one or several nested loops** for drawing a figure on the console. Logical deductions, simple calculations, and checks might be required. The problem checks the students' abilities to think logically and come up with simple algorithms for solving problems, i.e. to think algorithmically. Here are a few examples for an exam problem:

### Problem: Perfect Diamond

Write a program that reads an integer **n** from the console and draws a **perfect diamond** with size **n** as in the examples below.

#### Input Data

The input is an integer **n** within the range [1 ... 1000].



### Output Data

The diamond should be printed on the console as in the examples below.

### Sample Input and Output

| Input | Output              | Input | Output                       | Input | Output                                                                           | Input | Output                                                       |
|-------|---------------------|-------|------------------------------|-------|----------------------------------------------------------------------------------|-------|--------------------------------------------------------------|
| 2     | <pre>  *  *_*</pre> | 3     | <pre>   *   *_*  *_*_*</pre> | 5     | <pre>    *    *_*   *_*_*  *_*_*_* *_*_*_*_*  *_*_*_*   *_*_*    *_*     *</pre> | 4     | <pre>   *   *_*  *_*_* *_*_*_*  *_*_*_*   *_*_*    *_*</pre> |

### Hints and Guidelines

From the given input results, we can see that we are required to print repeating symbols. That's why in advance we type a method that we will use to repeat the same symbol that times as we need it.

```
static String repeatStr(String strToRepeat, int count) {
    String text = "";
    for (int i = 0; i < count; i++) {
        text = text + strToRepeat;
    }
    return text;
}
```

In problems for drawing figures, the most important thing to consider is the **sequence** in which we will draw. Which items are **repeated** and with what **steps**? We can see that the **top and bottom** parts of the diamond are the **same**. The easiest way to solve the problem is by creating a **loop** that draws the **upper part**, and then **another loop** that draws the **bottom part** (opposite to the top one).

We will read the number **n** from the console.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
```

We start painting the **top half** of the diamond. We see that **each row** starts with a **few empty spaces** and **\***. If we take a closer look, we will notice that the **empty spaces** are always equal to **n – the number of lines** (the first row is **n-1**, the second – **n-2**, etc.). We will start by drawing the number of **empty spaces**, and the **first asterisk**. Let's not forget to use **System.out.print(...)** instead of **System.out.println(...)**, to stay on the **same line**. At the end of the line, we write **System.out.println(...)**, to go to a **new line**.

Notice that we start counting from **1**, not from 0. Next, we will only add a few times **-\*** to finish the line.

Here is part of the code for the **top of the diamond**:

```
for (int i = 1; i <= n; i++) {
    System.out.print(repeatStr( strToRepeat: " ", count: n - i));
    System.out.print("*");
    // TODO
    System.out.println();
}
```

What remains is to **complete each line** with the required number of **-\*** elements. On each row, we have to add **i-1** such **items** (on the first 1-1 -> 0, the second -> 1, etc.)

Here is the complete code for drawing **the top of the diamond**:

```
for (int i = 1; i <= n; i++) {
    System.out.print(repeatStr( strToRepeat: " ", count: n - i));
    System.out.print("*");

    for (int j = 0; j < i - 1; j++) {
        System.out.print("-*");
    }

    System.out.println();
}
```

To draw the **bottom part** of the diamond, we have to reverse **the upper part**. We will count from **n-1**, because if we start from **n**, we will draw the middle row twice. Do not forget to change **the step** from **++** to **--**.

Here is the complete code for drawing **the top of the diamond**:

```
for (int i = n - 1; i >= 1; i--) {
    System.out.print(repeatStr( strToRepeat: " ", count: n - i));
    System.out.print("*");

    for (int j = 1; j < i; j++) {
        System.out.print("-*");
    }

    System.out.println();
}
```

What remains is to **assemble the whole program** by first reading the input, printing the top part of the diamond, and then the bottom part of the diamond.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#8>.

### Problem: Rectangle with Stars

Write a program that reads from the console an integer **n** and draws a **rectangle** with size **n** with **two asterisks** in its center as in the examples below.

#### Input Data

The input is an integer **n** in the range [2 ... 1000].

#### Output Data

The rectangle should be printed on the console as in the examples below.

#### Sample Input and Output

| Input | Output         | Input | Output                             | Input | Output                                       | Input | Output                                                                       |
|-------|----------------|-------|------------------------------------|-------|----------------------------------------------|-------|------------------------------------------------------------------------------|
| 2     | %%<br>**<br>%% | 3     | %%<br>%  %<br>% ** %<br>%  %<br>%% | 4     | %%<br>%      %<br>%  **  %<br>%      %<br>%% | 5     | %%<br>%          %<br>%      %<br>%  **  %<br>%      %<br>%          %<br>%% |

#### Hints and Guidelines

With this problem, we also notice that we will have to type the same symbol several times. To make things easier we type a method, with which we will type the same symbol fast and easily.

```
static String repeatStr(String strToRepeat, int count) {
    String text = "";
    for (int i = 0; i < count; i++) {
        text = text + strToRepeat;
    }
    return text;
}
```

We read the input data.

```
Scanner scanner = new Scanner(System.in);
int n = Integer.parseInt(scanner.nextLine());
```

The first thing we can easily notice is that the first and last rows contain **2 \* n** symbols **%**. We will start with this and then draw the middle part of the rectangle.

```
for (int i = 0; i < numRows; i++) {
    System.out.println("%");
    // TODO: Draw middle of the rectangle
    System.out.println("%");
}
```

From the examples, we see that **the middle** part of the figure always has an **odd number** of rows. Note that when an **even number** is set, the number of rows is equal to **the previous odd** number (2 -> 1, 4 -> 3, etc.). We create a variable that represents the number of rows that our rectangle will have, and correct it if the number **n** is even. Then we will draw a **rectangle without the asterisks**. Each row has for **the beginning and the end** the symbol **%** and between them **2 \* n - 2** empty spaces (the width is **2 \* n** and we subtract 2 for the two percent at the end). Do not forget to move the code for the **last line** after the loop.

```
int numRows = n;
if (n % 2 == 0) {
    numRows--;
}

for (int i = 0; i < numRows; i++) {
    System.out.print("%");
    System.out.print(repeatStr( strToRepeat: " ", count: n * 2 - 2));
    // TODO: Place the stars
    System.out.print("%");
    System.out.println();
}
```

We can **start and test the code so far**. Everything without the two asterisks in the middle should work correctly.

Now, **in the body** of the loop let's add the **asterisks**. We'll check if we're on the **middle row**. If we are in the middle, we will draw **the row together with the asterisks**, if not – we will draw **a normal row**. The line with the asterisks has **n-2 empty spaces** (**n** is half the length and we remove the asterisk and the percentage), **two stars**, and again **n-2 empty spaces**. We leave out of the check the two percent at the beginning and the end of the row.

```
for (int i = 0; i < numRows; i++) {
    System.out.print("%");

    if (i == numRows / 2) {
        System.out.print(repeatStr( strToRepeat: " ", count: n - 2));
        System.out.print("***");
        System.out.print(repeatStr( strToRepeat: " ", count: n - 2));
    } else {
        System.out.print(repeatStr( strToRepeat: " ", count: n * 2 - 2));
    }

    System.out.print("%");
    System.out.println();
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#9>.

## Nested Loops with More Complex Logic – Problems

The last (sixth) problem of the Practical Exam requires using of **several nested loops and more complex logic inside them**. The problems examine participants' ability to **think algorithmically** and to solve non-trivial coding problems that require nested loops with more complex logic and calculations. Here are a few examples of exam problems.

### Problem: Increasing 4 Numbers

For given pair of numbers **a** and **b** generate all four number **n1, n2, n3, n4**, for which **a ≤ n1 < n2 < n3 < n4 ≤ b**.

#### Input Data

The input contains two integers **a** and **b** in the range [0 ... 1000], one per line.

#### Output Data

The output contains all **numbers in batches of four**, in ascending order, one per line.

#### Sample Input and Output

| Input  | Output                                              | Input  | Output | Input    | Output      |
|--------|-----------------------------------------------------|--------|--------|----------|-------------|
| 3<br>7 | 3 4 5 6<br>3 4 5 7<br>3 4 6 7<br>3 5 6 7<br>4 5 6 7 | 5<br>7 | No     | 10<br>13 | 10 11 12 13 |

#### Hints and Guidelines

We will read the input data from the console. We also create the additional variable **count**, which will keep track of **existing number ranges**.

```
Scanner scanner = new Scanner(System.in);  
int a = Integer.parseInt(scanner.nextLine());  
int b = Integer.parseInt(scanner.nextLine());  
int count = 0;
```

It will be easier to solve the problem if we logically divide it **into parts**. If we are required to draw all the rows from a number between **a** and **b**, we will do it using **one loop** that takes all the numbers from **a** to **b**. Let's think about how to do this with a **series of two numbers**. The answer is easy – we will use **nested loops**.

```
for (int i = a; i <= b; i++) {  
    for (int j = i + 1; j < b; j++) {  
        System.out.printf("%d %d %n", i, j);  
    }  
}
```

We can test the incomplete program to see if it's accurate so far. It must print all pairs of numbers **i, j** for which  **$i \leq j$** .

Since each **next number** of the row must be **greater** than **the previous one**, the second loop will run around  **$i + 1$**  (the next greater number). Accordingly, if **there is no sequence** of two incremental numbers (**a** and **b** are equal), the second loop **will not be fulfilled**, and nothing will be printed on the console.

Similarly, what remains is to implement **the nested loops** for **four numbers**. We will add an **increase of the counter** that we initialized to know if **there is such a sequence**.

```
for (int i = a; i <= b; i++) {
    for (int j = i + 1; j <= b; j++) {
        for (int k = j + 1; k <= b; k++) {
            for (int l = k + 1; l <= b; l++) {
                System.out.printf("%d %d %d %d\n", i, j, k, l);
                count++;
            }
        }
    }
}
```

Finally, we will check if **the counter** is equal to **0** and we will print **"No"** on the console accordingly, if so.

```
if (count == 0) {
    System.out.print("No");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#10>.

## Problem: Generate Rectangles

By a given number **n** and a **minimum area m**, generate all possible rectangles with integer coordinates in the range  **$[-n...n]$**  with an area of at least **m**. The generated rectangles must be printed in the following format:

(left, top) (right, bottom) -> area

Rectangles are defined using the top left and bottom right corners. The following inequalities are in effect:

- $-n \leq \text{left} < \text{right} \leq n$
- $-n \leq \text{top} < \text{bottom} \leq n$

## Input Data

Two numbers, one per line, are entered from the console:

- An integer **n** in the range **[1 ... 100]** – sets the minimum and maximum coordinates of a peak.

- An integer **m** in the range [0 ... 50000] – sets the minimum area of the generated rectangles.

## Output Data

- The described rectangles should be printed on the console in a format such as in the examples below.
- If there are no rectangles for the specified **n** and **m**, then print "No".
- The order of rectangles in the output is not important.

## Sample Input and Output

| Input | Output               | Input | Output | Input | Output                |
|-------|----------------------|-------|--------|-------|-----------------------|
| 1     | (-1, -1) (0, 1) -> 2 | 2     | No     | 3     |                       |
| 2     | (-1, -1) (1, 0) -> 2 | 17    |        | 36    | (-3, -3) (3, 3) -> 36 |
|       | (-1, -1) (1, 1) -> 4 |       |        |       |                       |
|       | (-1, 0) (1, 1) -> 2  |       |        |       |                       |
|       | (0, -1) (1, 1) -> 2  |       |        |       |                       |

## Hints and Guidelines

Read the input data from the console. We will also create a **counter**, which will store the number of rectangles found.

```
Scanner scanner = new Scanner(System.in);
```

```
int n = Integer.parseInt(scanner.nextLine());
```

```
int m = Integer.parseInt(scanner.nextLine());
```

```
int count = 0;
```

It is very important to be able to imagine the problem before we begin to solve it. In our case, it is required to search for rectangles in a coordinate system. The thing we know is that the **left point** will always have the coordinate **x**, smaller than the **right** one. Accordingly, the **upper one** will always have a smaller **y** coordinate than the **lower one**. To find all the rectangles, we'll have to create a **loop** similar to the previous problem, but this time, **not every next loop** will start from the **next number** because some of the **coordinates** can be equal (for example **left** and **top**).

```
for (int left = -n; left < n; left++) {
    for (int top = -n; top < n; top++) {
        for (int right = left + 1; right <= n; right++) {
            for (int bottom = top + 1; bottom <= n; bottom++) {
                // TODO
            }
        }
    }
}
```

With the variables **left** and **right** we will track the coordinates on the horizontal, and with **top** and **bottom** - on the vertical way. The important thing here is knowing the corresponding coordinates so we can correctly calculate the sides of the rectangle. Now we have to find the area of the rectangle and check if it is greater than or equal to **m**. One side will be the difference between **left** and **right** and the other one - between **top** and **bottom**. Since the coordinates may be eventually interchanged, we will use absolute values. Again, we add the counter in the loop, counting only the rectangles we write. It is important to note that the writing order is **left, top, right, bottom**, as it is set in the problem's description.

```
for (int left = -n; left < n; left++) {
    for (int top = -n; top < n; top++) {
        for (int right = left + 1; right <= n; right++) {
            for (int bottom = top + 1; bottom <= n; bottom++) {
                int area = Math.abs(right - left) * Math.abs(bottom - top);

                if (area >= m) {
                    System.out.printf("(%d, %d) (%d, %d) -> %d%n",
                        left, top, right, bottom, area);
                    count++;
                }
            }
        }
    }
}
```

Finally, we print "No" if there are no such rectangles.

```
if (count == 0) {
    System.out.println("No");
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/661#11>.



## Chapter 8.2. Practical Exam Preparation – Part II

In this chapter, we will look at a **practical exam** held at SoftUni on December 18, 2016. The problems give a good idea of what we can expect at the practical exam in programming at SoftUni.

### Exam Problems

Traditionally, the Practical Exam of SoftUni consists of **6 practical programming problems**:

- Simple problem (without checks).
- A problem with a single condition.
- A problem with more complex conditions.
- A problem with a single loop.
- A problem with nested loops (drawing a figure on the console).
- A problem with nested loops and more complex logic.

Let's look at a **real exam topic**, the problems it contains, and their solutions.

### Problem: Distance

Write a program that calculates **what is the distance** passed by a car (in kilometers), for which we know **the initial speed** (km/h), **the initial time frame** in minutes, then the **speed is increased by 10%**, **the second time frame**, then the **speed is decreased by 5%**, and the **time until the end** of the trip. To calculate the distance, you need to **convert the minutes into hours** (e.g., 70 minutes = 1.1666 hours).

### Input Data

The input is read from the console and consists of **4 lines**:

- The **initial speed** in km/h – an integer within the range [1 ... 300].
- The **first time** in minutes – an integer within the range [1 ... 1000].
- The **second time** in minutes – an integer within the range [1 ... 1000].
- The **third time** in minutes – an integer within the range [1 ... 1000].

### Output Data

Print a number on the console: **kilometers passed**, formatted up to the **second character after the decimal point**.

Sample Input and Output

| Input                | Output | Comments                                                                                                                                                                                                                                                                                     |
|----------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 90<br>60<br>70<br>80 | 330.90 | Distance with initial speed: 90 km/h * 1 hour (60 min) = <b>90 km</b><br>After speed increase: 90 + 10% = 99.00 km/h * 1.166 hours (70 min) = <b>115.50 km</b><br>After speed decrease: 99 - 5% = 94.05 km/h * 1.33 hours (80 min) = <b>125.40 km</b><br>Total number of km passed: 330.9 km |

| Input                   | Output | Comments                                                                                                                                                                                                                                                                                                        |
|-------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 140<br>112<br>75<br>190 | 917.12 | Distance with initial speed: 140 km/h * 1.86 hours (112 min) = <b>261.33 km</b><br>After speed increase: 140 + 10% = 154.00 km/h * 1.25 hours (75 min) = <b>192.5 km</b><br>After speed decrease: 154.00 - 5% = 146.29 km/h * 3.16 hours (190 min) = <b>463.28 km</b><br>Total number of km passed: 917.1166 km |

Hints and Guidelines

Such a description may look **misleading** and incomplete at first glance, which **adds** to the **complexity** of a relatively easy problem. Let's **separate** the problem into a few **sub-problems** and try to **solve** each problem one by one, which will lead us to the final result:

- Our **initial** sub-problem will be to **read the input data** entered by the user and **store** them in appropriate variables.
- **Execution** of the main programming **logic**, which in our case is a batch of simple calculations of the data that we already have.
- **Caculate** and format the **result**.

The **main** part of the programming logic **is** to **calculate** what will be the **distance passed after all changes** in speed. As during **execution** of the program, part of the **data** that we have **is modified**, we could **separate** the program **code** into a few **logically separated parts**:

- Calculation of the **distance** passed with initial speed.
- Change of **speed** and calculation of the **distance** passed.
- Last change of **speed** and **calculation**.
- Addition of the above.

For **reading** data from the **console**, we use **Scanner**:

```
Scanner scan = new Scanner(System.in);
String initialSpeedString = scan.nextLine();
By the problem condition, input data is being entered on four separate rows, therefore,
we should execute the previous code a total of four times.
Scanner scan = new Scanner(System.in);
String initialSpeedString = scan.nextLine();
// String firstIntervalString = TODO
// String secondIntervalString = TODO
// String thirdIntervalString = TODO
```

To perform **calculations**, we choose to use a type **double**.

This way, we successfully solved the first sub-problem. The next step is to **convert the input data** into appropriate **types** to perform the needed calculations. We choose to use **Integer** or **int** type because the condition of the problem says that the input data must be within a particular range, for which this data type is sufficient. We will do the **conversion** in the following way:

```
int initialSpeed = Integer.parseInt(initialSpeedString);
// int firstInterval = TODO
// int secondInterval = TODO
// int thirdInterval = TODO
```

Initially, we **save** a variable that we will use multiple times. This approach of centralization gives us **flexibility** and the opportunity to change the overall result of the program with minimal effort. In case, we need to change the value, we need to do it **only in one place in the code**, which saves us time and effort.

```
double minutesPerHour = 60d;
```



**Avoiding repetitive code** (centralization of program logic) in the problems that we examine in the present book may look unnecessary at first glance, but this approach is very important upon building large-scale applications in a real work environment, and practicing it in an initial stage of training will help you build a quality programming style.

We calculate the **travel time** (in hours) by dividing the time by 60 (minutes in an hour). The **travel distance** is calculated by multiplying the starting speed by the time passed (in hours). After that, we change the speed by increasing it by **10%**, as per the problem description. Calculating the **percentage**, as well as the following **distances** passed, is done in the following way:

- The **time interval** (in hours) is calculated by dividing the provided time interval in minutes by the minutes in an hour (60).
- The **distance passed** is calculated by multiplying the time interval (in hours) by the speed obtained after the increase.
- The next step is to **decrease the speed** by **5%**, as per the problem description.

- We calculate the **remaining distance** as described in the first two points.

```
double firstIntervalHours = firstInterval / minutesPerHour;
double firstDistance = initialSpeed * firstIntervalHours;

double speedAfterIncrease = initialSpeed + ((initialSpeed * 10) / 100d);
double secondIntervalHours = secondInterval / minutesPerHour;
double secondDistance = speedAfterIncrease * secondIntervalHours;
```

So far, we solved two of the most important sub-problems, namely the **data input** and **their processing**. What remains is to **calculate the result**. Since it is required to **format it** up to 2 characters after the decimal point, we can do it as follows:

```
double finalDistance = firstDistance + secondDistance + thirdDistance;

System.out.println(String.format("%.2f", finalDistance));
```

If you have worked correctly and run the program with the input data from the problem condition, you will see that it works correctly.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#0>.

## Problem: Changing Tiles

Haralambi has some savings that he wants to use to **change the tiles on the bathroom floor**. The floor is rectangular, and the tiles are triangular. Write a program that **calculates** if his savings will be sufficient. Read from the console the width and length of the floor, **as well as one of the sides of the triangle with its height to it**. We must calculate how many tiles are needed, **to cover the floor**. The number of tiles must be rounded up to the higher integer **and 5 more tiles must be added as spare tiles**. Also, read from the console – the price per tile **and** the amount paid for the work\*\* of a workman.

### Input Data

The following 7 lines are read from the console:

- The savings
- The width of the floor
- The length of the floor
- The side of the triangle
- The height of the triangle
- The price of a tile
- The amount for the workman

All numbers are real numbers in the range [0.00 ... 5000.00].

## Output Data

The following must be printed on the console as a **single line**:

- If the money **is sufficient**:
  - "{The remaining money} lv left."
- If the money **IS NOT sufficient**:
  - "You'll need {Needed money} lv more."

The result must be **formatted up to the second character** after the decimal point.

## Sample Input and Output

| Input                                                | Output                             | Comments                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1000<br>5.55<br>8.95<br>0.90<br>0.85<br>13.99<br>321 | You'll need<br>1209.65 lv<br>more. | Floor area $\rightarrow 5.55 * 8.95 = 49.67249$<br>Tile area $\rightarrow 0.9 * 0.85 / 2 = 0.3825$<br>Needed tiles $\rightarrow 49.67249 / 0.3825 = 129.86... = 130 + 5$ spare tiles = 135<br>Total amount $\rightarrow 135 * 13.99 + 321$ (workman) = 2209.65<br>$2209.65 > 1000 \rightarrow 1209.65$ lv are insufficient |

| Input                                        | Output            | Comments                                                                                                                                                                                                                                                                          |
|----------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 500<br>3<br>2.5<br>0.5<br>0.7<br>7.80<br>100 | 25.60 lv<br>left. | Floor area $\rightarrow 3 * 2.5 = 7.5$<br>Tile area $\rightarrow 0.5 * 0.7 / 2 = 0.175$<br>Needed tiles $\rightarrow 7.5 / 0.175 = 42.857... = 43 + 5$ spare tiles = 48<br>Total amount $\rightarrow 48 * 7.8 + 100$ (workman) = 474.4<br>$474.4 < 500 \rightarrow 25.60$ lv left |

## Hints and Guidelines

The problem requires our program to accept more input data and perform more calculations, even though the solution is **identical**. Reading the input data is done **familiarly**. Note that the **Input** part of the condition states that all input data are **real numbers**, and for that reason, we would use the **decimal** type.

Now that we already have everything to execute the programming logic, we can move to the next part. How can we **calculate** what is the **needed** number of tiles that will be sufficient to cover the entire floor? The condition is that tiles have a **triangular** shape, which can confuse, but practically, the problem needs just **basic calculations**. We can calculate the **common part of the floor** by the formula for finding rectangle area, and the **area of a single tile** using the relevant formula for triangle area.

To calculate the **number of tiles** that are needed, we divide the floor area by the area of a **single tile** (we should not forget to add the 5 additional tiles, that were mentioned in the condition).



Pay attention that the condition state that we should round up the number of tiles, obtained upon the division, up to the higher number, and then we should add 5. Find more information about the system functionality that does that: `Math.ceil(...)`.

We can find the result by **calculating the total amount** needed to cover the entire floor, by **adding the tile price with the price for the workman**, which we have from the input data. We can guess that **the total costs** for tiles can be calculated by **multiplying the number of tiles by the price per tile**. Whether the amount that we have will be sufficient, we understand by comparing the savings (from the input data) and the total costs.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#1>.

## Problem: Flowers

A flowers shop offers 3 types of flowers: **chrysanthemums**, **roses**, and **tulips**. The prices depend on the season.

| Season          | Chrysanthemums | Roses      | Tulips     |
|-----------------|----------------|------------|------------|
| spring / summer | 2.00 lv/pc     | 4.10 lv/pc | 2.50 lv/pc |
| autumn / winter | 3.75 lv/pc     | 4.50 lv/pc | 4.15 lv/pc |

On holidays, the prices of all flowers are **increased by 15%**. The following **discounts** are offered:

- For purchasing more than 7 tulips in spring – **5% of the price** of the whole bouquet.
- For purchasing 10 or more roses in winter – **10% of the price** of the whole bouquet.
- For purchasing more than 20 flowers in total in any season – **20% of the price** of the whole bouquet.

Discounts are made in the above-described order and can be combined! All discounts are valid after the price increase for a holiday!

The price for arranging a bouquet is always **2 lv**. Write a program that calculates the **price of a bouquet**.

## Input Data

The input is read from the **console** and contains **exactly 5 lines**:

- The first line contains **the number of purchased chrysanthemums** – an integer within the range [0 ... 200].
- The second line contains **the number of purchased roses** – an integer within the range [0 ... 200].
- The third line contains **the number of purchased tulips** – an integer within the range [0 ... 200].
- The fourth line shows **the season** – [Spring, Summer, Autumn, Winter].
- The fifth line specifies **if the day is a holiday** – [Y = yes / N = no].

### Output Data

Print on the console 1 number – **the price of flowers**, formatted up to the second character after the decimal point.

### Sample Input and Output

| Input                      | Output | Comments                                                                                                                                                                                                                                                                            |
|----------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2<br>4<br>8<br>Spring<br>Y | 46.14  | Price: $2 \cdot 2.00 + 4 \cdot 4.10 + 8 \cdot 2.50 = 40.40$ lv<br>Holiday: $40.40 + 15\% = 46.46$ lv<br><b>5% discount</b> for more than 7 tulips in spring: 44.14lv<br>The flowers are in total 20 or less: <b>no discount</b><br>$44.14 + 2$ for arranging the bouquet = 46.14 lv |

| Input                       | Output | Comments                                                                                                                                                                                                                                                                                       |
|-----------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3<br>10<br>9<br>Winter<br>N | 69.39  | Price: $3 \cdot 3.75 + 10 \cdot 4.50 + 9 \cdot 4.15 = 93.60$ lv<br><b>Not a holiday</b> : no increase in price<br><b>10% discount</b> for 10 or more roses in winter: 84.24lv<br>Total flowers more than 20: <b>20% discount</b> = 67.39lv<br>$67.39 + 2$ for arranging the bouquet = 69.39 lv |

| Input                         | Output |
|-------------------------------|--------|
| 10<br>10<br>10<br>Autumn<br>N | 101.20 |

### Hints and Guidelines

After carefully reading the condition, we understand that we need to do **simple calculations**, but this time we will need **additional logical checks**. We need to pay more **attention** to the moment of **making changes** in the final price so that we can properly

build the logic of our program. Again, the bold text gives us sufficient **guidelines** on how to proceed. To begin with, we will separate the already **defined** values into **variables**, as we did in the previous problems:

```
// Initial price list
double roseAutumnWinterPrice = 4.50d;
double roseSpringSummerPrice = 4.10d;
double tulipAutumnWinterPrice = 4.15d;
double tulipSpringSummerPrice = 2.50d;
double chrysantemumAutumnWinterPrice = 3.75d;
double chrysantemumSpringSummerPrice = 2d;
double arrangePrice = 2d;
```

We do the same for the rest of the defined values:

```
// Price increases
int priceIncreasePercentage = 15;

// Price decreases
int tulipPriceDecreasePercentage = 5;
int rosePriceDecreasePercentage = 10;
int totalPriceDecreasePercentage = 20;

// Price decrease thresholds
int tulipPriceDecreaseThreshold = 7;
int rosePriceDecreaseThreshold = 10;
int totalPriceDecreaseThreshold = 20;
```

Our next sub-problem is to **read** correctly **the input** data from the console. We will do it **familiarly**, but this time we will **combine two** separate functions – one for **reading** a line from the console and another one for its **conversion** into a numeric data type:

```
int chrysantemumsPurchased = Integer.parseInt(scan.nextLine());
// int rosesPurchased = TODO
// int tulipsPurchased = TODO
// String season = TODO
// String isSpecialDay = TODO
```

Let's think of the most appropriate way to **structure** our programming logic. It is clear from the condition that the path of the program is divided mainly into two parts: **spring / summer** and **autumn / winter**. We can do the separation by conditional construction, by storing variables in advance for the **prices** of the individual flowers and the **final result**.



```

if (season.equals("Winter") || season.equals("Autumn")) {
    rosesPrice = rosesPurchased * roseAutumnWinterPrice;
    chrysantemumsPrice =
        chrysantemumsPrice * chrysantemumAutumnWinterPrice;

    tulipsPrice = tulipsPurchased * tulipAutumnWinterPrice;
    totalCost = rosesPrice + chrysantemumsPrice + tulipsPrice;
} else {
    // TODO
}

```

What remains is to perform a few checks regarding the discounts of the different types of flowers, depending on the season, and to modify the final result.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#2>.

## Problem: Grades

Write a program that calculates statistics for grades in an exam. In the beginning, the program reads the number of students who attended the exam and for each student – their grade. In the end, the program must print the percentage of students that have grades between 2.00 and 2.99, between 3.00 and 3.99, between 4.00 and 4.99, 5.00 or more, and the average grade of the exam.

### Input Data

Read from the console a sequence of numbers, each on a separate line:

- On the first line – the number of students who attended the exam – an integer within the range [1 ... 1000].
- For each student on a separate line – the grade on the exam – a real number within the range [2.00 ... 6.00].

### Output Data

Print on the console 5 lines that hold the following information:

- "Top students: {percentage of students with grade of 5.00 or more}%".
- "Between 4.00 and 4.99: {between 4.00 and 4.99 included}%".
- "Between 3.00 and 3.99: {between 3.00 and 3.99 included}%".
- "Fail: {less than 3.00}%".
- "Average: {average grade}%".

The results must be formatted up to the second symbol after the decimal point.

Sample Input and Output

| Input | Output                        | Comments                                                                                       |
|-------|-------------------------------|------------------------------------------------------------------------------------------------|
| 10    |                               |                                                                                                |
| 3.00  |                               | 5 or more: <b>3 students</b> = 30% of 10                                                       |
| 2.99  |                               | Between 4.00 and 4.99 - <b>3 students</b> = 30% of 10                                          |
| 5.68  | Top students: 30.00%          |                                                                                                |
| 3.01  | Between 4.00 and 4.99: 30.00% | Between 3.00 and 3.99 - <b>2 students</b> = 20% of 10                                          |
| 4     | Between 3.00 and 3.99: 20.00% |                                                                                                |
| 4     | Fail: 20.00%                  | Under 3 - <b>2 students</b> = 20% of 10                                                        |
| 6.00  | Average: 4.06                 | The average grade is: 3 + 2.99 + 5.68 + 3.01 + 4 + 4 + 6 + 4.50 + 2.44 + 5 = 40.62 / 10 = 4.06 |
| 4.50  |                               |                                                                                                |
| 2.44  |                               |                                                                                                |
| 5     |                               |                                                                                                |

| Input | Output                        |
|-------|-------------------------------|
| 6     |                               |
| 2     | Top students: 33.33%          |
| 3     | Between 4.00 and 4.99: 16.67% |
| 4     | Between 3.00 and 3.99: 16.67% |
| 5     | Fail: 33.33%                  |
| 6     | Average: 3.70                 |
| 2.2   |                               |

Hints and Guidelines

From the condition, we see that we will read the **number** of students, and then, **their grades**. For that reason, **first**, we will read the **number** of students and save it in a variable of **int** type. To read and process the grades themselves, we will use a **for** loop. The value of the **int** variable will be the **end** value of the **i** variable from the loop. This way, at **each** iteration of the loop, will read **each one of the grades**.

```
Scanner scan = new Scanner(System.in);
int numberOfStudents = Integer.parseInt(scan.nextLine());

for (int i = 0; i < numberOfStudents; i++) {
    // TODO
}
```

Before executing the code of the **for** loop, we will create variables where we will store **the number of students** for each group: poor results (up to 2.99), results from 3 to 3.99, from 4 to 4.99, and grades above 5. We will also need one more variable, where we will store **the sum of all grades**, via which we will calculate the average grade of all students.

```
double numberOfFailedStudents = 0;
double numberOfAverageStudents = 0;
double numberOfGoodStudents = 0;
double numberOfExcellentStudents = 0;
double totalResult = 0;
```

We run the **loop**, and inside it, we **declare one more** variable, in which we will store the **currently** entered grade. The variable will be **double** type, and upon each iteration, we will check **what is its value**. According to this value, we **increase** the number of students in the relevant group by **1**, as we should not forget to increase the **total** amount of the grades, which we also track.

```
for (int i = 0; i < numberOfStudents; i++) {
    double grade = Double.parseDouble(scan.nextLine());
    totalResult += grade;

    if (grade < 3) {
        numberOfFailedStudents++;
    } else // TODO: Check other groups
}
```

What **percentage** is occupied by a **group of students** from the total number, we can calculate by **multiplying the number of students** from the respective group by **100**, and then dividing by the **total number of students**.



Pay close attention to the numeric data type that you work with upon doing these calculations.

The **final result** is formed in the well know fashion – **up to the second character** after the decimal point.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#3>.

## Problem: Christmas Hat

Write a program that reads from the console an **integer n** and draws a **Christmas hat** with a width of  $4 * n + 1$  columns and a height of  $2 * n + 5$  rows, as in the examples below.

### Input Data

The input is read from the console – an **integer n** within the range [3 ... 100].

### Output Data

Print on the console a **Christmas hat**, exactly like in the examples.

### Sample Input and Output

[illegible]

## Hints and Guidelines

In problems requiring **drawing** on the console, most often the user inputs **an integer** that is related to the **total size of the figure** that we need to draw. As the problem requirements mention how the total length and width of the figure are calculated, we can use them as **starting points**. In the examples, it is clear that regardless of the input data, we always have the **first two rows** that are almost identical.

A diagram of a 2D lattice structure. It consists of two horizontal rows of dots. The top row has 11 dots, and the bottom row has 11 dots. Between the two rows, there are vertical lines connecting the dots. Specifically, the 4th dot from the left in the top row is connected to the 4th dot in the bottom row by a vertical line. The 5th dot in the top row is connected to the 5th dot in the bottom row by a vertical line. The 6th dot in the top row is connected to the 6th dot in the bottom row by a vertical line. The 7th dot in the top row is connected to the 7th dot in the bottom row by a vertical line. The 8th dot in the top row is connected to the 8th dot in the bottom row by a vertical line. The 9th dot in the top row is connected to the 9th dot in the bottom row by a vertical line. The 10th dot in the top row is connected to the 10th dot in the bottom row by a vertical line. The 11th dot in the top row is connected to the 11th dot in the bottom row by a vertical line. The vertical line connecting the 5th dots is highlighted with a thicker line.

We also notice that the **last three rows** are always present, as **two** of them are completely the same.

```
*****
*  *  *  *  *  *  *  *  *  *
*****
```

By these observations, we can come up with the **formula** for the **height of the variable part** of the Christmas hat. We use the formula specified in the problem to calculate the total height, by subtracting the size of the unchangeable part. We get  $(2 * n + 5) - 5$  or  $2 * n$ .

To **draw** the **dynamic** or the variable part of the figure, we will use a **loop**. The size of the loop will be from **0** to the **width** that we have by requirements, namely  $4 * n + 1$ . Since we will use this formula in **a few places** in the code, it is a good practice to declare it in a **separate variable**. Before running the loop, we should **declare variables** for the **number** of individual symbols that participate in the dynamic part: **dots** and **dashes**. By analyzing examples, we can also prepare formulas for the **starting values** of these variables. Initially, the **dashes** are **0**, but we can calculate the number of **dots** by subtracting **3** from the **total width** (the number of symbols that are building the top of the Christmas hat) and then **dividing by 2**, as the number of dots on both sides of the hat is the same.

```

.....***.....
.....*-*-*.....
.....*-*-*.....
.....*-*-*.....
....*-*-*-*.....
...*-*-*-*-*.....
..*-*-*-*-*.....
.*-*-*-*-*.....
*-*-*-*-*.....

```

All that remains is to execute the body of the loop, as **after each** drawing we **decrease** the number of dots by **1** and **increase the number of dashes** by **1**. Let's not forget to draw one **star** between each of them. The sequence of drawing in the body of the loop is the following:

- Symbol string of dots
- Star
- Symbol string of dashes
- Star
- Symbol string of dashes
- Star
- Symbol string of dots

In case we have worked correctly, we will obtain figures identical to those in the examples.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#4>.

## Problem: Letters Combinations

Write a program that prints on the console **all combinations of 3 letters** within a specified range by skipping the combinations **containing a certain letter**. Finally, print the number of printed combinations.

### Input Data

The input is read from the **console** and contains **exactly 3 lines**:

- A small letter from the English alphabet for a beginning of the range – between 'a' and 'z'.
- A small letter from the English alphabet for the end of the range – between the **first letter** and 'z'.
- A small letter from the English alphabet – from 'a' to 'z' – as the combinations containing this letter are **skipped**.

### Output Data

Print on a single line **all combinations**, corresponding to the requirements, followed by **their number**, separated by a space.

### Sample Input and Output

| Input       | Output                               | Comments                                                                                                                                                                                                                                                                        |
|-------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| a<br>c<br>b | aaa aac aca acc<br>caa cac cca ccc 8 | All possible combinations with letters 'a', 'b' and 'c' are:<br>aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb<br>bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc<br>The combinations <b>containing 'b' are not valid</b> .<br><b>8</b> valid combinations remain. |

| Input       | Output                                                                                                            |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| a<br>c<br>z | aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc<br>caa cab cac cba cbb cbc cca ccb ccc 27 |

| Input       | Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| f<br>k<br>h | fff ffg ffi ffj ffk fgf fgg fgi fgj fgk fif fig fii fij fik fjf fjg fji fjf fjk fkf fkg fki fkj fkk<br>gff gfg gfi gfj gfk ggf ggg ggi ggj gbk gif gig gii gjj gik gjf gjg gji gjj gjk gkf gkg<br>gki gkj gkk iff ifg ifi ifj ifk igf igg igi igj igk iif iig iii iij iik ijf ijj ijk ikf ikg iki<br>ikj ikk jff jfg jfi jfj jfk jgf jgg jgi jgj jgk jif jig jii jij jik jff jjg jji jjj jjk jkf jkg jki jkj jkk<br>kff kfg kfi kfj kfk kgf kgg kgi kgj kgk kif kig kii kij kik kjf kfg kji kjj kjk kkf kkg<br>kki kkj kkk 125 |

## Hints and Guidelines

According to the condition, we have as input data **3 lines**, each of which is represented by one character of the **ASCII table** (<https://www.asciitable.com>). We could use an already **defined function** in Java to read a single symbol from the console and save it in a variable with type **char**:

```
Scanner scan = new Scanner(System.in);
char startLetter = scan.next().charAt(0);
// TODO: Read other characters
```

Let's think of how we can achieve the **final result**. The problem condition is to print all characters from the starting to the end (by skipping a particular letter), what should we do?

The easiest and most efficient way is to use a **loop**, by iterating through **all characters** and printing those that are **different** from the **letter** that we need to skip. One of the advantages of Java is that we have the opportunity to use a different data type for a loop variable:

```
for (char i = 'a'; i <= 'z'; i++) {
    System.out.print(i + " ");
}
```

The result of running the code is all letters from **a** to **z** included, printed on a single line and separated by spaces. Does this look like the final result of our problem? We must find a **way** to print **3 characters**, as required, instead of **1**. The execution of the program is very similar to a gaming machine. We usually win there if we manage to arrange several identical symbols. Let's say that the machine has space for three characters. When we **stop** on a particular **character** in the first place, the other two places will **continue** rolling characters among all possible ones. In our case, **all possible characters** are the letters from the starting to the end one, entered by the user, and the solution of our program is identical to the way a gaming machine works.

We use a **loop** that runs through **all characters** from the starting to the end letter, included. On **each iteration** of the **first** loop, we run a **second** one with the same parameters (but **only if** the letter of the first loop is valid, i.e., does not match the one that we must exclude, by requirements). In each iteration of the **second** loop, we run **one** more

with the **same parameters** and the same **condition**. This way, we have three nested loops, as we will print the characters in the body of the **latter**.

```
for (int i = startLetter; i <= endLetter; i++) {  
    if (i != expectedLetter) {  
        // TODO: Second and third loop  
    }  
}
```

Let's not forget that we also need to print the **total number of valid combinations** that we have found, and they must be printed on the **same line**, separated by a space.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/662#5>.



# Chapter 9.1. Problems for Champions – Part I

In this chapter, we will offer the reader **more complex problems**. They aim to develop **algorithmic skills** and acquire **programming techniques** to solve problems with higher complexity.

## More Complex Problems on The Studied Material

We will solve several programming problems that cover the material studied in the book but are more difficult than the usual problems of the practical exams at SoftUni. If you want to become a **champion of the basics of programming**, we recommend this training to solve such complex problems to make it easy for you to take exams.

### Problem: Crossing Sequences

We have two sequences:

- a sequence of **Tribonacci** (by analogy with the Fibonacci sequence), where each number is **the sum of the previous three** (with given three numbers)
- a sequence generated by a **numerical spiral** defined by looping like a spiral (right, bottom, left, top, right, bottom, left, top, etc.) of a matrix of numbers starting from its center with a given starting number and incremental step, by storing the current numbers in the sequence each time we make a turn

Write a program that finds the first number that appears in **both sequences** defined as described above.

### Problem

Let the **Tribonacci sequence** start with **1, 2** and **3**. This means that **the first sequence** will contain the numbers 1, 2, 3, 6, 11, 20, 37, 68, 125, 230, 423, 778, 1431, 2632, 4841, 8904, 16377, 30122, 55403, 101902, and so on.

At the same time, let the **numbers in the spiral** begin with **5**, and the spiral increases by **2** at each step.

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 45  | ... | ... | ... | ... | 55  |
| ... | 17  | 19  | 21  | 23  | ... |
| ... | 15  | 5   | 7   | ... | ... |
| ... | 13  | 11  | 9   | ... | ... |
| 37  | ... | ... | ... | 29  | ... |
| ... | ... | ... | ... | ... | 65  |

Then **the second sequence** will contain the numbers 5, 7, 9, 13, 17, 23, 29, 37, and so on. We see that **37** is the first number to be found inside the Tribonacci sequence and the spiral one, and that is the desired solution to the problem.

Input Data

Input data should be read from the console.

- On the first three lines of the input, we will read **three integers**, representing **the first three numbers** in the Tribonacci sequence, positive non-zero numbers, sorted in ascending order.
- On the following two lines of the input, we will read **two integers** representing **the first number** and **the step** for each matrix's cell for the spiral of numbers. The numbers representing the spiral are positive non-zero numbers.

Input data will always be valid and will always be in the format described. There is no need to check.

Output Data

The result should be printed on the console.

On the single line of the output, we must print **the smallest number that occurs in both sequences**. If there is no number **in the range [1 ... 1 000 000]**, which can be found in both sequences, print "No".

Constraints

- All numbers in the input will be in the range [1 ... 1 000 000].
- Allowed program time: 0.3 seconds.
- Allowed memory: 16 MB.

Sample Input and Output

| Input                 | Output | Input                    | Output | Input                    | Output | Input                  | Output |
|-----------------------|--------|--------------------------|--------|--------------------------|--------|------------------------|--------|
| 1<br>2<br>3<br>5<br>2 | 37     | 13<br>25<br>99<br>5<br>2 | 13     | 99<br>99<br>99<br>2<br>2 | No     | 1<br>4<br>7<br>23<br>3 | 23     |

Hints and Guidelines

The problem seems quite complicated, so we will break it into simpler sub-problems.

## Processing The Input Data

The first step in solving the problem is to read and process the input. Input data consists of **5 integers**: **3** for the Tribonacci sequence and **2** for the numerical spiral.

```
Scanner scan = new Scanner(System.in);
int tribonacciFirst = Integer.parseInt(scan.nextLine());
// TODO: Read remaining numbers
```

Once we have the input data, we need to think about how we will generate the numbers in the two sequences.

## Generating Tribonacci Sequence

For the Tribonacci sequence, we will always **sum up the previous three values** and then move the values of those numbers (the three previous ones) to one position in the sequence, i.e., the value of the first one must accept the value of the second one, and so on. When we are ready with the number, we will store its value in **an array**. Since the problem description states that the numbers in the sequences do not exceed 1,000,000, we can stop generating this range at 1,000,000.

```
ArrayList<Integer> tribonacciNumbers = new ArrayList<Integer>();
tribonacciNumbers.add(tribonacciFirst);
tribonacciNumbers.add(tribonacciSecond);
tribonacciNumbers.add(tribonacciThird);

int tribonacciCurrent = tribonacciThird;

while (tribonacciCurrent < 1000000) {
    tribonacciCurrent = tribonacciFirst
        + tribonacciSecond + tribonacciThird;

    tribonacciNumbers.add(tribonacciCurrent);

    tribonacciFirst = tribonacciSecond;
    tribonacciSecond = tribonacciThird;
    tribonacciThird = tribonacciCurrent;
}
```

## Generating Numerical Spiral

We need to think of a **relationship** between numbers in the numerical spiral so we can easily generate every next number without having to look at matrices and loop through them. If we carefully look at the picture from the description, we will notice that **every 2 "turns" in the spiral, the numbers we skip are increased by 1**, i.e. from 5 to 7 and from 7 to 9, not a single number is skipped, but we directly **add with the step** of the sequence. From 9 to 13 and from 13 to 17 we skip a number, i.e. we add the step twice. From 17 to 23 and from 23 to 29 we skip two numbers, i.e. we add the step three times and so on.

Thus, we see that for the first two we have **the last number + 1 \* the step**, the next two we add with the **2 \* the step**, and so on. Every time we want to get to the next number of the spiral, we will have to make such calculations.

```
spiralCurrent += spiralStep * spiralStepMul;
```

What we have to take care of is **for every two numbers, our multiplier** (let's call it "coefficient") **must increase by 1** (**spiralStepMul++**), which can be achieved with a simple check (**spiralCount % 2 == 0**). The whole code from the generation of the spiral in **an array** is given below.

```
ArrayList<Integer> spiralNumbers = new ArrayList<Integer>();
spiralNumbers.add(spiralCurrent);
int spiralCount = 0;
int spiralStepMul = 1;

while (spiralCurrent < 1000000) {
    spiralCurrent += spiralStep * spiralStepMul;
    spiralNumbers.add(spiralCurrent);
    spiralCount++;

    if (spiralCount % 2 == 0)
    {
        spiralStepMul++;
    }
}
```

### Finding Common Number for The Sequences

Once we have generated the numbers in both sequences, we can combine them and build the final solution. How will it look? For **each of the numbers** in the first sequence (starting from the smaller one), we will check if it exists in the other one. The first number that meets this criterion will be **the answer** to the problem.

We will do a **linear** search in the second array. We will leave, the more curious readers, to optimize it using the technique called **binary search** because the second array is generated in sorted form, i.e., it meets the requirement to apply this type of search. The code for finding our solution will look like this:

```

boolean found = false;

for (int i = 0; i < tribonacciNumbers.size(); i++) {
    for (int j = 0; j < spiralNumbers.size(); j++) {
        if (tribonacciNumbers.get(i).equals(spiralNumbers.get(j))
            && tribonacciNumbers.get(i) <= 1000000) {
            System.out.println(tribonacciNumbers.get(i));
            found = true;
            break;
        }
    }

    if (found) {
        break;
    }
}

if (!found) {
    System.out.println("No");
}

```

The previous solution to the problem uses arrays to store the values. Arrays are not needed to solve the problem. There is an **alternative solution** that generates the numbers and works directly with them instead of keeping them in an array. On **every step**, we can check whether **the numbers in the two sequences match**. If this is the case, we will print the number on the console and terminate the execution of our program. Otherwise, we will see the current number of **which sequence is the smaller one**, and we will generate the next one where we are "lagging". The idea is that **we will generate numbers from the sequence that is "behind"** until we skip the current number of the other sequence and then vice versa, and if we find a match in the meantime, we will terminate the execution.

```

while (tribonacciCurrent <= 1000000 && spiralCurrent <= 1000000) {
    if (tribonacciCurrent == spiralCurrent) {
        // TODO: Print and stop execution
    }
    else if (tribonacciCurrent < spiralCurrent) {
        // TODO: Generate next Tribonacci number
    } else {
        // TODO: Generate next Spiral number
    }
}

```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/663#0>.

## Problem: Magic Dates

Date is given in a "dd-mm-yyyy" format, e.g. 17-04-2018. We calculate the weight of that date by taking all of its digits, multiplying each digit with the others after it, and finally summing up all the results obtained. In our case, we have 8 digits: 17032007, so the weight is  $1*7 + 1*0 + 1*3 + 1*2 + 1*0 + 1*0 + 1*7 + 7*0 + 7*3 + 7*2 + 7*0 + 7*0 + 7*7 + 0*3 + 0*2 + 0*0 + 0*0 + 0*7 + 3*2 + 3*0 + 3*0 + 3*7 + 2*0 + 2*0 + 2*7 + 0*0 + 0*7 + 0*7 = 144$ .

Our problem is to write a program that finds all the magical dates between two specific years (inclusively) corresponding to the given weight. Dates must be printed in ascending order (by date) in the format "dd-mm-yyyy". We will only use the valid dates in the traditional calendar (the leap years have 29 days in February).

### Input Data

Input data should be read from the console. It consists of 3 lines:

- The first line contains an integer: **start year**.
- The second line contains an integer: **end year**.
- The third line contains an integer: **the search weight** for the dates.

Input data will always be valid and will always be in the format described. There is no need to check.

### Output Data

The result should be printed on the console as consecutive dates in "dd-mm-yyyy" format, sorted by date in ascending order. Each string must be in a separate line. If there are no existing magic dates, print "No".

### Constraints

- The start and final years are integer numbers in the range [1900 ... 2100].
- Magic weight is an integer in the range [1 ... 1000].
- Allowed program time: 0.25 seconds.
- Allowed memory: 16 MB.

## Sample Input and Output

| Input | Output     | Input | Output |
|-------|------------|-------|--------|
| 2007  | 17-03-2007 | 2003  | No     |
| 2007  | 13-07-2007 | 2004  |        |
| 144   | 31-07-2007 | 1500  |        |

| Input              | Output     | Input              | Output                                               |
|--------------------|------------|--------------------|------------------------------------------------------|
| 2012<br>2014<br>80 | 09-01-2013 | 2011<br>2012<br>14 | 01-01-2011<br>10-01-2011<br>01-10-2011<br>10-10-2011 |
|                    | 17-01-2013 |                    |                                                      |
|                    | 23-03-2013 |                    |                                                      |
|                    | 11-07-2013 |                    |                                                      |
|                    | 01-09-2013 |                    |                                                      |
|                    | 10-09-2013 |                    |                                                      |
|                    | 09-10-2013 |                    |                                                      |
|                    | 17-10-2013 |                    |                                                      |
|                    | 07-11-2013 |                    |                                                      |
|                    | 24-11-2013 |                    |                                                      |
|                    | 14-12-2013 |                    |                                                      |
|                    | 23-11-2014 |                    |                                                      |
|                    | 13-12-2014 |                    |                                                      |
|                    | 31-12-2014 |                    |                                                      |

## Hints and Guidelines

We start with the input data. In this case, we have **3 integers** that should be read from the console, as this is the only entry and processing of input for the problem.

Having the start and end year, it is good to understand how we will go through each date, without being confused by how many days there are in the month and whether it is a leap year, and so on.

## Loop Through Dates

For looping through the dates, we will take advantage of the functionality that gives us the **LocalDate** class, in **Java**. We will define a **start date variable** that we can do using the constructor that accepts a year, month, and day. We know the year is the starting year we read from the console, and the month and the day must be January and 1st respectively. In Java, the "constructor" of **LocalDate** accepts as the first argument the year, as second argument the month, and as the third argument the day of the month:

```
LocalDate currentDate = LocalDate.of(
    startYear, month: 1, dayOfMonth: 1);
```

Once we have the start date, we want to create a **loop that runs until we exceed the final year** (or until we pass December 31 in the final year if we compare the full dates), increasing each day by one day.

To increase by one day in each rotation, we will use a method of **LocalDate – plusDays(...)**, which will add one day to the current date. The method will take care instead of us whether to skip the next month, how many days there is a month, and everything around the leap years.

```
currentDate = currentDate.plusDays(1);
```

**Caution:** since the **LocalDate.plusDays(...)** method returns the "new" date, it is important to assign the result, not just to call the method!

Finally, our loop may look like this:

```
while (currentDate.getYear() <= endYear) {
    // TODO: Do all the calculations
    currentDate = currentDate.plusDays(1);
}
```

**Note:** we can achieve the same result with a **for** loop: the **initialization** of the date goes to the first part of **for**, the condition is preserved and the **step** is the increase by 1 day.

## Calculating Date Weight

Each date consists of exactly 8 characters (digits) – 2 for the day (**d1, d2**), 2 for the month (**d3, d4**), and 4 for the year (**d5** to **d8**). It means that we will always have the same calculation every time, and we can benefit from this **to define the formula statically** (i.e., not to use loops, referring to different numbers from the date, but write the whole formula). To be able to write it, we will need **all digits from the date** in individual variables to make all the necessary multiplications. Using the operations of division and remainder on the individual components of the date, using the **getDayOfMonth()**, **getMonthValue()**, and **getYear()** properties, we can retrieve each digit.

```
int d1 = currentDate.getDayOfMonth() / 10; // First day digit
int d2 = currentDate.getDayOfMonth() % 10; // Second day digit

int d3 = currentDate.getMonthValue() / 10; // First month digit
int d4 = currentDate.getMonthValue() % 10; // Second month digit

int d5 = currentDate.getYear() / 1000; // First year digit
int d6 = (currentDate.getYear() / 100) % 10; // Second year digit
int d7 = currentDate.getYear() / 100 % 10; // Third year digit
int d8 = currentDate.getYear() % 100; // Fourth year digit
```

Let us also explain one of the more interesting lines here. Take, for example, the second digit of the year (**d6**). We divide the year by 100, and we take a remainder of 10. What do we do? First, we eliminate the last 2 digits of the year by dividing by 100 (Example:



**2018/100 = 20**). With the remainder of 10, we take the last digit of the resulting number (**20 % 10 = 0**) and so we get 0, which is the second digit of 2018.

What remains is to do the calculation that will give us the magical weight of a given date. In order **not to write all multiplications** as shown in the example, we will apply a grouping. What we need to do is multiply each digit with those that follow it. Instead of typing **d1 \* d2 + d1 \* d3 + ... + d1 \* d8**, we can shorten this expression to **d1 \* (d2 + d3 + ... + d8)** for grouping when we have multiplication and addition. Applying the same simplification for the other multiplications, we get the following formula:

```
int dateWeigth = d1 * (d2 + d3 + d4 + d5 + d6 + d7 + d8) +
                // d2 * ...
                // ...
```

### Printing The Result

Once we have the weight calculation of a given date, we need **to check and see if it matches the magical weight**, to know if it should be printed, or not. Checking can be done using a standard **if** block, taking care to print the date in the correct format. To format data as required in the problem, we will use **DateTimeFormatter** class.

```
if (dataWeight == numberToSearchFor) {
    DateTimeFormatter outputFormatter =
        DateTimeFormatter.ofPattern("dd-MM-yyyy");
    System.out.println(outputFormatter.format(currentDate));
    found = true;
}
```

**Caution:** Since we iterate over the dates from initial to final, they will always be ordered in ascending order as it is in the requirements.

Finally, if we have not found an eligible date, we will have a **false** value in the **found** variable, and we will be able to print **No**.

```
if (!found) {
    System.out.println("No");
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/663#1>.

## Problem: Five Special Letters

Two numbers are given: **start** and **end**. Write a program that **generates all combinations of 5 letters**, each among the sets of **{'a', 'b', 'c', 'd', 'e'}** so that the weight of these 5 letters is a number in the range **[start ... end]**, inclusive. Print them in alphabetical order, in a single row, separated by a space.

The weight of the letters is calculated as follows:

```
weight('a') = 5;
weight('b') = -12;
weight('c') = 47;
weight('d') = 7;
weight('e') = -32;
```

The weight of the sequence of the letters **c1, c2, ..., cn** is calculated by removing all the letters that are repeated (from right to left) and then calculating the formula:

$$\text{weight}(c_1c_2\dots c_n) = 1 * \text{weight}(c_1) + 2 * \text{weight}(c_2) + \dots + n * \text{weight}(c_n)$$

For example, the weight of **bcddc** is calculated as follows:

First, we remove the repeating letters and get **bcd**. Then we apply the formula:  $1 * \text{weight}('b') + 2 * \text{weight}('c') + 3 * \text{weight}('d') = 1 * (-12) + 2 * 47 + 3 * 7 = 103$ .

Another example:  $\text{weight}(\text{"cadae"}) = \text{weight}(\text{"cade"}) = 1 * 47 + 2 * 5 + 3 * 7 + 4 * (-32) = -50$ .

## Input Data

The input data is read from the console. It consists of two numbers:

- The number for our **start**.
- The number for our **end**.

Input data will always be valid and will always be in the format described. There is no need to check.

## Output Data

The result should be printed on the console as a sequence of strings, **sorted in alphabetical order**. Each string must be separated from the next one by a single space. If the weight of any of the 5 letter strings does not exist within the specified range, print "No".

## Constraints

- Numbers for **start** and **end** are integers in the range [-10000 ... 10000].
- Allowed program time: 0.25 seconds.
- Allowed memory: 16 MB.

## Sample Input and Output

| Input   | Output                                                                                                                                                                             |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -1<br>1 | bcdea cebda eaaad eaada<br>eaadd eaade eaaed eadaa<br>eadad eadae eadda eaddd<br>eadd eadea eaded eadee<br>eaead eaeda eaedd eaede<br>eaeed eeaad eeada eeadd<br>eeade eeaed eeead |

| Input      | Output                                                                                                                                                                     |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200<br>300 | baadc babdc badac badbc badca<br>badcb badcc badcd baddc bbadc<br>bbdac bdaac bdabc bdaca bdacb<br>bdacc bdacd bdadc bdbac bddac<br>beadc bedac eabdc ebadc ebdac<br>edbac |

| Input    | Output      | Comment                                      |
|----------|-------------|----------------------------------------------|
| 40<br>42 | bcead bdcea | weight("bcead") = 41<br>weight("bdcea") = 40 |

| Input      | Output |
|------------|--------|
| 300<br>400 | No     |

## Hints and Guidelines

Like every problem, we start the solution by **reading and processing the input data**. In this case, we have **two integers** that can be processed with a combination of the **`Integer.parseInt(...)`** and **`Scanner.nextLine()`** methods.

```
Scanner scan = new Scanner(System.in);
int firstNumber = Integer.parseInt(scan.nextLine());
int secondNumber = Integer.parseInt(scan.nextLine());
```

We have several main points in the problem – **generating all combinations** with a length of 5 including the 5 letters, **removing repeating letters**, and **calculating weight** for a simplified word. The answer will consist of every word whose weight is within the given range **`[firstNumber, secondNumber]`**.

## Generating All Combinations

To generate **all combinations with a length of 1** using 5 symbols, we would use a **loop from 0 to 4**, as we want each number of the loop to match one character. To generate **any combinations of length 2**, using 5 characters (i.e. "aa", "ab", "ac", ..., "ba", ...), we would create **two nested loops each running through the digits from 0 to 4**, and we will once again make sure that each digit matches a specific character. We will repeat this step 5 times, so we will finally have 5 nested loops with indexes **i1, i2, i3, i4, and i5**.

```

for (int i1 = 0; i1 < 5; i1++) {
    for (int i2 = 0; i2 < 5; i2++) {
        for (int i3 = 0; i3 < 5; i3++) {
            for (int i4 = 0; i4 < 5; i4++) {
                for (int i5 = 0; i5 < 5; i5++) {
                    // ...
                }
            }
        }
    }
}

```

Now that we have all 5-digit combinations, we must find a way to "turn" the five digits into a word with the letters from 'a' to 'e'. One of the ways to do that is to **predefine** a simple string that contains the letters that we have

```
String pattern = "abcde";
```

and for each digit, we take the letter from the particular position. This way, the number 00000 will become "aaaaa", and the number 02423 will become "acecd". We can create the 5-letter string in the following way.

```

String fullWord = "" + pattern.charAt(i1)
    + pattern.charAt(i2)
    + pattern.charAt(i3)
    + pattern.charAt(i4)
    + pattern.charAt(i5);

```

**Another way:** we can convert the digits to letters by using their arrangement in the **ASCII table**. The expression `'a' + i` return the result `'a'` in case `i = 0`, `'b'` in case `i = 1`, `'c'` in case `i = 2`, etc.

This way we already have generated all 5-letter combinations and can proceed with the following part of the problem.

**Attention:** as we have chosen a **pattern** that takes into consideration the alphabetical arrangement of the letters, and cycles are run appropriately, the algorithm will generate the works in alphabetical order and there is no need for additional sorting before printing the output.

### Removing Repetitive Letters

Once we have the finished string, we have to remove all the repeating symbols. We will do this by adding the letters from left to right in a new string, and each time before adding a letter, we will check if it already exists – if it does, we will skip it, and if it doesn't, we will add it. To begin with, we will add the first letter to the starting string.

```
String word = Character.toString(pattern.charAt(i1));
```

Then we will do the same with the other 4, checking each time with the following condition and the `.indexOf(...)` method. We can use a loop by `fullWord` (leaving it to the reader for exercise), and it can be done lazily by copy-paste.

```
if (word.indexOf(pattern.charAt(i2)) == -1)
    word += pattern.charAt(i2);
// ...
```

The `.indexOf(...)` method returns the index of the particular element if it is found, or `-1` if the item is not found. Therefore, every time we get `-1`, it means that we still do not have this letter in the new string with unique letters and we can add it, and if we get a value other than `-1`, this will mean we already have the letter, and we'll not add it.

### Calculating Weight

Calculating the weight is simply going through the unique word (`word`) obtained in the last step, and for each letter, we need to take its weight and multiply it by the position. For each letter, we need to calculate what value we will multiply its position by, for example, by using a `switch` construction.

```
int multiplier = 0;
switch (word.charAt(i)) {
    case 'a':
        multiplier = 5;
        break;
    case 'b':
        multiplier = -12;
        break;
    // TODO
}
```

Once we have the value of that letter, we should multiply it by its position. Because the indexes in the string differ by 1 from the actual positions, i.e., index 0 is position 1, index 1 is position 2, etc., we will add 1 to the indexes.

```
weight += multiplier * (i + 1);
```

All intermediate results obtained must be added to the total amount for each letter of the 5-letter combination.

### Preparing The Output

Whether a word needs to be printed is determined by its weight. We need a condition to determine if the current weight is in the range `[start ... end]` passed to the input at the start of the program. If this is the case, we print the full word (`fullWord`).

Be careful not to print the word with unique letters. It was only needed to calculate the weight!

The words are separated with a space, and we will accumulate them in an intermediate variable `result`, which is defined as an empty string at the beginning.

```
if (weight >= firstNumber && weight <= secondNumber) {  
    result += fullWord + " ";  
}
```

### Final Touches

The condition is met unless we do not have a single word in the entered range. To find out, if we have found a word, we can check whether the string **result** has its initial value (i.e., an empty string). If it does, we print **No** else print the whole string without the last space (using the **.trim ()**) method.

```
if (result.equals("")) {  
    System.out.println("No");  
} else {  
    System.out.println(result.trim());  
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/663#2>.

## Chapter 9.2. Problems for Champions – Part II

In this chapter, we will review three additional problems that belong to the category "for champions", i.e., they are more complex than the rest of the problems in this book.

### More Complex Problems on The Studied Material

Before moving on to particular problems, we must clarify that we can solve them more efficiently with **additional knowledge in programming with Java** (methods, arrays, collections, recursion, etc.). Each solution that we will provide now only uses the material covered in this book. The goal is to learn how to construct **more complex algorithms** based on your knowledge up to the present moment.

#### Problem: Passion Days

Lina has a real shopping passion. When she has some money, she immediately goes to the closest shopping center (mall) and tries to spend as much as she can on clothes, bags, and shoes. But her favorite thing is winter sales. Our problem is to analyze her strange behavior, **calculate the purchases** that Lina does when she enters the mall, as well as the **money she has left** when the shopping is over. All prices and money are in BGN (Bulgarian leva, lv).

The **first line** of the input will pass the **amount** that Lina has **before** she starts shopping. Then, upon reading the **"mall.Enter"** command, Lina enters the mall and starts shopping we receive the **"mall.Exit"** command. When Lina starts shopping, **on each line** of the input, we will receive strings that are **actions performed by Lina**. Each **symbol** in the string is a **purchase or another action**. String commands contain only symbols of the **ASCII table**. The ASCII code of each sign is **related to what Lina must pay** for each of the goods. You need to interpret the symbols in the following way:

- If the symbol is a **capital letter**, Lina gets a **50% discount**, which means that you must decrease the money she has by 50% of the numeric representation of the symbol from the ASCII table.
- If the symbol is a **lowercase letter**, Lina gets a **70% discount**, which means that you must decrease the money she has by 30% of the numeric representation of the symbol from the ASCII table.
- If the symbol is **"%"**, Lina makes a **purchase** that decreases her money in half.
- If the symbol is **"\*"**, Lina **withdraws money from her debit card** and adds 10 lv. to her available funds.
- If the symbol is **different from all above**, Lina just purchases without discount, and in this case, you should subtract the value of the symbol from the ASCII table from her available funds.

If any of the purchase's values is **greater than** her current available funds, Lina **DOES NOT** make the purchase. Lina's funds **cannot be less than 0**.

The shopping ends when the `"mall.Exit"` command is given. When this happens, you need to **print the number of purchases made and the money** that Lina has left.

Input Data

We will read input data from the console. The **first line** of the input will indicate the **amount that Lina has before starting to purchase**. On each of the following lines, there will be a particular command. After we receive the `"mall. Enter"` command, then on each of the following lines, we will receive commands (strings) that contain **information about the purchases/actions** that Lina wants to perform. We will receive strings until the `"mall.Exit"` command is given.

We will receive only one `"mall.Enter"` command and only one `"mall.Exit"` command.

Output Data

The output data must be **printed on the console**. When shopping is over, you must print on the console a particular output depending on what purchases have been made.

- If no purchases have been made – `"No purchases. Money left: {remaining funds} lv."`
- If at least one purchase is made – `"{number of purchases} purchases. Money left: {remaining funds} lv."`

The funds must be printed with an accuracy of up to 2 symbols after the decimal point.

Constraints

- Money is a **float** number within the range:  $[0 - 7.9 \times 10^{28}]$ .
- The number of strings between `"mall.Enter"` and `"mall.Exit"` will be within the range: `[1-20]`.
- The number of symbols in each string that represents a command will be within the range: `[1-20]`.
- Allowed execution time: **0.1 seconds**.
- Allowed memory: **16 MB**.

Sample Input and Output

| Input                               | Output                                   | Comment                                                                                                                                                                           |
|-------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 110<br>mall.Enter<br>d<br>mall.Exit | 1 purchases.<br>Money left:<br>80.00 lv. | 'd' has an ASCII code of 100. 'd' is a small letter, this is why Lina gets a 70% discount. She spends 30% of 100, which is 30 lv. After this purchase, she has: 110 - 30 = 80 lv. |



| Input                               | Output                                   |
|-------------------------------------|------------------------------------------|
| 110<br>mall.Enter<br>%<br>mall.Exit | 1 purchases.<br>Money left:<br>55.00 lv. |

| Input                                      | Output                                   |
|--------------------------------------------|------------------------------------------|
| 100<br>mall.Enter<br>Ab<br>**<br>mall.Exit | 2 purchases.<br>Money left:<br>58.10 lv. |

## Hints and Guidelines

We will separate the solution of the problem into three main parts:

- Processing of the **input**.
- **Algorithm** for solving the problem.
- Formatting the **output**.

Let's examine each of the parts in detail.

### Processing The Input Data

The input of our problem consists of a few components:

- On the **first line**, we have **all the money** that Lina has for shopping.
- On **each of the following lines**, we will have a **command**.

The first part of the reading is trivial. We should keep in mind that we will be using the **BigDecimal** type because we are working with money, and we need precise accuracy:

```
Scanner scanner = new Scanner(System.in);
BigDecimal shoppingMoney = new BigDecimal(scanner.nextLine());
```

The second one contains a detail that we need to take into consideration. The requirements state the following:

After we receive the **"mall. Enter"** command, then on each of the following lines, we will receive commands (strings) that contain information about the purchases/actions that Lina wants to perform.

It is where we need to take into consideration the fact that from the **second input line on**, we need to start reading commands, but only after we read the command **"mall.Enter"**, we must begin processing them. How can we do this? Using a **while** or a **do-while** loop is a good option. Here is an exemplary solution of how to skip all commands before processing the command **"mall.Enter"**:

```
String command = scanner.nextLine();
while (!command.equals("mall.Enter")) {
    command = scanner.nextLine();
}
command = scanner.nextLine();
```

Here is the place to point out that calling `scanner.nextLine()` after the end of the loop is used for **moving to the first command** for processing.

### Algorithm for Solving the Problem

The algorithm for solving the problem is a direct one – we continue **reading commands** from the console **until the command "mall.Exit" is passed**. In the meantime, we **process** each symbol (**char**) of each one of the commands according to the rules specified in the problem requirements, and in parallel, we **modify the amount** that Lina has and **store the number of purchases**.

Let's examine the first two sub-problems for our algorithm. The first sub-problem concerns the way we read the commands until we reach the **"mall.Exit"** command. The solution that we previously saw uses a **while-loop**. The second sub-problem is to **access each symbol** of the command passed. Keeping in mind that the input data with the commands is a **String** type, the easiest way to access each symbol inside the strings is via a **foreach** loop.

Here is what the code will look like when are using those two types of loops:

```
while (!command.equals("mall.Exit")) {  
    for (char action : command.toCharArray()) {  
          
    }  
    command = scanner.nextLine();  
}
```

The next part of the algorithm, is to **process the symbols from the commands**, according to the following rules in the requirements:

- If the symbol is a **capital letter**, Lina gets a 50% discount, which means that you must decrease the money she has by 50% of the numeric representation of the symbol from the ASCII table.
- If the symbol is a **lowercase letter**, Lina gets a 70% discount, which means that you must decrease the money she has by 30% of the numeric representation of the symbol from the ASCII table.
- If the symbol is **"%"**, Lina purchases that decrease her money in half.
- If the symbol is **"\*"**, Lina withdraws money from her debit card and adds 10 lv. to her available funds.
- If the symbol is **different from all above**, Lina just purchases without discount, and in this case, you should subtract the value of the symbol from the ASCII table from her available funds.

Let's examine the problems that we will be facing in the first condition. The first one is how to distinguish if a particular **symbol is a capital letter**. We can use one of the following ways:

- Keeping in mind that the letters in the alphabet have a particular order, we can use the following condition **action >= 'A' && action <= 'Z'** to check if our symbol is within the capital letters range.
- We can use the **char.IsUpper(..)** function.

The other problem is how to skip a particular symbol if it is not an operation that requires more money than Lina has. It is doable using the **continue** construction.

An example for the first part of the requirements looks like this:

```
if (action >= 65 && action <= 90) {
    price = new BigDecimal((int) action).
        multiply(new BigDecimal( val: 0.5));
    if (shoppingMoney.compareTo(price) < 0) {
        continue;
    }

    shoppingMoney=shoppingMoney.subtract(price);
    purchases++;
}
```

**Note:** the variable “**purchases**” is of **int** type, in which we store the number of all purchases.

We believe the reader should not have difficulties implementing all the other conditions because they are very similar to the first one.

### Formatting The Output

At the end of our problem, we must **print** a particular **output**, depending on the following condition:

- If no purchases have been made – “No purchases. Money left: {remaining funds} lv.”
- If at least one purchase is made – “{number of purchases} purchases. Money left: {remaining funds} lv.”

The printing operations are trivial. The only thing we need to keep in mind is that **the amount has to be printed with an accuracy of up to 2 symbols after the decimal point.**

How can we do that? We will leave the answer to this question to the reader.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/664#0>.

## Problem: X Expression

Bonny is an extremely powerful witch. Because her natural power is not sufficient to successfully fight vampires and werewolves, she has started to master the power of Expressions. An expression is hard to master because the spell relies on the ability to quickly solve mathematical expressions.

To use an "Expression spell" the witch must know the result of a mathematical expression in advance. An **Expression spell** consists of a few simple mathematical expressions. Each mathematical expression can contain operators for **addition**, **subtraction**, **multiplication**, and/or **division**.

The expression is solved without considering the mathematical rules for calculating numerical expressions. This means that the priority is applied according to the sequence of the operators, and not the type of calculation that they do. The expression **can contain brackets**, as **everything inside the brackets is calculated first**. Every expression can contain multiple brackets but no nested brackets:

- An expression containing (...(...)) is an invalid one.
- An expression containing (...)...(...) is a valid one.

## Problem

The expression

$$4 + 6 / 5 + (4 * 9 - 8) / 7 * 2$$

is solved in the following way:

$$\begin{aligned} &4 + 6 / 5 + (4 * 9 - 8) / 7 * 2 = \\ &10 / 5 + (4 * 9 - 8) / 7 * 2 = \\ &2 + (4 * 9 - 8) / 7 * 2 = \\ &2 + (36 - 8) / 7 * 2 = \\ &2 + 28 / 7 * 2 = \\ &30 / 7 * 2 = \\ &4.285714285714286 * 2 = \\ &8.571728571428571 = \\ &8.57 \end{aligned}$$

Bonny is pretty but not as wise, so she will need our help to master the power of Expressions.

## Input Data

The input data consists of a single text line passed from the console. It contains a **mathematical expression for calculation**. The line **always ends with the "=" symbol**. The "=" symbol means the **end of the mathematical expression**.

The input data is always valid and always in the described format. There is no need to validate it.

## Output Data

The output data must be printed on the console. The output consists of one line: the **result** of the calculated mathematical expression rounded up to the **second digit after the decimal point**.

## Constraints

- The expressions will consist of a maximum of **2500 symbols**.
- The numbers of each mathematical expression will be within the range **[1 ... 9]**.
- The operators in the mathematical expressions will always be among **+** (addition), **-** (subtraction), **/** (division) or **\*** (multiplication).
- The result of the mathematical expression will be within the range **[-100000.00 ... 100000.00]**.
- Allowed execution time: **0.1 seconds**.
- Allowed memory: **16 MB**.

## Sample Input and Output

| Input                | Output |
|----------------------|--------|
| $4+6/5+(4*9-8)/7*2=$ | 8.57   |

| Input                            | Output |
|----------------------------------|--------|
| $3+(6/5)+(2*3/7)*7/2*(9/4+4*1)=$ | 110.63 |

## Hints and Guidelines

As usual, we will first read and process the input, then we will solve the problem, and finally, we will print the result, formatted as required.

### Reading the Input Data

The input data will consist of only one text line read from the console. Here we have **two ways** to process the input. The first way is by **reading the entire line using the `bufferedReader.readLine()`** (where **`bufferedReader`** is a variable from type **`BufferedReader`**) and accessing each symbol (**`char`**) of the line via a **`foreach`** loop. The second one is by **reading the input symbol by symbol using the `bufferedReader.read()` command** and processing each symbol.

We will use the second option to solve the problem.

```
BufferedReader bufferedReader =
    new BufferedReader(new InputStreamReader(System.in));
int symbol = bufferedReader.read();
```

### Algorithm for Solving the Problem

For the problems of our problem, we will need two variables:

- One variable that will hold the **current result**.
- Another variable to hold our **current operator**.

```
double result = 0;
int expressionOperator = '+';
```

About the code above we must clarify one detail. The default value of the operator is **+** so that the first detected number is added to our result.

After we have declared our starting variables we must think of **what our structure** will look like of our program. From the condition of our problem, we know that **every expression is ending with =**. It means we have to read and process symbols until we match with **=**. After that, we will type a **while** loop as shown below.

```
while (symbol != '=') {
    symbol = bufferReader.read();
}
```

The next step is the processing of our **symbol** variable. We have 3 possible cases for it:

- If the symbol is a **start of a sub-expression placed in brackets**, i.e., the found symbol is a **(**.
- If the symbol is a **digit between 0 and 9**. But how can we check this? How can we check if our symbol is a digit? We can use for assistance the **ASCII code** of the symbol, via which we can use the following formula: **[ASCII code of our symbol] - [ASCII code of the symbol 0] = [the digit that represents the symbol]**. If the result of this condition is between 0 and 9, then our symbol is a **number**. (We can also use directly **'0'** and **'9'** symbols or their **ASCII codes**.)
- If the symbol is an **operator**, i.e., it is **+**, **-**, **\*** or **/**.

```
if (symbol == '(') {

} else if (0 <= symbol - '0' && symbol - '0' <= 9) {

} else if (symbol == '+' ||
           symbol == '-' ||
           symbol == '/' ||
           symbol == '*') {

}
```

Let's examine the actions that we need to undertake in the relevant cases that we defined:

- If our symbol is an **operator**, then the only thing we need to do is to **set a new value for the expressionOperator** variable.
- If our symbol is a **digit**, then we need to **change the current result of the expression depending on the current operator**, i.e., if **expressionOperator** is a **-**, then we must **decrease the result by the numerical representation of the current symbol**. We can get the numerical representation of the current symbol via the formula that we used upon checking the condition for this case (the **[ASCII code of our symbol] - [the ASCII code of the symbol 0] = [the digit that represents the symbol]**)

```

} else if (0 <= symbol - '0' && symbol - '0' <= 9) {
    switch (expressionOperator) {
        case '+':
            result += symbol - '0';
            break;
        case '-':
            result -= symbol - '0';
            break;
        case '*':
            result *= symbol - '0';
            break;
        case '/':
            result /= symbol - '0';
            break;
    }
} else if (symbol == '+' ||
    symbol == '-' ||
    symbol == '/' ||
    symbol == '*') {
    expressionOperator = symbol;
}

```

- If our symbol is a `(`, this indicates the **beginning of a sub-expression** (an expression in brackets). By definition, **the sub-expression must be calculated before modifying the result of the whole expression** (the actions in brackets are performed first). It means that we will have a local result for the sub-expression and a local operator.

```

if (symbol == '(') {
    double innerResult = 0;
    int innerOperator = '+';
    symbol = bufferReader.read();
}

```

After that, to **calculate the sub-expression value**, we will use the same methods that we used for calculating the main expression – we use a **while loop** to **read symbols** (until we reach an `)` symbol). Depending on whether the read symbol is a number or an operator, we modify the result of the sub-expression. The implementation of these operations is identical to the above-described logic for calculating expressions. It is why we believe the reader should not have a problem with it.

After completing the calculation of the result of our sub-expression, we **modify the result of the whole expression** depending on the value of the **expressionOperator**.

```

switch (expressionOperator) {
    case '+':
        result += innerResult;
        break;
    case '-':
        result -= innerResult;
        break;
    case '*':
        result *= innerResult;
        break;
    case '/':
        result /= innerResult;
        break;
}

```

### Formatting The Output

The only output that the program must print on the console is the **result of solving the expression with an accuracy of up to two digits after the decimal point**. How can we format the output this way? We will leave the answer to this question to the reader.

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/664#1>.

## Problem: Bulls and Cows

We all know the game called "Bulls and Cows" ([https://en.wikipedia.org/wiki/Bulls\\_and\\_cows](https://en.wikipedia.org/wiki/Bulls_and_cows)). Upon having a particular 4-digit **secret number** and a 4-digit **suggested number**, the following rules are applied:

- If a digit in the suggested number matches a digit in the secret number and is located at the **same position**, we have a **bull**.
- If a digit in the suggested number matches a digit in the secret number but is located at a **different position**, we have a **cow**.

| Secret number    | 1 | 4 | 8 | 1 | Comment               |
|------------------|---|---|---|---|-----------------------|
| Suggested number | 8 | 8 | 1 | 1 | Bulls = 1<br>Cows = 2 |

| Secret number    | 2 | 2 | 4 | 1 | Comment               |
|------------------|---|---|---|---|-----------------------|
| Suggested number | 9 | 9 | 2 | 4 | Bulls = 0<br>Cows = 2 |



Upon having a particular secret number and number of bulls and cows, our problem is **to find all possible suggested numbers\*** in ascending order.

If there are **no suggested numbers** that match the criteria provided from the console, we must print "No".

## Input Data

Our input data consists of 3 arguments:

- The first contains **the secret number**.
- The second contains **the number of bulls**.
- The third contains **the number of cows**.

The input data will always be valid. There is no need to verify them.

## Output Data

The output data must be printed on the console. The output must consist of **a single line** holding **all suggested numbers**, space-separated. If there are **no suggested numbers** that match the criteria provided from the console, we must **print "No"**.

## Constraints

- The secret number will always consist of **4 digits in the range [1..9]**.
- The number of **cows and bulls** will always be in the range **[0..9]**.
- Allowed execution time: **0.15 seconds**.
- Allowed memory: **16 MB**.

## Sample Input and Output

| Input          | Output                                                                                                                                                                                                                     |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2228<br>2<br>1 | 1222 2122 2212 2232 2242 2252 2262 2272 2281 2283 2284 2285<br>2286 2287 2289 2292 2322 2422 2522 2622 2722 2821 2823 2824<br>2825 2826 2827 2829 2922 3222 4222 5222 6222 7222 8221 8223<br>8224 8225 8226 8227 8229 9222 |

| Input          | Output                                                                                                                                                                |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1234<br>3<br>0 | 1134 1214 1224 1231 1232 1233 1235 1236 1237 1238 1239 1244<br>1254 1264 1274 1284 1294 1334 1434 1534 1634 1734 1834 1934<br>2234 3234 4234 5234 6234 7234 8234 9234 |

| Input          | Output |
|----------------|--------|
| 1234<br>3<br>1 | No     |

## Hints and Guidelines

We will solve the problem in a few steps:

- We will read the **input data**.
- We will generate all possible **four-digit combinations** (candidates for verification).
- For each generated combination we will calculate **how many bulls** and **how many cows** it has according to the secret number. Upon matching the needed bulls and cows, we will **print the combination**.

## Reading The Input Data

We have 3 lines as input data:

- Secret number.
- A Number for desired **bulls**.
- A Number for desired **cows**.

Reading the input data is trivial:

```
Scanner scanner = new Scanner(System.in);  
int guessNumber = Integer.parseInt(scanner.nextLine());  
int targetBulls = Integer.parseInt(scanner.nextLine());  
int targetCows = Integer.parseInt(scanner.nextLine());
```

## Algorithm for Solving the Problem

Before starting to write the algorithm for solving our problem, we must **declare a flag** that indicates whether a solution has been found:

```
boolean solutionFound = false;
```

If, after finishing our algorithm, this flag is still **false**, then we will print **No** on the console, as specified in the requirements.

```
if (!solutionFound) {  
    System.out.println("No");  
}
```

Let's start analyzing our problem. What we need to do is **analyze all numbers from 1111 to 9999**, excluding those containing zeroes (for example **9011**, **3401**, etc. are invalid). What is the easiest way to **generate all these numbers**? We will use **nested loops**. As we have a **4-digit number**, we will have **4 nested loops**, as each of them will generate an individual digit in our number for testing.

```
for (int digit1 = 1; digit1 <= 9; digit1++) {
    for (int digit2 = 1; digit2 <= 9; digit2++) {
        for (int digit3 = 1; digit3 <= 9; digit3++) {
            for (int digit4 = 1; digit4 <= 9; digit4++) {
```

Thanks to these loops, we have access to every digit of all numbers that we need to check. Our next step is to **separate the secret number into digits**. It can be achieved very easily using a combination of integer division and modular division.

```
int guessDigit1 = (guessNumber / 1000) % 10;
int guessDigit2 = (guessNumber / 100) % 10;
int guessDigit3 = (guessNumber / 10) % 10;
int guessDigit4 = (guessNumber / 1) % 10;
```

Only two last steps remain before we begin to analyze how many cows and bulls there are in a particular number. Accordingly, the first step is the **declaration of counter variables** in the nested loops, to **count the cows and bulls** for the current number. The second step is to make **copies of the digits of the current number** that we will analyze, to prevent problems upon working with nested loops, in case we make changes to them.

```
int digitToCheck1 = digit1;
int digitToCheck2 = digit2;
int digitToCheck3 = digit3;
int digitToCheck4 = digit4;

int currentBulls = 0;
int currentCows = 0;
```

We are ready to start analyzing the generated numbers. What logic can we use? The easiest way to check how many cows and bulls there are inside a number is via a **sequence of if-else conditions**. Yes, it is not the best way, but to use knowledge beyond the scope of this book, we will use this approach.

What conditions do we need?

The condition for the bulls is simple – we check whether the **first digit** of the generated number matches the **same digit** in the secret number. We remove the digits that are already checked to avoid repetitions of bulls and cows.

```
// Find all bulls, count them and remove them (assign -1 and -2)
if (digit1 == guessDigit1) {
    // Bull at position #1 found -> count it and remove it
    currentBulls++;
    guessDigit1 = -1;
    digit1 = -2;
}
```

We repeat the action for the second, third, and fourth digits.

We will apply the following condition for the cows – first, we will check whether the **first digit** of the generated number **matches the second one**, the **third one**, or the **fourth digit** of the secret number. An example for the implementation:

```
// Find all cows for digit1, count them and remove them (assign -1)
if (digit1 == guessDigit2) {
    // Cow at position #2 found -> count it and remove it
    currentCows++;
    guessDigit2 = -1;
} else if (digit1 == guessDigit3) {
    // Cow at position #3 found -> count it and remove it
    currentCows++;
    guessDigit3 = -1;
} else if (digit1 == guessDigit4) {
    // Cow at position #4 found -> count it and remove it
    currentCows++;
    guessDigit4 = -1;
}
```

Then, we check: whether the **second digit** of the generated number matches the **first one**, the **third one**, or the **fourth digit** of the secret number; whether the **third digit** of the generated number matches the **first one**, the **second one**, or the **fourth digit** of the secret number; and finally, we check whether the **fourth digit** of the generated number matches the **first one**, the **second one** or the **third digit** of the secret number.

### Printing The Result

After completing all conditions, we need to check whether the bulls and cows in the currently generated number match the desired bulls and cows read from the console. If this is true, we print the current number on the console.

```
if (currentBulls == targetBulls && currentCows == targetCows) {
    if (solutionFound) {
        System.out.println(" ");
    }
    System.out.print("" + digit1 + digit2 + digit3 + digit4);
    solutionFound = true;
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/664#2>.

# Chapter 10. Methods

In this chapter, we will introduce you to **methods**. We will learn what they are, what is their **base concepts** when working with them. We will learn why it is a **good practice** to use them, how to **declare** and **call** them. We will also introduce you to the **parameters** and **return value of a method** and how to use that return value. At the end of this chapter, we will look at the **established practices** when using methods.

## Video

Watch the video lesson about what we will learn in this chapter:

<https://www.youtube.com/watch?v=dYgouc4KMiU>.

## What is a Method?

So far, we have found out that while **writing** a code for a program that solves a problem, **separating** the problem into different **parts** favors us a lot. Every part is responsible for a **corresponding action** and by doing so, it is **easier** to solve the problem, and the **readability** and the debugging of the code are better.

A block of code designed to perform a particular problem, which we have logically separated is called a **method**. **Methods – pieces of code, that are named** by us in a certain way and could be called as many times as we need.

A **method** can be **called** as many times as we think we need to solve a problem. That **saves** us from repeating the same code several times, **reduces** the possibility of making a mistake while editing the repeated code.

## Simple Methods

Simple methods are responsible for running a specific **action** that will **help** us to solve a problem. Those actions can be printing a string on the console, performing some checks, performing a loop, etc.

Let's look at the following **example of a simple method**:

```
static void printHeader() {
    System.out.println("-----");
}
```

This **method** has the problem to print a header containing a series of the symbol **-**. Because of this, the name of the method is **printHeader**. The parentheses **(** and **)** always follow the name, no matter how we have named the method. Later we will take a look at how we should name methods. For now, we will only say that the **name of a method must describe the action** he performs.

The **body** of the method consists of **programming code**, which is located between the curly brackets **{** and **}**. Between them, we place code, that will solve our problem, described by the name of the method.

## Why Should We Use Methods?

Up to this moment, we have found out that, methods help us with **separating big problems into smaller parts**, which leads to a **simple solution** of the corresponding problem. It makes our program, not just well structured, **easily readable** but also more understandable.

Using methods, we **escape the repeating** of programming code. **Repeating** code is a bad practice, as it makes it very difficult to maintain the program and leads to errors. If one part of code exists in our program more than once and we have to fix it, we will have to change it at every occurrence. The probability of us forgetting one of the repeated places is high, which will lead to incorrect behavior of our program. It is why it's a **good practice** to define a fragment that will be used **more than once** as a **separate method**.

Methods offer us an **opportunity** to use **code several times**. By solving more and more problems, we will conclude that using already defined methods saves us much time and effort.

## Declaring Methods

In Java language, you can **declare** methods inside a class, i.e., between the opening **{** and closing **}** brackets of the class but not in the declaration of another method. The declaration is the registration of the method in the program, and thus the method will be recognized in the rest of it. The best-known example is the **main(...)** method, which we use in every program we write.

```
public class Methods {  
    public static void main(String[] args) {  
  
    }  
}
```

With the next example, we will look at the mandatory elements in the declaration of a method.

```
static double getSquare(double side) {  
    return side * side;  
}
```

- **The type of the returned value.** In this case, the type is **double**, which means that the method will **return a result**, which is of **double** type. The returned value can be **int, double, string, etc.**, and also **void**. If the type is **void**, this means that the method **doesn't return a result**, but only **does a particular operation**.
- **Method name.** The name of the method is **defined by us**, but we should not forget that it has to **describe the function**, executed by the code in its body. In the example, the name is **GetSquare**, which tells us that this method calculates the area of a square.
- **Parameters list** is declared between the parentheses **(** and **)** that we write after its name. Here we list all the **parameters** that the method will use. There can be **only**

one parameter, multiple ones, or it could be an **empty** list. If there are no parameters, we will write only the parentheses **()**. In this example, we declare one parameter with a name **side** and with a type **double**.

- **static** declaration in the method description. For the moment, we can accept that we write **static** always when we declare a method. Later, when we get familiar with object-oriented programming (OOP), we will learn about the difference between **static methods** (shared for the whole class) and **methods of an object**, which work on the data of a particular instance of the class (object).

When declaring methods, you must follow the **sequence** of its base elements – first is the keyword **static**, then the **type of the returned value**, then **method name**, and finally a **list of parameters**, surrounded by parentheses **()**.

After we have declared a method, its **implementation (body)** follows. In the body of the method, we write down **the algorithm**, by which it solves a problem, i.e., the body contains the code (program block), which applies the method's **logic**. In the example above we calculate the area of a square, which is **side \* side**.

When declaring a variable in the body of a method, it is called a **local** variable. The area where this variable exists and can be used starts from the line where we have declared it and reaches the closing curly bracket **}** of the body of the method. This area is called **variable scope**.

## Invoking a Method

Invoking a method means **starting to execute the code**, which is in the **body of the method**. It happens by writing its **name**, followed by parentheses **()** and the semicolon sign **;** to end the line. If the method needs input data, it is given in the parentheses **()**, and the succession of the parameters should be the same as one of the given parameters when declaring the method. Here is an example:

```
// declaring method
static void printHeader() {
    System.out.println("-----");
}

public static void main() {
    // invoking the declared method
    printHeader();
}
```

A method can be called from **several places** in the program. One way is to call it inside the **main** method.

```
public static void main(String[] args) {
    // invoking the declared method
    // from the main method
    printHeader();
}
```

A method can also be called from **the body of another method**, which is **not** the main method of the program.

```
static void printHeader() {  
    // invoking methods from another method  
    printHeaderTop();  
    printHeaderBottom();  
}
```

A method can be called from **its own body**. It is called **recursion** and you can find more about it on [Wikipedia](#) or google it.

It is important to know that if a method is declared in a class, it can be called before the line, on which it has been declared.

### Problem: Blank Receipt

Write a method that prints a blank receipt. The method should call another three methods: one to print the upper part, one for the middle part of the receipt, and one for the bottom part.

| Part of cash receipt | Text                                |
|----------------------|-------------------------------------|
| Upper part           | CASH RECEIPT<br>-----               |
| Middle part          | Charged to_____<br>Received by_____ |
| Bottom part          | -----<br>(c) SoftUni                |

### Sample Input and Output

| Input      | Output                                                                                       |
|------------|----------------------------------------------------------------------------------------------|
| (no input) | CASH RECEIPT<br>-----<br><br>Charged to_____<br>Received by_____<br>-----<br><br>(c) SoftUni |



## Hints and Guidelines

The first step is to create a **void** method to **print the upper part** of the receipt (header). Let's give it a meaningful name, which describes what the method does, e.g., **PrintReceiptHeader**. In its body, write the code from the example below:

```
static void printReceiptHeader() {
    System.out.println("CASH RECEIPT");
    System.out.println("-----");
}
```

In the same way, we'll create two more methods: one to **print the middle part** of the receipt (body) **PrintReceiptBody** and another to **print the bottom part** of the receipt (footer) **PrintReceiptFooter**.

After this we will create **another method**, which will call the three methods we already wrote, one after the other:

```
static void printReceipt() {
    printReceiptHeader();
    printReceiptBody();
    printReceiptFooter();
}
```

Finally, we'll call the **PrintReceipt** method from the body of the **Main** method of our program:

```
public static void main(String[] args) {
    printReceipt();
}
```

## Testing in The Judge System

The program with a total of five methods that call one another is ready and we can **run and debug it**. After that, we will send it for test in the Judge system:

<https://judge.softuni.org/Contests/Practice/Index/665#0>.

## Methods with Parameters

Often to solve a problem, the method by which we achieve this needs **additional information**, which depends on its purpose. This information represents the **parameters of the method**, and its behavior depends on them.

## Using Parameters in Methods

As we noted above, the **parameters can be zero, one, or more**. When declaring them, you should divide them with a comma. They can be of any type (**int**, **string**, etc.), and below is an example of how they are used by the method.

We **declare** the method and its **list of parameters**, then we write the code that the method executes.

```
static void printNumbers(int start, int end) {  
    for (int i = start; i <= end; i++) {  
        System.out.printf("%d", i);  
    }  
}
```

After that **call** the method and **give it particular values**:

```
public static void main(String[] args) {  
    printNumbers(5, 10);  
}
```

When **declaring parameters**, we can use **various** types of variables, and we should be careful that every parameter has a **type** and **name**. It is important to note that when invoking the method, we must pass to its **values** for the parameters in **the order**, in which they are **declared**. If the parameters are first **int** and after that **string**, when invoking the method, we can't give it first a **string** and then **int**.

Let's look at the example for declaring a method, which has several parameters of different types.

```
static void printStudent(  
    String name, int age, double grade) {  
    System.out.printf("Student: %s; Age: %d; Grade: %.2f",  
        name, age, grade);  
}
```

## Problem: Sign of Integer Number

Create a method that prints the sign of an integer **n**.

### Sample Input and Output

| Input | Output                     |
|-------|----------------------------|
| 2     | The number 2 is positive.  |
| -5    | The number -5 is negative. |
| 0     | The number 0 is zero.      |

### Hints and Guidelines

The first step is **creating** a method and giving it a descriptive name, e.g., **PrintSign**. This method will have only one parameter of **int** type.

```
static void printSign(int n) {  
}
```

The next step is **implementing** the logic by which the program will check what the sign of the number is. You can see from the examples that there are three cases – the number is greater than, equals to, or less than zero, which means that we'll make three verifications in our method.

The next step is to read the input number and to call the new method from the body of the **main** method.

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    int n = Integer.parseInt(scanner.nextLine());  
    printSign(n);  
}
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#1>.

Problem: Printing Triangle

Create a method that prints a triangle as shown in the examples.

Sample Input and Output

| Input | Output | Input | Output  |
|-------|--------|-------|---------|
| 3     | 1      | 4     | 1       |
|       | 1 2    |       | 1 2     |
|       | 1 2 3  |       | 1 2 3   |
|       | 1 2    |       | 1 2 3 4 |
|       | 1      |       | 1 2 3   |
|       |        |       | 1 2     |
|       |        |       | 1       |

Hints and Guidelines

Before creating a method to print a row with a given beginning and an end, we must read the input number from the console. After that, we choose a meaningful name, which describes its purpose, e.g., **printLine**, and implements it.

```
static void printLine(int start, int end) {
    for (int i = start; i <= end; i++) {
        System.out.print(i + " ");
    }
    System.out.println();
}
```

From the drawing exercises, we remember that it is a good practice **to divide the figure into several parts**. To make it easier, we will divide the triangle into three parts – upper, middle, and bottom.

The next step is to print **the upper half** of the triangle using a loop:

```
for (int i = 0; i < n; i++) {
    printLine(1, i);
}
```

After that, we will print the **middle part**:

```
printLine(1, n);
```

Finally, we will print the **bottom part** from the triangle, but this time with a reverse loop.

```
for (int i = n - 1; i > 0; i--) {
    printLine(1, i);
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#2>.

## Problem: Draw a Filled Square

Print on the console a filled square with side **n**, as in the examples below.

### Sample Input and Output

| Input | Output                                           | Input | Output                                           | Input | Output                         |
|-------|--------------------------------------------------|-------|--------------------------------------------------|-------|--------------------------------|
| 4     | <pre> ----- -\\\\/- -\\\\/- -\\\\/- ----- </pre> | 3     | <pre> ----- -\\\\/- -\\\\/- -\\\\/- ----- </pre> | 2     | <pre> ----- ----- ----- </pre> |

### Hints and Guidelines

The first step is to read the input from the console. After that, we need to create a method, which will print the first and the last rows because they are the same. Let's

remember that we must give it a **descriptive name** and give it as a **parameter** the length of the side. We will use the constructor's **new String**.

```
static void printHeaderFooter(int n) {
    System.out.println(new String(new char[2 * n])
        .replace("\0", "-"));
}
```

Our next step is to create a method that will draw the middle rows on the console. Again, give it a descriptive name i.e., **printMiddleRow**.

```
static void printMiddleRow(int n) {
    System.out.print("-");
    for (int i = 0; i < n - 1; i++) {
        System.out.print("\\\\");
    }
    System.out.println("-");
}
```

Finally, call the methods in the **main()** method of the program to draw the whole square:

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int n = Integer.parseInt(scanner.nextLine());

    printHeaderFooter(n);
    // TODO: Draw the rest of the square
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#3>.

## Returning Result from a Method

So far, we have looked at methods that do a specific problem, for example, printing a text, number, or figure on the console. There is another type of method that can **return** a **result** from their problem. We will look at this type of method in the following lines.

## Types of Returned Values

So far, we saw some examples, in which when declaring methods, we used the keyword **void**, which shows that the method **does not** return a result, but just executes a certain action.

```
static void addOne(int n) {
    n += 1;
    System.out.println();
}
```

If we replace **void** with a type of some variable, this will tell the program that the method should return a value of the said type. This returned value could be of any type – **int**, **String**, **double**, etc.



For a method to return a **result** we need to write the type of returned value we expect when declaring the method, in place of the **void**.

```
static int plusOne (int n) {  
    return n + 1;  
}
```

We should note that the **result** returned by the method can be of a type, compatible with the type of the returned value of the method. For example, if the declared type of the returned value is **double**, we can return a value of **int** type.

## The **return** Operator

To obtain a result from the method, we need to use the **return** operator. It should be used in the **body** of the method and tells the program to **stop its execution** and to **return** to the method's caller, a **value** that is defined by the expression after the **return** operator.

In the example below, there is a method that reads two names from the console, concatenates them, and returns them as a result. The return value is of **String** type:

```
static String readFullName(String firstName,  
    String lastName) {  
    return firstName + " " + lastName;  
}
```

The **return** operator can also be used in **void** methods. This way, the method will stop its execution without returning a value, and after it, there shouldn't be an expression, which should be returned. In this case, we use **return** only to exit the method.

There are cases where **return** can be called, from multiple places in the method, but only if there are **certain** input conditions.

We have a method in the example below, which compares two numbers and returns a result respectively **-1**, **0**, or **1** depending on if the first argument is less than, equals to, or greater than the second argument, given to the method. The method uses the keyword **return** in three different places, to return three different values according to the logic of comparing the numbers:

```
static int compareTo(int number1, int number2) {
    if (number1 > number2) {
        return 1;
    } else if (number1 == number2) {
        return 0;
    } else {
        return -1;
    }
}
```

### Code After a return is Unreachable

When the **return** operator is located inside a conditional statement such as **if**, after the statement in the same block, we must **not** have rows with code because IntelliJ IDEA will display a warning telling us that it had found an **unreachable** statement.

```
private static int someFunction() {
    return 5;
    return 2;
}
```

Unreachable statement



In programming, we can't have 2 times the operator **return** one after another because the first return won't allow us to use the second. Sometimes programmers joke with the phrase *write return; return; and let's go home*, to explain that the logic of the program is wrongly typed.

### Using The return Value of a Method

After a method is executed and has returned a value, we can use the value in several ways.

The first is to **assign** the result to a variable:

```
int max = getMax(5, 10);
```

The second is to be used inside **an expression**:

```
double total = getPrice() * quantity * 1.2;
```

The third is to **pass** the result from one method to **another method**:

```
Scanner scanner = new Scanner(System.in);
int age = Integer.parseInt(scanner.nextLine());
```

## Problem: Calculate Triangle Area

Create a method that calculates the area of a triangle using the given base and height and returns it as a result.

### Sample Input and Output

| Input  | Output |
|--------|--------|
| 3<br>4 | 6      |

### Hints and Guidelines

The first step is to read the input. After that, **create** a method, but this time be careful when **declaring** to give it the correct **type** of data we want the method to return, which is **double**

```
static double getTriangleArea(  
    double length, double height) {  
    return (length * height) / 2;  
}
```

The next step is to **call the new** method from the **main()** method and to **store the** returned value in a suitable variable.

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    DecimalFormat format = new DecimalFormat("#.#####");  
  
    double l = Double.parseDouble(scanner.nextLine());  
    double h = Double.parseDouble(scanner.nextLine());  
    double area = getTriangleArea(l, h);  
  
    System.out.println(format.format(area));  
}
```

### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#4>.

## Problem: Math Power

Write a method that calculates and returns the result the power of a number.



## Sample Input and Output

| Input | Output | Input | Output |
|-------|--------|-------|--------|
| 2     | 256    | 3     | 81     |
| 8     |        | 4     |        |

## Hints and Guidelines

Our first step is to read the input data from the console. The next step is to create a method that will take two parameters (the number and the power) and return, as a result, a number of a **double** type.

```
static double CalculatePower(
    double number, double power) {
    double result = 0.0;
    // TODO: Calculate result
    // Use a loop or Math.pow(...)
}
```

After we have done the calculations, we have to only print the result in the **main()** method of the program.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#5>.

# Method Overloading

In many programming languages, the same methods can be declared in **different variants** with the **same name and different parameters**. It is known as “**method overloading**”. Now let's see how to write these overloaded methods.

## Method Signature

In programming, methods are **identified** through the elements of their declaration: **name** of the method + a list of its **parameters**. These two elements define its specification, the so-called “**method signature**”.

```
static void print(String text) {
    System.out.println(text);
}
```

In this example, the method's signature is its name (**print**), together with its parameter (**String text**).

If our program holds several **methods with the same name**, but with **different lists of parameters (signatures)**, we can say that we use **method overloading**.

## Overloading Methods

As we mentioned, if you use the same name for several methods with different signatures, this means that you are **overloading a method**. The code below shows how three different methods can use the same name but have different signatures and execute different actions.

```
static void print(String text) {
    System.out.println(text);
}

static void print(int number) {
    System.out.println(number);
}

static void print(String text, int number) {
    System.out.println(text + " " + number);
}
```

## Signature and the return Value Type

It is important to say that the returned type as a result of the method is not a part of its signature. If the returned type was a part of the signature, then the compiler doesn't know which method exactly to call (there is ambiguity).

Let's look at the following example: we have two methods with different return types. Even though IntelliJ IDEA shows that there is an error, our compiler won't know which of the two methods to invoke because they are with the same name **print()**.

```
static void print(String text) {
    System.out.println(text);
}

static String print(String anotherText) {
    return anotherText;
}
```

## Problem: Greater of Two Values

The input is two values of the same type. The values can be of **int**, **char**, or **String** type. Create a method **getMax()** that returns, as a result, the greater of the two values.

### Sample Input and Output

| Input    | Output | Input     | Output | Input          | Output |
|----------|--------|-----------|--------|----------------|--------|
| int<br>2 | 16     | char<br>a | z      | string<br>Ivan | Todor  |

| Input | Output |
|-------|--------|
| 16    |        |

| Input | Output |
|-------|--------|
| z     |        |

| Input | Output |
|-------|--------|
| Todor |        |

## Hints and Guidelines

To complete our problem, we need to create three methods with the same name and different signatures. First, we create a method, which will compare integers.

```
static int getMax(int first, int second) {
    if (first >= second) {
        // TODO: Return value
    }
    // TODO: Handle other cases
}
```

Following the logic of the previous method, we create another one with the same name, but this one will compare characters.

```
static char getMax(char first, char second) {
    // TODO: Create logic
}
```

The next method we need to create will compare strings. The logic here is a bit different from the previous two methods because variables of **String** type cannot be compared with the operators **<** and **>**. We will use the method **compareTo(...)**, which returns a numerical value: greater than 0 (the compared object is larger), less than 0 (the compared object is smaller), and 0 (the two objects are the same).

```
static String getMax(
    String first, String second) {
    if (first.compareTo(second) >= 0) {
        // TODO: Return value
    }
    // TODO: Return value
}
```

The last step is to read the input data, use the appropriate variables, and invoke the method **getMax()** from the body of the **main()** method.

```
Scanner scanner = new Scanner(System.in);
String type = scanner.nextLine();

if (type.equals("int")) {
    int first = Integer.parseInt(scanner.nextLine());
    int second = Integer.parseInt(scanner.nextLine());
    int max = getMax(first, second);
    System.out.println(max);
}
```

```
} else if (type.equals("char")) {  
    // TODO: Call getMax() with char arguments  
} else if (type.equals("string")) {  
    // TODO: Call getMax() with string arguments  
}
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#6>.

## Naming Methods. Best Practices When Working with Methods

In this chapter, we will introduce you to some of the **best practices** when working with methods. They will be related to naming, organizing our code and its structure.

### Naming methods

When we name a method, it's recommended to use a **logical name**. This is good because every method must **correspond** to a specific part of our problem. We must take into consideration the **problem** which the method will do. This is why it's a good practice **for** the name to describe the purpose of the method.

It is required for the name of the method to start with a **small letter** and to be a verb or a combination of a verb and a noun. Formatting the names must be done following **Lower Camel Case** convention - **every word except the first to start with an upper letter**. Round brackets ( and ) are always after the name of the method.

Every method must do an independent problem, and the name must describe its role.

Here are some examples of **correctly** named methods:

- **findStudent**
- **loadReport**
- **sine**

Some examples of **badly** named methods:

- **Method1**
- **DoSomething**
- **HandleStuff**
- **SampleMethod**
- **DirtyHack**

If we can't come up with a good name, it's probably because our method does more than one job or it doesn't have a clear problem. We must think of a way to divide the method into several separate methods.

## Naming Parameters of Methods

When naming the parameters of a method you can apply almost the same rules as with the methods themselves. The difference here is that it is good for the names of the parameters to use a noun or a couple of an adjective and a noun, and when naming the parameters, we use the **lowerCamelCase** convention, i.e., **each word except for the first one starts with uppercase**. We should note that it is a good practice that the name of the parameter **shows** what **unit** is used when working with it.

Here are some examples of **correctly** named parameters of methods:

- `firstName`
- `report`
- `speedKmh`
- `usersList`
- `fontSizeInPixels`
- `font`

Some examples of **incorrectly** named parameters:

- `p`
- `p1`
- `p2`
- `populate`
- `LastName`
- `last_name`

## Good Practices When Working with Methods

We must keep in mind that a method must do **only one** specific **problem**. If this can't be achieved, then we must come up with a way to **divide** the functionality into different parts. As we have already said, the name must be clear and descriptive. Another **good practice** is to **avoid** methods that are longer than our screen. If this happens, it's recommended to **split** the method into smaller ones, as shown in the example below.

```
static void printReceipt() {  
    printReceiptHeader();  
    printReceiptBody();  
    printReceiptFooter();  
}
```

## Structure and Formatting of The Code

When writing methods, we must keep in mind to follow a correct **indentation** (offset blocks of code).

Here are some examples of **correctly** formatted Java code:

```
public static void main(String[] args) {  
    // some code here...  
    // some more code...  
}
```

Some examples of **incorrectly** formatted Java code:

```
public static void main(String[] args)  
{  
    // some code here...  
    // some more code...  
}
```

When the headline row of the method is **too long**, it's recommended to divide it into several rows, moving each row after the first one with two tabs to the right (for better readability):

```
static void printStudent(  
    String name, int age, double grade) {  
    System.out.printf("Student: %s; Age: %d; Grade: %.2f",  
        name, age, grade);  
}
```

Another good practice is to **leave an empty row** between methods, after loops, and conditional statements. You should **avoid** writing **long rows and complicated expressions**. Over time you will find out that improves the readability and saves time.

We recommend you to always **use curly brackets for the body of conditional statements and loops**. The brackets don't just improve the readability but also reduce the possibility of making a mistake.

## What Have We Learned from This Chapter?

In this chapter, we have learned base concepts about working with methods:

- We learned that **the goal** of a method is to **split** programs with many rows of code into smaller and shorter parts.
- We got acquainted with the **structure** of methods and how to **declare** and **call** them by their name.
- We looked into different examples of methods with **parameters** and how to use them in our program.
- We learned what the **signature** and the **return value** of the method are and what the role of the **return** operator is in the methods.
- We learned about **good practices** for working with methods, how to name them and their parameters, how to format our code, and others.

## Problems

To improve what we have learned, we will solve a few exercises. In them, it is required to write a method with specific functionality and then call it with values read from the console, as shown in the example.

### Problem: Hello, Name!

Write a method that takes a name as a parameter and prints on the console "Hello, \!".

#### Sample Input and Output

| Input | Output        |
|-------|---------------|
| Peter | Hello, Peter! |

#### Hints and Guidelines

Define a method `printName(String name)` and implement it, after which read a name from the console in the main program and invoke the method by giving him the name.

#### Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#7>.

### Problem: Min Method

Create a method `getMin(int a, int b)`, which returns the smaller of two numbers. Write a program, which takes as input three numbers and prints the smallest of them. Use the method `getMin(...)`, which you have already created.

#### Sample Input and Output

| Input | Output | Input | Output |
|-------|--------|-------|--------|
| 1     | 1      | -100  | -102   |
| 2     |        | -101  |        |
| 3     |        | -102  |        |

#### Hints and Guidelines

Define a method `getMin(int a, int b)` and implement it. After that, call it from the main method, as shown in the example below. To find the minimum of 3 numbers, first, find the minimum of the first two and then the result from their result and the third one.

```
int min = getMin(getMin(num1, num2), num3);
```

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#8>.

## Problem: String Repeater

Create a method **repeatString(str, count)**, which takes parameters of type **String** and an integer **n** and returns the string, repeated **n** times. After this, print the result on the console.

### Sample Input and Output

| Input    | Output | Input     | Output                   |
|----------|--------|-----------|--------------------------|
| str<br>2 | strstr | roki<br>6 | rokirokirokirokirokiroki |

### Hints and Guidelines

Write method and add the input string to the result in the for loop:

```
static String repeatString(  
    String str, int count) {  
    String repeatedString = "";  
    for (int i = 0; i < count; i++) {  
        // TODO: Logic  
    }  
}
```

Keep in mind that in Java concatenating strings in loops leads to bad performance and is not recommended. Learn more at: <https://stackoverflow.com/questions/1235179>.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#9>.

## Problem: Nth Digit

Create a method **findNthDigit(number, index)**, which takes a number and index **N** as parameters and prints the **N**-th digit of the number (counting from right to left and starting from 1). After that, print the result on the console.



## Sample Input and Output

| Input      | Output |
|------------|--------|
| 83746<br>2 | 4      |

| Input         | Output |
|---------------|--------|
| 93847837<br>6 | 8      |

| Input     | Output |
|-----------|--------|
| 2435<br>4 | 2      |

## Hints and Guidelines

To do the algorithm, use a **while** loop until the given number equals 0. At each iteration of the **while** loop, check if the current index of the digit is equal to the index you are looking for. If it is, return as a result of the digit at this index (**number % 10**). If not, remove the last digit in the number (**number = number / 10**). You should count which digit you are checking by index (from right to left and starting from 1). When you find the number, return the index.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#10>.

## Problem: Integer to Base

Write a method **integerToBase(number, toBase)**, which takes as parameters an integer and a base of a numeral system and returns the integer converted to the given numeral system. After this, the result should be printed on the console. The input number will always be in the decimal numeral system, and the base parameter will be between 2 and 10.

## Sample Input and Output

| Input  | Output |
|--------|--------|
| 3<br>2 | 11     |

| Input  | Output |
|--------|--------|
| 4<br>4 | 10     |

| Input  | Output |
|--------|--------|
| 9<br>7 | 12     |

## Hints and Guidelines

To solve the problem, we will declare a string, in which we will keep the result. After this, we need to do the following calculations to convert the number.

- We calculate **the remainder** of the number, divided by the base.
- We **insert the remainder** at the beginning of the string.
- We **divide** the number by the base.
- We **repeat** the algorithm until the input integer equals 0.

Write the missing logic in the method below:

```
static String integerToBase(int number, int toBase) {
```

```
String result = "";
while (number != 0) {
    // Implement the missing conversion logic
}
return result;
}
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#11>.

Problem: Notifications

Write a program, which takes an integer **n** and **n** input messages and prints **n** output messages, based on the input. For each message read a few lines. Each message starts with **messageType**: “**success**”, “**warning**” or “**error**”:

- When **messageType** is a “**success**”, read **operation** + **message** (each from a new line).
- When **messageType** is a “**warning**” read-only **message** (from a new line).
- When **messageType** is an “**error**” read **operation** + **message** + **errorCode** (each from a new line).

Print on the console **each read message** formatted depending on its **messageType**. After the headline of the message print as much **=**, as the **length** of the said **headline** and print **an empty line** after each message (to understand in detail look at the examples).

The problem should be solved by defining four methods: **showSuccessMessage()**, **showWarningMessage()**, **showErrorMessage()** and **readAndProcessMessage()**, so that only the last method is called in the **main()** method:

```
static void showSuccessMessage(String operation,
                                String message) {...}

static void showWarningMessage(String message) {...}

static void showErrorMessage(String operation,
                              String message, int errorCode) {...}

static void readAndProcessMessage () {...}
```

Sample Input and Output

| Input      | Output                                                  |
|------------|---------------------------------------------------------|
| 4<br>error | Error: Failed to execute credit card purchase.<br>===== |

| Input                                                           | Output                                                                                  |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| credit card purchase<br>Invalid customer<br>address<br>500      | Reason: Invalid customer address.<br>Error code: 500.                                   |
| warning<br>Email not confirmed                                  | Warning: Email not confirmed.<br>=====                                                  |
| success<br>user registration<br>User registered<br>successfully | Successfully executed user registration.<br>=====                                       |
| warning<br>Customer has not email<br>assigned                   | User registered successfully.<br><br>Warning: Customer has not email assigned.<br>===== |

## Hints and Guidelines

Define and implement the four shown methods.

In **ReadAndProcessMessage()** read the type of message from the console and according to the read, type read the rest of the data (one, two, or three more lines). After that call the method for printing the given type of message.

If you are reading from the console a few separate methods, you should keep in mind that you should **create a Scanner object only once** (for example in the main method) and pass it to the other methods as a parameter. Creating more than one **Scanner** which reads from the console might not work as expected.

## Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#12>.

## Problem: \* Numbers to Words

Write a method **letterize(number)**, which reads an integer and prints it in words in English according to the conditions below:

- Print in words the hundreds, the tens, and the ones (and the eventual minus) according to the rules of the English language.
- If the number is greater than 999, you must print "too large".
- If the number is less than -999, you must print "too small".
- If the number is **negative**, you must print "minus" before it.
- If the number is not built up of three digits, you shouldn't print it.

Sample Input and Output

| Input                    | Output                                                                     | Input          | Output                             |
|--------------------------|----------------------------------------------------------------------------|----------------|------------------------------------|
| 3<br>999<br>-420<br>1020 | nine-hundred and ninety nine<br>minus four-hundred and twenty<br>too large | 2<br>15<br>350 | fifteen<br>three-hundred and fifty |

| Input                           | Output                                                                                      | Input                | Output                                               |
|---------------------------------|---------------------------------------------------------------------------------------------|----------------------|------------------------------------------------------|
| 4<br>311<br>418<br>509<br>-9945 | three-hundred and eleven<br>four-hundred and eighteen<br>five-hundred and nine<br>too small | 3<br>500<br>123<br>9 | five-hundred<br>one-hundred and twenty three<br>nine |

Hints and Guidelines

We can first print **the hundreds** as a text -  $(\text{the number} / 100) \% 10$ , after that **the tens** -  $(\text{the number} / 10) \% 10$  and at the end **the ones** -  $(\text{the number} \% 10)$ .

The first special case is when the number is exactly **rounded to 100** (e.g. 101, 305, 609, etc.). In this case we print "one-hundred", "two-hundred", "three-hundred" etc.

The second special case is when the number formed by the last two digits of the input number is **less than 10** (e.g. 101, 305, 609, etc.). In this case, we print "one-hundred and one", "three-hundred and five", "six-hundred and nine" etc.

The third special case is when the number formed by the last two digits of the input number is **greater than 10 and less than 20** (e.g. 111, 814, 919, etc.). In this case, we print "one-hundred and eleven", "eight-hundred and fourteen", "nine-hundred and nineteen" etc.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#13>.

Problem: \* String Encryption

Write a method `encrypt(char letter)`, which encrypts a given letter in the following way:

- It takes the first and the last digit from the ASCII code of the letter and concatenates them into a string, which will represent the result.

- At the beginning of the string, which represents the result, we will insert the symbol which matches the following condition:
  - ASCII code of the letter + the last digit of the ASCII code of the letter.
- After that, at the end of the string, which represents the result, you concatenate the character which matches the following condition:
  - ASCII code of the letter - the first digit of the ASCII code of the letter.
- The method should return the encrypted string.

Example:

- $j \rightarrow p16i$ 
  - ASCII code of  $j$  is **106**  $\rightarrow$  First digit – **1**, last digit – **6**.
  - We concatenate the first and the last digit  $\rightarrow$  **16**.
  - At **the beginning** of the string, which represents the result, concatenate the symbol, which you get from the sum of the ASCII code + the last digit  $\rightarrow 106 + 6 \rightarrow 112 \rightarrow$  **p**.
  - At **the end** of the string, which represents the result, concatenate the symbol, which you get from subtracting the ASCII code – the first digit  $\rightarrow 106 - 1 \rightarrow 105 \rightarrow$  **i**.

Using the method shown above, write a program that takes **a sequence of characters, encrypts them**, and prints the result on one line.

The input data will always be valid. The Main method must read the data given by the user – an integer **n**, followed by a character for each of the following **n** lines.

Encrypt the symbols and add them to the encrypted string. In the end, as a result, you must print **an encrypted string** as in the following example.

Example:

- S, o, f, t, U, n, i  $\rightarrow$  V83Kp11nh12ez16sZ85Mn10mn15h

### Sample Input and Output

| Input                                | Output                       |
|--------------------------------------|------------------------------|
| 7<br>S<br>o<br>f<br>t<br>U<br>n<br>i | V83Kp11nh12ez16sZ85Mn10mn15h |

| Input                                | Output                       |
|--------------------------------------|------------------------------|
| 7<br>B<br>i<br>r<br>a<br>H<br>a<br>x | H66<n15hv14qh97XJ72Ah97xx10w |

Hints and Guidelines

Our variable from type **String**, which will store the value of the result, will have an initial value of **String str = ""** (Empty String). The loop should iterate **n** times, and in every iteration, we will add to our variable the encrypted symbol.

To find the first and the last digit from the ASCII code, we will use the algorithm that we have used when solving the "Integer to Base" problem.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/665#14>.

# Chapter 11. Tricks and Hacks

In the current chapter, we are going to see some tricks, hacks, and techniques, which will make our work with C# easier in the IntelliJ IDEA. In particular, we will see:

- How to properly **format our code**
- Conventions for **naming elements in the code**
- Some **keyboard shortcuts**
- Some **code snippets**
- Techniques to **debug our code**

## Code Formatting

The right formatting of our code will make it easier to read and understand in case someone else needs to work with it. This is important because in practice we will need to work in a team with other people and it is highly important to write our code in a way that our colleagues can **quickly understand** it.

There are some defined rules for correct formatting of the code, which are collected in one place and are called **conventions**. The conventions are a group of rules, generally accepted by the programmers using a given language, which are massively used. These conventions help to build norms in given languages – what is the best way to write and what are good practices. It is accepted that if a programmer follows them then his code is easy to read and understand.

The Java language is developed by **Sun Microsystems** (a tech company, later acquired by **Oracle**). Sun is the one that lays the foundations of the current well-established conventions in Java. You should know that even if you don't follow the conventions given by **Microsoft**, your code will work (as long as it is properly written), but it will not be easy to understand. This, of course, is not fatal at a base level, but the faster you get used to writing quality code the better.

## The Official Java Code Conventions

The most widely adopted modern **Java convention** is called [Google Java Style Guide](#) and this book will be closely following it.

For code formatting in the Java ecosystem, the following is recommended for the **curly brackets {}** - the opening bracket is supposed to be on the same line and the closing bracket should be right underneath the construct:

```
if (someCondition) {  
    System.out.println("Inside the if statement");  
}
```

You can see that the command **System.out.println(...)** in the example is **indented by 4 white spaces (one tab)**, which is a standard in Java. If the given block enclosed in curly

brackets is indented by any number of tabs, then the closing curly bracket `}` must be right below the beginning of the construct, as in the example below:

```
if (someCondition) {
    if (anotherCondition) {
        System.out.println("Inside the if statement");
    }
}
```

Below you can see an example for badly formatted code according to the accepted conventions for writing code in Java:

```
if(someCondition)
{
    System.out.println("Inside the if statement");}
```

The first thing that we see is the curly brackets `{}`. The first (opening) bracket should be just after the `if` condition, and the second (closing) bracket – below the command `System.out.println(...)`, at a new and empty line. In addition, the command inside the `if` construction should be offset by 4 white spaces (one tab). Just after the keyword `if`, before the condition and preceding the `{` you should put a space.

The same rule applies for the `for` loops and all other constructions with curly brackets `{}`. Here are some more examples:

Correct formatting:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Wrong formatting:

```
for(int i=0;i<5;i++)
{
    System.out.println(i);
}
```

## Code Formatting Shortcuts in IntelliJ IDEA

For your comfort, there are keyboard shortcuts in IntelliJ IDEA, which we will explain later in this chapter, but for now, we are interested in two specific combinations. One of the combinations is for formatting the code in the whole document, and the other one – is for formatting a part of the code. If we want to format the whole code we need to press [CTRL + ALT + L]. In case we need to format only a part of the code, we need to mark this part with the mouse and press [Ctrl + Shift + Alt + L].



Let's use the wrongly formatted example from earlier:

```
for(int i=0;i<5;i++)
{
System.out.println(i);
}
```

If we press [CTRL + ALT + L], which is the combination to format the whole document, we will have a code, formatted according to the accepted conventions for Java, which will look as follows:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

This key combination in IntelliJ IDEA can help us if we work with a badly formatted code.

## Naming Code Elements

In this section, we will focus on the accepted conventions for naming projects, files, and variables, defined by Microsoft.

### Naming Projects and Files

It is recommended to use a descriptive name for naming projects and files, which suggests the role of the respective file / project, and at the same time, the **PascalCase** convention is also recommended. This is a convention for naming elements, in which each word, including the first one, starts with an uppercase character, for example **ExpressionCalculator**.

Example: This course starts with the **First Steps in Coding** lecture, therefore an exemplary name for the solution for this lecture can be **FirstStepsInCoding**. The same convention applies to the files in a project. If we take for example the first problem in the **First steps in Coding** lecture, it is called **Hello World**, therefore our file in the project will be called **HelloWorld**.

### Naming Variables

In programming variables keep data, and for the code to be more understandable, the name of a variable should suggest its purpose. Here are some recommendations for naming variables:

- The name should be **short and descriptive** and explain what the variable serves for.
- The name should only contain the letters a-z, A-Z, the numbers 0-9, and the symbols '\$' and '\_'.
- It is accepted in Java for the variables to always **begin with a lowercase letter** and to contain lowercase letters, and each next word in them should start with an uppercase letter (this naming is also known as **camelCase** convention).

- You should be careful about uppercase and lowercase letters because Java distinguishes them. For example, **age** and **Age** are different variables.
- The names of the variables cannot coincide with keywords in the Java language, for example **int** is an invalid name for a variable.



Although using the symbol `_` in the names of variables is allowed, in Java it is not recommended and is considered a bad style of naming.

## Naming – Examples

Here are some examples for **well-named** variables:

- **firstName**
- **age**
- **startIndex**
- **lastNegativeNumberIndex**

Here are some examples for **badly named variables**, even though the names are correct according to the Java compiler:

- **\_firstName** (starts with '\_')
- **last\_name** (contains '\_')
- **AGE** (written in uppercase, which is a **badly named variable** but **well-named constant**)
- **Start\_Index** (starts with an uppercase letter and contains '\_')
- **lastNegativeNumber\_Index** (contains '\_')

At first look, all these rules can seem meaningless and unnecessary, but with time passed and experience gaining you will see the need for **conventions for writing quality code** to be able to work more easily and faster in a team. You will understand that the work with a code, which is written without complying with any rules for code quality, is annoying.

## Shortcuts in IntelliJ IDEA

In the previous section, we mentioned two of the combinations that are used for formatting code. One of them [Ctrl + Alt + L] is used for **formatting the whole code in a file**, and the second one [Ctrl + Shift + Alt + L] serves if we want to **format just a piece of the code**. These combinations are called **shortcuts** and now we will give more thorough information about them.

Shortcuts are **combinations** that give us the possibility to do some things in an **easier and faster** way, and each IDE has its shortcuts, even though most of them are recurring. Now we will look at some of the **shortcuts** in **IntelliJ IDEA**:

| Combination        | Action                                                                                     |
|--------------------|--------------------------------------------------------------------------------------------|
| [CTRL + F]         | Opens the <b>search window</b> , by which we can <b>search in our code</b> .               |
| [CTRL + /]         | <b>Comments</b> part of the code. / <b>Uncomments a code</b> , which is already commented. |
| [CTRL + Z]         | <b>Brings back one change</b> (so-called Undo).                                            |
| [Ctrl + Shift + Z] | The combination is opposite of [CTRL + Z] (the so-called Redo).                            |
| [CTRL + ALT + L]   | <b>Formats the code</b> according the default conventions.                                 |
| [CTRL + Backspace] | <b>Deletes</b> the word to the left of the cursor.                                         |
| [CTRL + Del]       | <b>Deletes</b> the word to the right of the cursor.                                        |
| [CTRL + Shift + S] | <b>Saves</b> all files in the project.                                                     |
| [CTRL + S]         | <b>Saves</b> the current file.                                                             |
| [CTRL + D]         | <b>Copies</b> the current line or the selected fragment.                                   |
| [CTRL + Y]         | <b>Deletes</b> the current line.                                                           |

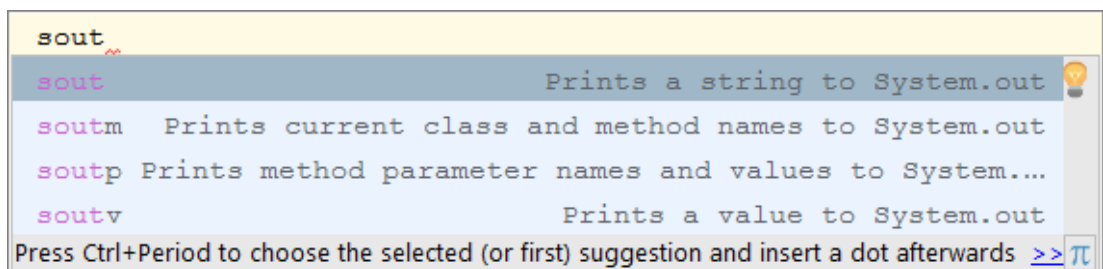
More about the **shortcuts in IntelliJ IDEA\*** can be found here:  
<https://www.jetbrains.com/help/idea/keyboard-shortcuts-by-category.html>.

## Code Snippets in IntelliJ IDEA

In IntelliJ IDEA there are the so-called **code snippets**, which write a block of code by using a code template. For example, by writing the shortcode **sout** + [Enter] the following code is generated in the body of our program, in the place of the shortcode:

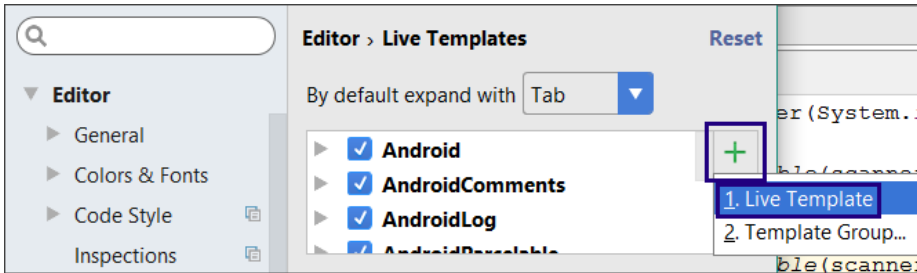
```
System.out.println();
```

This is called “unfolding a code snippet”. The **fori** + [Enter] snippet works in the same way. On the figure below you can see the **sout** snippet in action:

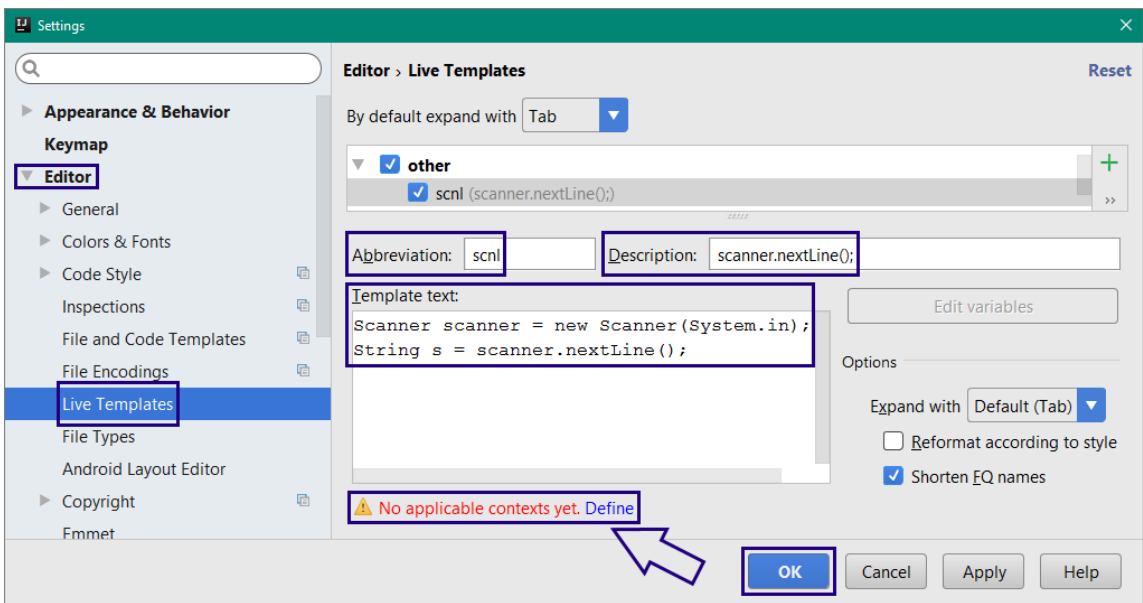


## Creating Your Code Snippet

In this section, we are going to show you how to make your code snippet. We will see how to make a code snippet for the `scanner.nextLine();` command. To begin we must create a new empty project in IntelliJ IDEA and go to [File → Settings → Editor → Live Templates], and choose [+ → Live Template] as shown on the picture:



A new window that looks like the one on the image below pops up:

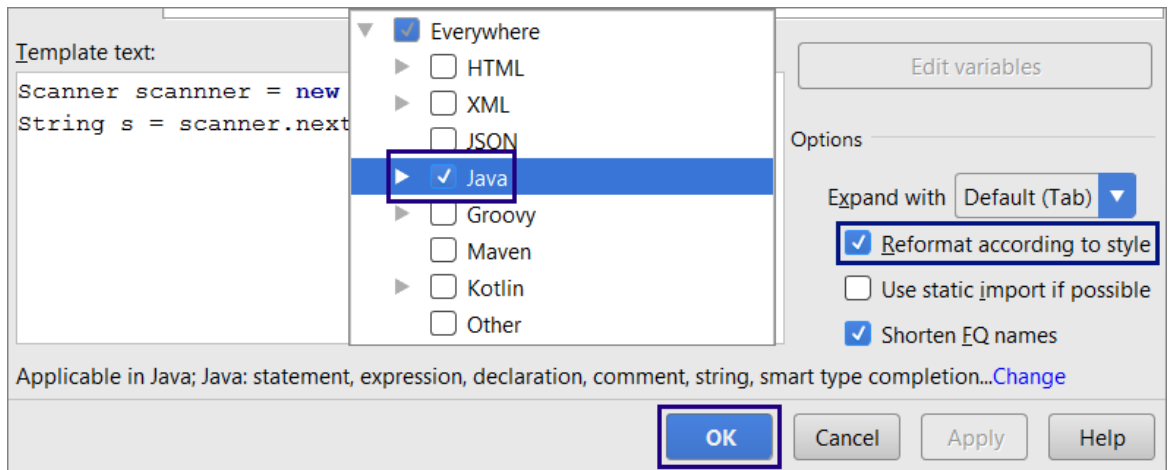


The following information should be entered:

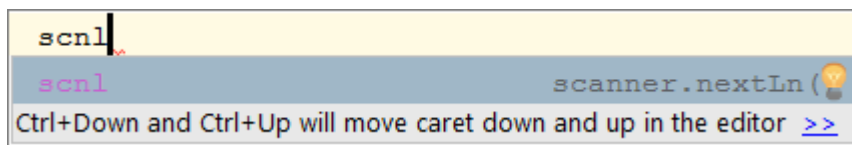
- [Abbreviation] - we specify the code snippet that we wish to use. In our case this is **scnl**.
- [Description] - this is the place for our snippet's description. In our case this is **scanner.nextLine()**.
- [Template text] - we enter the code to be generated in the event of snippet usage. In our case this is:

```
Scanner scanner = new scanner(System.in);
String s = scanner.nextLine();
```

Next, we select **[Reformat according to style]** and choose **[Java]** from **[Define]** list. To finish the procedure press **[OK]** as it's shown in the picture below:



Now when we write **scnl** in IntelliJ IDEA, our new snippet is going to appear:



## Code Debugging Techniques

Debugging plays an important role in the process of creating software, which allows us to follow the implementation of our program **step by step**. With this technique, we can follow the values of the local variables, because they are changing during the execution of the program, and to remove possible errors (bugs). The process of debugging includes:

- Finding the problems (bugs).
- Locating the code, which causes the problems.
- Correcting the code, which causes the problems, so that the program works correctly.
- Testing to make sure that the program works correctly after the corrections we have made.

## Debugging in IntelliJ IDEA

IntelliJ IDEA gives us a **built-in debugger**, thanks to which we can place **breakpoints** at places we have chosen. When it reaches a **breakpoint**, the program **stops running** and allows **step-by-step running** of the remaining lines. Debugging allows us to **get into the details of the program** and see where exactly the errors occur and what is the reason for this.

To demonstrate how to use the debugger we will use the following program:

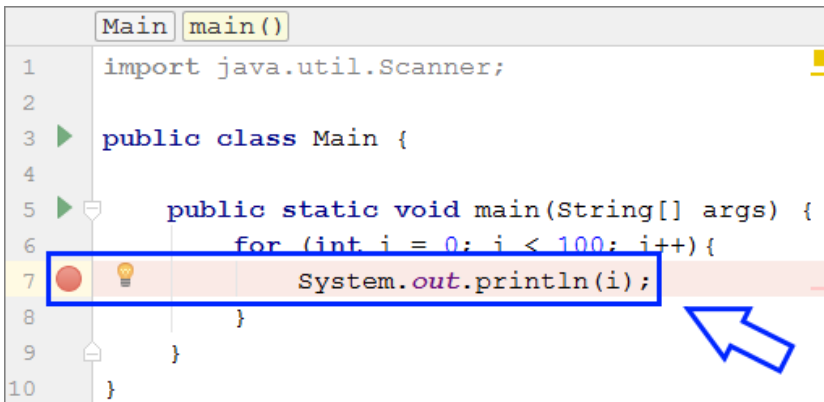
```
public static void main(String[] args) {
```

```

    for (int i = 0; i < 100; i++) {
        System.out.println(i);
    }
}

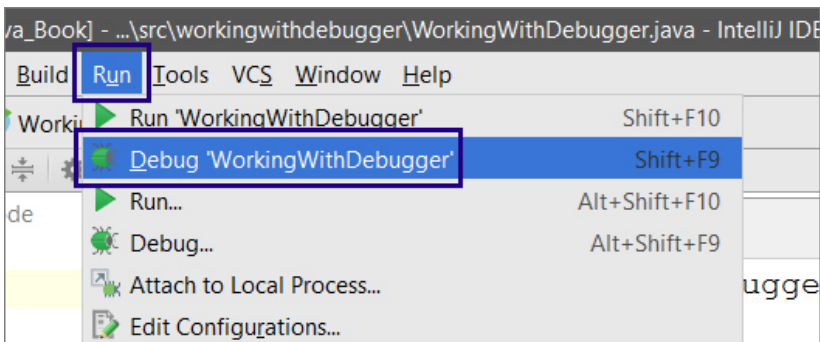
```

We will place a **breakpoint** at the function `System.out.println(...)`. For this, we will need to move our cursor to the line, which prints on the console, and press `[Ctrl + F8]`, alternatively we can simply click using the left mouse button on the right side of the line number. A **breakpoint** appears, showing where the program will **stop** running:

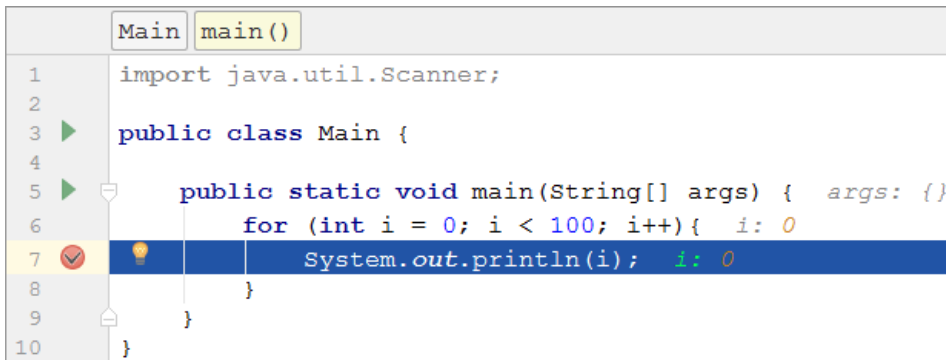


## Starting The Program in Debug Mode

To start the program in debug mode, we choose `[Run] -> [Debug <class name>]` or press `[SHIFT + F9]`:



After starting the program, we can see that it **stops executing** at line 8, where we placed our breakpoint. The code in the current line is colored in additional color and we can run it step by step. To go to the next line, we use the key `[F8]`. We can see that the code on the current line hasn't been executed yet. It will execute when we go ahead with debugging the next line. The current value of the variable is depicted in green, in this case, `i = 0` can be seen in the following picture.



From the [Variables] window we can observe the changes in the local variables.



## Tricks for Java Developers

In this section, we will recall some **tricks and techniques** in programming with Java, already seen in this book, which can be very useful if you attend an exam for beginner programming.

### Formatted Output with printf()

For printing long and complex sequences of elements, we can use the **printf(...)** method. This method is an abbreviation of "Print Formatted". The main idea of **printf(...)** is to take a **special string** formatted with special **formatting symbols** and a comma-separated list of values that have to substitute the **formatting specifiers**.

```
printf(<formatted string>, <param1>, <param2>, <param3>, ...)
```

Example:

```
String str = "some text";
System.out.printf("%s", str);
// This will print on the console "some text"
```

The first argument of the **printf(...)** method is the formatting string. In this case **%s** means that **%s** is going to be substituted by the string **str** and the value of **str** is what you'll see printed on the console.

```
String str1 = "some text";
int number = 5;
String str2 = "some more text";
System.out.printf("%s %d %s \n", str1, number, str2);
// This will print on the console "some text 5 some more text"
```

Notice that in this example we can pass **non-exclusively string variables**. The first argument is a **formatting string**. Following it comes to a **sequence of arguments** that are replacing any instance of a formatting specifier (meaning % followed by a single character, e.g. %s or %d). The first %s means that the first argument that is passed after the formatting string, is going to take its place, in our case that's **str1**. After that, there's %d which means that it's going to be substituted by the first integer number that is among the arguments. The last special symbol is %s which means that it'll be replaced by the next string that can be found in the arguments. Finally, there's \n and that is a special symbol that denotes a new line. A single variable can be used multiple times.

```
String str = "some text";
int number = 5;
System.out.printf("%s %d %s \n", str, number, str);
// This will print on the console "some text 5 some text"
```

## Rounding of Floating-Point Numbers

Real numbers are represented in Java with the types **float** and **double**.

```
double number = 5.432432;
System.out.println(Math.round(number));
// This will print on the console "5"
```

If the digit at the first decimal place is less than 5, just like in the example above, then the number is rounded down, otherwise - it's rounded up as you can see in the example below:

```
double number = 5.543;
System.out.println(Math.round(number));
// This will print on the console "6"
```

## Other Rounding Methods

In case we always want to round down instead of **Math.round(...)** we can use another method - **Math.floor(...)**, which always rounds down. For example, if we have the number 5.99 and we use **Math.floor(5.99)**, we will get the number 5.0\*.

We can also do the exact opposite - to always round up using the method **Math.ceil(...)**. Again, if we have for example 5.11 and we use **Math.ceil(5.11)**, we will get 6.0. Here are some examples:

```
double numberToFloor = 5.99;
System.out.println(Math.floor(numberToFloor));
// This will print on the console 5.0

double numberToCeiling = 5.11;
```



```
System.out.println(Math.ceil(numberToCeiling));
// This will print on the console 6.0
```

## Formatting with 2 Digits After The Decimal Point

When we print numbers, we often need to round them to 2 digits after the decimal point, e.g.

```
double number = 5.432432;
System.out.printf("%.2f%n", number);
// This will print on the console 5.43
```

In the given example a formatting string `%.2f` is used, which rounds the number variable to two decimal places. We have to take into account that the number before the letter **f** means the number of decimal places up to which the result will be rounded (e.g. the formatting string can very well be `%.3f` or `%.5f`). When formatting a string using `printf()`, it's recommended to use `%n` as a symbol for new line and not `/n`.

## How to Write a Conditional Statement?

The conditional **if** construction contains the following elements:

- Keyword **if**
- A Boolean expression (condition)
- Body of the conditional construction
- Optional: **else** clause

```
if (condition) {
    // body
} else (condition) {
    // body
}
```

To make it easier we can use a **code snippet** for an **if** construction:

- **if** + [Enter]

## How to Write a for Loop?

For a **for** loop, we need a couple of things:

- **Initializing block**, in which the counter variable is declared (**int i**) and its initial value is set
- **Condition for repetition** (**i <= 10**)
- **Loop variable (counter) updating statement** (**i++**)
- **Body of the loop**, holding statements

```
for (int i = 0; i <= 10; i++) {
```

```
// body  
}
```

It's important to know that the three elements of the **for** loop are **optional** and can be omitted. **for( ; ; ) { ... }** is a valid **for** loop statement.

To make it easier we can use a code snippet for a **for** loop:

- **fori** + [Enter]

## What Have We Learned from This Chapter?

In **this** chapter, we have learned how to **correctly format** and name the elements of our **code**, some **shortcuts** in IntelliJ IDEA, some **code snippets**, and we analyzed how to **debug** the code.

# Conclusion

If you have **read the entire** book and you've solved all the problems from the exercises and reached the present conclusion, **congratulations!** You've already made the **first step** in learning the **profession of a programmer**, but there is a **long way** to go until you become **really good** and make **software writing** your **profession**.

## Developer Skills

Remember the [four main groups of skills](#) that each programmer must have to work in the industry:

- Skill #1 – **writing the program code** (20% of programmer's skills) – covered to a large degree by this book, but you must learn additional basic data structures, classes, objects, functions, strings, and other elements of code writing.
- Skill #2 – **algorithmic thinking** (30% of programmer's skills) – covered partially by this book and developed mostly by solving a large amount of diverse algorithmic problems.
- Skill #3 – **a fundamental understanding of the profession** (25% of programmer's skills) – acquired for a few years in combination with learning and practice (reading books, watching video lessons, attending courses, and mostly by writing diverse projects in various technological areas).
- Skill #4 – **programming languages and software technologies** (25% of programmer's skills) – acquired in a long period, by a lot of practice, consistent reading, and writing projects. Such knowledge and skills quickly get outdated and need to be updated frequently. Good programmers are involved in studying new technologies every day.

## This Book is Only the First Step!

The **present** book on programming basics is just the **first step** in building the skills of a programmer. If you were able to solve **all problems**, this means you have **obtained valuable knowledge** in the programming principles with Java on a **basic level**. You are about to start **in-depth** studying of programming and develop **your algorithmic thinking**, and then add **technological knowledge** regarding the Java language and the Java ecosystem (Java EE, Spring, Android, etc.), front-end technologies (HTML, CSS, JavaScript), many other concepts, technologies, and instruments for software development.

If you were **not able** to solve all problems or a large part of them, go back and solve them! Remember that **becoming a programmer** requires **a lot of work and effort**. This profession is not for lazy people. There is no way to learn it unless **you seriously practice** programming for years!

As we have already explained, the programmer's first and most basic skill is easily writing code. This is namely the mission of this book: to teach you how to code.

## How to Proceed After This Book?

This book **gives you a solid foundation**, thanks to which it will be easy for you to continue developing as a programmer. If you wonder how to continue your development, you have the following possibilities:

- to study for a [software developer at SoftUni](#) and make programming your profession;
- to continue developing as a programmer **in your way**, for example, through self-training or via online lessons;
- to **stay at the coder level**, without going more seriously into programming.

## Study Software Engineering in SoftUni

The first option, which we recommend, is to enroll in a [Software Developer program](#) to master programming on a professional level. The SoftUni curriculum is carefully developed by **Dr. Svetlin Nakov and his team**, to provide you consequently and with gradually increasing complexity, all the skills that a software engineer must possess, **to start a career as a software developer** in an IT company.

### It Takes at Least a Year of Intense Code Writing to Become a Programmer

Keep in mind that **to become a programmer takes a lot of effort**, writing tens of thousands of lines of code, and solving hundreds, even thousands of practical problems, and this takes years! If someone offers an easier program with the promise of you becoming a programmer and landing a job within 3-4 months of starting the course, they are either lying to you or providing a significantly lower-quality education that will not prepare you enough for even an intern position. There are exceptions, of course, for example, if you are not starting from scratch, or if you have extremely well-developed engineering thinking, or if you apply for a very low position (for example technical support), but in general, **you cannot become a programmer if you haven't spent at least 1 year of intense learning and code writing!**

### The SoftUni Curriculum for Software Engineers

SoftUni's [self-paced programs](#) are designed as a sequence of modules giving you fundamental knowledge in software development and practical skills to prepare you to work as a programmer with the latest software technologies. The classes are divided into learning theory (30%) and doing exercises and projects (70%).

### How Many Hours Per Day Does the Training Take?

The training for software engineers at SoftUni is a **serious occupation** and you need to spend it **at least 4-5 hours every day**, preferably your entire attention and time. Combining **training with other obligations** is not always easy. Of course, it is always best if you can dedicate all your time and attention to mastering programming but SoftUni's self-paced [Software Developer programs](#) make it possible to learn at your own pace. They are great for **high school pupils, university students, and people with full or part-time jobs**. And remember, you cannot become a successful software developer by only practicing 2-

4 hours a week. As a minimum, you need to spend at least 20-30 hours a week learning to code!

## SoftUni for People Who Work and Study

We recommend to everyone who gets an excellent score at Judge Problems and is passionate about making programming their profession, to consider leaving the rest of their commitments aside to **spend more time on learning the profession of a software engineer** and start making a living through it.

We recommend to all who have trouble in solving the programming problems from the book to spend more time on better learning, understanding, and most of all, practicing the material studied in the present book. If you cannot easily solve the problems in this book, you will not be able to cope with programming and software development in the future.

## Study Software Engineering in Your Way

Another possibility to develop after this book is to **continue studying programming outside of SoftUni**. You can enroll or subscribe to **video training** that goes into more details in programming with Java or other languages and development platforms. You can **read books** on programming and software technologies, follow **online tutorials** and other online resources – there are plenty of free materials on the Internet. However, keep in mind that the most important thing about the profession of a programmer is **to do practical projects!**

You cannot become a programmer without a lot of code writing and intense practice. Allocate **sufficient time** to it. You cannot become a programmer for a month or two. On the Internet, you will find a wide variety of **free resources**, such as books, manuals, video lessons, online and on-site courses on programming and software development. However, you need to invest **at least a year or two** to acquire the foundation level needed for starting a job.

After you gain some experience, find a way to start **an internship in a company** (which will be almost impossible unless you'd spent at least a year of intense code writing before that) or come up with **your practical project**, on which you need to spend a few months, even a year, to learn based on the trial-and-error principle.



Keep in mind that there are many ways to become a programmer, but they all have something in common: **intense code writing and years of practice!**

## Recommended Resources for Developers

A **huge amount of resources** is available on the Web for developers: online trainings, courses, tutorials, books, interactive training sites, etc.

We can not mention all of them because they change over time and because the list might be huge. What we recommend is to **join the developer communities** in your region, because this will help you a lot when you study.

## Online Communities for Beginners in Programming

Regardless of the path, you have chosen, if you are seriously involved in programming, we recommend subscribing to specialized **online forums, discussion groups, and communities**, from which you can get assistance from your colleagues and track the novelties in the software industry.

If you will study programming seriously **surround yourselves with people who are involved in programming** seriously. Join **communities of software developers**, attend software conferences, go to events for programmers, find friends with whom you can talk about programming, discuss problems and bugs, find an environment that can help you. In most large towns, there are free events for programmers a few times a week. In smaller localities, you have the Internet and access to the entire online community.

Here are some recommended **resources** that will be useful for your development as a programmer:

- <https://learn.softuni.org> – the official page for SoftUni's self-paced Software Developer programs and open courses
- <https://www.facebook.com/softuni.global> – official Facebook page of SoftUni. By it, you will learn about new courses, seminars, and events related to programming and software development.
- <https://introprogramming.info> – official website of the books "Programming Basics" with C#, Java, JavaScript, and Python by Dr. Svetlin Nakov and his team. The books examine in-depth programming basics, basic data structures and algorithms, object-oriented programming, and other basic skills and are an excellent continuation for reading after the current book. However, **besides reading, you need to do intense code writing**, do not forget that!
- <https://stackoverflow.com> – **Stack Overflow** is one of the **widest** discussion forums for programmers worldwide, where you will find assistance for any possible question in the world of programming. If you are fluent in English, look up at StackOverflow and ask your questions there.
- <https://www.udemy.com> – **Udemy** is one of the biggest **marketplaces** for technical training, offered free or at affordable prices.
- <https://www.meetup.com> – look for **tech meetups** around your city and involve in communities that you like. Most tech meetups are free, and newbies are welcome.

## Good Luck to All!

On behalf of the entire authors' team, we **wish you endless success in the profession and your life!** We will be really happy, if with our help you are **passionate about programming** and we have inspired you to boldly take up the profession of "software engineer", which will: bring you a good job, working with pleasure, quality life, and prosperity, as well as great perspectives for development and possibilities for making impressive projects with inspiration and passion.

*Sofia, December 5, 2017*



In your hands you hold something more than a **programming book, textbook, or tutorial**. This modern teaching resource guides you through the **first steps in programming** using a little text and many code examples with explanations, combined with carefully selected **practical problems** and a **judge system** for immediate **automatic code evaluation**.

The teaching content was designed personally by **Svetlin Nakov**, who has helped more than **150 000 beginners** in the last 20 years to **start coding** and start learning the **software engineering profession**.

Svetlin Nakov is known for his **inspiration** and ability to teach programming skills gradually, using **small learning units**, full of real-world examples and **practical problem solving**.

Remember that **programming skills** can only be learned through a lot of **code writing** and a lot of **practical problem solving**, not through reading books and watching videos, so be sure that you solve the exercises in each book section. Good luck!

Web site: [java-book.softuni.org](http://java-book.softuni.org)



## AUTHORS TEAM:

Asya Dikova-Kirova  
Daniel Tsvetkov  
Dimitar Dalev  
Ivaylo Ivanov  
Kristiyan Pamidov  
Milena Georgieva  
Nikolay Bankin  
Petar Ivanov  
Rositsa Nenova  
Svetlin Nakov  
Teodor Kurtev  
Tsvetan Georgiev  
Vaydin Kachanov  
Ventsislav Petrov  
Zhulieta Atanasova

ISBN 978-619-00-1402-7



9 786190 014027 >